

The Java[®] Virtual Machine Specification

Java SE 26 Edition

Tim Lindholm
Frank Yellin
Gilad Bracha
Alex Buckley
Daniel Smith

2026-02-17

Specification: JSR-401 Java SE 26

Version: 26

Status: Final Release

Release: March 2026

Copyright © 1997, 2026, Oracle America, Inc.

All rights reserved.

The Specification provided herein is provided to you only under the Limited License Grant included herein as Appendix A. Please see Appendix A, *Limited License Grant*.

Table of Contents

1 Introduction 1

- 1.1 A Bit of History 1
- 1.2 The Java Virtual Machine 2
- 1.3 Organization of the Specification 5
- 1.4 Notation 5
- 1.5 Preview Features 6
 - 1.5.1 Restrictions on the Use of Preview Features 7
 - 1.5.2 Current Preview VM Features 8
- 1.6 Feedback 8

2 The Structure of the Java Virtual Machine 9

- 2.1 The class File Format 9
- 2.2 Data Types 10
- 2.3 Primitive Types and Values 10
 - 2.3.1 Integral Types and Values 11
 - 2.3.2 Floating-Point Types and Values 12
 - 2.3.3 The returnAddress Type and Values 13
 - 2.3.4 The boolean Type 14
- 2.4 Reference Types and Values 14
- 2.5 Run-Time Data Areas 15
 - 2.5.1 The pc Register 15
 - 2.5.2 Java Virtual Machine Stacks 15
 - 2.5.3 Heap 16
 - 2.5.4 Method Area 17
 - 2.5.5 Run-Time Constant Pool 17
 - 2.5.6 Native Method Stacks 18
- 2.6 Frames 19
 - 2.6.1 Local Variables 19
 - 2.6.2 Operand Stacks 20
 - 2.6.3 Dynamic Linking 21
 - 2.6.4 Normal Method Invocation Completion 21
 - 2.6.5 Abrupt Method Invocation Completion 22
- 2.7 Representation of Objects 22
- 2.8 Floating-Point Arithmetic 22
- 2.9 Special Methods 26
 - 2.9.1 Instance Initialization Methods 26
 - 2.9.2 Class Initialization Methods 26
 - 2.9.3 Signature Polymorphic Methods 27
- 2.10 Exceptions 28
- 2.11 Instruction Set Summary 30

- 2.11.1 Types and the Java Virtual Machine 31
- 2.11.2 Load and Store Instructions 33
- 2.11.3 Arithmetic Instructions 34
- 2.11.4 Type Conversion Instructions 35
- 2.11.5 Object Creation and Manipulation 37
- 2.11.6 Operand Stack Management Instructions 38
- 2.11.7 Control Transfer Instructions 38
- 2.11.8 Method Invocation and Return Instructions 38
- 2.11.9 Throwing Exceptions 39
- 2.11.10 Synchronization 39
- 2.12 Class Libraries 40
- 2.13 Public Design, Private Implementation 41

3 Compiling for the Java Virtual Machine 43

- 3.1 Format of Examples 43
- 3.2 Use of Constants, Local Variables, and Control Constructs 44
- 3.3 Arithmetic 49
- 3.4 Accessing the Run-Time Constant Pool 50
- 3.5 More Control Examples 51
- 3.6 Receiving Arguments 54
- 3.7 Invoking Methods 55
- 3.8 Working with Class Instances 57
- 3.9 Arrays 59
- 3.10 Compiling Switches 61
- 3.11 Operations on the Operand Stack 63
- 3.12 Throwing and Handling Exceptions 63
- 3.13 Compiling `finally` 67
- 3.14 Synchronization 70
- 3.15 Annotations 71
- 3.16 Modules 72

4 The class File Format 75

- 4.1 The `ClassFile` Structure 76
- 4.2 Names 81
 - 4.2.1 Binary Class and Interface Names 81
 - 4.2.2 Unqualified Names 82
 - 4.2.3 Module and Package Names 82
- 4.3 Descriptors 83
 - 4.3.1 Grammar Notation 83
 - 4.3.2 Field Descriptors 83
 - 4.3.3 Method Descriptors 85
- 4.4 The Constant Pool 86
 - 4.4.1 The `CONSTANT_Class_info` Structure 89
 - 4.4.2 The `CONSTANT_Fieldref_info`, `CONSTANT_Methodref_info`, and `CONSTANT_InterfaceMethodref_info` Structures 90
 - 4.4.3 The `CONSTANT_String_info` Structure 91

- 4.4.4 The CONSTANT_Integer_info and CONSTANT_Float_info Structures 91
- 4.4.5 The CONSTANT_Long_info and CONSTANT_Double_info Structures 93
- 4.4.6 The CONSTANT_NameAndType_info Structure 94
- 4.4.7 The CONSTANT_Utf8_info Structure 95
- 4.4.8 The CONSTANT_MethodHandle_info Structure 97
- 4.4.9 The CONSTANT_MethodType_info Structure 99
- 4.4.10 The CONSTANT_Dynamic_info and CONSTANT_InvokeDynamic_info Structures 99
- 4.4.11 The CONSTANT_Module_info Structure 100
- 4.4.12 The CONSTANT_Package_info Structure 101
- 4.5 Fields 102
- 4.6 Methods 104
- 4.7 Attributes 108
 - 4.7.1 Defining and Naming New Attributes 115
 - 4.7.2 The ConstantValue Attribute 116
 - 4.7.3 The Code Attribute 117
 - 4.7.4 The StackMapTable Attribute 120
 - 4.7.5 The Exceptions Attribute 128
 - 4.7.6 The InnerClasses Attribute 129
 - 4.7.7 The EnclosingMethod Attribute 131
 - 4.7.8 The Synthetic Attribute 133
 - 4.7.9 The Signature Attribute 134
 - 4.7.9.1 Signatures 135
 - 4.7.10 The SourceFile Attribute 139
 - 4.7.11 The SourceDebugExtension Attribute 140
 - 4.7.12 The LineNumberTable Attribute 141
 - 4.7.13 The LocalVariableTable Attribute 142
 - 4.7.14 The LocalVariableTypeTable Attribute 144
 - 4.7.15 The Deprecated Attribute 145
 - 4.7.16 The RuntimeVisibleAnnotations Attribute 146
 - 4.7.16.1 The element_value structure 148
 - 4.7.17 The RuntimeInvisibleAnnotations Attribute 151
 - 4.7.18 The RuntimeVisibleParameterAnnotations Attribute 152
 - 4.7.19 The RuntimeInvisibleParameterAnnotations Attribute 154
 - 4.7.20 The RuntimeVisibleTypeAnnotations Attribute 155
 - 4.7.20.1 The target_info union 161
 - 4.7.20.2 The type_path structure 166
 - 4.7.21 The RuntimeInvisibleTypeAnnotations Attribute 172
 - 4.7.22 The AnnotationDefault Attribute 173
 - 4.7.23 The BootstrapMethods Attribute 174
 - 4.7.24 The MethodParameters Attribute 175
 - 4.7.25 The Module Attribute 177
 - 4.7.26 The ModulePackages Attribute 184
 - 4.7.27 The ModuleMainClass Attribute 185
 - 4.7.28 The NestHost Attribute 186
 - 4.7.29 The NestMembers Attribute 186

- 4.7.30 The Record Attribute 188
- 4.7.31 The PermittedSubclasses Attribute 189
- 4.8 Format Checking 191
- 4.9 Constraints on Java Virtual Machine Code 191
 - 4.9.1 Static Constraints 192
 - 4.9.2 Structural Constraints 195
- 4.10 Verification of class Files 199
 - 4.10.1 Verification by Type Checking 200
 - 4.10.1.1 Accessors for Java Virtual Machine Artifacts 202
 - 4.10.1.2 Verification Type System 206
 - 4.10.1.3 Instruction Representation 210
 - 4.10.1.4 Stack Map Frames and Type Transitions 212
 - 4.10.1.5 Type Checking Methods 216
 - 4.10.1.6 Type Checking Code Streams 220
 - 4.10.1.7 Type Checking Load and Store Instructions 222
 - 4.10.1.8 Type Checking for protected Members 224
 - 4.10.1.9 Type Checking Instructions 228
 - 4.10.2 Verification by Type Inference 348
 - 4.10.2.1 The Process of Verification by Type Inference 348
 - 4.10.2.2 The Bytecode Verifier 348
 - 4.10.2.3 Values of Types long and double 351
 - 4.10.2.4 Instance Initialization Methods and Newly Created Objects 352
 - 4.10.2.5 Exceptions and finally 353
- 4.11 Limitations of the Java Virtual Machine 355

5 Loading, Linking, and Initializing 359

- 5.1 The Run-Time Constant Pool 359
- 5.2 Java Virtual Machine Startup 362
- 5.3 Creation and Loading 363
 - 5.3.1 Loading Using the Bootstrap Class Loader 365
 - 5.3.2 Loading Using a User-defined Class Loader 366
 - 5.3.3 Creating Array Classes 367
 - 5.3.4 Loading Constraints 368
 - 5.3.5 Deriving a Class from a class File Representation 369
 - 5.3.6 Modules and Layers 373
- 5.4 Linking 375
 - 5.4.1 Verification 376
 - 5.4.2 Preparation 376
 - 5.4.3 Resolution 377
 - 5.4.3.1 Class and Interface Resolution 379
 - 5.4.3.2 Field Resolution 379
 - 5.4.3.3 Method Resolution 380
 - 5.4.3.4 Interface Method Resolution 382
 - 5.4.3.5 Method Type and Method Handle Resolution 384
 - 5.4.3.6 Dynamically-Computed Constant and Call Site Resolution 388

- 5.4.4 Access Control 393
- 5.4.5 Method Overriding 395
- 5.4.6 Method Selection 396
- 5.5 Initialization 397
- 5.6 Binding Native Method Implementations 400
- 5.7 Java Virtual Machine Termination 400

6 The Java Virtual Machine Instruction Set 403

- 6.1 Assumptions: The Meaning of "Must" 403
- 6.2 Reserved Opcodes 404
- 6.3 Virtual Machine Errors 404
- 6.4 Format of Instruction Descriptions 405
 - mnemonic 406
- 6.5 Instructions 408
 - aaload* 409
 - aastore* 410
 - aconst_null* 411
 - aload* 412
 - aload_<n>* 413
 - anewarray* 414
 - areturn* 415
 - arraylength* 416
 - astore* 417
 - astore_<n>* 418
 - athrow* 419
 - baload* 421
 - bastore* 422
 - bipush* 423
 - caload* 424
 - castore* 425
 - checkcast* 426
 - d2f* 429
 - d2i* 430
 - d2l* 431
 - dadd* 432
 - daload* 434
 - dastore* 435
 - dcmp<op>* 436
 - dconst_<d>* 438
 - ddiv* 439
 - dload* 441
 - dload_<n>* 442
 - dmul* 443
 - dneg* 444
 - drem* 445
 - dreturn* 447
 - dstore* 448

dstore_<n> 449
dsub 450
dup 451
dup_x1 452
dup_x2 453
dup2 454
dup2_x1 455
dup2_x2 456
f2d 458
f2i 459
f2l 460
fadd 461
faload 463
fastore 464
fcmp<op> 465
fconst_<f> 467
fdiv 468
fload 470
fload_<n> 471
fmul 472
fneg 473
frem 474
freturn 476
fstore 477
fstore_<n> 478
fsub 479
getfield 480
getstatic 481
goto 483
goto_w 484
i2b 485
i2c 486
i2d 487
i2f 488
i2l 489
i2s 490
iadd 491
iaload 492
iand 493
iastore 494
iconst_<i> 495
idiv 496
if_acmp<cond> 497
if_icmp<cond> 498
if<cond> 500
ifnonnull 502
ifnull 503
iinc 504

iload 505
iload_<n> 506
imul 507
ineg 508
instanceof 509
invokedynamic 511
invokeinterface 513
invokespecial 517
invokestatic 522
invokevirtual 525
ior 532
irem 533
ireturn 534
ishl 536
ishr 537
istore 538
istore_<n> 539
isub 540
iushr 541
ixor 542
jsr 543
jsr_w 544
l2d 545
l2f 546
l2i 547
ladd 548
laload 549
land 550
lastore 551
lcmp 552
lconst_<l> 553
ldc 554
ldc_w 556
ldc2_w 558
ldiv 560
lload 561
lload_<n> 562
lmul 563
lneg 564
lookupswitch 565
lor 567
lrem 568
lreturn 569
lshl 570
lshr 571
lstore 572
lstore_<n> 573
lsub 574

lushr 575
lxor 576
monitorenter 577
monitorexit 579
multianewarray 581
new 583
newarray 585
nop 587
pop 588
pop2 589
putfield 590
putstatic 592
ret 594
return 595
saload 596
sastore 597
sipush 598
swap 599
tableswitch 600
wide 602

7 Opcode Mnemonics by Opcode 605

A Limited License Grant 609

Introduction

1.1 A Bit of History

The Java® programming language is a general-purpose, concurrent, object-oriented language. Its syntax is similar to C and C++, but it omits many of the features that make C and C++ complex, confusing, and unsafe. The Java platform was initially developed to address the problems of building software for networked consumer devices. It was designed to support multiple host architectures and to allow secure delivery of software components. To meet these requirements, compiled code had to survive transport across networks, operate on any client, and assure the client that it was safe to run.

The popularization of the World Wide Web made these attributes much more interesting. Web browsers enabled millions of people to surf the Net and access media-rich content in simple ways. At last there was a medium where what you saw and heard was essentially the same regardless of the machine you were using and whether it was connected to a fast network or a slow modem.

Web enthusiasts soon discovered that the content supported by the Web's HTML document format was too limited. HTML extensions, such as forms, only highlighted those limitations, while making it clear that no browser could include all the features users wanted. Extensibility was the answer.

The HotJava browser first showcased the interesting properties of the Java programming language and platform by making it possible to embed programs inside HTML pages. Programs are transparently downloaded into the browser along with the HTML pages in which they appear. Before being accepted by the browser, programs are carefully checked to make sure they are safe. Like HTML pages, compiled programs are network- and host-independent. The programs behave the same way regardless of where they come from or what kind of machine they are being loaded into and run on.

A Web browser incorporating the Java platform is no longer limited to a predetermined set of capabilities. Visitors to Web pages incorporating dynamic content can be assured that their machines cannot be damaged by that content. Programmers can write a program once, and it will run on any machine supplying a Java run-time environment.

1.2 The Java Virtual Machine

The Java Virtual Machine is the cornerstone of the Java platform. It is the component of the technology responsible for its hardware- and operating system-independence, the small size of its compiled code, and its ability to protect users from malicious programs.

The Java Virtual Machine is an abstract computing machine. Like a real computing machine, it has an instruction set and manipulates various memory areas at run time. It is reasonably common to implement a programming language using a virtual machine; the best-known virtual machine may be the P-Code machine of UCSD Pascal.

The first prototype implementation of the Java Virtual Machine, done at Sun Microsystems, Inc., emulated the Java Virtual Machine instruction set in software hosted by a handheld device that resembled a contemporary Personal Digital Assistant (PDA). Oracle's current implementations emulate the Java Virtual Machine on mobile, desktop and server devices, but the Java Virtual Machine does not assume any particular implementation technology, host hardware, or host operating system. It is not inherently interpreted, but can just as well be implemented by compiling its instruction set to that of a silicon CPU. It may also be implemented in microcode or directly in silicon.

The Java Virtual Machine knows nothing of the Java programming language, only of a particular binary format, the `class` file format. A `class` file contains Java Virtual Machine instructions (or *bytecodes*) and a symbol table, as well as other ancillary information.

For the sake of security, the Java Virtual Machine imposes strong syntactic and structural constraints on the code in a `class` file. However, any language with functionality that can be expressed in terms of a valid `class` file can be hosted by the Java Virtual Machine. Attracted by a generally available, machine-independent platform, implementors of other languages can turn to the Java Virtual Machine as a delivery vehicle for their languages.

The class file format is versioned: every class file declares a version number, of the form *major.minor*, which indicates the file's dependency on a particular release of Java SE, and influences the interpretation of the file by the Java Virtual Machine.

The Java Virtual Machine specified here is compatible with Java SE 26, and supports the Java programming language specified in *The Java Language Specification, Java SE 26 Edition*. It supports class files with major version numbers 45 through 70, inclusive.

Tools that generate class files will typically adopt the latest major version number in order to take advantage of the latest features; but a class file with an older major version number can generally expect to be supported in future Java Virtual Machine releases.

For reference, the following table shows the class file major version numbers supported by each release of Java SE, up to Java SE 26. The third column, "Earliest", shows the earliest class file major version number supported by the Java Virtual Machine in that release. The fourth column, "Latest", shows the latest class file major version number supported by the Java Virtual Machine in that release. (For very early releases, the JDK version is shown instead of the Java SE release.)

Table 1.2-A. Java SE releases & class file major versions

Java SE	Released	Earliest	Latest
1.0.2	May 1996	45	45
1.1	February 1997	45	45
1.2	December 1998	45	46
1.3	May 2000	45	47
1.4	February 2002	45	48
5.0	September 2004	45	49
6	December 2006	45	50
7	July 2011	45	51
8	March 2014	45	52
9	September 2017	45	53
10	March 2018	45	54
11	September 2018	45	55
12	March 2019	45	56
13	September 2019	45	57
14	March 2020	45	58
15	September 2020	45	59
16	March 2021	45	60
17	September 2021	45	61
18	March 2022	45	62
19	September 2022	45	63
20	March 2023	45	64
21	September 2023	45	65
22	March 2024	45	66
23	September 2024	45	67
24	March 2025	45	68
25	September 2025	45	69
26	March 2026	45	70

1.3 Organization of the Specification

Chapter 2 gives an overview of the Java Virtual Machine architecture.

Chapter 3 introduces compilation of code written in the Java programming language into the instruction set of the Java Virtual Machine.

Chapter 4 specifies the class file format, the hardware- and operating system-independent binary format used to represent compiled classes and interfaces.

Chapter 5 specifies the start-up of the Java Virtual Machine and the loading, linking, and initialization of classes and interfaces.

Chapter 6 specifies the instruction set of the Java Virtual Machine, presenting the instructions in alphabetical order of opcode mnemonics.

Chapter 7 gives a table of Java Virtual Machine opcode mnemonics indexed by opcode value.

In the Second Edition of *The Java® Virtual Machine Specification*, Chapter 2 gave an overview of the Java programming language that was intended to support the specification of the Java Virtual Machine but was not itself a part of the specification. In *The Java Virtual Machine Specification, Java SE 26 Edition*, the reader is referred to *The Java Language Specification, Java SE 26 Edition* for information about the Java programming language.

In the Second Edition of *The Java® Virtual Machine Specification*, Chapter 8 detailed the low-level actions that explained the interaction of Java Virtual Machine threads with a shared main memory. In *The Java Virtual Machine Specification, Java SE 26 Edition*, the reader is referred to Chapter 17 of *The Java Language Specification, Java SE 26 Edition* for information about threads and locks. Chapter 17 reflects *The Java Memory Model and Thread Specification* produced by the JSR 133 Expert Group.

1.4 Notation

Throughout this specification, we refer to classes and interfaces drawn from the Java SE Platform API. Whenever we refer to a class or interface (other than those declared in an example) using a single identifier *N*, the intended reference is to the class or interface named *N* in the package `java.lang`. We use the fully qualified name for classes or interfaces from packages other than `java.lang`.

Whenever we refer to a class or interface that is declared in the package `java` or any of its subpackages, the intended reference is to that class or interface as loaded by the bootstrap class loader (§5.3.1).

Whenever we refer to a subpackage of a package named `java`, the intended reference is to that subpackage as determined by the bootstrap class loader.

A cross-reference within this specification is shown as (§x.y). Occasionally, we refer to concepts in the *The Java Language Specification, Java SE 26 Edition* via cross-references of the form (JLS §x.y).

The use of fonts in this specification is as follows:

- A fixed width font is used for Java Virtual Machine data types, exceptions, errors, class file structures, Prolog code, and Java code fragments.
- *Italic* is used for Java Virtual Machine "assembly language", its opcodes and operands, as well as items in the Java Virtual Machine's run-time data areas. It is also used to introduce new terms and simply for emphasis.

Non-normative text, designed to clarify the normative text of this specification, is given in smaller, indented text.

This is non-normative text. It provides intuition, rationale, advice, examples, etc.

1.5 Preview Features

A *preview feature* is:

- a new feature of the Java programming language ("preview language feature"), or
- a new feature of the Java Virtual Machine ("preview VM feature"), or
- a new module, package, class, interface, field, method, constructor, or enum constant in the `java.*` or `javax.*` namespace ("preview API")

that is fully specified, fully implemented, and yet impermanent. It is available in implementations of a given release of the Java SE Platform to provoke developer feedback based on real world use; this may lead to it becoming permanent in a future release of the Java SE Platform.

The preview features defined by a given release of the Java SE Platform are enumerated in the Java SE Platform Specification for that release. The preview features are specified as follows:

- Preview language features are specified in standalone documents that indicate changes ("diffs") to *The Java® Language Specification* for that release. The specifications of preview language features are incorporated into *The Java® Language Specification* by reference, and made a part thereof, if and only if preview features are enabled at compile time.
- Preview VM features are specified in standalone documents that indicate changes ("diffs") to *The Java® Virtual Machine Specification* for that release. The specifications of preview VM features are incorporated into *The Java® Virtual Machine Specification* by reference, and made a part thereof, if and only if preview features are enabled at run time.
- Preview APIs are specified within the Java SE API Specification for that release.

1.5.1 Restrictions on the Use of Preview Features

Implementations of the Java SE Platform disable, at both compile time and run time, the preview features defined by a given release, unless the user indicates via the host system, at both compile time and run time, that preview features are enabled. Implementations do not provide a way to enable only some of the given release's preview features.

A `class` file *depends on the preview features of Java SE N ($N \geq 12$)* if:

- the `class` file's major version number is the latest major version number supported by Java SE N in Table 1.2-A, and
- the `class` file's minor version number is 65535

For example, a `class` file with version number 66.65535 depends on the preview features of Java SE 22, because 66 is the latest major version number supported by Java SE 22.

Compilers may need to emit `class` files that depend on the preview features of a given release even if the `class` files do not use any preview VM features. For example, a Java source file that uses a preview language feature of the release must be compiled to a `class` file that depends on the preview features of the release.

At run time, the rules for loading a `class` file that depends on the preview features of a given Java SE Platform release are specified in §4.1. Such a `class` file is tied to that release of the Java SE Platform, and cannot be loaded on any other release (even if preview features are enabled) because the preview features it depends on may be different or missing in the other release.

1.5.2 Current Preview VM Features

Java SE 26 does not define any preview VM features.

1.6 Feedback

Readers are invited to report technical errors and ambiguities in *The Java® Virtual Machine Specification* to java-jvm-spec-comments@openjdk.org.

Questions concerning the generation and manipulation of `class` files by `javac` (the reference compiler for the Java programming language) may be sent to compiler-dev@openjdk.org.

The Structure of the Java Virtual Machine

THIS document specifies an abstract machine. It does not describe any particular implementation of the Java Virtual Machine.

To implement the Java Virtual Machine correctly, you need only be able to read the `class` file format and correctly perform the operations specified therein. Implementation details that are not part of the Java Virtual Machine's specification would unnecessarily constrain the creativity of implementors. For example, the memory layout of run-time data areas, the garbage-collection algorithm used, and any internal optimization of the Java Virtual Machine instructions (for example, translating them into machine code) are left to the discretion of the implementor.

All references to Unicode in this specification are given with respect to *The Unicode Standard, Version 17.0.0*, available at <https://www.unicode.org/>.

2.1 The `class` File Format

Compiled code to be executed by the Java Virtual Machine is represented using a hardware- and operating system-independent binary format, typically (but not necessarily) stored in a file, known as the `class` file format. The `class` file format precisely defines the representation of a class or interface, including details such as byte ordering that might be taken for granted in a platform-specific object file format.

Chapter 4, "The `class` File Format", covers the `class` file format in detail.

2.2 Data Types

Like the Java programming language, the Java Virtual Machine operates on two kinds of types: *primitive types* and *reference types*. There are, correspondingly, two kinds of values that can be stored in variables, passed as arguments, returned by methods, and operated upon: *primitive values* and *reference values*.

The Java Virtual Machine expects that nearly all type checking is done prior to run time, typically by a compiler, and does not have to be done by the Java Virtual Machine itself. Values of primitive types need not be tagged or otherwise be inspectable to determine their types at run time, or to be distinguished from values of reference types. Instead, the instruction set of the Java Virtual Machine distinguishes its operand types using instructions intended to operate on values of specific types. For instance, *iadd*, *ladd*, *fadd*, and *dadd* are all Java Virtual Machine instructions that add two numeric values and produce numeric results, but each is specialized for its operand type: *int*, *long*, *float*, and *double*, respectively. For a summary of type support in the Java Virtual Machine instruction set, see §2.11.1.

The Java Virtual Machine contains explicit support for objects. An object is either a dynamically allocated class instance or an array. A reference to an object is considered to have Java Virtual Machine type reference. References are polymorphic: a single reference may also be a value of multiple class types, interface types, or array types. Values of type reference can be thought of as pointers to objects. More than one reference to an object may exist. Objects are always operated on, passed, and tested via values of type reference.

2.3 Primitive Types and Values

The primitive data types supported by the Java Virtual Machine are the *numeric types*, the *boolean type* (§2.3.4), and the *returnAddress type* (§2.3.3).

The numeric types consist of the *integral types* (§2.3.1) and the *floating-point types* (§2.3.2).

The integral types are:

- *byte*, whose values are 8-bit signed two's-complement integers, and whose default value is zero
- *short*, whose values are 16-bit signed two's-complement integers, and whose default value is zero

- `int`, whose values are 32-bit signed two's-complement integers, and whose default value is zero
- `long`, whose values are 64-bit signed two's-complement integers, and whose default value is zero
- `char`, whose values are 16-bit unsigned integers representing Unicode code points in the Basic Multilingual Plane, encoded with UTF-16, and whose default value is the null code point (`'\u0000'`)

The floating-point types are:

- `float`, whose values exactly correspond to the values representable in the 32-bit IEEE 754 binary32 format, and whose default value is positive zero
- `double`, whose values exactly correspond to the values of the 64-bit IEEE 754 binary64 format, and whose default value is positive zero

The values of the `boolean` type encode the truth values `true` and `false`, and the default value is `false`.

The First Edition of *The Java® Virtual Machine Specification* did not consider `boolean` to be a Java Virtual Machine type. However, `boolean` values do have limited support in the Java Virtual Machine. The Second Edition of *The Java® Virtual Machine Specification* clarified the issue by treating `boolean` as a type.

The values of the `returnAddress` type are pointers to the opcodes of Java Virtual Machine instructions. Of the primitive types, only the `returnAddress` type is not directly associated with a Java programming language type.

2.3.1 Integral Types and Values

The values of the integral types of the Java Virtual Machine are:

- For `byte`, from -128 to 127 (-2^7 to $2^7 - 1$), inclusive
- For `short`, from -32768 to 32767 (-2^{15} to $2^{15} - 1$), inclusive
- For `int`, from -2147483648 to 2147483647 (-2^{31} to $2^{31} - 1$), inclusive
- For `long`, from -9223372036854775808 to 9223372036854775807 (-2^{63} to $2^{63} - 1$), inclusive
- For `char`, from 0 to 65535 inclusive

2.3.2 Floating-Point Types and Values

The floating-point types are `float` and `double`, which are conceptually associated with the 32-bit binary32 and 64-bit binary64 floating-point formats for IEEE 754 values and operations, as specified in the IEEE 754 Standard (JLS §1.7).

In Java SE 15 and later, the Java Virtual Machine uses the 2019 version of the IEEE 754 Standard. Prior to Java SE 15, the Java Virtual Machine used the 1985 version of the IEEE 754 Standard, where the binary32 format was known as the single format and the binary64 format was known as the double format.

IEEE 754 includes not only positive and negative numbers that consist of a sign and magnitude, but also positive and negative zeros, positive and negative *infinities*, and special *Not-a-Number* values (hereafter abbreviated NaN). A NaN value is used to represent the result of certain invalid operations such as dividing zero by zero. NaN constants of both `float` and `double` type are predefined as `Float.NaN` and `Double.NaN`.

The finite nonzero values of a floating-point type can all be expressed in the form $s \cdot m \cdot 2^{(e - N + 1)}$, where:

- s is +1 or -1,
- m is a positive integer less than 2^N ,
- e is an integer between $E_{min} = -(2^{K-1}-2)$ and $E_{max} = 2^{K-1}-1$, inclusive, and
- N and K are parameters that depend on the type.

Some values can be represented in this form in more than one way. For example, supposing that a value v of a floating-point type might be represented in this form using certain values for s , m , and e , then if it happened that m were even and e were less than 2^{K-1} , one could halve m and increase e by 1 to produce a second representation for the same value v .

A representation in this form is called *normalized* if $m \geq 2^{N-1}$; otherwise the representation is said to be *subnormal*. If a value of a floating-point type cannot be represented in such a way that $m \geq 2^{N-1}$, then the value is said to be a *subnormal value*, because its magnitude is below the magnitude of the smallest normalized value.

The constraints on the parameters N and K (and on the derived parameters E_{min} and E_{max}) for `float` and `double` are summarized in Table 2.3.2-A.

Table 2.3.2-A. Floating-point parameters

Parameter	float	double
N	24	53
K	8	11
E_{max}	+127	+1023
E_{min}	-126	-1022

Except for NaN, floating-point values are *ordered*. When arranged from smallest to largest, they are negative infinity, negative finite nonzero values, positive and negative zero, positive finite nonzero values, and positive infinity.

IEEE 754 allows multiple distinct NaN values for each of its binary32 and binary64 floating-point formats. However, the Java SE Platform generally treats NaN values of a given floating-point type as though collapsed into a single canonical value, and hence this specification normally refers to an arbitrary NaN as though to a canonical value.

Under IEEE 754, a floating-point operation with non-NaN arguments may generate a NaN result. IEEE 754 specifies a set of NaN bit patterns, but does not mandate which particular NaN bit pattern is used to represent a NaN result; this is left to the hardware architecture. A programmer can create NaNs with different bit patterns to encode, for example, retrospective diagnostic information. These NaN values can be created with the `Float.intBitsToFloat` and `Double.longBitsToDouble` methods for `float` and `double`, respectively. Conversely, to inspect the bit patterns of NaN values, the `Float.floatToRawIntBits` and `Double.doubleToRawLongBits` methods can be used for `float` and `double`, respectively.

Positive zero and negative zero compare equal, but there are other operations that can distinguish them; for example, dividing `1.0` by `0.0` produces positive infinity, but dividing `1.0` by `-0.0` produces negative infinity.

NaN is *unordered*, so numerical comparisons and tests for numerical equality have the value `false` if either or both of their operands are NaN. In particular, a test for numerical equality of a value against itself has the value `false` if and only if the value is NaN. A test for numerical inequality has the value `true` if either operand is NaN.

2.3.3 The `returnAddress` Type and Values

The `returnAddress` type is used by the Java Virtual Machine's `jsr`, `ret`, and `jsr_w` instructions (`$jsr`, `$ret`, `$jsr_w`). The values of the `returnAddress` type are pointers to the opcodes of Java Virtual Machine instructions. Unlike the numeric primitive

types, the `returnAddress` type does not correspond to any Java programming language type and cannot be modified by the running program.

2.3.4 The boolean Type

Although the Java Virtual Machine defines a boolean type, it only provides very limited support for it. There are no Java Virtual Machine instructions solely dedicated to operations on boolean values. Instead, expressions in the Java programming language that operate on boolean values are compiled to use values of the Java Virtual Machine `int` data type.

The Java Virtual Machine does directly support boolean arrays. Its *`newarray`* instruction (*`$newarray`*) enables creation of boolean arrays. Arrays of type boolean are accessed and modified using the byte array instructions *`baload`* and *`bastore`* (*`$baload`*, *`$bastore`*).

In Oracle's Java Virtual Machine implementation, boolean arrays in the Java programming language are encoded as Java Virtual Machine byte arrays, using 8 bits per boolean element.

The Java Virtual Machine encodes boolean array components using 1 to represent true and 0 to represent false. Where Java programming language boolean values are mapped by compilers to values of Java Virtual Machine type `int`, the compilers must use the same encoding.

2.4 Reference Types and Values

There are three kinds of reference types: class types, array types, and interface types. Their values are references to dynamically created class instances, arrays, or class instances or arrays that implement interfaces, respectively.

An array type consists of a *component type* with a single dimension (whose length is not given by the type). The component type of an array type may itself be an array type. If, starting from any array type, one considers its component type, and then (if that is also an array type) the component type of that type, and so on, eventually one must reach a component type that is not an array type; this is called the *element type* of the array type. The element type of an array type is necessarily either a primitive type, or a class type, or an interface type.

A reference value may also be the special null reference, a reference to no object, which will be denoted here by `null`. The null reference initially has no run-time type, but may be cast to any type. The default value of a reference type is `null`.

This specification does not mandate a concrete value encoding `null`.

2.5 Run-Time Data Areas

The Java Virtual Machine defines various run-time data areas that are used during execution of a program. Some of these data areas are created on Java Virtual Machine start-up and are destroyed only when the Java Virtual Machine terminates. Other data areas are per thread. Per-thread data areas are created when a thread is created and destroyed when the thread terminates.

2.5.1 The pc Register

The Java Virtual Machine can support many threads of execution at once (JLS §17). Each Java Virtual Machine thread has its own pc (program counter) register. At any point, each Java Virtual Machine thread is executing the code of a single method, namely the current method (§2.6) for that thread. If that method is not native, the pc register contains the address of the Java Virtual Machine instruction currently being executed. If the method currently being executed by the thread is native, the value of the Java Virtual Machine's pc register is undefined. The Java Virtual Machine's pc register is wide enough to hold a `returnAddress` or a native pointer on the specific platform.

2.5.2 Java Virtual Machine Stacks

Each Java Virtual Machine thread has a private *Java Virtual Machine stack*, created at the same time as the thread. A Java Virtual Machine stack stores frames (§2.6). A Java Virtual Machine stack is analogous to the stack of a conventional language such as C: it holds local variables and partial results, and plays a part in method invocation and return. Because the Java Virtual Machine stack is never manipulated directly except to push and pop frames, frames may be heap allocated. The memory for a Java Virtual Machine stack does not need to be contiguous.

In the First Edition of *The Java® Virtual Machine Specification*, the Java Virtual Machine stack was known as the *Java stack*.

This specification permits Java Virtual Machine stacks either to be of a fixed size or to dynamically expand and contract as required by the computation. If the Java Virtual Machine stacks are of a fixed size, the size of each Java Virtual Machine stack may be chosen independently when that stack is created.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of Java Virtual Machine stacks, as well as, in the case of dynamically expanding or contracting Java Virtual Machine stacks, control over the maximum and minimum sizes.

The following exceptional conditions are associated with Java Virtual Machine stacks:

- If the computation in a thread requires a larger Java Virtual Machine stack than is permitted, the Java Virtual Machine throws a `StackOverflowError`.
- If Java Virtual Machine stacks can be dynamically expanded, and expansion is attempted but insufficient memory can be made available to effect the expansion, or if insufficient memory can be made available to create the initial Java Virtual Machine stack for a new thread, the Java Virtual Machine throws an `OutOfMemoryError`.

2.5.3 Heap

The Java Virtual Machine has a *heap* that is shared among all Java Virtual Machine threads. The heap is the run-time data area from which memory for all class instances and arrays is allocated.

The heap is created on virtual machine start-up. Heap storage for objects is reclaimed by an automatic storage management system (known as a *garbage collector*); objects are never explicitly deallocated. The Java Virtual Machine assumes no particular type of automatic storage management system, and the storage management technique may be chosen according to the implementor's system requirements. The heap may be of a fixed size or may be expanded as required by the computation and may be contracted if a larger heap becomes unnecessary. The memory for the heap does not need to be contiguous.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the heap, as well as, if the heap can be dynamically expanded or contracted, control over the maximum and minimum heap size.

The following exceptional condition is associated with the heap:

- If a computation requires more heap than can be made available by the automatic storage management system, the Java Virtual Machine throws an `OutOfMemoryError`.

2.5.4 Method Area

The Java Virtual Machine has a *method area* that is shared among all Java Virtual Machine threads. The method area is analogous to the storage area for compiled code of a conventional language or analogous to the "text" segment in an operating system process. It stores per-class structures such as the run-time constant pool, field and method data, and the code for methods and constructors, including the special methods used in class and interface initialization and in instance initialization (§2.9).

The method area is created on virtual machine start-up. Although the method area is logically part of the heap, simple implementations may choose not to either garbage collect or compact it. This specification does not mandate the location of the method area or the policies used to manage compiled code. The method area may be of a fixed size or may be expanded as required by the computation and may be contracted if a larger method area becomes unnecessary. The memory for the method area does not need to be contiguous.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the method area, as well as, in the case of a varying-size method area, control over the maximum and minimum method area size.

The following exceptional condition is associated with the method area:

- If memory in the method area cannot be made available to satisfy an allocation request, the Java Virtual Machine throws an `OutOfMemoryError`.

2.5.5 Run-Time Constant Pool

A *run-time constant pool* is a per-class or per-interface run-time representation of the `constant_pool` table in a class file (§4.4). It contains several kinds of constants, ranging from numeric literals known at compile-time to method and field references that must be resolved at run-time. The run-time constant pool serves a function similar to that of a symbol table for a conventional programming language, although it contains a wider range of data than a typical symbol table.

Each run-time constant pool is allocated from the Java Virtual Machine's method area (§2.5.4). The run-time constant pool for a class or interface is constructed when the class or interface is created (§5.3) by the Java Virtual Machine.

The following exceptional condition is associated with the construction of the run-time constant pool for a class or interface:

- When creating a class or interface, if the construction of the run-time constant pool requires more memory than can be made available in the method area of the Java Virtual Machine, the Java Virtual Machine throws an `OutOfMemoryError`.

See §5 (*Loading, Linking, and Initializing*) for information about the construction of the run-time constant pool.

2.5.6 Native Method Stacks

An implementation of the Java Virtual Machine may use conventional stacks, colloquially called "C stacks," to support native methods (methods written in a language other than the Java programming language). Native method stacks may also be used by the implementation of an interpreter for the Java Virtual Machine's instruction set in a language such as C. Java Virtual Machine implementations that cannot load native methods and that do not themselves rely on conventional stacks need not supply native method stacks. If supplied, native method stacks are typically allocated per thread when each thread is created.

This specification permits native method stacks either to be of a fixed size or to dynamically expand and contract as required by the computation. If the native method stacks are of a fixed size, the size of each native method stack may be chosen independently when that stack is created.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the native method stacks, as well as, in the case of varying-size native method stacks, control over the maximum and minimum method stack sizes.

The following exceptional conditions are associated with native method stacks:

- If the computation in a thread requires a larger native method stack than is permitted, the Java Virtual Machine throws a `StackOverflowError`.
- If native method stacks can be dynamically expanded and native method stack expansion is attempted but insufficient memory can be made available, or if insufficient memory can be made available to create the initial native method stack for a new thread, the Java Virtual Machine throws an `OutOfMemoryError`.

2.6 Frames

A *frame* is used to store data and partial results, as well as to perform dynamic linking, return values for methods, and dispatch exceptions.

A new frame is created each time a method is invoked. A frame is destroyed when its method invocation completes, whether that completion is normal or abrupt (it throws an uncaught exception). Frames are allocated from the Java Virtual Machine stack (§2.5.2) of the thread creating the frame. Each frame has its own array of local variables (§2.6.1), its own operand stack (§2.6.2), and a reference to the run-time constant pool (§2.5.5) of the class of the current method.

A frame may be extended with additional implementation-specific information, such as debugging information.

The sizes of the local variable array and the operand stack are determined at compile-time and are supplied along with the code for the method associated with the frame (§4.7.3). Thus the size of the frame data structure depends only on the implementation of the Java Virtual Machine, and the memory for these structures can be allocated simultaneously on method invocation.

Only one frame, the frame for the executing method, is active at any point in a given thread of control. This frame is referred to as the *current frame*, and its method is known as the *current method*. The class in which the current method is defined is the *current class*. Operations on local variables and the operand stack are typically with reference to the current frame.

A frame ceases to be current if its method invokes another method or if its method completes. When a method is invoked, a new frame is created and becomes current when control transfers to the new method. On method return, the current frame passes back the result of its method invocation, if any, to the previous frame. The current frame is then discarded as the previous frame becomes the current one.

Note that a frame created by a thread is local to that thread and cannot be referenced by any other thread.

2.6.1 Local Variables

Each frame (§2.6) contains an array of variables known as its *local variables*. The length of the local variable array of a frame is determined at compile-time and supplied in the binary representation of a class or interface along with the code for the method associated with the frame (§4.7.3).

A single local variable can hold a value of type `int`, `float`, `reference`, or `returnAddress`. A pair of local variables can hold a value of type `long` or `double`.

Local variables are addressed by indexing. The index of the first local variable is zero. An integer is considered to be an index into the local variable array if and only if that integer is between zero and one less than the size of the local variable array.

A value of type `long` or type `double` occupies two consecutive local variables. Such a value may only be addressed using the lesser index. For example, a value of type `double` stored in the local variable array at index n actually occupies the local variables with indices n and $n+1$; however, the local variable at index $n+1$ cannot be loaded from. It can be stored into. However, doing so invalidates the contents of local variable n .

The Java Virtual Machine does not require n to be even. In intuitive terms, values of types `long` and `double` need not be 64-bit aligned in the local variables array. Implementors are free to decide the appropriate way to represent such values using the two local variables reserved for the value.

The Java Virtual Machine uses local variables to pass parameters on method invocation. On class method invocation, any parameters are passed in consecutive local variables starting from local variable 0 . On instance method invocation, local variable 0 is always used to pass a reference to the object on which the instance method is being invoked (`this` in the Java programming language). Any parameters are subsequently passed in consecutive local variables starting from local variable 1 .

2.6.2 Operand Stacks

Each frame (§2.6) contains a last-in-first-out (LIFO) stack known as its *operand stack*. The maximum depth of the operand stack of a frame is determined at compile-time and is supplied along with the code for the method associated with the frame (§4.7.3).

Where it is clear by context, we will sometimes refer to the operand stack of the current frame as simply the operand stack.

The operand stack is empty when the frame that contains it is created. The Java Virtual Machine supplies instructions to load constants or values from local variables or fields onto the operand stack. Other Java Virtual Machine instructions take operands from the operand stack, operate on them, and push the result back onto the operand stack. The operand stack is also used to prepare parameters to be passed to methods and to receive method results.

For example, the *iadd* instruction (§*iadd*) adds two `int` values together. It requires that the `int` values to be added be the top two values of the operand stack, pushed there by previous instructions. Both of the `int` values are popped from the operand stack. They are added, and their sum is pushed back onto the operand stack. Subcomputations may be nested on the operand stack, resulting in values that can be used by the encompassing computation.

Each entry on the operand stack can hold a value of any Java Virtual Machine type, including a value of type `long` or type `double`.

Values from the operand stack must be operated upon in ways appropriate to their types. It is not possible, for example, to push two `int` values and subsequently treat them as a `long` or to push two `float` values and subsequently add them with an *iadd* instruction. A small number of Java Virtual Machine instructions (the *dup* instructions (§*dup*) and *swap* (§*swap*)) operate on run-time data areas as raw values without regard to their specific types; these instructions are defined in such a way that they cannot be used to modify or break up individual values. These restrictions on operand stack manipulation are enforced through `class` file verification (§4.10).

At any point in time, an operand stack has an associated depth, where a value of type `long` or `double` contributes two units to the depth and a value of any other type contributes one unit.

2.6.3 Dynamic Linking

Each frame (§2.6) contains a reference to the run-time constant pool (§2.5.5) for the type of the current method to support *dynamic linking* of the method code. The `class` file code for a method refers to methods to be invoked and variables to be accessed via symbolic references. Dynamic linking translates these symbolic method references into concrete method references, loading classes as necessary to resolve as-yet-undefined symbols, and translates variable accesses into appropriate offsets in storage structures associated with the run-time location of these variables.

This late binding of the methods and variables makes changes in other classes that a method uses less likely to break this code.

2.6.4 Normal Method Invocation Completion

A method invocation *completes normally* if that invocation does not cause an exception (§2.10) to be thrown, either directly from the Java Virtual Machine or as a result of executing an explicit `throw` statement. If the invocation of the current method completes normally, then a value may be returned to the invoking method. This occurs when the invoked method executes one of the return instructions

(§2.11.8), the choice of which must be appropriate for the type of the value being returned (if any).

The current frame (§2.6) is used in this case to restore the state of the invoker, including its local variables and operand stack, with the program counter of the invoker appropriately incremented to skip past the method invocation instruction. Execution then continues normally in the invoking method's frame with the returned value (if any) pushed onto the operand stack of that frame.

2.6.5 Abrupt Method Invocation Completion

A method invocation *completes abruptly* if execution of a Java Virtual Machine instruction within the method causes the Java Virtual Machine to throw an exception (§2.10), and that exception is not handled within the method. Execution of an *athrow* instruction (§*athrow*) also causes an exception to be explicitly thrown and, if the exception is not caught by the current method, results in abrupt method invocation completion. A method invocation that completes abruptly never returns a value to its invoker.

2.7 Representation of Objects

The Java Virtual Machine does not mandate any particular internal structure for objects.

In some of Oracle's implementations of the Java Virtual Machine, a reference to a class instance is a pointer to a *handle* that is itself a pair of pointers: one to a table containing the methods of the object and a pointer to the `Class` object that represents the type of the object, and the other to the memory allocated from the heap for the object data.

2.8 Floating-Point Arithmetic

The Java Virtual Machine incorporates a subset of the floating-point arithmetic specified in the IEEE 754 Standard (JLS §1.7).

In Java SE 15 and later, the Java Virtual Machine uses the 2019 version of the IEEE 754 Standard. Prior to Java SE 15, the Java Virtual Machine used the 1985 version of the IEEE 754 Standard, where the binary32 format was known as the single format and the binary64 format was known as the double format.

Many of the Java Virtual Machine instructions for arithmetic (§2.11.3) and type conversion (§2.11.4) work with floating-point numbers. These instructions typically correspond to IEEE 754 operations (Table 2.8-A), except for certain instructions described below.

Table 2.8-A. Correspondence with IEEE 754 operations

Instruction	IEEE 754 operation
<i>dcmp</i> < <i>op</i> > (§ <i>dcmp</i> < <i>op</i> >), (<i>\$fcmp</i> < <i>op</i> >)	compareQuietLess, compareQuietLessEqual, compareQuietGreater, compareQuietGreaterEqual, compareQuietEqual, compareQuietNotEqual
<i>dadd</i> (§ <i>dadd</i>), <i>fadd</i> (§ <i>fadd</i>)	addition
<i>dsub</i> (§ <i>dsub</i>), <i>fsub</i> (§ <i>fsub</i>)	subtraction
<i>dmul</i> (§ <i>dmul</i>), <i>fmul</i> (§ <i>fmul</i>)	multiplication
<i>ddiv</i> (§ <i>ddiv</i>), <i>fdiv</i> (§ <i>fdiv</i>)	division
<i>dneg</i> (§ <i>dneg</i>), <i>fneg</i> (§ <i>fneg</i>)	negate
<i>i2d</i> (§ <i>i2d</i>), <i>i2f</i> (§ <i>i2f</i>), <i>l2d</i> (§ <i>l2d</i>), <i>l2f</i> (§ <i>l2f</i>)	convertFromInt
<i>d2i</i> (§ <i>d2i</i>), <i>d2l</i> (§ <i>d2l</i>), <i>f2i</i> (§ <i>f2i</i>), <i>f2l</i> (§ <i>f2l</i>)	convertToIntegerTowardZero
<i>d2f</i> (§ <i>d2f</i>), <i>f2d</i> (§ <i>f2d</i>)	convertFormat

The key differences between the floating-point arithmetic supported by the Java Virtual Machine and the IEEE 754 Standard are:

- The floating-point remainder instructions *drem* (§*drem*) and *frem* (§*frem*) do not correspond to the IEEE 754 remainder operation. The instructions are based on an implied division using the round toward zero rounding policy; the IEEE 754 remainder is instead based on an implied division using the round to nearest rounding policy. (Rounding policies are discussed below.)
- The floating-point negate instructions *dneg* (§*dneg*) and *fneg* (§*fneg*) do not correspond precisely to the IEEE 754 negate operation. In particular, the instructions do not require the sign bit of a NaN operand to be inverted.
- The floating-point instructions of the Java Virtual Machine do not throw exceptions, trap, or otherwise signal the IEEE 754 exceptional conditions of invalid operation, division by zero, overflow, underflow, or inexact.
- The Java Virtual Machine does not support IEEE 754 signaling floating-point comparisons, and has no signaling NaN value.

- IEEE 754 includes rounding-direction attributes that do not correspond to a rounding policy in the Java Virtual Machine. The Java Virtual Machine does not provide any means to change the rounding policy used by a given floating-point instruction.
- The Java Virtual Machine does not support the binary32 extended and binary64 extended floating-point formats defined by IEEE 754. Neither extended range nor extended precision beyond those specified for the `float` and `double` types may be used when operating on or storing floating-point values.

Some IEEE 754 operations without corresponding instructions in the Java Virtual Machine are provided via methods in the `Math` and `StrictMath` classes, including the `sqrt` method for the IEEE 754 `squareRoot` operation, the `fma` method for the IEEE 754 fusedMultiplyAdd operation, and the `IEEEremainder` method for the IEEE 754 remainder operation.

The Java Virtual Machine requires support of IEEE 754 *subnormal* floating-point numbers and *gradual underflow*, which make it easier to prove desirable properties of particular numerical algorithms.

Floating-point arithmetic is an approximation to real arithmetic. While there are an infinite number of real numbers, a particular floating-point format only has a finite number of values. In the Java Virtual Machine, a *rounding policy* is a function used to map from a real number to a floating-point value in a given format. For real numbers in the representable range of a floating-point format, a continuous segment of the real number line is mapped to a single floating-point value. The real number whose value is numerically equal to a floating-point value is mapped to that floating-point value; for example, the real number 1.5 is mapped to the floating-point value 1.5 in a given format. The Java Virtual Machine defines two rounding policies, as follows:

- The *round to nearest* rounding policy applies to all floating-point instructions except for (i) conversion to an integer value and (ii) remainder. Under the round to nearest rounding policy, inexact results must be rounded to the representable value nearest to the infinitely precise result; if the two nearest representable values are equally near, then the value whose least significant bit is zero is chosen.

The round to nearest rounding policy corresponds to the default rounding-direction attribute for binary arithmetic in IEEE 754, *roundTiesToEven*.

The *roundTiesToEven* rounding-direction attribute was known as the "round to nearest" rounding mode in the 1985 version of the IEEE 754 Standard. The rounding policy in the Java Virtual Machine is named after this rounding mode.

- The *round toward zero* rounding policy applies to (i) conversion of a floating-point value to an integer value by the *d2i*, *d2l*, *f2i*, and *f2l* instructions (§*d2i*, §*d2l*, §*f2i*, §*f2l*), and (ii) the floating-point remainder instructions *drem* and *frem* (§*drem*, §*frem*). Under the round toward zero rounding policy, inexact results are rounded to the nearest representable value that is not greater in magnitude than the infinitely precise result. For conversion to integer, the round toward zero rounding policy is equivalent to truncation where fractional significand bits are discarded.

The round toward zero rounding policy corresponds to the *roundTowardZero* rounding-direction attribute for binary arithmetic in IEEE 754.

The *roundTowardZero* rounding-direction attribute was known as the "round toward zero" rounding mode in the 1985 version of the IEEE 754 Standard. The rounding policy in the Java Virtual Machine is named after this rounding mode.

The Java Virtual Machine requires that every floating-point instruction rounds its floating-point result to the result precision. The rounding policy used by each instruction is either round to nearest or round toward zero, as specified above.

Java 1.0 and 1.1 required *strict* evaluation of floating-point expressions. Strict evaluation means that each `float` operand corresponds to a value representable in the IEEE 754 binary32 format, each `double` operand corresponds to a value representable in the IEEE 754 binary64 format, and each floating-point operator with a corresponding IEEE 754 operation matches the IEEE 754 result for the same operands.

Strict evaluation provides predictable results, but caused performance problems in the Java Virtual Machine implementations for some processor families common in the Java 1.0/1.1 era. Consequently, in Java 1.2 through Java SE 16, the Java SE Platform allowed a Java Virtual Machine implementation to have one or two *value sets* associated with each floating-point type. The `float` type was associated with the *float value set* and the *float-extended-exponent value set*, while the `double` type was associated with the *double value set* and the *double-extended-exponent value set*. The float value set corresponded to the values representable in the IEEE 754 binary32 format; the float-extended-exponent value set had the same number of precision bits but larger exponent range. Similarly, the double value set corresponded to the values representable in the IEEE 754 binary64 format; the double-extended-exponent value set had the same number of precision bits but larger exponent range. Allowing use of the extended-exponent value sets by default ameliorated the performance problems on some processor families.

For compatibility, Java 1.2 allowed a `class` file to forbid an implementation from using the extended-exponent value sets. A `class` file expressed this by setting the `ACC_STRICT` flag on the declaration of a method. `ACC_STRICT` constrained the floating-point semantics of the method's instructions to use the float value set for `float` operands and the double value set for `double` operands, ensuring the results of such instructions were fully predictable. Methods flagged as `ACC_STRICT` thus had the same floating-point semantics as specified in Java 1.0 and 1.1.

In Java SE 17 and later, the Java SE Platform always requires strict evaluation of floating-point expressions. Newer members of the processor families that had performance problems implementing strict evaluation no longer have that difficulty. This specification no longer associates `float` and `double` with the four value sets described above, and the `ACC_STRICT` flag no longer affects the evaluation of floating-point operations. For compatibility, the bit pattern assigned to denote `ACC_STRICT` in a `class` file whose major version number is 46-60 is unassigned (that is, does not denote any flag) in a `class` file whose major version number is greater than 60 (§4.6). Future versions of the Java Virtual Machine may assign a different meaning to the bit pattern in future `class` files.

2.9 Special Methods

2.9.1 Instance Initialization Methods

A class has zero or more *instance initialization methods*, each typically corresponding to a constructor written in the Java programming language.

A method is an instance initialization method if all of the following are true:

- It is defined in a class (not an interface).
- It has the special name `<init>`.
- It is void (§4.3.3).

In a class, any non-void method named `<init>` is not an instance initialization method. In an interface, any method named `<init>` is not an instance initialization method. Such methods cannot be invoked by any Java Virtual Machine instruction (§4.4.2, §4.9.2) and are rejected by format checking (§4.6, §4.8).

The declaration and use of an instance initialization method is constrained by the Java Virtual Machine. For the declaration, the method's `access_flags` item and code array are constrained (§4.6, §4.9.2). For a use, an instance initialization method may be invoked only by the *invokespecial* instruction on an uninitialized class instance (§4.10.1.9).

Because the name `<init>` is not a valid identifier in the Java programming language, it cannot be used directly in a program written in the Java programming language.

2.9.2 Class Initialization Methods

A class or interface has at most one *class or interface initialization method* and is initialized by the Java Virtual Machine invoking that method (§5.5).

A method is a *class or interface initialization method* if all of the following are true:

- It has the special name `<clinit>`.
- It is `void` (§4.3.3).
- In a class file whose version number is 51.0 or above, the method has its `ACC_STATIC` flag set and takes no arguments (§4.6).

The requirement for `ACC_STATIC` was introduced in Java SE 7, and for taking no arguments in Java SE 9. In a class file whose version number is 50.0 or below, a method named `<clinit>` that is `void` is considered the class or interface initialization method regardless of the setting of its `ACC_STATIC` flag or whether it takes arguments.

Other methods named `<clinit>` in a class file are not class or interface initialization methods. They are never invoked by the Java Virtual Machine itself, cannot be invoked by any Java Virtual Machine instruction (§4.9.1), and are rejected by format checking (§4.6, §4.8).

Because the name `<clinit>` is not a valid identifier in the Java programming language, it cannot be used directly in a program written in the Java programming language.

2.9.3 Signature Polymorphic Methods

A method is *signature polymorphic* if all of the following are true:

- It is declared in the `java.lang.invoke.MethodHandle` class or the `java.lang.invoke.VarHandle` class.
- It has a single formal parameter of type `Object[]`.
- It has the `ACC_VARARGS` and `ACC_NATIVE` flags set.

The Java Virtual Machine gives special treatment to signature polymorphic methods in the *invokevirtual* instruction (§*invokevirtual*), in order to effect invocation of a *method handle* or to effect access to a variable referenced by an instance of `java.lang.invoke.VarHandle`.

A method handle is a dynamically strongly typed and directly executable reference to an underlying method, constructor, field, or similar low-level operation (§5.4.3.5), with optional transformations of arguments or return values. An instance of `java.lang.invoke.VarHandle` is a dynamically strongly typed reference to a variable or family of variables, including static fields, non-static fields, array elements, or components of an off-heap data structure. See the `java.lang.invoke` package in the Java SE Platform API for more information.

2.10 Exceptions

An exception in the Java Virtual Machine is represented by an instance of the class `Throwable` or one of its subclasses. Throwing an exception results in an immediate nonlocal transfer of control from the point where the exception was thrown.

Most exceptions occur synchronously as a result of an action by the thread in which they occur. An asynchronous exception, by contrast, can potentially occur at any point in the execution of a program. The Java Virtual Machine throws an exception for one of three reasons:

- An *athrow* instruction (§*athrow*) was executed.
- An abnormal execution condition was synchronously detected by the Java Virtual Machine. These exceptions are not thrown at an arbitrary point in the program, but only synchronously after execution of an instruction that either:
 - Specifies the exception as a possible result, such as:
 - › When the instruction embodies an operation that violates the semantics of the Java programming language, for example indexing outside the bounds of an array.
 - › When an error occurs in loading or linking part of the program.
 - Causes some limit on a resource to be exceeded, for example when too much memory is used.
- An asynchronous exception occurred because an internal error occurred in the Java Virtual Machine implementation (§6.3).

A Java Virtual Machine implementation may permit a small but bounded amount of execution to occur before an asynchronous exception is thrown. This delay is permitted to allow optimized code to detect and throw these exceptions at points where it is practical to handle them while obeying the semantics of the Java programming language.

A simple implementation might poll for asynchronous exceptions at the point of each control transfer instruction. Since a program has a finite size, this provides a bound on the total delay in detecting an asynchronous exception. Since no asynchronous exception will occur between control transfers, the code generator has some flexibility to reorder computation between control transfers for greater performance. The paper *Polling Efficiently on Stock Hardware* by Marc Feeley, *Proc. 1993 Conference on Functional Programming and Computer Architecture*, Copenhagen, Denmark, pp. 179–187, is recommended as further reading.

Exceptions thrown by the Java Virtual Machine are precise: when the transfer of control takes place, all effects of the instructions executed before the point from which the exception is thrown must appear to have taken place. No instructions that occur after the point from which the exception is thrown may appear to have been evaluated. If optimized code has speculatively executed some of the instructions which follow the point at which the exception occurs, such code must be prepared to hide this speculative execution from the user-visible state of the program.

Each method in the Java Virtual Machine may be associated with zero or more *exception handlers*. An exception handler specifies the range of offsets into the Java Virtual Machine code implementing the method for which the exception handler is active, describes the type of exception that the exception handler is able to handle, and specifies the location of the code that is to handle that exception. An exception matches an exception handler if the offset of the instruction that caused the exception is in the range of offsets of the exception handler and the exception type is the same class as or a subclass of the class of exception that the exception handler handles. When an exception is thrown, the Java Virtual Machine searches for a matching exception handler in the current method. If a matching exception handler is found, the system branches to the exception handling code specified by the matched handler.

If no such exception handler is found in the current method, the current method invocation completes abruptly (§2.6.5). On abrupt completion, the operand stack and local variables of the current method invocation are discarded, and its frame is popped, reinstating the frame of the invoking method. The exception is then rethrown in the context of the invoker's frame and so on, continuing up the method invocation chain. If no suitable exception handler is found before the top of the method invocation chain is reached, the execution of the thread in which the exception was thrown is terminated. Before termination of the thread, the uncaught exception is handled according to the following rules:

- If the thread has an uncaught exception handler set, then that handler is executed.
- Otherwise, the method `uncaughtException` is invoked for the `ThreadGroup` that is the parent of the thread. If the `ThreadGroup` and its parent `ThreadGroups` do not override `uncaughtException`, then the default handler's `uncaughtException` method is invoked.

The order in which the exception handlers of a method are searched for a match is important. Within a `class` file, the exception handlers for each method are stored in a table (§4.7.3). At run time, when an exception is thrown, the Java Virtual Machine searches the exception handlers of the current method in the order that they appear

in the corresponding exception handler table in the class file, starting from the beginning of that table.

Note that the Java Virtual Machine does not enforce nesting of or any ordering of the exception table entries of a method. The exception handling semantics of the Java programming language are implemented only through cooperation with the compiler (§3.12). When class files are generated by some other means, the defined search procedure ensures that all Java Virtual Machine implementations will behave consistently.

2.11 Instruction Set Summary

A Java Virtual Machine instruction consists of a one-byte *opcode* specifying the operation to be performed, followed by zero or more *operands* supplying arguments or data that are used by the operation. Many instructions have no operands and consist only of an opcode.

Ignoring exceptions, the inner loop of a Java Virtual Machine interpreter is effectively

```
do {
    atomically calculate pc and fetch opcode at pc;
    if (operands) fetch operands;
    execute the action for the opcode;
} while (there is more to do);
```

The number and size of the operands are determined by the opcode. If an operand is more than one byte in size, then it is stored in *big-endian* order - high-order byte first. For example, an unsigned 16-bit index into the local variables is stored as two unsigned bytes, *byte1* and *byte2*, such that its value is $(byte1 \ll 8) \mid byte2$.

The bytecode instruction stream is only single-byte aligned. The two exceptions are the *lookupswitch* and *tableswitch* instructions (*\$lookupswitch*, *\$tableswitch*), which are padded to force internal alignment of some of their operands on 4-byte boundaries.

The decision to limit the Java Virtual Machine opcode to a byte and to forgo data alignment within compiled code reflects a conscious bias in favor of compactness, possibly at the cost of some performance in naive implementations. A one-byte opcode also limits the size of the instruction set. Not assuming data alignment means that immediate data larger than a byte must be constructed from bytes at run time on many machines.

2.11.1 Types and the Java Virtual Machine

Most of the instructions in the Java Virtual Machine instruction set encode type information about the operations they perform. For instance, the *iload* instruction (§*iload*) loads the contents of a local variable, which must be an `int`, onto the operand stack. The *fload* instruction (§*fload*) does the same with a `float` value. The two instructions may have identical implementations, but have distinct opcodes.

For the majority of typed instructions, the instruction type is represented explicitly in the opcode mnemonic by a letter: *i* for an `int` operation, *l* for `long`, *s* for `short`, *b* for `byte`, *c* for `char`, *f* for `float`, *d* for `double`, and *a* for `reference`. Some instructions for which the type is unambiguous do not have a type letter in their mnemonic. For instance, *arraylength* always operates on an object that is an array. Some instructions, such as *goto*, an unconditional control transfer, do not operate on typed operands.

Given the Java Virtual Machine's one-byte opcode size, encoding types into opcodes places pressure on the design of its instruction set. If each typed instruction supported all of the Java Virtual Machine's run-time data types, there would be more instructions than could be represented in a byte. Instead, the instruction set of the Java Virtual Machine provides a reduced level of type support for certain operations. In other words, the instruction set is intentionally not orthogonal. Separate instructions can be used to convert between unsupported and supported data types as necessary.

Table 2.11.1-A summarizes the type support in the instruction set of the Java Virtual Machine. A specific instruction, with type information, is built by replacing the *T* in the instruction template in the opcode column by the letter in the type column. If the type column for some instruction template and type is blank, then no instruction exists supporting that type of operation. For instance, there is a load instruction for type `int`, *iload*, but there is no load instruction for type `byte`.

Note that most instructions in Table 2.11.1-A do not have forms for the integral types `byte`, `char`, and `short`. None have forms for the `boolean` type. Whenever values of types `byte` and `short` are loaded onto the operand stack, they are implicitly converted by sign extension to values of type `int`. Similarly, whenever values of types `boolean` and `char` are loaded onto the operand stack, they are implicitly converted by zero extension to values of type `int`. Thus, most operations on values originally stored in variables of types `boolean`, `byte`, `char`, and `short` are correctly performed by instructions operating on values of computational type `int`.

Table 2.11.1-A. Type support in the Java Virtual Machine instruction set

opcode	byte	short	int	long	float	double	char	reference
<i>Tipush</i>	<i>bipush</i>	<i>sipush</i>						
<i>Tconst</i>			<i>iconst</i>	<i>lconst</i>	<i>fconst</i>	<i>dconst</i>		<i>aconst</i>
<i>Tload</i>			<i>iload</i>	<i>lload</i>	<i>fload</i>	<i>dload</i>		<i>aload</i>
<i>Tstore</i>			<i>istore</i>	<i>lstore</i>	<i>fstore</i>	<i>dstore</i>		<i>astore</i>
<i>Tinc</i>			<i>iinc</i>					
<i>Taload</i>	<i>baload</i>	<i>saload</i>	<i>iaload</i>	<i>laload</i>	<i>faload</i>	<i>daload</i>	<i>caload</i>	<i>aaload</i>
<i>Tastore</i>	<i>bastore</i>	<i>sastore</i>	<i>iastore</i>	<i>lastore</i>	<i>fastore</i>	<i>dastore</i>	<i>castore</i>	<i>aastore</i>
<i>Tadd</i>			<i>iadd</i>	<i>ladd</i>	<i>fadd</i>	<i>dadd</i>		
<i>Tsub</i>			<i>isub</i>	<i>lsub</i>	<i>fsub</i>	<i>dsub</i>		
<i>Tmul</i>			<i>imul</i>	<i>lmul</i>	<i>fmul</i>	<i>dmul</i>		
<i>Tdiv</i>			<i>idiv</i>	<i>ldiv</i>	<i>fdiv</i>	<i>ddiv</i>		
<i>Trem</i>			<i>irem</i>	<i>lrem</i>	<i>frem</i>	<i>drem</i>		
<i>Tneg</i>			<i>ineg</i>	<i>lneg</i>	<i>fneg</i>	<i>dneg</i>		
<i>Tshl</i>			<i>ishl</i>	<i>lshl</i>				
<i>Tshr</i>			<i>ishr</i>	<i>lshr</i>				
<i>Tushr</i>			<i>iushr</i>	<i>lushr</i>				
<i>Tand</i>			<i>iand</i>	<i>land</i>				
<i>Tor</i>			<i>ior</i>	<i>lor</i>				
<i>Txor</i>			<i>ixor</i>	<i>lxor</i>				
<i>i2T</i>	<i>i2b</i>	<i>i2s</i>		<i>i2l</i>	<i>i2f</i>	<i>i2d</i>	<i>i2c</i>	
<i>l2T</i>			<i>l2i</i>		<i>l2f</i>	<i>l2d</i>		
<i>f2T</i>			<i>f2i</i>	<i>f2l</i>		<i>f2d</i>		
<i>d2T</i>			<i>d2i</i>	<i>d2l</i>	<i>d2f</i>			
<i>Tcmp</i>				<i>lcmp</i>				
<i>Tcmpl</i>					<i>fcmpl</i>	<i>dcmpl</i>		
<i>Tcmpg</i>					<i>fcmpg</i>	<i>dcmpg</i>		
<i>if_TcmpOP</i>			<i>if_icmpOP</i>					<i>if_acmpOP</i>
<i>Treturn</i>			<i>ireturn</i>	<i>lreturn</i>	<i>freturn</i>	<i>dreturn</i>		<i>areturn</i>

The mapping between Java Virtual Machine storage types and Java Virtual Machine computational types is summarized by Table 2.11.1-B.

Certain Java Virtual Machine instructions such as *pop* and *swap* operate on the operand stack without regard to type; however, such instructions are constrained to use only on values of certain categories of computational types, also given in Table 2.11.1-B.

Table 2.11.1-B. Storage and Computational types in the Java Virtual Machine

Storage type	Computational type	Category
boolean	int	1
byte	int	1
char	int	1
short	int	1
int	int	1
float	float	1
reference	reference	1
returnAddress	returnAddress	1
long	long	2
double	double	2

2.11.2 Load and Store Instructions

The load and store instructions transfer values between the local variables (§2.6.1) and the operand stack (§2.6.2) of a Java Virtual Machine frame (§2.6):

- Load a local variable onto the operand stack: *iload*, *iload_<n>*, *lload*, *lload_<n>*, *fload*, *fload_<n>*, *dload*, *dload_<n>*, *aload*, *aload_<n>*.
- Store a value from the operand stack into a local variable: *istore*, *istore_<n>*, *lstore*, *lstore_<n>*, *fstore*, *fstore_<n>*, *dstore*, *dstore_<n>*, *astore*, *astore_<n>*.
- Load a constant on to the operand stack: *bipush*, *sipush*, *ldc*, *ldc_w*, *ldc2_w*, *acnst_null*, *iconst_m1*, *iconst_<i>*, *lconst_<l>*, *fconst_<f>*, *dconst_<d>*.
- Gain access to more local variables using a wider index, or to a larger immediate operand: *wide*.

Instructions that access fields of objects and elements of arrays (§2.11.5) also transfer data to and from the operand stack.

Instruction mnemonics shown above with trailing letters between angle brackets (for instance, *iload_<n>*) denote families of instructions (with members *iload_0*, *iload_1*, *iload_2*, and *iload_3* in the case of *iload_<n>*). Such families of instructions are specializations of an additional generic instruction (*iload*) that takes one operand. For the specialized instructions, the operand is implicit and does not need to be stored or fetched. The semantics are otherwise the same (*iload_0* means the same thing as *iload* with the operand 0). The letter between the angle brackets specifies the type of the implicit operand for that family of instructions: for *<n>*, a nonnegative integer; for *<i>*, an *int*; for *<l>*, a *long*; for *<f>*, a *float*; and for *<d>*, a *double*.

This notation for instruction families is used throughout this specification.

2.11.3 Arithmetic Instructions

The arithmetic instructions compute a result that is typically a function of two values on the operand stack, pushing the result back on the operand stack. There are two main kinds of arithmetic instructions: those operating on integer values and those operating on floating-point values. Within each of these kinds, the arithmetic instructions are specialized to Java Virtual Machine numeric types. There is no direct support for integer arithmetic on values of the *byte*, *short*, and *char* types (§2.11.1), or for values of the *boolean* type; those operations are handled by instructions operating on type *int*. Integer and floating-point instructions also differ in their behavior on overflow and divide-by-zero. The arithmetic instructions are as follows:

- Add: *iadd*, *ladd*, *fadd*, *dadd*.
- Subtract: *isub*, *lsub*, *fsub*, *dsub*.
- Multiply: *imul*, *lmul*, *fmul*, *dmul*.
- Divide: *idiv*, *ldiv*, *fdiv*, *ddiv*.
- Remainder: *irem*, *lrem*, *frem*, *drem*.
- Negate: *ineg*, *lneg*, *fneg*, *dneg*.
- Shift: *ishl*, *ishr*, *iushr*, *lshl*, *lshr*, *lushr*.
- Bitwise OR: *ior*, *lor*.
- Bitwise AND: *iand*, *land*.
- Bitwise exclusive OR: *ixor*, *lxor*.
- Local variable increment: *iinc*.

- Comparison: *dcmpg*, *dcmpl*, *fcmpg*, *fcmpl*, *lcmp*.

The semantics of the Java programming language operators on integer and floating-point values (JLS §4.2.2, JLS §4.2.4) are directly supported by the semantics of the Java Virtual Machine instruction set.

The Java Virtual Machine does not indicate overflow during operations on integer data types. The only integer operations that can throw an exception are the integer divide instructions (*idiv* and *ldiv*) and the integer remainder instructions (*irem* and *lrem*), which throw an `ArithmeticException` if the divisor is zero.

The Java Virtual Machine does not indicate overflow or underflow during operations on floating-point data types. That is, floating-point instructions never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception). An operation that overflows produces a signed infinity; an operation that underflows produces a subnormal value or a signed zero; an operation that has no unique mathematically defined result produces NaN. All numeric operations with NaN as an operand produce NaN as a result.

Comparisons on values of type `long` (*lcmp*) perform a signed comparison.

Comparisons on values of floating-point types (*dcmpg*, *dcmpl*, *fcmpg*, *fcmpl*) are performed using IEEE 754 nonsignaling comparisons.

2.11.4 Type Conversion Instructions

The type conversion instructions allow conversion between Java Virtual Machine numeric types. These may be used to implement explicit conversions in user code or to mitigate the lack of orthogonality in the instruction set of the Java Virtual Machine.

The Java Virtual Machine directly supports the following widening numeric conversions:

- `int` to `long`, `float`, or `double`
- `long` to `float` or `double`
- `float` to `double`

The widening numeric conversion instructions are *i2l*, *i2f*, *i2d*, *l2f*, *l2d*, and *f2d*. The mnemonics for these opcodes are straightforward given the naming conventions for typed instructions and the punning use of 2 to mean "to." For instance, the *i2d* instruction converts an `int` value to a `double`.

Most widening numeric conversions do not lose information about the overall magnitude of a numeric value. Indeed, conversions widening from `int` to `long` and `int` to `double` do not lose any information at all; the numeric value is preserved exactly. Conversions widening from `float` to `double` also preserve the numeric value exactly.

Conversions from `int` to `float`, or from `long` to `float`, or from `long` to `double`, may lose *precision*, that is, may lose some of the least significant bits of the value; the resulting floating-point value is a correctly rounded version of the integer value, using the round to nearest rounding policy (§2.8).

Despite the fact that loss of precision may occur, widening numeric conversions never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception).

A widening numeric conversion of an `int` to a `long` simply sign-extends the two's-complement representation of the `int` value to fill the wider format. A widening numeric conversion of a `char` to an integral type zero-extends the representation of the `char` value to fill the wider format.

Note that widening numeric conversions do not exist from integral types `byte`, `char`, and `short` to type `int`. As noted in §2.11.1, values of type `byte`, `char`, and `short` are internally widened to type `int`, making these conversions implicit.

The Java Virtual Machine also directly supports the following narrowing numeric conversions:

- `int` to `byte`, `short`, or `char`
- `long` to `int`
- `float` to `int` or `long`
- `double` to `int`, `long`, or `float`

The narrowing numeric conversion instructions are *i2b*, *i2c*, *i2s*, *l2i*, *f2i*, *f2l*, *d2i*, *d2l*, and *d2f*. A narrowing numeric conversion can result in a value of different sign, a different order of magnitude, or both; it may thereby lose precision.

A narrowing numeric conversion of an `int` or `long` to an integral type τ simply discards all but the n lowest-order bits, where n is the number of bits used to represent type τ . This may cause the resulting value not to have the same sign as the input value.

In a narrowing numeric conversion of a floating-point value to an integral type τ , where τ is either `int` or `long`, the floating-point value is converted as follows:

- If the floating-point value is NaN, the result of the conversion is an `int` or `long` 0.

- Otherwise, if the floating-point value is not an infinity, the floating-point value is rounded to an integer value v using the round toward zero rounding policy (§2.8). There are two cases:
 - If τ is `long` and this integer value can be represented as a `long`, then the result is the `long` value v .
 - If τ is of type `int` and this integer value can be represented as an `int`, then the result is the `int` value v .
- Otherwise:
 - Either the value must be too small (a negative value of large magnitude or negative infinity), and the result is the smallest representable value of type `int` or `long`.
 - Or the value must be too large (a positive value of large magnitude or positive infinity), and the result is the largest representable value of type `int` or `long`.

A narrowing numeric conversion from `double` to `float` behaves in accordance with IEEE 754. The result is correctly rounded using the round to nearest rounding policy (§2.8). A value too small to be represented as a `float` is converted to a positive or negative zero of type `float`; a value too large to be represented as a `float` is converted to a positive or negative infinity. A `double` NaN is always converted to a `float` NaN.

Despite the fact that overflow, underflow, or loss of precision may occur, narrowing conversions among numeric types never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception).

2.11.5 Object Creation and Manipulation

Although both class instances and arrays are objects, the Java Virtual Machine creates and manipulates class instances and arrays using distinct sets of instructions:

- Create a new class instance: *new*.
- Create a new array: *newarray*, *anewarray*, *multianewarray*.
- Access fields of classes (static fields, known as class variables) and fields of class instances (non-static fields, known as instance variables): *getstatic*, *putstatic*, *getfield*, *putfield*.
- Load an array component onto the operand stack: *baload*, *caload*, *saload*, *iaload*, *laload*, *faload*, *daload*, *aaload*.

- Store a value from the operand stack as an array component: *bastore*, *castore*, *sastore*, *iastore*, *lastore*, *fastore*, *dastore*, *aastore*.
- Get the length of array: *arraylength*.
- Check properties of class instances or arrays: *instanceof*, *checkcast*.

2.11.6 Operand Stack Management Instructions

A number of instructions are provided for the direct manipulation of the operand stack: *pop*, *pop2*, *dup*, *dup2*, *dup_x1*, *dup2_x1*, *dup_x2*, *dup2_x2*, *swap*.

2.11.7 Control Transfer Instructions

The control transfer instructions conditionally or unconditionally cause the Java Virtual Machine to continue execution with an instruction other than the one following the control transfer instruction. They are:

- Conditional branch: *ifeq*, *ifne*, *iflt*, *ifle*, *ifgt*, *ifge*, *ifnull*, *ifnonnull*, *if_icmpeq*, *if_icmpne*, *if_icmplt*, *if_icmple*, *if_icmpgt*, *if_icmpge*, *if_acmpeq*, *if_acmpne*.
- Compound conditional branch: *tableswitch*, *lookupswitch*.
- Unconditional branch: *goto*, *goto_w*, *jsr*, *jsr_w*, *ret*.

The Java Virtual Machine has distinct sets of instructions that conditionally branch on comparison with data of `int` and reference types. It also has distinct conditional branch instructions that test for the null reference and thus it is not required to specify a concrete value for `null` (§2.4).

Conditional branches on comparisons between data of types `boolean`, `byte`, `char`, and `short` are performed using `int` comparison instructions (§2.11.1). A conditional branch on a comparison between data of types `long`, `float`, or `double` is initiated using an instruction that compares the data and produces an `int` result of the comparison (§2.11.3). A subsequent `int` comparison instruction tests this result and effects the conditional branch. Because of its emphasis on `int` comparisons, the Java Virtual Machine provides a rich complement of conditional branch instructions for type `int`.

All `int` conditional control transfer instructions perform signed comparisons.

2.11.8 Method Invocation and Return Instructions

The following five instructions invoke methods:

- *invokevirtual* invokes an instance method of an object, dispatching on the (virtual) type of the object.
- *invokeinterface* invokes an interface method, searching the methods implemented by the particular run-time object to find the appropriate method.
- *invokespecial* invokes an instance method requiring special handling, either an instance initialization method (§2.9.1) or a method of the current class or its supertypes.
- *invokestatic* invokes a class (static) method in a named class.
- *invokedynamic* invokes the method which is the target of the call site object bound to the *invokedynamic* instruction. The call site object was bound to a specific lexical occurrence of the *invokedynamic* instruction by the Java Virtual Machine as a result of running a bootstrap method before the first execution of the instruction. Therefore, each occurrence of an *invokedynamic* instruction has a unique linkage state, unlike the other instructions which invoke methods.

The method return instructions, which are distinguished by return type, are *ireturn* (used for return types boolean, byte, char, short, and int), *lreturn*, *freturn*, *dreturn*, and *areturn*. In addition, the *return* instruction is used to return from methods declared to be void, instance initialization methods, and class or interface initialization methods.

2.11.9 Throwing Exceptions

An exception is thrown programmatically using the *athrow* instruction. Exceptions can also be thrown by various Java Virtual Machine instructions if they detect an abnormal condition.

2.11.10 Synchronization

The Java Virtual Machine supports synchronization of both methods and sequences of instructions within a method by a single synchronization construct: the *monitor*.

Method-level synchronization is performed implicitly, as part of method invocation and return (§2.11.8). A synchronized method is distinguished in the run-time constant pool's *method_info* structure (§4.6) by the *ACC_SYNCHRONIZED* flag, which is checked by the method invocation instructions. When invoking a method for which *ACC_SYNCHRONIZED* is set, the executing thread enters a monitor, invokes the method itself, and exits the monitor whether the method invocation completes normally or abruptly. During the time the executing thread owns the monitor, no other thread may enter it. If an exception is thrown during invocation of

the synchronized method and the synchronized method does not handle the exception, the monitor for the method is automatically exited before the exception is rethrown out of the synchronized method.

Synchronization of sequences of instructions is typically used to encode the synchronized block of the Java programming language. The Java Virtual Machine supplies the *monitorenter* and *monitorexit* instructions to support such language constructs. Proper implementation of synchronized blocks requires cooperation from a compiler targeting the Java Virtual Machine (§3.14).

Structured locking is the situation when, during a method invocation, every exit on a given monitor matches a preceding entry on that monitor. Since there is no assurance that all code submitted to the Java Virtual Machine will perform structured locking, implementations of the Java Virtual Machine are permitted but not required to enforce both of the following two rules guaranteeing structured locking. Let T be a thread and M be a monitor. Then:

1. The number of monitor entries performed by T on M during a method invocation must equal the number of monitor exits performed by T on M during the method invocation whether the method invocation completes normally or abruptly.
2. At no point during a method invocation may the number of monitor exits performed by T on M since the method invocation exceed the number of monitor entries performed by T on M since the method invocation.

Note that the monitor entry and exit automatically performed by the Java Virtual Machine when invoking a synchronized method are considered to occur during the calling method's invocation.

2.12 Class Libraries

The Java Virtual Machine must provide sufficient support for the implementation of the class libraries of the Java SE Platform. Some of the classes in these libraries cannot be implemented without the cooperation of the Java Virtual Machine.

Classes that might require special support from the Java Virtual Machine include those that support:

- Reflection, such as the classes in the package `java.lang.reflect` and the class `Class`.

- Loading and creation of a class or interface. The most obvious example is the class `ClassLoader`.
- Linking and initialization of a class or interface. The example classes cited above fall into this category as well.
- The module system, such as the class `ModuleLayer`.
- Multithreading, such as the class `Thread`.
- Weak references, such as the classes in the package `java.lang.ref`.

The list above is meant to be illustrative rather than comprehensive. An exhaustive list of these classes or of the functionality they provide is beyond the scope of this specification. See the specifications of the Java SE Platform class libraries for details.

2.13 Public Design, Private Implementation

Thus far this specification has sketched the public view of the Java Virtual Machine: the `class` file format and the instruction set. These components are vital to the hardware-, operating system-, and implementation-independence of the Java Virtual Machine. The implementor may prefer to think of them as a means to securely communicate fragments of programs between hosts each implementing the Java SE Platform, rather than as a blueprint to be followed exactly.

It is important to understand where the line between the public design and the private implementation lies. A Java Virtual Machine implementation must be able to read `class` files and must exactly implement the semantics of the Java Virtual Machine code therein. One way of doing this is to take this document as a specification and to implement that specification literally. But it is also perfectly feasible and desirable for the implementor to modify or optimize the implementation within the constraints of this specification. So long as the `class` file format can be read and the semantics of its code are maintained, the implementor may implement these semantics in any way. What is "under the hood" is the implementor's business, as long as the correct external interface is carefully maintained.

There are some exceptions: debuggers, profilers, and just-in-time code generators can each require access to elements of the Java Virtual Machine that are normally considered to be "under the hood." Where appropriate, Oracle works with other Java Virtual Machine implementors and with tool vendors to develop common interfaces to the Java Virtual Machine for use by such tools, and to promote those interfaces across the industry.

The implementor can use this flexibility to tailor Java Virtual Machine implementations for high performance, low memory use, or portability. What makes sense in a given implementation depends on the goals of that implementation. The range of implementation options includes the following:

- Translating Java Virtual Machine code at load-time or during execution into the instruction set of another virtual machine.
- Translating Java Virtual Machine code at load-time or during execution into the native instruction set of the host CPU (sometimes referred to as *just-in-time*, or *JIT*, code generation).

The existence of a precisely defined virtual machine and object file format need not significantly restrict the creativity of the implementor. The Java Virtual Machine is designed to support many different implementations, providing new and interesting solutions while retaining compatibility between implementations.

Compiling for the Java Virtual Machine

THE Java Virtual Machine machine is designed to support the Java programming language. Oracle's JDK software contains a compiler from source code written in the Java programming language to the instruction set of the Java Virtual Machine, and a run-time system that implements the Java Virtual Machine itself. Understanding how one compiler utilizes the Java Virtual Machine is useful to the prospective compiler writer, as well as to one trying to understand the Java Virtual Machine itself. The numbered sections in this chapter are not normative.

Note that the term "compiler" is sometimes used when referring to a translator from the instruction set of the Java Virtual Machine to the instruction set of a specific CPU. One example of such a translator is a just-in-time (JIT) code generator, which generates platform-specific instructions only after Java Virtual Machine code has been loaded. This chapter does not address issues associated with code generation, only those associated with compiling source code written in the Java programming language to Java Virtual Machine instructions.

3.1 Format of Examples

This chapter consists mainly of examples of source code together with annotated listings of the Java Virtual Machine code that the `javac` compiler in Oracle's JDK release 1.0.2 generates for the examples. The Java Virtual Machine code is written in the informal "virtual machine assembly language" output by Oracle's `javap` utility, distributed with the JDK release. You can use `javap` to generate additional examples of compiled methods.

The format of the examples should be familiar to anyone who has read assembly code. Each instruction takes the form:

```
<index> <opcode> [ <operand1> [ <operand2>... ] ] [<comment>]
```

The <index> is the index of the opcode of the instruction in the array that contains the bytes of Java Virtual Machine code for this method. Alternatively, the <index> may be thought of as a byte offset from the beginning of the method. The <opcode> is the mnemonic for the instruction's opcode, and the zero or more <operandN> are the operands of the instruction. The optional <comment> is given in end-of-line comment syntax:

```
8    bipush 100      // Push int constant 100
```

Some of the material in the comments is emitted by `javap`; the rest is supplied by the authors. The <index> prefacing each instruction may be used as the target of a control transfer instruction. For instance, a `goto 8` instruction transfers control to the instruction at index 8. Note that the actual operands of Java Virtual Machine control transfer instructions are offsets from the addresses of the opcodes of those instructions; these operands are displayed by `javap` (and are shown in this chapter) as more easily read offsets into their methods.

We preface an operand representing a run-time constant pool index with a hash sign and follow the instruction by a comment identifying the run-time constant pool item referenced, as in:

```
10   ldc #1          // Push float constant 100.0
```

or:

```
9    invokevirtual #4 // Method Example.addTwo(II)I
```

For the purposes of this chapter, we do not worry about specifying details such as operand sizes.

3.2 Use of Constants, Local Variables, and Control Constructs

Java Virtual Machine code exhibits a set of general characteristics imposed by the Java Virtual Machine's design and use of types. In the first example we encounter many of these, and we consider them in some detail.

The `spin` method simply spins around an empty for loop 100 times:

```
void spin() {
```

```
    int i;
    for (i = 0; i < 100; i++) {
        ;    // Loop body is empty
    }
}
```

A compiler might compile `spin` to:

```
0   iconst_0           // Push int constant 0
1   istore_1           // Store into local variable 1 (i=0)
2   goto 8             // First time through don't increment
5   iinc 1 1           // Increment local variable 1 by 1 (i++)
8   iload_1            // Push local variable 1 (i)
9   bipush 100         // Push int constant 100
11  if_icmplt 5         // Compare and loop if less than (i < 100)
14  return             // Return void when done
```

The Java Virtual Machine is stack-oriented, with most operations taking one or more operands from the operand stack of the Java Virtual Machine's current frame or pushing results back onto the operand stack. A new frame is created each time a method is invoked, and with it is created a new operand stack and set of local variables for use by that method (§2.6). At any one point of the computation, there are thus likely to be many frames and equally many operand stacks per thread of control, corresponding to many nested method invocations. Only the operand stack in the current frame is active.

The instruction set of the Java Virtual Machine distinguishes operand types by using distinct bytecodes for operations on its various data types. The method `spin` operates only on values of type `int`. The instructions in its compiled code chosen to operate on typed data (*iconst_0*, *istore_1*, *iinc*, *iload_1*, *if_icmplt*) are all specialized for type `int`.

The two constants in `spin`, `0` and `100`, are pushed onto the operand stack using two different instructions. The `0` is pushed using an *iconst_0* instruction, one of the family of *iconst_<i>* instructions. The `100` is pushed using a *bipush* instruction, which fetches the value it pushes as an immediate operand.

The Java Virtual Machine frequently takes advantage of the likelihood of certain operands (`int` constants `-1`, `0`, `1`, `2`, `3`, `4` and `5` in the case of the *iconst_<i>* instructions) by making those operands implicit in the opcode. Because the *iconst_0* instruction knows it is going to push an `int 0`, *iconst_0* does not need to store an operand to tell it what value to push, nor does it need to fetch or decode an operand. Compiling the push of `0` as *bipush 0* would have been correct, but would have made the compiled code for `spin` one byte longer. A simple virtual machine would have also spent additional time fetching and decoding the explicit operand

each time around the loop. Use of implicit operands makes compiled code more compact and efficient.

The `int i` in `spin` is stored as Java Virtual Machine local variable `1`. Because most Java Virtual Machine instructions operate on values popped from the operand stack rather than directly on local variables, instructions that transfer values between local variables and the operand stack are common in code compiled for the Java Virtual Machine. These operations also have special support in the instruction set. In `spin`, values are transferred to and from local variables using the `istore_1` and `iload_1` instructions, each of which implicitly operates on local variable `1`. The `istore_1` instruction pops an `int` from the operand stack and stores it in local variable `1`. The `iload_1` instruction pushes the value in local variable `1` on to the operand stack.

The use (and reuse) of local variables is the responsibility of the compiler writer. The specialized load and store instructions should encourage the compiler writer to reuse local variables as much as is feasible. The resulting code is faster, more compact, and uses less space in the frame.

Certain very frequent operations on local variables are catered to specially by the Java Virtual Machine. The `iinc` instruction increments the contents of a local variable by a one-byte signed value. The `iinc` instruction in `spin` increments the first local variable (its first operand) by `1` (its second operand). The `iinc` instruction is very handy when implementing looping constructs.

The `for` loop of `spin` is accomplished mainly by these instructions:

```

5   iinc 1 1      // Increment local variable 1 by 1 (i++)
8   iload_1      // Push local variable 1 (i)
9   bipush 100    // Push int constant 100
11  if_icmplt 5    // Compare and loop if less than (i < 100)
```

The `bipush` instruction pushes the value `100` onto the operand stack as an `int`, then the `if_icmplt` instruction pops that value off the operand stack and compares it against `i`. If the comparison succeeds (the variable `i` is less than `100`), control is transferred to index `5` and the next iteration of the `for` loop begins. Otherwise, control passes to the instruction following the `if_icmplt`.

If the `spin` example had used a data type other than `int` for the loop counter, the compiled code would necessarily change to reflect the different data type. For instance, if instead of an `int` the `spin` example uses a `double`, as shown:

```

void dspin() {
    double i;
    for (i = 0.0; i < 100.0; i++) {
        ;    // Loop body is empty
    }
}
```



```
    }  
}
```

the compiled code is:

```
Method void dspin()  
0   dconst_0      // Push double constant 0.0  
1   dstore_1      // Store into local variables 1 and 2  
2   goto 9        // First time through don't increment  
5   dload_1       // Push local variables 1 and 2  
6   dconst_1      // Push double constant 1.0  
7   dadd          // Add; there is no dinc instruction  
8   dstore_1      // Store result in local variables 1 and 2  
9   dload_1       // Push local variables 1 and 2  
10  ldc2_w #4     // Push double constant 100.0  
13  dcmpg         // There is no if_dcmplt instruction  
14  iflt 5        // Compare and loop if less than (i < 100.0)  
17  return        // Return void when done
```

The instructions that operate on typed data are now specialized for type double. (The *ldc2_w* instruction will be discussed later in this chapter.)

Recall that double values occupy two local variables, although they are only accessed using the lesser index of the two local variables. This is also the case for values of type long. Again for example,

```
double doubleLocals(double d1, double d2) {  
    return d1 + d2;  
}
```

becomes

```
Method double doubleLocals(double, double)  
0   dload_1       // First argument in local variables 1 and 2  
1   dload_3       // Second argument in local variables 3 and 4  
2   dadd          // Add  
3   dreturn
```

Note that local variables of the local variable pairs used to store double values in *doubleLocals* must never be manipulated individually.

The Java Virtual Machine's opcode size of 1 byte results in its compiled code being very compact. However, 1-byte opcodes also mean that the Java Virtual Machine instruction set must stay small. As a compromise, the Java Virtual Machine does not provide equal support for all data types: it is not completely orthogonal (Table 2.11.1-A).

For example, the comparison of values of type *int* in the *for* statement of example *spin* can be implemented using a single *if_icmplt* instruction; however, there is

no single instruction in the Java Virtual Machine instruction set that performs a conditional branch on values of type `double`. Thus, `dspin` must implement its comparison of values of type `double` using a `dcmpg` instruction followed by an `iflt` instruction.

The Java Virtual Machine provides the most direct support for data of type `int`. This is partly in anticipation of efficient implementations of the Java Virtual Machine's operand stacks and local variable arrays. It is also motivated by the frequency of `int` data in typical programs. Other integral types have less direct support. There are no `byte`, `char`, or `short` versions of the `store`, `load`, or `add` instructions, for instance. Here is the `spin` example written using a `short`:

```
void sspin() {
    short i;
    for (i = 0; i < 100; i++) {
        ;    // Loop body is empty
    }
}
```

It must be compiled for the Java Virtual Machine, as follows, using instructions operating on another type, most likely `int`, converting between `short` and `int` values as necessary to ensure that the results of operations on `short` data stay within the appropriate range:

```
Method void sspin()
0   iconst_0
1   istore_1
2   goto 10
5   iload_1           // The short is treated as though an int
6   iconst_1
7   iadd
8   i2s              // Truncate int to short
9   istore_1
10  iload_1
11  bipush 100
13  if_icmplt 5
16  return
```

The lack of direct support for `byte`, `char`, and `short` types in the Java Virtual Machine is not particularly painful, because values of those types are internally promoted to `int` (`byte` and `short` are sign-extended to `int`, `char` is zero-extended). Operations on `byte`, `char`, and `short` data can thus be done using `int` instructions. The only additional cost is that of truncating the values of `int` operations to valid ranges.

The long and floating-point types have an intermediate level of support in the Java Virtual Machine, lacking only the full complement of conditional control transfer instructions.

3.3 Arithmetic

The Java Virtual Machine generally does arithmetic on its operand stack. (The exception is the *iinc* instruction, which directly increments the value of a local variable.) For instance, the `align2grain` method aligns an `int` value to a given power of 2:

```
int align2grain(int i, int grain) {
    return ((i + grain-1) & ~(grain-1));
}
```

Operands for arithmetic operations are popped from the operand stack, and the results of operations are pushed back onto the operand stack. Results of arithmetic subcomputations can thus be made available as operands of their nesting computation. For instance, the calculation of $\sim(\text{grain}-1)$ is handled by these instructions:

```
5   iload_2          // Push grain
6   iconst_1         // Push int constant 1
7   isub             // Subtract; push result
8   iconst_m1        // Push int constant -1
9   ixor             // Do XOR; push result
```

First $\text{grain}-1$ is calculated using the contents of local variable 2 and an immediate `int` value 1. These operands are popped from the operand stack and their difference pushed back onto the operand stack. The difference is thus immediately available for use as one operand of the *ixor* instruction. (Recall that $\sim x == -1 \wedge x$.) Similarly, the result of the *ixor* instruction becomes an operand for the subsequent *iand* instruction.

The code for the entire method follows:

```
Method int align2grain(int,int)
0   iload_1
1   iload_2
2   iadd
3   iconst_1
4   isub
5   iload_2
6   iconst_1
7   isub
```

```

8   iconst_m1
9   ixor
10  iand
11  ireturn

```

3.4 Accessing the Run-Time Constant Pool

Many numeric constants, as well as objects, fields, and methods, are accessed via the run-time constant pool of the current class. Object access is considered later (§3.8). Data of types `int`, `long`, `float`, and `double`, as well as references to instances of class `String`, are managed using the *ldc*, *ldc_w*, and *ldc2_w* instructions.

The *ldc* and *ldc_w* instructions are used to access values in the run-time constant pool (including instances of class `String`) of types other than `double` and `long`. The *ldc_w* instruction is used in place of *ldc* only when there is a large number of run-time constant pool items and a larger index is needed to access an item. The *ldc2_w* instruction is used to access all values of types `double` and `long`; there is no non-wide variant.

Integral constants of types `byte`, `char`, or `short`, as well as small `int` values, may be compiled using the *bipush*, *sipush*, or *iconst_<i>* instructions (§3.2). Certain small floating-point constants may be compiled using the *fconst_<f>* and *dconst_<d>* instructions.

In all of these cases, compilation is straightforward. For instance, the constants for:

```

void useManyNumeric() {
    int i = 100;
    int j = 1000000;
    long l1 = 1;
    long l2 = 0xffffffff;
    double d = 2.2;
    ...do some calculations...
}

```

are set up as follows:

```

Method void useManyNumeric()
0   bipush 100    // Push small int constant with bipush
2   istore_1
3   ldc #1        // Push large int constant (1000000) with ldc
5   istore_2
6   lconst_1     // A tiny long value uses small fast lconst_1
7   lstore_3
8   ldc2_w #6     // Push long 0xffffffff (that is, an int -1)

```

```

        // Any long constant value can be pushed with ldc2_w
11  lstore 5
13  ldc2_w #8      // Push double constant 2.200000
        // Uncommon double values are also pushed with ldc2_w
16  dstore 7
    ...do those calculations...

```

3.5 More Control Examples

Compilation of `for` statements was shown in an earlier section (§3.2). Most of the Java programming language's other control constructs (`if-then-else`, `do`, `while`, `break`, and `continue`) are also compiled in the obvious ways. The compilation of `switch` statements is handled in a separate section (§3.10), as are the compilation of exceptions (§3.12) and the compilation of `finally` clauses (§3.13).

As a further example, a `while` loop is compiled in an obvious way, although the specific control transfer instructions made available by the Java Virtual Machine vary by data type. As usual, there is more support for data of type `int`, for example:

```

void whileInt() {
    int i = 0;
    while (i < 100) {
        i++;
    }
}

```

is compiled to:

```

Method void whileInt()
0   iconst_0
1   istore_1
2   goto 8
5   iinc 1 1
8   iload_1
9   bipush 100
11  if_icmplt 5
14  return

```

Note that the test of the `while` statement (implemented using the `if_icmplt` instruction) is at the bottom of the Java Virtual Machine code for the loop. (This was also the case in the `spin` examples earlier.) The test being at the bottom of the loop forces the use of a `goto` instruction to get to the test prior to the first iteration of the loop. If that test fails, and the loop body is never entered, this extra instruction is wasted. However, `while` loops are typically used when their body is expected to be run, often for many iterations. For subsequent iterations, putting the test at

the bottom of the loop saves a Java Virtual Machine instruction each time around the loop: if the test were at the top of the loop, the loop body would need a trailing *goto* instruction to get back to the top.

Control constructs involving other data types are compiled in similar ways, but must use the instructions available for those data types. This leads to somewhat less efficient code because more Java Virtual Machine instructions are needed, for example:

```
void whileDouble() {
    double i = 0.0;
    while (i < 100.1) {
        i++;
    }
}
```

is compiled to:

```
Method void whileDouble()
0   dconst_0
1   dstore_1
2   goto 9
5   dload_1
6   dconst_1
7   dadd
8   dstore_1
9   dload_1
10  ldc2_w #4          // Push double constant 100.1
13  dcmpg           // To compare and branch we have to use...
14  iflt 5          // ...two instructions
17  return
```

Each floating-point type has two comparison instructions: *fcmpl* and *fcmpg* for type float, and *dcmpl* and *dcmpg* for type double. The variants differ only in their treatment of NaN. NaN is unordered (§2.3.2), so all floating-point comparisons fail if either of their operands is NaN. The compiler chooses the variant of the comparison instruction for the appropriate type that produces the same result whether the comparison fails on non-NaN values or encounters a NaN. For instance:

```
int lessThan100(double d) {
    if (d < 100.0) {
        return 1;
    } else {
        return -1;
    }
}
```

compiles to:

```

Method int lessThan100(double)
0   dload_1
1   ldc2_w #4          // Push double constant 100.0
4   dcmprg             // Push 1 if d is NaN or d > 100.0;
                        // push 0 if d == 100.0
5   ifge 10            // Branch on 0 or 1
8   iconst_1
9   ireturn
10  iconst_m1
11  ireturn

```

If *d* is not NaN and is less than 100.0, the *dcmprg* instruction pushes an int *-1* onto the operand stack, and the *ifge* instruction does not branch. Whether *d* is greater than 100.0 or is NaN, the *dcmprg* instruction pushes an int *1* onto the operand stack, and the *ifge* branches. If *d* is equal to 100.0, the *dcmprg* instruction pushes an int *0* onto the operand stack, and the *ifge* branches.

The *dcmpl* instruction achieves the same effect if the comparison is reversed:

```

int greaterThan100(double d) {
    if (d > 100.0) {
        return 1;
    } else {
        return -1;
    }
}

```

becomes:

```

Method int greaterThan100(double)
0   dload_1
1   ldc2_w #4          // Push double constant 100.0
4   dcmlpl             // Push -1 if d is NaN or d < 100.0;
                        // push 0 if d == 100.0
5   ifle 10            // Branch on 0 or -1
8   iconst_1
9   ireturn
10  iconst_m1
11  ireturn

```

Once again, whether the comparison fails on a non-NaN value or because it is passed a NaN, the *dcmlpl* instruction pushes an int value onto the operand stack that causes the *ifle* to branch. If both of the *dcmp* instructions did not exist, one of the example methods would have had to do more work to detect NaN.

3.6 Receiving Arguments

If n arguments are passed to an instance method, they are received, by convention, in the local variables numbered 1 through n of the frame created for the new method invocation. The arguments are received in the order they were passed. For example:

```
int addTwo(int i, int j) {  
    return i + j;  
}
```

compiles to:

```
Method int addTwo(int,int)  
0  iload_1          // Push value of local variable 1 (i)  
1  iload_2          // Push value of local variable 2 (j)  
2  iadd             // Add; leave int result on operand stack  
3  ireturn          // Return int result
```

By convention, an instance method is passed a reference to its instance in local variable 0 . In the Java programming language the instance is accessible via the `this` keyword.

Class (static) methods do not have an instance, so for them this use of local variable 0 is unnecessary. A class method starts using local variables at index 0 . If the `addTwo` method were a class method, its arguments would be passed in a similar way to the first version:

```
static int addTwoStatic(int i, int j) {  
    return i + j;  
}
```

compiles to:

```
Method int addTwoStatic(int,int)  
0  iload_0  
1  iload_1  
2  iadd  
3  ireturn
```

The only difference is that the method arguments appear starting in local variable 0 rather than 1 .

3.7 Invoking Methods

The normal method invocation for a instance method dispatches on the run-time type of the object. (They are virtual, in C++ terms.) Such an invocation is implemented using the *invokevirtual* instruction, which takes as its argument an index to a run-time constant pool entry giving the internal form of the binary name of the class type of the object, the name of the method to invoke, and that method's descriptor (§4.3.3). To invoke the *addTwo* method, defined earlier as an instance method, we might write:

```
int add12and13() {  
    return addTwo(12, 13);  
}
```

This compiles to:

```
Method int add12and13()  
0  aload_0          // Push local variable 0 (this)  
1  bipush 12         // Push int constant 12  
3  bipush 13         // Push int constant 13  
5  invokevirtual #4  // Method Example.addtwo(II)I  
8  ireturn           // Return int on top of operand stack;  
                        // it is the int result of addTwo()
```

The invocation is set up by first pushing a reference to the current instance, *this*, on to the operand stack. The method invocation's arguments, *int* values 12 and 13, are then pushed. When the frame for the *addTwo* method is created, the arguments passed to the method become the initial values of the new frame's local variables. That is, the reference for *this* and the two arguments, pushed onto the operand stack by the invoker, will become the initial values of local variables 0, 1, and 2 of the invoked method.

Finally, *addTwo* is invoked. When it returns, its *int* return value is pushed onto the operand stack of the frame of the invoker, the *add12and13* method. The return value is thus put in place to be immediately returned to the invoker of *add12and13*.

The return from *add12and13* is handled by the *ireturn* instruction of *add12and13*. The *ireturn* instruction takes the *int* value returned by *addTwo*, on the operand stack of the current frame, and pushes it onto the operand stack of the frame of the invoker. It then returns control to the invoker, making the invoker's frame current. The Java Virtual Machine provides distinct return instructions for many of its numeric and reference data types, as well as a *return* instruction for methods with no return value. The same set of return instructions is used for all varieties of method invocations.

The operand of the *invokevirtual* instruction (in the example, the run-time constant pool index #4) is not the offset of the method in the class instance. The compiler does not know the internal layout of a class instance. Instead, it generates symbolic references to the methods of an instance, which are stored in the run-time constant pool. Those run-time constant pool items are resolved at run-time to determine the actual method location. The same is true for all other Java Virtual Machine instructions that access class instances.

Invoking `addTwoStatic`, a class (static) variant of `addTwo`, is similar, as shown:

```
int add12and13() {
    return addTwoStatic(12, 13);
}
```

although a different Java Virtual Machine method invocation instruction is used:

```
Method int add12and13()
0  bipush 12
2  bipush 13
4  invokestatic #3      // Method Example.addTwoStatic(II)I
7  ireturn
```

Compiling an invocation of a class (static) method is very much like compiling an invocation of an instance method, except this is not passed by the invoker. The method arguments will thus be received beginning with local variable 0 (§3.6). The *invokestatic* instruction is always used to invoke class methods.

The *invokespecial* instruction must be used to invoke instance initialization methods (§3.8). It is also used when invoking methods in the superclass (super). For instance, given classes `Near` and `Far` declared as:

```
class Near {
    int it;
    int getItNear() {
        return it;
    }
}
class Far extends Near {
    int getItFar() {
        return super.getItNear();
    }
}
```

The method `Far.getItFar` (which invokes a superclass method) becomes:

```
Method int getItFar()
0  aload_0
1  invokespecial #4      // Method Near.getItNear()I
4  ireturn
```

Note that methods called using the *invokespecial* instruction always pass this to the invoked method as its first argument. As usual, it is received in local variable 0.

To invoke the target of a method handle, a compiler must form a method descriptor that records the actual argument and return types. A compiler may not perform method invocation conversions on the arguments; instead, it must push them on the stack according to their own unconverted types. The compiler arranges for a reference to the method handle object to be pushed on the stack before the arguments, as usual. The compiler emits an *invokevirtual* instruction that references a descriptor which describes the argument and return types. By special arrangement with method resolution (§5.4.3.3), an *invokevirtual* instruction which invokes the *invokeExact* or *invoke* methods of `java.lang.invoke.MethodHandle` will always link, provided the method descriptor is syntactically well-formed and the types named in the descriptor can be resolved.

3.8 Working with Class Instances

Java Virtual Machine class instances are created using the Java Virtual Machine's *new* instruction. Recall that at the level of the Java Virtual Machine, a constructor appears as a method with the compiler-supplied name `<init>`. This specially named method is known as the instance initialization method (§2.9). Multiple instance initialization methods, corresponding to multiple constructors, may exist for a given class. Once the class instance has been created and its instance variables, including those of the class and all of its superclasses, have been initialized to their default values, an instance initialization method of the new class instance is invoked. For example:

```
Object create() {
    return new Object();
}
```

compiles to:

```
Method java.lang.Object create()
0   new #1                // Class java.lang.Object
3   dup
4   invokespecial #4       // Method java.lang.Object.<init>()V
7   areturn
```

Class instances are passed and returned (as reference types) very much like numeric values, although type reference has its own complement of instructions, for example:

```

int i;                                // An instance variable
MyObj example() {
    MyObj o = new MyObj();
    return silly(o);
}
MyObj silly(MyObj o) {
    if (o != null) {
        return o;
    } else {
        return o;
    }
}

```

becomes:

```

Method MyObj example()
0   new #2                          // Class MyObj
3   dup
4   invokespecial #5               // Method MyObj.<init>()V
7   astore_1
8   aload_0
9   aload_1
10  invokevirtual #4               // Method Example.silly(LMyObj;)LMyObj;
13  return

Method MyObj silly(MyObj)
0   aload_1
1   ifnull 6
4   aload_1
5   areturn
6   aload_1
7   areturn

```

The fields of a class instance (instance variables) are accessed using the *getfield* and *putfield* instructions. If *i* is an instance variable of type *int*, the methods *setIt* and *getIt*, defined as:

```

void setIt(int value) {
    i = value;
}
int getIt() {
    return i;
}

```

become:

```

Method void setIt(int)
0   aload_0
1   iload_1
2   putfield #4                   // Field Example.i I
5   return

```

```

Method int getIt()
0  aload_0
1  getfield #4      // Field Example.i I
4  ireturn

```

As with the operands of method invocation instructions, the operands of the *putfield* and *getfield* instructions (the run-time constant pool index #4) are not the offsets of the fields in the class instance. The compiler generates symbolic references to the fields of an instance, which are stored in the run-time constant pool. Those run-time constant pool items are resolved at run-time to determine the location of the field within the referenced object.

3.9 Arrays

Java Virtual Machine arrays are also objects. Arrays are created and manipulated using a distinct set of instructions. The *newarray* instruction is used to create an array of a numeric type. The code:

```

void createBuffer() {
    int buffer[];
    int bufsz = 100;
    int value = 12;
    buffer = new int[bufsz];
    buffer[10] = value;
    value = buffer[11];
}

```

might be compiled to:

```

Method void createBuffer()
0  bipush 100      // Push int constant 100 (bufsz)
2  istore_2       // Store bufsz in local variable 2
3  bipush 12      // Push int constant 12 (value)
5  istore_3       // Store value in local variable 3
6  iload_2        // Push bufsz...
7  newarray int   // ...and create new int array of that length
9  astore_1       // Store new array in buffer
10 aload_1        // Push buffer
11 bipush 10      // Push int constant 10
13 iload_3        // Push value
14 iastore        // Store value at buffer[10]
15 aload_1        // Push buffer
16 bipush 11      // Push int constant 11
18 iaload         // Push value at buffer[11]...
19 istore_3       // ...and store it in value
20 return

```

The *anewarray* instruction is used to create a one-dimensional array of object references, for example:

```
void createThreadArray() {
    Thread threads[];
    int count = 10;
    threads = new Thread[count];
    threads[0] = new Thread();
}
```

becomes:

```
Method void createThreadArray()
0  bipush 10           // Push int constant 10
2  istore_2           // Initialize count to that
3  iload_2            // Push count, used by anewarray
4  anewarray class #1 // Create new array of class Thread
7  astore_1           // Store new array in threads
8  aload_1            // Push value of threads
9  iconst_0           // Push int constant 0
10 new #1             // Create instance of class Thread
13 dup               // Make duplicate reference...
14 invokespecial #5   // ...for Thread's constructor
                       // Method java.lang.Thread.<init>()V
17 astore            // Store new Thread in array at 0
18 return
```

The *anewarray* instruction can also be used to create the first dimension of a multidimensional array. Alternatively, the *multianewarray* instruction can be used to create several dimensions at once. For example, the three-dimensional array:

```
int[][][] create3DArray() {
    int grid[][][];
    grid = new int[10][5][];
    return grid;
}
```

is created by:

```
Method int create3DArray()[][][]
0  bipush 10           // Push int 10 (dimension one)
2  iconst_5           // Push int 5 (dimension two)
3  multianewarray #1 dim #2 // Class [[[I, a three-dimensional
                           // int array; only create the
                           // first two dimensions
7  astore_1           // Store new array...
8  aload_1            // ...then prepare to return it
9  areturn
```

The first operand of the *multianewarray* instruction is the run-time constant pool index to the array class type to be created. The second is the number of dimensions

of that array type to actually create. The *multianewarray* instruction can be used to create all the dimensions of the type, as the code for `create3DArray` shows. Note that the multidimensional array is just an object and so is loaded and returned by an *aload_1* and *areturn* instruction, respectively. For information about array class names, see §4.4.1.

All arrays have associated lengths, which are accessed via the *arraylength* instruction.

3.10 Compiling Switches

Compilation of switch statements uses the *tableswitch* and *lookupswitch* instructions. The *tableswitch* instruction is used when the cases of the switch can be efficiently represented as indices into a table of target offsets. The default target of the switch is used if the value of the expression of the switch falls outside the range of valid indices. For instance:

```
int chooseNear(int i) {
    switch (i) {
        case 0: return 0;
        case 1: return 1;
        case 2: return 2;
        default: return -1;
    }
}
```

compiles to:

```
Method int chooseNear(int)
0   iload_1           // Push local variable 1 (argument i)
1   tableswitch 0 to 2: // Valid indices are 0 through 2
    0: 28             // If i is 0, continue at 28
    1: 30             // If i is 1, continue at 30
    2: 32             // If i is 2, continue at 32
    default:34        // Otherwise, continue at 34
28  iconst_0          // i was 0; push int constant 0...
29  ireturn           // ...and return it
30  iconst_1          // i was 1; push int constant 1...
31  ireturn           // ...and return it
32  iconst_2          // i was 2; push int constant 2...
33  ireturn           // ...and return it
34  iconst_m1         // otherwise push int constant -1...
35  ireturn           // ...and return it
```

The Java Virtual Machine's *tableswitch* and *lookupswitch* instructions operate only on int data. Because operations on byte, char, or short values are internally

promoted to `int`, a switch whose expression evaluates to one of those types is compiled as though it evaluated to type `int`. If the `chooseNear` method had been written using type `short`, the same Java Virtual Machine instructions would have been generated as when using type `int`. Other numeric types must be narrowed to type `int` for use in a switch.

Where the cases of the switch are sparse, the table representation of the *tableswitch* instruction becomes inefficient in terms of space. The *lookupswitch* instruction may be used instead. The *lookupswitch* instruction pairs `int` keys (the values of the case labels) with target offsets in a table. When a *lookupswitch* instruction is executed, the value of the expression of the switch is compared against the keys in the table. If one of the keys matches the value of the expression, execution continues at the associated target offset. If no key matches, execution continues at the default target. For instance, the compiled code for:

```
int chooseFar(int i) {
    switch (i) {
        case -100: return -1;
        case 0:   return 0;
        case 100: return 1;
        default:  return -1;
    }
}
```

looks just like the code for `chooseNear`, except for the *lookupswitch* instruction:

```
Method int chooseFar(int)
0   iload_1
1   lookupswitch 3:
      -100: 36
       0: 38
      100: 40
      default: 42
36  iconst_m1
37  ireturn
38  iconst_0
39  ireturn
40  iconst_1
41  ireturn
42  iconst_m1
43  ireturn
```

The Java Virtual Machine specifies that the table of the *lookupswitch* instruction must be sorted by key so that implementations may use searches more efficient than a linear scan. Even so, the *lookupswitch* instruction must search its keys for a match rather than simply perform a bounds check and index into a table like *tableswitch*. Thus, a *tableswitch* instruction is probably more efficient than a *lookupswitch* where space considerations permit a choice.

3.11 Operations on the Operand Stack

The Java Virtual Machine has a large complement of instructions that manipulate the contents of the operand stack as untyped values. These are useful because of the Java Virtual Machine's reliance on deft manipulation of its operand stack. For instance:

```
public long nextIndex() {
    return index++;
}

private long index = 0;
```

is compiled to:

```
Method long nextIndex()
0  aload_0          // Push this
1  dup             // Make a copy of it
2  getfield #4      // One of the copies of this is consumed
                       // pushing long field index,
                       // above the original this
5  dup2_x1         // The long on top of the operand stack is
                       // inserted into the operand stack below the
                       // original this
6  lconst_1        // Push long constant 1
7  ladd            // The index value is incremented...
8  putfield #4     // ...and the result stored in the field
11 lreturn         // The original value of index is on top of
                       // the operand stack, ready to be returned
```

Note that the Java Virtual Machine never allows its operand stack manipulation instructions to modify or break up individual values on the operand stack.

3.12 Throwing and Handling Exceptions

Exceptions are thrown from programs using the `throw` keyword. Its compilation is simple:

```
void cantBeZero(int i) throws TestExc {
    if (i == 0) {
        throw new TestExc();
    }
}
```

becomes:

```

Method void cantBeZero(int)
0  iload_1          // Push argument 1 (i)
1  ifne 12          // If i==0, allocate instance and throw
4  new #1           // Create instance of TestExc
7  dup             // One reference goes to its constructor
8  invokespecial #7 // Method TestExc.<init>()V
11 athrow          // Second reference is thrown
12 return          // Never get here if we threw TestExc

```

Compilation of try-catch constructs is straightforward. For example:

```

void catchOne() {
    try {
        tryItOut();
    } catch (TestExc e) {
        handleExc(e);
    }
}

```

is compiled as:

```

Method void catchOne()
0  aload_0          // Beginning of try block
1  invokevirtual #6  // Method Example.tryItOut()V
4  return           // End of try block; normal return
5  astore_1         // Store thrown value in local var 1
6  aload_0          // Push this
7  aload_1          // Push thrown value
8  invokevirtual #5  // Invoke handler method:
                      // Example.handleExc(LTestExc;)V
11 return           // Return after handling TestExc

```

Exception table:

From	To	Target	Type
0	4	5	Class TestExc

Looking more closely, the try block is compiled just as it would be if the try were not present:

```

Method void catchOne()
0  aload_0          // Beginning of try block
1  invokevirtual #6  // Method Example.tryItOut()V
4  return           // End of try block; normal return

```

If no exception is thrown during the execution of the try block, it behaves as though the try were not there: tryItOut is invoked and catchOne returns.

Following the try block is the Java Virtual Machine code that implements the single catch clause:

```

5  astore_1         // Store thrown value in local var 1
6  aload_0          // Push this

```

```

7   aload_1           // Push thrown value
8   invokevirtual #5   // Invoke handler method:
                        // Example.handleExc(LTestExc;)V
11  return             // Return after handling TestExc
Exception table:
From    To    Target    Type
0       4      5          Class TestExc

```

The invocation of `handleExc`, the contents of the catch clause, is also compiled like a normal method invocation. However, the presence of a catch clause causes the compiler to generate an exception table entry (§2.10, §4.7.3). The exception table for the `catchOne` method has one entry corresponding to the one argument (an instance of class `TestExc`) that the catch clause of `catchOne` can handle. If some value that is an instance of `TestExc` is thrown during execution of the instructions between indices 0 and 4 in `catchOne`, control is transferred to the Java Virtual Machine code at index 5, which implements the block of the catch clause. If the value that is thrown is not an instance of `TestExc`, the catch clause of `catchOne` cannot handle it. Instead, the value is rethrown to the invoker of `catchOne`.

A try may have multiple catch clauses:

```

void catchTwo() {
    try {
        tryItOut();
    } catch (TestExc1 e) {
        handleExc(e);
    } catch (TestExc2 e) {
        handleExc(e);
    }
}

```

Multiple catch clauses of a given try statement are compiled by simply appending the Java Virtual Machine code for each catch clause one after the other and adding entries to the exception table, as shown:

```

Method void catchTwo()
0   aload_0           // Begin try block
1   invokevirtual #5   // Method Example.tryItOut()V
4   return             // End of try block; normal return
5   astore_1           // Beginning of handler for TestExc1;
                        // Store thrown value in local var 1
6   aload_0           // Push this
7   aload_1           // Push thrown value
8   invokevirtual #7   // Invoke handler method:
                        // Example.handleExc(LTestExc1;)V
11  return            // Return after handling TestExc1
12  astore_1           // Beginning of handler for TestExc2;
                        // Store thrown value in local var 1
13  aload_0           // Push this

```

```

14  aload_1          // Push thrown value
15  invokevirtual #7  // Invoke handler method:
                        // Example.handleExc(LTestExc2;)V
18  return           // Return after handling TestExc2

```

Exception table:

From	To	Target	Type
0	4	5	Class TestExc1
0	4	12	Class TestExc2

If during the execution of the try clause (between indices 0 and 4) a value is thrown that matches the parameter of one or more of the catch clauses (the value is an instance of one or more of the parameters), the first (innermost) such catch clause is selected. Control is transferred to the Java Virtual Machine code for the block of that catch clause. If the value thrown does not match the parameter of any of the catch clauses of catchTwo, the Java Virtual Machine rethrows the value without invoking code in any catch clause of catchTwo.

Nested try-catch statements are compiled very much like a try statement with multiple catch clauses:

```

void nestedCatch() {
    try {
        try {
            tryItOut();
        } catch (TestExc1 e) {
            handleExc1(e);
        }
    } catch (TestExc2 e) {
        handleExc2(e);
    }
}

```

becomes:

```

Method void nestedCatch()
0   aload_0          // Begin try block
1   invokevirtual #8  // Method Example.tryItOut()V
4   return           // End of try block; normal return
5   astore_1         // Beginning of handler for TestExc1;
                        // Store thrown value in local var 1
6   aload_0          // Push this
7   aload_1          // Push thrown value
8   invokevirtual #7  // Invoke handler method:
                        // Example.handleExc1(LTestExc1;)V
11  return           // Return after handling TestExc1
12  astore_1         // Beginning of handler for TestExc2;
                        // Store thrown value in local var 1
13  aload_0          // Push this
14  aload_1          // Push thrown value
15  invokevirtual #6  // Invoke handler method:
                        // Example.handleExc2(LTestExc2;)V

```

```

18  return                                // Return after handling TestExc2
Exception table:
From    To    Target    Type
0       4     5         Class TestExc1
0       12    12        Class TestExc2

```

The nesting of catch clauses is represented only in the exception table. The Java Virtual Machine does not enforce nesting of or any ordering of the exception table entries (§2.10). However, because try-catch constructs are structured, a compiler can always order the entries of the exception handler table such that, for any thrown exception and any program counter value in that method, the first exception handler that matches the thrown exception corresponds to the innermost matching catch clause.

For instance, if the invocation of `tryItOut` (at index 1) threw an instance of `TestExc1`, it would be handled by the catch clause that invokes `handleExc1`. This is so even though the exception occurs within the bounds of the outer catch clause (catching `TestExc2`) and even though that outer catch clause might otherwise have been able to handle the thrown value.

As a subtle point, note that the range of a catch clause is inclusive on the "from" end and exclusive on the "to" end (§4.7.3). Thus, the exception table entry for the catch clause catching `TestExc1` does not cover the *return* instruction at offset 4. However, the exception table entry for the catch clause catching `TestExc2` does cover the *return* instruction at offset 11. Return instructions within nested catch clauses are included in the range of instructions covered by nesting catch clauses.

3.13 Compiling finally

(This section assumes a compiler generates class files with version number 50.0 or below, so that the *jsr* instruction may be used. See also §4.10.2.5.)

Compilation of a try-finally statement is similar to that of try-catch. Prior to transferring control outside the try statement, whether that transfer is normal or abrupt, because an exception has been thrown, the *finally* clause must first be executed. For this simple example:

```

void tryFinally() {
    try {
        tryItOut();
    } finally {
        wrapItUp();
    }
}

```

the compiled code is:

```

Method void tryFinally()
0  aload_0          // Beginning of try block
1  invokevirtual #6  // Method Example.tryItOut()V
4  jsr 14           // Call finally block
7  return          // End of try block
8  astore_1        // Beginning of handler for any throw
9  jsr 14           // Call finally block
12 aload_1         // Push thrown value
13 athrow          // ...and rethrow value to the invoker
14 astore_2        // Beginning of finally block
15 aload_0         // Push this
16 invokevirtual #5  // Method Example.wrapItUp()V
19 ret 2           // Return from finally block
Exception table:
From    To    Target    Type
0       4       8         any

```

There are four ways for control to pass outside of the try statement: by falling through the bottom of that block, by returning, by executing a break or continue statement, or by raising an exception. If `tryItOut` returns without raising an exception, control is transferred to the finally block using a `jsr` instruction. The `jsr 14` instruction at index 4 makes a "subroutine call" to the code for the finally block at index 14 (the finally block is compiled as an embedded subroutine). When the finally block completes, the `ret 2` instruction returns control to the instruction following the `jsr` instruction at index 4.

In more detail, the subroutine call works as follows: The `jsr` instruction pushes the address of the following instruction (`return` at index 7) onto the operand stack before jumping. The `astore_2` instruction that is the jump target stores the address on the operand stack into local variable 2. The code for the finally block (in this case the `aload_0` and `invokevirtual` instructions) is run. Assuming execution of that code completes normally, the `ret` instruction retrieves the address from local variable 2 and resumes execution at that address. The `return` instruction is executed, and `tryFinally` returns normally.

A try statement with a finally clause is compiled to have a special exception handler, one that can handle any exception thrown within the try statement. If `tryItOut` throws an exception, the exception table for `tryFinally` is searched for an appropriate exception handler. The special handler is found, causing execution to continue at index 8. The `astore_1` instruction at index 8 stores the thrown value into local variable 1. The following `jsr` instruction does a subroutine call to the code for the finally block. Assuming that code returns normally, the `aload_1` instruction at index 12 pushes the thrown value back onto the operand stack, and the following `athrow` instruction rethrows the value.

Compiling a try statement with both a catch clause and a finally clause is more complex:

```
void tryCatchFinally() {
    try {
        tryItOut();
    } catch (TestExc e) {
        handleExc(e);
    } finally {
        wrapItUp();
    }
}
```

becomes:

```
Method void tryCatchFinally()
0  aload_0                // Beginning of try block
1  invokevirtual #4        // Method Example.tryItOut()V
4  goto 16                // Jump to finally block
7  astore_3               // Beginning of handler for TestExc;
                        // Store thrown value in local var 3
8  aload_0                // Push this
9  aload_3                // Push thrown value
10 invokevirtual #6        // Invoke handler method:
                        // Example.handleExc(LTestExc;)V
13 goto 16                // This goto is unnecessary, but was
                        // generated by javac in JDK 1.0.2
16 jsr 26                 // Call finally block
19 return                 // Return after handling TestExc
20 astore_1               // Beginning of handler for exceptions
                        // other than TestExc, or exceptions
                        // thrown while handling TestExc
21 jsr 26                 // Call finally block
24 aload_1                // Push thrown value...
25 athrow                 // ...and rethrow value to the invoker
26 astore_2               // Beginning of finally block
27 aload_0                // Push this
28 invokevirtual #5        // Method Example.wrapItUp()V
31 ret 2                  // Return from finally block

Exception table:
From    To    Target    Type
0        4        7        Class TestExc
0       16       20       any
```

If the try statement completes normally, the *goto* instruction at index 4 jumps to the subroutine call for the finally block at index 16. The finally block at index 26 is executed, control returns to the *return* instruction at index 19, and *tryCatchFinally* returns normally.

If *tryItOut* throws an instance of *TestExc*, the first (innermost) applicable exception handler in the exception table is chosen to handle the exception. The

code for that exception handler, beginning at index 7, passes the thrown value to `handleExc` and on its return makes the same subroutine call to the `finally` block at index 26 as in the normal case. If an exception is not thrown by `handleExc`, `tryCatchFinally` returns normally.

If `tryItOut` throws a value that is not an instance of `TestExc` or if `handleExc` itself throws an exception, the condition is handled by the second entry in the exception table, which handles any value thrown between indices 0 and 16. That exception handler transfers control to index 20, where the thrown value is first stored in local variable 1. The code for the `finally` block at index 26 is called as a subroutine. If it returns, the thrown value is retrieved from local variable 1 and rethrown using the `athrow` instruction. If a new value is thrown during execution of the `finally` clause, the `finally` clause aborts, and `tryCatchFinally` returns abruptly, throwing the new value to its invoker.

3.14 Synchronization

Synchronization in the Java Virtual Machine is implemented by monitor entry and exit, either explicitly (by use of the *monitorenter* and *monitorexit* instructions) or implicitly (by the method invocation and return instructions).

For code written in the Java programming language, perhaps the most common form of synchronization is the `synchronized` method. A `synchronized` method is not normally implemented using *monitorenter* and *monitorexit*. Rather, it is simply distinguished in the run-time constant pool by the `ACC_SYNCHRONIZED` flag, which is checked by the method invocation instructions (§2.11.10).

The *monitorenter* and *monitorexit* instructions enable the compilation of `synchronized` statements. For example:

```
void onlyMe(Foo f) {
    synchronized(f) {
        doSomething();
    }
}
```

is compiled to:

```
Method void onlyMe(Foo)
0  aload_1          // Push f
1  dup              // Duplicate it on the stack
2  astore_2         // Store duplicate in local variable 2
3  monitorenter     // Enter the monitor associated with f
4  aload_0          // Holding the monitor, pass this and...
```



```

5  invokevirtual #5      // ...call Example.doSomething()V
8  aload_2              // Push local variable 2 (f)
9  monitorexit          // Exit the monitor associated with f
10 goto 18              // Complete the method normally
13 astore_3             // In case of any throw, end up here
14 aload_2              // Push local variable 2 (f)
15 monitorexit          // Be sure to exit the monitor!
16 aload_3              // Push thrown value...
17 athrow               // ...and rethrow value to the invoker
18 return               // Return in the normal case

```

Exception table:

From	To	Target	Type
4	10	13	any
13	16	13	any

The compiler ensures that at any method invocation completion, a *monitorexit* instruction will have been executed for each *monitorenter* instruction executed since the method invocation. This is the case whether the method invocation completes normally (§2.6.4) or abruptly (§2.6.5). To enforce proper pairing of *monitorenter* and *monitorexit* instructions on abrupt method invocation completion, the compiler generates exception handlers (§2.10) that will match any exception and whose associated code executes the necessary *monitorexit* instructions.

3.15 Annotations

The representation of annotations in class files is described in §4.7.16-§4.7.22. These sections make it clear how to represent annotations on declarations of classes, interfaces, fields, methods, method parameters, and type parameters, as well as annotations on types used in those declarations. Annotations on package declarations require additional rules, given here.

When the compiler encounters an annotated package declaration that must be made available at run time, it emits a class file with the following properties:

- The class file represents an interface, that is, the `ACC_INTERFACE` and `ACC_ABSTRACT` flags of the `ClassFile` structure are set (§4.1).
- If the class file version number is less than 50.0, then the `ACC_SYNTHETIC` flag is unset; if the class file version number is 50.0 or above, then the `ACC_SYNTHETIC` flag is set.
- The interface has package access (JLS §6.6.1).

- The interface's name is the internal form (§4.2.1) of *package-name.package-info*.
- The interface has no superinterfaces.
- The interface's only members are those implied by *The Java Language Specification, Java SE 26 Edition* (JLS §9.2).
- The annotations on the package declaration are stored as `RuntimeVisibleAnnotations` and `RuntimeInvisibleAnnotations` attributes in the attributes table of the `ClassFile` structure.

3.16 Modules

A compilation unit that contains a module declaration (JLS §7.7) is compiled to a class file that contains a `Module` attribute.

By convention, the name of a compilation unit that contains a module declaration is `module-info.java`, echoing the `package-info.java` convention for a compilation unit that contains solely a package declaration. Consequently, by convention, the name for the compiled form of a module declaration is `module-info.class`.

A flag in the `access_flags` item of the `ClassFile` structure, `ACC_MODULE` (0x8000), indicates that this class file declares a module. `ACC_MODULE` plays a similar role to `ACC_ANNOTATION` (0x2000) and `ACC_ENUM` (0x4000) in flagging this class file as "not an ordinary class". `ACC_MODULE` does *not* describe accessibility of a class or interface.

The `Module` attribute is explicit about the module's dependences; there are no implicit requires directives at the `ClassFile` level. If the `requires_count` item is zero, then the Java SE Platform does *not* infer the existence of a `requires` table nor any particular entry therein. `java.base` is the only module in which a zero `requires_count` is legal, because it is the primordial module. For every other module, the `Module` attribute must have a `requires` table of at least length one, because every other module depends on `java.base`. If a compilation unit contains a module declaration (except `java.base`) that does not state its dependence on `java.base` explicitly, then a compiler must emit an entry for `java.base` in the `requires` table and flag it as `ACC_MANDATED` to denote that it was implicitly declared.

For encapsulation, the `Module` attribute is explicit about the packages exported and opened by a normal module; there are no implicit `exports` or `opens` directives at the

`ClassFile` level for a normal module. If the `exports_count` item or `opens_count` item is zero, then the Java SE Platform does *not* infer the existence of an `exports` table or `opens` table, nor any particular entry therein. On the other hand, for an open module, the `Module` attribute is implicit about the packages opened by the module. All packages of an open module are opened to all other modules, even though the `opens_count` item is zero.

The `Module` attribute is explicit about the module's consumption and provision of services; there are no implicit uses or provides directives at the `ClassFile` level.

The class File Format

THIS chapter describes the class file format of the Java Virtual Machine. Each class file contains the definition of a single class, interface, or module. Although a class, interface, or module need not have an external representation literally contained in a file (for instance, because the class is generated by a class loader), we will colloquially refer to any valid representation of a class, interface, or module as being in the class file format.

A class file consists of a stream of 8-bit bytes. 16-bit and 32-bit quantities are constructed by reading in two and four consecutive 8-bit bytes, respectively. Multibyte data items are always stored in big-endian order, where the high bytes come first. This chapter defines the data types u1, u2, and u4 to represent an unsigned one-, two-, or four-byte quantity, respectively.

In the Java SE Platform API, the class file format is supported by interfaces `java.io.DataInput` and `java.io.DataOutput` and classes such as `java.io.DataInputStream` and `java.io.DataOutputStream`. For example, values of the types u1, u2, and u4 may be read by methods such as `readUnsignedByte`, `readUnsignedShort`, and `readInt` of the interface `java.io.DataInput`.

This chapter presents the class file format using pseudostructures written in a C-like structure notation. To avoid confusion with the fields of classes and class instances, etc., the contents of the structures describing the class file format are referred to as *items*. Successive items are stored in the class file sequentially, without padding or alignment.

Tables, consisting of zero or more variable-sized items, are used in several class file structures. Although we use C-like array syntax to refer to table items, the fact that tables are streams of varying-sized structures means that it is not possible to translate a table index directly to a byte offset into the table.

Where we refer to a data structure as an **array**, it consists of zero or more contiguous fixed-sized items and can be indexed like an array.

Reference to an ASCII character in this chapter should be interpreted to mean the Unicode code point corresponding to the ASCII character.

4.1 The ClassFile Structure

A class file consists of a single ClassFile structure:

```
ClassFile {
    u4          magic;
    u2          minor_version;
    u2          major_version;
    u2          constant_pool_count;
    cp_info     constant_pool[constant_pool_count-1];
    u2          access_flags;
    u2          this_class;
    u2          super_class;
    u2          interfaces_count;
    u2          interfaces[interfaces_count];
    u2          fields_count;
    field_info  fields[fields_count];
    u2          methods_count;
    method_info methods[methods_count];
    u2          attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items in the ClassFile structure are as follows:

magic

The magic item supplies the magic number identifying the class file format; it has the value 0xCAFEBAFE.

minor_version, major_version

The values of the minor_version and major_version items are the minor and major version numbers of this class file. Together, a major and a minor version number determine the version of the class file format. If a class file has major version number M and minor version number m , we denote the version of its class file format as $M.m$.

The major_version item must be a value in the range 45 through 70, inclusive. A Java Virtual Machine implementation which conforms to Java SE 26 supports precisely these major version numbers.

For a class file whose major_version is 56 or above, the minor_version must be 0 or 65535.

For a class file whose `major_version` is between 45 and 55 inclusive, the `minor_version` may be any value.

A class file with version number 70.65535 depends on the preview features of Java SE 26 (§1.5.1). Such a class file may be loaded only when the user has indicated via the host system that preview features are enabled.

A class file with version number $M.65535$, where $56 \leq M < 70$, depends on the preview features of an older release of the Java SE Platform. Such a class file may not be loaded, regardless of whether preview features are enabled. Instead, the class file may be loaded only by a Java Virtual Machine implementation that conforms to the older release.

A class file with any other version number does not depend on preview features, and may be loaded regardless of whether preview features are enabled.

A historical perspective is warranted on JDK use of the class file `minor_version`. JDK 1.0.2 supported versions 45.0 through 45.3 inclusive. JDK 1.1 supported versions 45.0 through 45.65535 inclusive. When JDK 1.2 introduced support for major version 46, the only minor version supported under that major version was 0. Later JDKs continued the practice of introducing support for a new major version (47, 48, etc.) but supporting only a minor version of 0 under the new major version. Finally, the introduction of preview features (§1.5) in Java SE 12 motivated a standard role for the minor version of the class file format, so JDK 12 supported minor versions of 0 and 65535 under major version 56. Subsequent JDKs introduce support for $N.0$ and $N.65535$ where N is the corresponding major version of the implemented Java SE Platform. For example, JDK 13 supports 57.0 and 57.65535.

`constant_pool_count`

The value of the `constant_pool_count` item is equal to the number of entries in the `constant_pool` table plus one. A `constant_pool` index is considered valid if it is greater than zero and less than `constant_pool_count`, with the exception for constants of type `long` and `double` noted in §4.4.5.

`constant_pool[]`

The `constant_pool` is a table of structures (§4.4) representing various string constants, class and interface names, field names, and other constants that are referred to within the `ClassFile` structure and its substructures. The format of each `constant_pool` table entry is indicated by its first "tag" byte.

The `constant_pool` table is indexed from 1 to `constant_pool_count - 1`.

access_flags

The value of the `access_flags` item is a mask of flags used to denote access permissions to and properties of this class or interface. The interpretation of each flag, when set, is specified in Table 4.1-B.

Table 4.1-B. Class access and property modifiers

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared <code>public</code> ; may be accessed from outside its package.
ACC_FINAL	0x0010	Declared <code>final</code> ; no subclasses allowed.
ACC_SUPER	0x0020	Treat superclass methods specially when invoked by the <i>invokespecial</i> instruction.
ACC_INTERFACE	0x0200	Is an interface, not a class.
ACC_ABSTRACT	0x0400	Declared <code>abstract</code> ; must not be instantiated.
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.
ACC_ANNOTATION	0x2000	Declared as an annotation interface.
ACC_ENUM	0x4000	Declared as an <code>enum</code> class.
ACC_MODULE	0x8000	Is a module, not a class or interface.

The `ACC_MODULE` flag indicates that this class file defines a module, not a class or interface. If the `ACC_MODULE` flag is set, then special rules apply to the class file which are given at the end of this section. If the `ACC_MODULE` flag is not set, then the rules immediately below the current paragraph apply to the class file.

An interface is distinguished by the `ACC_INTERFACE` flag being set. If the `ACC_INTERFACE` flag is not set, this class file defines a class, not an interface or module.

If the `ACC_INTERFACE` flag is set, the `ACC_ABSTRACT` flag must also be set, and the `ACC_FINAL`, `ACC_SUPER`, `ACC_ENUM`, and `ACC_MODULE` flags set must not be set.

If the `ACC_INTERFACE` flag is not set, any of the other flags in Table 4.1-B may be set except `ACC_ANNOTATION` and `ACC_MODULE`. However, such a class file must not have both its `ACC_FINAL` and `ACC_ABSTRACT` flags set (JLS §8.1.1.2).

The `ACC_SUPER` flag indicates which of two alternative semantics is to be expressed by the *invokespecial* instruction (*\$invokespecial*) if it appears in this class or interface. Compilers to the instruction set of the Java Virtual Machine should set the `ACC_SUPER` flag. In Java SE 8 and above, the Java

Virtual Machine considers the `ACC_SUPER` flag to be set in every class file, regardless of the actual value of the flag in the class file and the version of the class file.

The `ACC_SUPER` flag exists for backward compatibility with code compiled by older compilers for the Java programming language. Prior to JDK 1.0.2, the compiler generated `access_flags` in which the flag now representing `ACC_SUPER` had no assigned meaning, and Oracle's Java Virtual Machine implementation ignored the flag if it was set.

The `ACC_SYNTHETIC` flag indicates that this class or interface was generated by a compiler and does not appear in source code.

An annotation interface (JLS §9.6) must have its `ACC_ANNOTATION` flag set. If the `ACC_ANNOTATION` flag is set, the `ACC_INTERFACE` flag must also be set.

The `ACC_ENUM` flag indicates that this class or its superclass is declared as an enum class (JLS §8.9).

All bits of the `access_flags` item not assigned in Table 4.1-B are reserved for future use. They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

`this_class`

The value of the `this_class` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing the class or interface defined by this class file.

`super_class`

For a class, the value of the `super_class` item either must be zero or must be a valid index into the `constant_pool` table. If the value of the `super_class` item is nonzero, the `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure representing the direct superclass of the class defined by this class file. Neither the direct superclass nor any of its superclasses may have the `ACC_FINAL` flag set in the `access_flags` item of its `ClassFile` structure.

If the value of the `super_class` item is zero, then this class file must represent the class object, the only class or interface without a direct superclass.

For an interface, the value of the `super_class` item must always be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure representing the class `Object`.

interfaces_count

The value of the `interfaces_count` item gives the number of direct superinterfaces of this class or interface type.

interfaces[]

Each value in the `interfaces` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at each value of `interfaces[i]`, where $0 \leq i < \text{interfaces_count}$, must be a `CONSTANT_Class_info` structure representing an interface that is a direct superinterface of this class or interface type, in the left-to-right order given in the source for the type.

fields_count

The value of the `fields_count` item gives the number of `field_info` structures in the `fields` table. The `field_info` structures represent all fields, both class variables and instance variables, declared by this class or interface type.

fields[]

Each value in the `fields` table must be a `field_info` structure (§4.5) giving a complete description of a field in this class or interface. The `fields` table includes only those fields that are declared by this class or interface. It does not include items representing fields that are inherited from superclasses or superinterfaces.

methods_count

The value of the `methods_count` item gives the number of `method_info` structures in the `methods` table.

methods[]

Each value in the `methods` table must be a `method_info` structure (§4.6) giving a complete description of a method in this class or interface. If neither of the `ACC_NATIVE` and `ACC_ABSTRACT` flags are set in the `access_flags` item of a `method_info` structure, the Java Virtual Machine instructions implementing the method are also supplied.

The `method_info` structures represent all methods declared by this class or interface type, including instance methods, class methods, instance initialization methods (§2.9.1), and any class or interface initialization method (§2.9.2). The `methods` table does not include items representing methods that are inherited from superclasses or superinterfaces.

attributes_count

The value of the `attributes_count` item gives the number of attributes in the attributes table of this class.

attributes[]

Each value of the attributes table must be an `attribute_info` structure (§4.7).

The attributes defined by this specification as appearing in the attributes table of a `ClassFile` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the attributes table of a `ClassFile` structure are given in §4.7.

The rules concerning non-predefined attributes in the attributes table of a `ClassFile` structure are given in §4.7.1.

If the `ACC_MODULE` flag is set in the `access_flags` item, then no other flag in the `access_flags` item may be set, and the following rules apply to the rest of the `ClassFile` structure:

- `major_version, minor_version`: ≥ 53.0 (i.e., Java SE 9 and above)
- `this_class`: `module-info`
- `super_class, interfaces_count, fields_count, methods_count`: zero
- `attributes`: One Module attribute must be present. Except for `Module`, `ModulePackages`, `ModuleMainClass`, `InnerClasses`, `SourceFile`, `SourceDebugExtension`, `RuntimeVisibleAnnotations`, and `RuntimeInvisibleAnnotations`, none of the pre-defined attributes (§4.7) may appear.

4.2 Names

4.2.1 Binary Class and Interface Names

Class and interface names that appear in class file structures are always represented in a fully qualified form known as *binary names* (JLS §13.1). Such names are always represented as `CONSTANT_Utf8_info` structures (§4.4.7) and thus may be drawn, where not further constrained, from the entire Unicode codespace. Class and interface names are referenced from those `CONSTANT_NameAndType_info` structures (§4.4.6) which have such names as part of their descriptor (§4.3), and from all `CONSTANT_Class_info` structures (§4.4.1).

For historical reasons, the syntax of binary names that appear in class file structures differs from the syntax of binary names documented in JLS §13.1. In this internal form, the ASCII periods (.) that normally separate the identifiers which make up the binary name are replaced by ASCII forward slashes (/). The identifiers themselves must be unqualified names (§4.2.2).

For example, the normal binary name of class `Thread` is `java.lang.Thread`. In the internal form used in descriptors in the class file format, a reference to the name of class `Thread` is implemented using a `CONSTANT_Utf8_info` structure representing the string `java/lang/Thread`.

4.2.2 Unqualified Names

Names of methods, fields, local variables, and formal parameters are stored as *unqualified names*. An unqualified name must contain at least one Unicode code point and must not contain any of the ASCII characters `.` `;` `[` `/` (that is, period or semicolon or left square bracket or forward slash).

Method names are further constrained so that, with the exception of the special method names `<init>` and `<clinit>` (§2.9), they must not contain the ASCII characters `<` or `>` (that is, left angle bracket or right angle bracket).

Note that no method invocation instruction may reference `<clinit>`, and only the *invokespecial* instruction (*\$invokespecial*) may reference `<init>`.

4.2.3 Module and Package Names

Module names referenced from the `Module` attribute are stored in `CONSTANT_Module_info` structures in the constant pool (§4.4.11). A `CONSTANT_Module_info` structure wraps a `CONSTANT_Utf8_info` structure that denotes the module name. Module names are not encoded in "internal form" like class and interface names, that is, the ASCII periods (.) that separate the identifiers in a module name are not replaced by ASCII forward slashes (/).

Module names may be drawn from the entire Unicode codespace, subject to the following constraints:

- A module name must not contain any code point in the range `'\u0000'` to `'\u001F'` inclusive.
- The ASCII backslash (`\`) is reserved for use as an escape character in module names. It must not appear in a module name unless it is followed by an ASCII backslash, an ASCII colon (`:`), or an ASCII at-sign (`@`). The ASCII character sequence `\\` may be used to encode a backslash in a module name.

- The ASCII colon (:) and at-sign (@) are reserved for future use in module names. They must not appear in module names unless they are escaped. The ASCII character sequences \: and \@ may be used to encode a colon and an at-sign in a module name.

Package names referenced from the `Module` attribute are stored in `CONSTANT_Package_info` structures in the constant pool (§4.4.12). A `CONSTANT_Package_info` structure wraps a `CONSTANT_Utf8_info` structure that represents a package name encoded in internal form.

4.3 Descriptors

A *descriptor* is a string representing the type of a field or method. Descriptors are represented in the class file format using modified UTF-8 strings (§4.4.7) and thus may be drawn, where not further constrained, from the entire Unicode codespace.

4.3.1 Grammar Notation

Descriptors are specified using a grammar. The grammar is a set of productions that describe how sequences of characters can form syntactically correct descriptors of various kinds. Terminal symbols of the grammar are shown in fixed width font, and should be interpreted as ASCII characters. Nonterminal symbols are shown in *italic* type. The definition of a nonterminal is introduced by the name of the nonterminal being defined, followed by a colon. One or more alternative definitions for the nonterminal then follow on succeeding lines.

The syntax `{x}` on the right-hand side of a production denotes zero or more occurrences of `x`.

The phrase (*one of*) on the right-hand side of a production signifies that each of the terminal symbols on the following line or lines is an alternative definition.

4.3.2 Field Descriptors

A *field descriptor* represents the type of a field, parameter, local variable, or value.

FieldDescriptor:
FieldType

FieldType:

BaseType

ClassType

ArrayType

BaseType:

(one of)

B C D F I J S Z

ClassType:

L *ClassName* ;

ArrayType:

[*ComponentType*

ComponentType:

FieldType

ClassName represents a binary class or interface name encoded in internal form (§4.2.1).

A field descriptor *mentions* a class or interface name if the name appears as a *ClassName* in the descriptor. This includes a *ClassName* nested in the *ComponentType* of an *ArrayType*.

The interpretation of field descriptors as types is shown in Table 4.3-A. See §2.2, §2.3, and §2.4 for the meaning of these types.

A field descriptor representing an array type is valid only if it represents a type with 255 or fewer dimensions.

Table 4.3-A. Interpretation of field descriptors

<i>FieldType</i> term	Type
B	byte
C	char
D	double
F	float
I	int
J	long
L <i>ClassName</i> ;	Named class or interface type
S	short
Z	boolean
[<i>ComponentType</i>	Array of given component type

The field descriptor of an instance variable of type `int` is simply `I`.

The field descriptor of an instance variable of type `Object` is `L java/lang/Object ;`. Note that the internal form of the binary name for class `Object` is used.

The field descriptor of an instance variable of the multidimensional array type `double[][]` is `[[[D`.

4.3.3 Method Descriptors

A *method descriptor* contains zero or more *parameter descriptors*, representing the types of parameters that the method takes, and a *return descriptor*, representing the type of the value (if any) that the method returns.

MethodDescriptor:

({*ParameterDescriptor*}) *ReturnDescriptor*

ParameterDescriptor:

FieldType

ReturnDescriptor:

FieldType

VoidDescriptor

VoidDescriptor:

V

The character V indicates that the method returns no value (its result is void).

A method descriptor *mentions* a class or interface name if the name appears as a *ClassName* in the *FieldType* of a parameter descriptor or return descriptor.

The method descriptor for the method:

```
Object m(int i, double d, Thread t) {...}
```

is:

```
(IDLjava/lang/Thread;)Ljava/lang/Object;
```

Note that the internal forms of the binary names of Thread and Object are used.

A method descriptor is valid only if it represents method parameters with a total length of 255 or less, where that length includes the contribution for this in the case of instance or interface method invocations. The total length is calculated by summing the contributions of the individual parameters, where a parameter of type long or double contributes two units to the length and a parameter of any other type contributes one unit.

A method descriptor is the same whether the method it describes is a class method or an instance method. Although an instance method is passed this, a reference to the object on which the method is being invoked, in addition to its intended arguments, that fact is not reflected in the method descriptor. The reference to this is passed implicitly by the Java Virtual Machine instructions which invoke instance methods (§2.6.1, §4.11).

4.4 The Constant Pool

Java Virtual Machine instructions do not rely on the run-time layout of classes, interfaces, class instances, or arrays. Instead, instructions refer to symbolic information in the constant_pool table.

All constant_pool table entries have the following general format:

```
cp_info {  
    u1 tag;  
    u1 info[];  
}
```


Each entry in the `constant_pool` table must begin with a 1-byte tag indicating the kind of constant denoted by the entry. There are 17 kinds of constant, listed in Table 4.4-A with their corresponding tags, and ordered by their section number in this chapter. Each tag byte must be followed by two or more bytes giving information about the specific constant. The format of the additional information depends on the tag byte, that is, the content of the `info` array varies with the value of tag.

Table 4.4-A. Constant pool tags (by section)

Constant Kind	Tag	Section
CONSTANT_Class	7	§4.4.1
CONSTANT_Fieldref	9	§4.4.2
CONSTANT_Methodref	10	§4.4.2
CONSTANT_InterfaceMethodref	11	§4.4.2
CONSTANT_String	8	§4.4.3
CONSTANT_Integer	3	§4.4.4
CONSTANT_Float	4	§4.4.4
CONSTANT_Long	5	§4.4.5
CONSTANT_Double	6	§4.4.5
CONSTANT_NameAndType	12	§4.4.6
CONSTANT_Utf8	1	§4.4.7
CONSTANT_MethodHandle	15	§4.4.8
CONSTANT_MethodType	16	§4.4.9
CONSTANT_Dynamic	17	§4.4.10
CONSTANT_Invokedynamic	18	§4.4.10
CONSTANT_Module	19	§4.4.11
CONSTANT_Package	20	§4.4.12

In a class file whose version number is `v`, each entry in the `constant_pool` table must have a tag that was first defined in version `v` or earlier of the class file format (§4.1). That is, each entry must denote a kind of constant that is approved for use in the class file. Table 4.4-B lists each tag with the first version of the class file format in which it was defined. Also shown is the version of the Java SE Platform which introduced that version of the class file format.

Table 4.4-B. Constant pool tags (by tag)

Constant Kind	Tag	class file format	Java SE
CONSTANT_Utf8	1	45.3	1.0.2
CONSTANT_Integer	3	45.3	1.0.2
CONSTANT_Float	4	45.3	1.0.2
CONSTANT_Long	5	45.3	1.0.2
CONSTANT_Double	6	45.3	1.0.2
CONSTANT_Class	7	45.3	1.0.2
CONSTANT_String	8	45.3	1.0.2
CONSTANT_Fieldref	9	45.3	1.0.2
CONSTANT_Methodref	10	45.3	1.0.2
CONSTANT_InterfaceMethodref	11	45.3	1.0.2
CONSTANT_NameAndType	12	45.3	1.0.2
CONSTANT_MethodHandle	15	51.0	7
CONSTANT_MethodType	16	51.0	7
CONSTANT_Dynamic	17	55.0	11
CONSTANT_Invokedynamic	18	51.0	7
CONSTANT_Module	19	53.0	9
CONSTANT_Package	20	53.0	9

Some entries in the `constant_pool` table are *loadable* because they represent entities that can be pushed onto the stack at run time to enable further computation. In a class file whose version number is *v*, an entry in the `constant_pool` table is loadable if it has a tag that was first deemed to be loadable in version *v* or earlier of the class file format. Table 4.4-C lists each tag with the first version of the class file format in which it was deemed to be loadable. Also shown is the version of the Java SE Platform which introduced that version of the class file format.

In every case except `CONSTANT_Class`, a tag was first deemed to be loadable in the same version of the class file format that first defined the tag.

Table 4.4-C. Loadable constant pool tags

Constant Kind	Tag	class file format	Java SE
CONSTANT_Integer	3	45.3	1.0.2
CONSTANT_Float	4	45.3	1.0.2
CONSTANT_Long	5	45.3	1.0.2
CONSTANT_Double	6	45.3	1.0.2
CONSTANT_Class	7	49.0	5.0
CONSTANT_String	8	45.3	1.0.2
CONSTANT_MethodHandle	15	51.0	7
CONSTANT_MethodType	16	51.0	7
CONSTANT_Dynamic	17	55.0	11

4.4.1 The CONSTANT_Class_info Structure

The CONSTANT_Class_info structure is used to represent a class or an interface:

```
CONSTANT_Class_info {
    u1 tag;
    u2 name_index;
}
```

The items of the CONSTANT_Class_info structure are as follows:

tag

The tag item has the value CONSTANT_Class (7).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing a valid binary class or interface name encoded in internal form (§4.2.1).

Because arrays are objects, the opcodes *anewarray* and *multianewarray* - but not the opcode *new* - can reference array "classes" via CONSTANT_Class_info structures in the constant_pool table. For such array classes, the name of the class is the descriptor of the array type (§4.3.2).

For example, the class name representing the two-dimensional array type `int[][]` is `[[I`, while the class name representing the type `Thread[]` is `[Ljava/lang/Thread;`.

An array type descriptor is valid only if it represents 255 or fewer dimensions.

4.4.2 The `CONSTANT_Fieldref_info`, `CONSTANT_Methodref_info`, and `CONSTANT_InterfaceMethodref_info` Structures

Fields, methods, and interface methods are represented by similar structures:

```
CONSTANT_Fieldref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}

CONSTANT_Methodref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}

CONSTANT_InterfaceMethodref_info {
    u1 tag;
    u2 class_index;
    u2 name_and_type_index;
}
```

The items of these structures are as follows:

tag

The tag item of a `CONSTANT_Fieldref_info` structure has the value `CONSTANT_Fieldref` (9).

The tag item of a `CONSTANT_Methodref_info` structure has the value `CONSTANT_Methodref` (10).

The tag item of a `CONSTANT_InterfaceMethodref_info` structure has the value `CONSTANT_InterfaceMethodref` (11).

class_index

The value of the `class_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class or interface type that has the field or method as a member.

In a `CONSTANT_Fieldref_info` structure, the `class_index` item may be either a class type or an interface type.

In a `CONSTANT_Methodref_info` structure, the `class_index` item should be a class type, not an interface type.

In a `CONSTANT_InterfaceMethodref_info` structure, the `class_index` item should be an interface type, not a class type.

`name_and_type_index`

The value of the `name_and_type_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_NameAndType_info` structure (§4.4.6). This `constant_pool` entry indicates the name and descriptor of the field or method.

In a `CONSTANT_Fieldref_info` structure, the indicated descriptor must be a field descriptor (§4.3.2). Otherwise, the indicated descriptor must be a method descriptor (§4.3.3).

If the name of the method in a `CONSTANT_Methodref_info` structure begins with a '`<`' (`'\u003c'`), then the name must be the special name `<init>`, representing an instance initialization method (§2.9.1). The return type of such a method must be `void`.

4.4.3 The `CONSTANT_String_info` Structure

The `CONSTANT_String_info` structure is used to represent constant objects of the type `String`:

```
CONSTANT_String_info {
    u1 tag;
    u2 string_index;
}
```

The items of the `CONSTANT_String_info` structure are as follows:

`tag`

The `tag` item has the value `CONSTANT_String` (8).

`string_index`

The value of the `string_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the sequence of Unicode code points to which the `String` object is to be initialized.

4.4.4 The `CONSTANT_Integer_info` and `CONSTANT_Float_info` Structures

The `CONSTANT_Integer_info` and `CONSTANT_Float_info` structures represent 4-byte numeric (`int` and `float`) constants:

```

CONSTANT_Integer_info {
    u1 tag;
    u4 bytes;
}

CONSTANT_Float_info {
    u1 tag;
    u4 bytes;
}

```

The items of these structures are as follows:

tag

The tag item of the `CONSTANT_Integer_info` structure has the value `CONSTANT_Integer` (3).

The tag item of the `CONSTANT_Float_info` structure has the value `CONSTANT_Float` (4).

bytes

The bytes item of the `CONSTANT_Integer_info` structure represents the value of the `int` constant. The bytes of the value are stored in big-endian (high byte first) order.

The bytes item of the `CONSTANT_Float_info` structure represents the value of the `float` constant in IEEE 754 binary32 floating-point format (§2.3.2). The bytes of the item are stored in big-endian (high byte first) order.

The value represented by the `CONSTANT_Float_info` structure is determined as follows. The bytes of the value are first converted into an `int` constant *bits*. Then:

- If *bits* is `0x7f800000`, the `float` value will be positive infinity.
- If *bits* is `0xff800000`, the `float` value will be negative infinity.
- If *bits* is in the range `0x7f800001` through `0x7fffffff` or in the range `0xff800001` through `0xffffffff`, the `float` value will be NaN.
- In all other cases, let *s*, *e*, and *m* be three values that might be computed from *bits*:

```

int s = ((bits >> 31) == 0) ? 1 : -1;
int e = ((bits >> 23) & 0xff);
int m = (e == 0) ?
    (bits & 0x7fffffff) << 1 :
    (bits & 0x7fffffff) | 0x800000;

```

Then the float value equals the result of the mathematical expression $s \cdot m \cdot 2^{e-150}$.

4.4.5 The CONSTANT_Long_info and CONSTANT_Double_info Structures

The CONSTANT_Long_info and CONSTANT_Double_info represent 8-byte numeric (long and double) constants:

```
CONSTANT_Long_info {
    u1 tag;
    u4 high_bytes;
    u4 low_bytes;
}

CONSTANT_Double_info {
    u1 tag;
    u4 high_bytes;
    u4 low_bytes;
}
```

All 8-byte constants take up two entries in the constant_pool table of the class file. If a CONSTANT_Long_info or CONSTANT_Double_info structure is the entry at index n in the constant_pool table, then the next usable entry in the table is located at index $n+2$. The constant_pool index $n+1$ must be valid but is considered unusable.

In retrospect, making 8-byte constants take two constant pool entries was a poor choice.

The items of these structures are as follows:

tag

The tag item of the CONSTANT_Long_info structure has the value CONSTANT_Long (5).

The tag item of the CONSTANT_Double_info structure has the value CONSTANT_Double (6).

high_bytes, low_bytes

The unsigned high_bytes and low_bytes items of the CONSTANT_Long_info structure together represent the value of the long constant

$$((\text{long}) \text{high_bytes} \ll 32) + \text{low_bytes}$$

where the bytes of each of high_bytes and low_bytes are stored in big-endian (high byte first) order.

The `high_bytes` and `low_bytes` items of the `CONSTANT_Double_info` structure together represent the double value in IEEE 754 binary64 floating-point format (§2.3.2). The bytes of each item are stored in big-endian (high byte first) order.

The value represented by the `CONSTANT_Double_info` structure is determined as follows. The `high_bytes` and `low_bytes` items are converted into the long constant *bits*, which is equal to

$$((\text{long}) \text{high_bytes} \ll 32) + \text{low_bytes}$$

Then:

- If *bits* is `0x7ff0000000000000L`, the double value will be positive infinity.
- If *bits* is `0xfff0000000000000L`, the double value will be negative infinity.
- If *bits* is in the range `0x7ff0000000000001L` through `0x7fffffffffffffffffL` or in the range `0xfff0000000000001L` through `0xfffffffffffffffffL`, the double value will be NaN.
- In all other cases, let *s*, *e*, and *m* be three values that might be computed from *bits*:

```
int s = ((bits >> 63) == 0) ? 1 : -1;
int e = (int)((bits >> 52) & 0x7ffL);
long m = (e == 0) ?
    (bits & 0xffffffffffffffffL) << 1 :
    (bits & 0xffffffffffffffffL) | 0x100000000000000L;
```

Then the floating-point value equals the double value of the mathematical expression $s \cdot m \cdot 2^{e-1075}$.

4.4.6 The `CONSTANT_NameAndType_info` Structure

The `CONSTANT_NameAndType_info` structure is used to represent a field or method, without indicating which class or interface type it belongs to:

```
CONSTANT_NameAndType_info {
    u1 tag;
    u2 name_index;
    u2 descriptor_index;
}
```

The items of the `CONSTANT_NameAndType_info` structure are as follows:

tag

The tag item has the value `CONSTANT_NameAndType` (12).

name_index

The value of the name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing either a valid unqualified name denoting a field or method (§4.2.2), or the special method name <init> (§2.9.1).

descriptor_index

The value of the descriptor_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing a valid field descriptor or method descriptor (§4.3.2, §4.3.3).

4.4.7 The CONSTANT_Utf8_info Structure

The CONSTANT_Utf8_info structure is used to represent constant string values:

```
CONSTANT_Utf8_info {
    u1 tag;
    u2 length;
    u1 bytes[length];
}
```

The items of the CONSTANT_Utf8_info structure are as follows:

tag

The tag item has the value CONSTANT_Utf8 (1).

length

The value of the length item gives the number of bytes in the bytes array (not the length of the resulting string).

bytes[]

The bytes array contains the bytes of the string.

No byte may have the value (byte)0.

No byte may lie in the range (byte)0xf0 to (byte)0xff.

String content is encoded in modified UTF-8. Modified UTF-8 strings are encoded so that code point sequences that contain only non-null ASCII characters can be represented using only 1 byte per code point, but all code points in the Unicode codespace can be represented. Modified UTF-8 strings are not null-terminated. The encoding is as follows:

- Code points in the range '\u0001' to '\u007F' are represented by a single byte:

0	bits 6-0
---	----------

The 7 bits of data in the byte give the value of the code point represented.

- The null code point ('\u0000') and code points in the range '\u0080' to '\u07FF' are represented by a pair of bytes x and y :

x:	1	1	0	bits 10-6
y:	1	0	bits 5-0	

The two bytes represent the code point with the value:

$$((x \& 0x1f) \ll 6) + (y \& 0x3f)$$

- Code points in the range '\u0800' to '\uFFFF' are represented by 3 bytes x, y, and z :

x:	1	1	1	0	bits 15-12
y:	1	0	bits 11-6		
z:	1	0	bits 5-0		

The three bytes represent the code point with the value:

$$((x \& 0xf) \ll 12) + ((y \& 0x3f) \ll 6) + (z \& 0x3f)$$

- Characters with code points above U+FFFF (so-called *supplementary characters*) are represented by separately encoding the two surrogate code units of their UTF-16 representation. Each of the surrogate code units is represented by

three bytes. This means supplementary characters are represented by six bytes, u, v, w, x, y, and z :

u:	1	1	1	0	1	1	0	1
v:	1	0	1	0	(bits 20-16)-1			
w:	1	0	bits 15-10					
x:	1	1	1	0	1	1	0	1
y:	1	0	1	1	bits 9-6			
z:	1	0	bits 5-0					

The six bytes represent the code point with the value:

$$0 \times 10000 + ((v \& 0 \times 0f) \ll 16) + ((w \& 0 \times 3f) \ll 10) + ((y \& 0 \times 0f) \ll 6) + (z \& 0 \times 3f)$$

The bytes of multibyte characters are stored in the class file in big-endian (high byte first) order.

There are two differences between this format and the "standard" UTF-8 format. First, the null character (char)0 is encoded using the 2-byte format rather than the 1-byte format, so that modified UTF-8 strings never have embedded nulls. Second, only the 1-byte, 2-byte, and 3-byte formats of standard UTF-8 are used. The Java Virtual Machine does not recognize the four-byte format of standard UTF-8; it uses its own two-times-three-byte format instead.

For more information regarding the standard UTF-8 format, see Section 3.9 *Unicode Encoding Forms of The Unicode Standard, Version 17.0.0*.

4.4.8 The CONSTANT_MethodHandle_info Structure

The CONSTANT_MethodHandle_info structure is used to represent a method handle:

```
CONSTANT_MethodHandle_info {
    u1 tag;
    u1 reference_kind;
    u2 reference_index;
}
```

The items of the CONSTANT_MethodHandle_info structure are the following:

tag

The tag item has the value CONSTANT_MethodHandle (15).

reference_kind

The value of the `reference_kind` item must be in the range 1 to 9. The value denotes the *kind* of this method handle, which characterizes its bytecode behavior (§5.4.3.5).

reference_index

The value of the `reference_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be as follows:

- If the value of the `reference_kind` item is 1 (`REF_getField`), 2 (`REF_getStatic`), 3 (`REF_putField`), or 4 (`REF_putStatic`), then the `constant_pool` entry at that index must be a `CONSTANT_Fieldref_info` structure (§4.4.2) representing a field for which a method handle is to be created.
- If the value of the `reference_kind` item is 5 (`REF_invokeVirtual`) or 8 (`REF_newInvokeSpecial`), then the `constant_pool` entry at that index must be a `CONSTANT_Methodref_info` structure (§4.4.2) representing a class's method or constructor (§2.9.1) for which a method handle is to be created.
- If the value of the `reference_kind` item is 6 (`REF_invokeStatic`) or 7 (`REF_invokeSpecial`), then if the class file version number is less than 52.0, the `constant_pool` entry at that index must be a `CONSTANT_Methodref_info` structure representing a class's method for which a method handle is to be created; if the class file version number is 52.0 or above, the `constant_pool` entry at that index must be either a `CONSTANT_Methodref_info` structure or a `CONSTANT_InterfaceMethodref_info` structure (§4.4.2) representing a class's or interface's method for which a method handle is to be created.
- If the value of the `reference_kind` item is 9 (`REF_invokeInterface`), then the `constant_pool` entry at that index must be a `CONSTANT_InterfaceMethodref_info` structure representing an interface's method for which a method handle is to be created.

If the value of the `reference_kind` item is 5 (`REF_invokeVirtual`), 6 (`REF_invokeStatic`), 7 (`REF_invokeSpecial`), or 9 (`REF_invokeInterface`), the name of the method represented by a `CONSTANT_Methodref_info` structure or a `CONSTANT_InterfaceMethodref_info` structure must not be `<init>` or `<clinit>`.

If the value is 8 (`REF_newInvokeSpecial`), the name of the method represented by a `CONSTANT_Methodref_info` structure must be `<init>`.

4.4.9 The CONSTANT_MethodType_info Structure

The CONSTANT_MethodType_info structure is used to represent a method type:

```
CONSTANT_MethodType_info {  
    u1 tag;  
    u2 descriptor_index;  
}
```

The items of the CONSTANT_MethodType_info structure are as follows:

tag

The tag item has the value CONSTANT_MethodType (16).

descriptor_index

The value of the descriptor_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing a method descriptor (§4.3.3).

4.4.10 The CONSTANT_Dynamic_info and CONSTANT_Invokedynamic_info Structures

Most structures in the constant_pool table represent entities directly, by combining names, descriptors, and values recorded statically in the table. In contrast, the CONSTANT_Dynamic_info and CONSTANT_Invokedynamic_info structures represent entities indirectly, by pointing to code which computes an entity dynamically. The code, called a *bootstrap method*, is invoked by the Java Virtual Machine during resolution of symbolic references derived from these structures (§5.1, §5.4.3.6). Each structure specifies a bootstrap method as well as an auxiliary name and type that characterize the entity to be computed. In more detail:

- The CONSTANT_Dynamic_info structure is used to represent a *dynamically-computed constant*, an arbitrary value that is produced by invocation of a bootstrap method in the course of an *ldc* instruction (§*ldc*), among others. The auxiliary type specified by the structure constrains the type of the dynamically-computed constant.
- The CONSTANT_Invokedynamic_info structure is used to represent a *dynamically-computed call site*, an instance of `java.lang.invoke.CallSite` that is produced by invocation of a bootstrap method in the course of an *invokedynamic* instruction (§*invokedynamic*). The auxiliary type specified by the structure constrains the method type of the dynamically-computed call site.

```

CONSTANT_Dynamic_info {
    u1 tag;
    u2 bootstrap_method_attr_index;
    u2 name_and_type_index;
}

CONSTANT_Invokedynamic_info {
    u1 tag;
    u2 bootstrap_method_attr_index;
    u2 name_and_type_index;
}

```

The items of these structures are as follows:

tag

The tag item of a `CONSTANT_Dynamic_info` structure has the value `CONSTANT_Dynamic` (17).

The tag item of a `CONSTANT_Invokedynamic_info` structure has the value `CONSTANT_Invokedynamic` (18).

bootstrap_method_attr_index

The value of the `bootstrap_method_attr_index` item must be a valid index into the `bootstrap_methods` array of the bootstrap method table of this class file (§4.7.23).

`CONSTANT_Dynamic_info` structures are unique in that they are syntactically allowed to refer to themselves via the bootstrap method table. Rather than mandating that such cycles are detected when classes are loaded (a potentially expensive check), we permit cycles initially but mandate a failure at resolution (§5.4.3.6).

name_and_type_index

The value of the `name_and_type_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_NameAndType_info` structure (§4.4.6). This `constant_pool` entry indicates a name and descriptor.

In a `CONSTANT_Dynamic_info` structure, the indicated descriptor must be a field descriptor (§4.3.2).

In a `CONSTANT_Invokedynamic_info` structure, the indicated descriptor must be a method descriptor (§4.3.3).

4.4.11 The `CONSTANT_Module_info` Structure

The `CONSTANT_Module_info` structure is used to represent a module:

```
CONSTANT_Module_info {  
    u1 tag;  
    u2 name_index;  
}
```

The items of the `CONSTANT_Module_info` structure are as follows:

tag

The tag item has the value `CONSTANT_Module` (19).

name_index

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid module name (§4.2.3).

A `CONSTANT_Module_info` structure is permitted only in the constant pool of a class file that declares a module, that is, a `ClassFile` structure where the `access_flags` item has the `ACC_MODULE` flag set. In all other class files, a `CONSTANT_Module_info` structure is illegal.

4.4.12 The `CONSTANT_Package_info` Structure

The `CONSTANT_Package_info` structure is used to represent a package exported or opened by a module:

```
CONSTANT_Package_info {  
    u1 tag;  
    u2 name_index;  
}
```

The items of the `CONSTANT_Package_info` structure are as follows:

tag

The tag item has the value `CONSTANT_Package` (20).

name_index

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid package name encoded in internal form (§4.2.3).

A `CONSTANT_Package_info` structure is permitted only in the constant pool of a class file that declares a module, that is, a `ClassFile` structure where the

`access_flags` item has the `ACC_MODULE` flag set. In all other class files, a `CONSTANT_Package_info` structure is illegal.

4.5 Fields

Each field is described by a `field_info` structure.

No two fields in one class file may have the same name and descriptor (§4.3.2).

The structure has the following format:

```
field_info {  
    u2          access_flags;  
    u2          name_index;  
    u2          descriptor_index;  
    u2          attributes_count;  
    attribute_info attributes[attributes_count];  
}
```

The items of the `field_info` structure are as follows:

`access_flags`

The value of the `access_flags` item is a mask of flags used to denote access permission to and properties of this field. The interpretation of each flag, when set, is specified in Table 4.5-A.

Table 4.5-A. Field access and property flags

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared public ; may be accessed from outside its package.
ACC_PRIVATE	0x0002	Declared private ; accessible only within the defining class and other classes belonging to the same nest (§5.4.4).
ACC_PROTECTED	0x0004	Declared protected ; may be accessed within subclasses.
ACC_STATIC	0x0008	Declared static .
ACC_FINAL	0x0010	Declared final ; never directly assigned to after object construction (JLS §17.5).
ACC_VOLATILE	0x0040	Declared volatile ; cannot be cached.
ACC_TRANSIENT	0x0080	Declared transient ; not written or read by a persistent object manager.
ACC_SYNTHETIC	0x1000	Declared synthetic ; not present in the source code.
ACC_ENUM	0x4000	Declared as an element of an enum class.

Fields of classes may set any of the flags in Table 4.5-A. However, each field of a class may have at most one of its ACC_PUBLIC, ACC_PRIVATE, and ACC_PROTECTED flags set (JLS §8.3.1), and must not have both its ACC_FINAL and ACC_VOLATILE flags set (JLS §8.3.1.4).

Fields of interfaces must have their ACC_PUBLIC, ACC_STATIC, and ACC_FINAL flags set; they may have their ACC_SYNTHETIC flag set and must not have any of the other flags in Table 4.5-A set (JLS §9.3).

The ACC_SYNTHETIC flag indicates that this field was generated by a compiler and does not appear in source code.

The ACC_ENUM flag indicates that this field is used to hold an element of an enum class (JLS §8.9).

All bits of the `access_flags` item not assigned in Table 4.5-A are reserved for future use. They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a

`CONSTANT_Utf8_info` structure (§4.4.7) which represents a valid unqualified name denoting a field (§4.2.2).

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) which represents a valid field descriptor (§4.3.2).

`attributes_count`

The value of the `attributes_count` item indicates the number of additional attributes of this field.

`attributes[]`

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A field can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a `field_info` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `field_info` structure are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a `field_info` structure are given in §4.7.1.

4.6 Methods

Each method, including each instance initialization method (§2.9.1) and the class or interface initialization method (§2.9.2), is described by a `method_info` structure.

No two methods in one class file may have the same name and descriptor (§4.3.3).

The structure has the following format:

```
method_info {
    u2          access_flags;
    u2          name_index;
    u2          descriptor_index;
    u2          attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items of the `method_info` structure are as follows:

access_flags

The value of the `access_flags` item is a mask of flags used to denote access permission to and properties of this method. The interpretation of each flag, when set, is specified in Table 4.6-A.

Table 4.6-A. Method access and property flags

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Declared <code>public</code> ; may be accessed from outside its package.
ACC_PRIVATE	0x0002	Declared <code>private</code> ; accessible only within the defining class and other classes belonging to the same nest (§5.4.4).
ACC_PROTECTED	0x0004	Declared <code>protected</code> ; may be accessed within subclasses.
ACC_STATIC	0x0008	Declared <code>static</code> .
ACC_FINAL	0x0010	Declared <code>final</code> ; must not be overridden (§5.4.5).
ACC_SYNCHRONIZED	0x0020	Declared <code>synchronized</code> ; invocation is wrapped by a monitor use.
ACC_BRIDGE	0x0040	A bridge method, generated by the compiler.
ACC_VARARGS	0x0080	Declared with variable number of arguments.
ACC_NATIVE	0x0100	Declared <code>native</code> ; implemented in a language other than the Java programming language.
ACC_ABSTRACT	0x0400	Declared <code>abstract</code> ; no implementation is provided.
ACC_STRICT	0x0800	In a <code>class</code> file whose major version number is at least 46 and at most 60: Declared <code>strictfp</code> .
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.

The value 0x0800 is interpreted as the `ACC_STRICT` flag only in a `class` file whose major version number is at least 46 and at most 60. For methods in such a `class` file, the rules below determine whether the `ACC_STRICT` flag may be set in combination with other flags. (Setting the `ACC_STRICT` flag constrained a method's floating-point instructions in Java SE 1.2 through 16 (§2.8).) For methods in a `class` file whose major version number is less than 46 or greater than 60, the value 0x0800 is not interpreted as the `ACC_STRICT` flag, but rather is unassigned; it is not meaningful to "set the `ACC_STRICT` flag" in such a `class` file.

Methods of classes may have any of the flags in Table 4.6-A set. However, each method of a class may have at most one of its `ACC_PUBLIC`, `ACC_PRIVATE`, and `ACC_PROTECTED` flags set (JLS §8.4.3).

Methods of interfaces may have any of the flags in Table 4.6-A set except `ACC_PROTECTED`, `ACC_FINAL`, `ACC_SYNCHRONIZED`, and `ACC_NATIVE` (JLS §9.4). In a `class` file whose version number is less than 52.0, each method of an interface must have its `ACC_PUBLIC` and `ACC_ABSTRACT` flags set; in a `class` file whose version number is 52.0 or above, each method of an interface must have exactly one of its `ACC_PUBLIC` and `ACC_PRIVATE` flags set.

If a method of a class or interface has its `ACC_ABSTRACT` flag set, it must not have any of its `ACC_PRIVATE`, `ACC_STATIC`, `ACC_FINAL`, `ACC_SYNCHRONIZED`, or `ACC_NATIVE` flags set, nor (in a `class` file whose major version number is at least 46 and at most 60) have its `ACC_STRICT` flag set.

An instance initialization method (§2.9.1) may have at most one of its `ACC_PUBLIC`, `ACC_PRIVATE`, and `ACC_PROTECTED` flags set, and may also have its `ACC_VARARGS` and `ACC_SYNTHETIC` flags set, and may also (in a `class` file whose major version number is at least 46 and at most 60) have its `ACC_STRICT` flag set, but must not have any of the other flags in Table 4.6-A set.

In a `class` file whose version number is 51.0 or above, a method whose name is `<clinit>` must have its `ACC_STATIC` flag set.

A class or interface initialization method (§2.9.2) is called implicitly by the Java Virtual Machine. The value of its `access_flags` item is ignored except for the setting of the `ACC_STATIC` flag and (in a `class` file whose major version number is at least 46 and at most 60) the `ACC_STRICT` flag, and the method is exempt from the preceding rules about legal combinations of flags.

The `ACC_BRIDGE` flag is used to indicate a bridge method generated by a compiler for the Java programming language.

The `ACC_VARARGS` flag indicates that this method takes a variable number of arguments at the source code level. A method declared to take a variable number of arguments must be compiled with the `ACC_VARARGS` flag set to 1. All other methods must be compiled with the `ACC_VARARGS` flag set to 0.

The `ACC_SYNTHETIC` flag indicates that this method was generated by a compiler and does not appear in source code, unless it is one of the methods named in §4.7.8.

All bits of the `access_flags` item not assigned in Table 4.6-A are reserved for future use. (This includes the bit corresponding to 0x0800 in a `class` file

whose major version number is less than 46 or greater than 60.) They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing either a valid unqualified name denoting a method (§4.2.2), or (if this method is in a class rather than an interface) the special method name `<init>`, or the special method name `<clinit>`.

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing a valid method descriptor (§4.3.3). Furthermore:

- If this method is in a class rather than an interface, and the name of the method is `<init>`, then the descriptor must denote a void method.
- If the name of the method is `<clinit>`, then the descriptor must denote a void method, and, in a class file whose version number is 51.0 or above, a method that takes no arguments.

A future edition of this specification may require that the last parameter descriptor of the method descriptor is an array type if the `ACC_VARARGS` flag is set in the `access_flags` item.

`attributes_count`

The value of the `attributes_count` item indicates the number of additional attributes of this method.

`attributes[]`

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A method can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a `method_info` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `method_info` structure are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a `method_info` structure are given in §4.7.1.

4.7 Attributes

Attributes are used in the `ClassFile`, `field_info`, `method_info`, `Code_attribute`, and `record_component_info` structures of the class file format (§4.1, §4.5, §4.6, §4.7.3, §4.7.30).

All attributes have the following general format:

```
attribute_info {
    u2 attribute_name_index;
    u4 attribute_length;
    u1 info[attribute_length];
}
```

For all attributes, the `attribute_name_index` item must be a valid unsigned 16-bit index into the constant pool of the class. The `constant_pool` entry at `attribute_name_index` must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the name of the attribute. The value of the `attribute_length` item indicates the length of the subsequent information in bytes. The length does not include the initial six bytes that contain the `attribute_name_index` and `attribute_length` items.

30 attributes are predefined by this specification. They are listed three times, for ease of navigation:

- Table 4.7-A is ordered by the attributes' section numbers in this chapter. Each attribute is shown with the first version of the class file format in which it was defined. Also shown is the version of the Java SE Platform which introduced that version of the class file format (§4.1).
- Table 4.7-B is ordered by the first version of the class file format in which each attribute was defined.
- Table 4.7-C is ordered by the location in a class file where each attribute is defined to appear.

Within the context of their use in this specification, that is, in the `attributes` tables of the class file structures in which they appear, the names of these predefined attributes are reserved.

Any conditions on the presence of a predefined attribute in an `attributes` table are specified explicitly in the section which describes the attribute. If no conditions are specified, then the attribute may appear any number of times in an `attributes` table.

The predefined attributes are categorized into three groups according to their purpose:

1. Seven attributes are critical to correct interpretation of the `class` file by the Java Virtual Machine:
 - `ConstantValue`
 - `Code`
 - `StackMapTable`
 - `BootstrapMethods`
 - `NestHost`
 - `NestMembers`
 - `PermittedSubclasses`

In a `class` file whose version number is `v`, each of these attributes must be recognized and correctly read by an implementation of the Java Virtual Machine if the implementation supports version `v` of the `class` file format, and the attribute was first defined in version `v` or earlier of the `class` file format, and the attribute appears in a location where it is defined to appear.

2. Ten attributes are not critical to correct interpretation of the `class` file by the Java Virtual Machine, but are either critical to correct interpretation of the

class file by the class libraries of the Java SE Platform, or are useful for tools (in which case the section that specifies an attribute describes it as "optional"):

- Exceptions
- InnerClasses
- EnclosingMethod
- Synthetic
- Signature
- Record
- SourceFile
- LineNumberTable
- LocalVariableTable
- LocalVariableTypeTable

In a class file whose version number is *v*, each of these attributes must be recognized and correctly read by an implementation of the Java Virtual Machine if the implementation supports version *v* of the class file format, and the attribute was first defined in version *v* or earlier of the class file format, and the attribute appears in a location where it is defined to appear.

3. Thirteen attributes are not critical to correct interpretation of the class file by the Java Virtual Machine, but contain metadata about the class file that is either exposed by the class libraries of the Java SE Platform, or made available

by tools (in which case the section that specifies an attribute describes it as "optional"):

- SourceDebugExtension
- Deprecated
- RuntimeVisibleAnnotations
- RuntimeInvisibleAnnotations
- RuntimeVisibleParameterAnnotations
- RuntimeInvisibleParameterAnnotations
- RuntimeVisibleTypeAnnotations
- RuntimeInvisibleTypeAnnotations
- AnnotationDefault
- MethodParameters
- Module
- ModulePackages
- ModuleMainClass

An implementation of the Java Virtual Machine may use the information that these attributes contain, or otherwise must silently ignore these attributes.

Table 4.7-A. Predefined class file attributes (by section)

Attribute	Section	class file	Java SE
ConstantValue	§4.7.2	45.3	1.0.2
Code	§4.7.3	45.3	1.0.2
StackMapTable	§4.7.4	50.0	6
Exceptions	§4.7.5	45.3	1.0.2
InnerClasses	§4.7.6	45.3	1.1
EnclosingMethod	§4.7.7	49.0	5.0
Synthetic	§4.7.8	45.3	1.1
Signature	§4.7.9	49.0	5.0
SourceFile	§4.7.10	45.3	1.0.2
SourceDebugExtension	§4.7.11	49.0	5.0
LineNumberTable	§4.7.12	45.3	1.0.2
LocalVariableTable	§4.7.13	45.3	1.0.2
LocalVariableTypeTable	§4.7.14	49.0	5.0
Deprecated	§4.7.15	45.3	1.1
RuntimeVisibleAnnotations	§4.7.16	49.0	5.0
RuntimeInvisibleAnnotations	§4.7.17	49.0	5.0
RuntimeVisibleParameterAnnotations	§4.7.18	49.0	5.0
RuntimeInvisibleParameterAnnotations	§4.7.19	49.0	5.0
RuntimeVisibleTypeAnnotations	§4.7.20	52.0	8
RuntimeInvisibleTypeAnnotations	§4.7.21	52.0	8
AnnotationDefault	§4.7.22	49.0	5.0
BootstrapMethods	§4.7.23	51.0	7
MethodParameters	§4.7.24	52.0	8
Module	§4.7.25	53.0	9
ModulePackages	§4.7.26	53.0	9
ModuleMainClass	§4.7.27	53.0	9
NestHost	§4.7.28	55.0	11
NestMembers	§4.7.29	55.0	11
Record	§4.7.30	60.0	16
PermittedSubclasses	§4.7.31	61.0	17

Table 4.7-B. Predefined class file attributes (by class file format)

Attribute	class file	Java SE	Section
ConstantValue	45.3	1.0.2	§4.7.2
Code	45.3	1.0.2	§4.7.3
Exceptions	45.3	1.0.2	§4.7.5
SourceFile	45.3	1.0.2	§4.7.10
LineNumberTable	45.3	1.0.2	§4.7.12
LocalVariableTable	45.3	1.0.2	§4.7.13
InnerClasses	45.3	1.1	§4.7.6
Synthetic	45.3	1.1	§4.7.8
Deprecated	45.3	1.1	§4.7.15
EnclosingMethod	49.0	5.0	§4.7.7
Signature	49.0	5.0	§4.7.9
SourceDebugExtension	49.0	5.0	§4.7.11
LocalVariableTypeTable	49.0	5.0	§4.7.14
RuntimeVisibleAnnotations	49.0	5.0	§4.7.16
RuntimeInvisibleAnnotations	49.0	5.0	§4.7.17
RuntimeVisibleParameterAnnotations	49.0	5.0	§4.7.18
RuntimeInvisibleParameterAnnotations	49.0	5.0	§4.7.19
AnnotationDefault	49.0	5.0	§4.7.22
StackMapTable	50.0	6	§4.7.4
BootstrapMethods	51.0	7	§4.7.23
RuntimeVisibleTypeAnnotations	52.0	8	§4.7.20
RuntimeInvisibleTypeAnnotations	52.0	8	§4.7.21
MethodParameters	52.0	8	§4.7.24
Module	53.0	9	§4.7.25
ModulePackages	53.0	9	§4.7.26
ModuleMainClass	53.0	9	§4.7.27
NestHost	55.0	11	§4.7.28
NestMembers	55.0	11	§4.7.29
Record	60.0	16	§4.7.30
PermittedSubclasses	61.0	17	§4.7.31

Table 4.7-C. Predefined class file attributes (by location)

Attribute	Location	class file
SourceFile	ClassFile	45.3
InnerClasses	ClassFile	45.3
EnclosingMethod	ClassFile	49.0
SourceDebugExtension	ClassFile	49.0
BootstrapMethods	ClassFile	51.0
Module, ModulePackages, ModuleMainClass	ClassFile	53.0
NestHost, NestMembers	ClassFile	55.0
Record	ClassFile	60.0
PermittedSubclasses	ClassFile	61.0
ConstantValue	field_info	45.3
Code	method_info	45.3
Exceptions	method_info	45.3
RuntimeVisibleParameterAnnotations, RuntimeInvisibleParameterAnnotations	method_info	49.0
AnnotationDefault	method_info	49.0
MethodParameters	method_info	52.0

Table 4.7-C (cont.). Predefined class file attributes (by location)

Attribute	Location	class file
Synthetic	ClassFile, field_info, method_info	45.3
Deprecated	ClassFile, field_info, method_info	45.3
Signature	ClassFile, field_info, method_info, record_component_info	49.0
RuntimeVisibleAnnotations, RuntimeInvisibleAnnotations	ClassFile, field_info, method_info, record_component_info	49.0
LineNumberTable	Code	45.3
LocalVariableTable	Code	45.3
LocalVariableTypeTable	Code	49.0
StackMapTable	Code	50.0
RuntimeVisibleTypeAnnotations, RuntimeInvisibleTypeAnnotations	ClassFile, field_info, method_info, Code, record_component_info	52.0

4.7.1 Defining and Naming New Attributes

Compilers are permitted to define and emit class files containing new attributes in the attributes tables of class file structures, field_info structures, method_info structures, and Code attributes (§4.7.3). Java Virtual Machine implementations are permitted to recognize and use new attributes found in these attributes tables. However, any attribute not defined as part of this specification must not affect the semantics of the class file. Java Virtual Machine implementations are required to silently ignore attributes they do not recognize.

For instance, defining a new attribute to support vendor-specific debugging is permitted. Because Java Virtual Machine implementations are required to ignore

attributes they do not recognize, `class` files intended for that particular Java Virtual Machine implementation will be usable by other implementations even if those implementations cannot make use of the additional debugging information that the `class` files contain.

Java Virtual Machine implementations are specifically prohibited from throwing an exception or otherwise refusing to use `class` files simply because of the presence of some new attribute. Of course, tools operating on `class` files may not run correctly if given `class` files that do not contain all the attributes they require.

Two attributes that are intended to be distinct, but that happen to use the same attribute name and are of the same length, will conflict on implementations that recognize either attribute. Attributes defined other than in this specification should have names chosen according to the package naming convention described in *The Java Language Specification, Java SE 26 Edition* (JLS §6.1).

Future versions of this specification may define additional attributes.

4.7.2 The `ConstantValue` Attribute

The `ConstantValue` attribute is a fixed-length attribute in the `attributes` table of a `field_info` structure (§4.5). A `ConstantValue` attribute represents the value of a constant expression (JLS §15.28), and is used as follows:

- If the `ACC_STATIC` flag in the `access_flags` item of the `field_info` structure is set, then the field represented by the `field_info` structure is assigned the value represented by its `ConstantValue` attribute as part of the initialization of the class or interface declaring the field (§5.5). This occurs prior to the invocation of the class or interface initialization method of that class or interface (§2.9.2).
- Otherwise, the Java Virtual Machine must silently ignore the attribute.

There may be at most one `ConstantValue` attribute in the `attributes` table of a `field_info` structure.

The `ConstantValue` attribute has the following format:

```
ConstantValue_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 constantvalue_index;  
}
```

The items of the `ConstantValue_attribute` structure are as follows:

attribute_name_index

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "ConstantValue".

attribute_length

The value of the `attribute_length` item must be two.

constantvalue_index

The value of the `constantvalue_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index gives the value represented by this attribute. The `constant_pool` entry must be of a type appropriate to the field, as specified in Table 4.7.2-A.

Table 4.7.2-A. Constant value attribute types

Field Type	Entry Type
int, short, char, byte, boolean	CONSTANT_Integer
float	CONSTANT_Float
long	CONSTANT_Long
double	CONSTANT_Double
String	CONSTANT_String

4.7.3 The Code Attribute

The Code attribute is a variable-length attribute in the attributes table of a `method_info` structure (§4.6). A Code attribute contains the Java Virtual Machine instructions and auxiliary information for a method, including an instance initialization method and a class or interface initialization method (§2.9.1, §2.9.2).

If the method is either native or abstract, and is not a class or interface initialization method, then its `method_info` structure must not have a Code attribute in its attributes table. Otherwise, its `method_info` structure must have exactly one Code attribute in its attributes table.

The Code attribute has the following format:

```

Code_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 max_stack;
    u2 max_locals;
    u4 code_length;
    u1 code[code_length];
    u2 exception_table_length;
    {    u2 start_pc;
        u2 end_pc;
        u2 handler_pc;
        u2 catch_type;
    } exception_table[exception_table_length];
    u2 attributes_count;
    attribute_info attributes[attributes_count];
}

```

The items of the `Code_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Code".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`max_stack`

The value of the `max_stack` item gives the maximum depth of the operand stack of this method (§2.6.2) at any point during execution of the method.

`max_locals`

The value of the `max_locals` item gives the number of local variables in the local variable array allocated upon invocation of this method (§2.6.1), including the local variables used to pass parameters to the method on its invocation.

The greatest local variable index for a value of type `long` or `double` is `max_locals - 2`. The greatest local variable index for a value of any other type is `max_locals - 1`.

`code_length`

The value of the `code_length` item gives the number of bytes in the code array for this method.

The value of `code_length` must be greater than zero (as the code array must not be empty) and less than 65536.

`code[]`

The code array gives the actual bytes of Java Virtual Machine code that implement the method.

When the code array is read into memory on a byte-addressable machine, if the first byte of the array is aligned on a 4-byte boundary, the *tableswitch* and *lookupswitch* 32-bit offsets will be 4-byte aligned. (Refer to the descriptions of those instructions for more information on the consequences of code array alignment.)

The detailed constraints on the contents of the code array are extensive and are given in a separate section (§4.9).

`exception_table_length`

The value of the `exception_table_length` item gives the number of entries in the `exception_table` array.

`exception_table[]`

Each entry in the `exception_table` array describes one exception handler in the code array. The order of the handlers in the `exception_table` array is significant (§2.10).

Each `exception_table` entry contains the following four items:

`start_pc`, `end_pc`

The values of the two items `start_pc` and `end_pc` indicate the ranges in the code array at which the exception handler is active. The value of `start_pc` must be a valid index into the code array of the opcode of an instruction. The value of `end_pc` either must be a valid index into the code array of the opcode of an instruction or must be equal to `code_length`, the length of the code array. The value of `start_pc` must be less than the value of `end_pc`.

The `start_pc` is inclusive and `end_pc` is exclusive; that is, the exception handler must be active while the program counter is within the interval [`start_pc`, `end_pc`).

The fact that `end_pc` is exclusive is a historical mistake in the design of the Java Virtual Machine: if the Java Virtual Machine code for a method is exactly 65535 bytes long and ends with an instruction that is 1 byte long, then that instruction cannot be protected by an exception handler. A compiler writer can work around this bug by limiting the maximum size of the generated Java Virtual Machine code for any method, instance initialization method, or static initializer (the size of any code array) to 65534 bytes.

handler_pc

The value of the `handler_pc` item indicates the start of the exception handler. The value of the item must be a valid index into the code array and must be the index of the opcode of an instruction.

catch_type

If the value of the `catch_type` item is nonzero, it must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class of exceptions that this exception handler is designated to catch. The exception handler will be called only if the thrown exception is an instance of the given class or one of its subclasses.

The verifier checks that the class is `Throwable` or a subclass of `Throwable` (§4.9.2).

If the value of the `catch_type` item is zero, this exception handler is called for all exceptions.

This is used to implement `finally` (§3.13).

attributes_count

The value of the `attributes_count` item indicates the number of attributes of the Code attribute.

attributes[]

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A Code attribute can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a Code attribute are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a Code attribute are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a Code attribute are given in §4.7.1.

4.7.4 The StackMapTable Attribute

The `StackMapTable` attribute is a variable-length attribute in the `attributes` table of a Code attribute (§4.7.3). A `StackMapTable` attribute is used during the process of verification by type checking (§4.10.1).

There may be at most one `StackMapTable` attribute in the attributes table of a Code attribute.

In a class file whose version number is 50.0 or above, if a method's Code attribute does not have a `StackMapTable` attribute, it has an *implicit stack map attribute* (§4.10.1). This implicit stack map attribute is equivalent to a `StackMapTable` attribute with `number_of_entries` equal to zero.

The `StackMapTable` attribute has the following format:

```
StackMapTable_attribute {
    u2          attribute_name_index;
    u4          attribute_length;
    u2          number_of_entries;
    stack_map_frame entries[number_of_entries];
}
```

The items of the `StackMapTable_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "StackMapTable".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`number_of_entries`

The value of the `number_of_entries` item gives the number of `stack_map_frame` entries in the `entries` table.

`entries[]`

Each entry in the `entries` table describes one stack map frame of the method. The order of the stack map frames in the `entries` table is significant.

A *stack map frame* specifies (either explicitly or implicitly) the bytecode offset at which it applies, and the verification types of local variables and operand stack entries for that offset.

Each stack map frame described in the `entries` table relies on the previous frame for some of its semantics. The first stack map frame of a method is implicit, and computed from the method descriptor by the type checker (§4.10.1.5). The `stack_map_frame` structure at `entries[0]` therefore describes the second stack map frame of the method.

The *bytecode offset* at which a stack map frame applies is calculated by taking the value `offset_delta` specified in the frame (either explicitly or implicitly), and adding `offset_delta + 1` to the bytecode offset of the previous frame, unless the previous frame is the initial frame of the method. In that case, the bytecode offset at which the stack map frame applies is the value `offset_delta` specified in the frame.

By using an offset delta rather than storing the actual bytecode offset, we ensure, by definition, that stack map frames are in the correctly sorted order. Furthermore, by consistently using the formula `offset_delta + 1` for all explicit frames (as opposed to the implicit first frame), we guarantee the absence of duplicates.

We say that an instruction in the bytecode has a *corresponding stack map frame* if the instruction starts at offset *i* in the code array of a Code attribute, and the Code attribute has a StackMapTable attribute whose `entries` array contains a stack map frame that applies at bytecode offset *i*.

A *verification type* specifies the type of either one or two locations, where a *location* is either a single local variable or a single operand stack entry. A verification type is represented by a discriminated union, `verification_type_info`, that consists of a one-byte tag, indicating which item of the union is in use, followed by zero or more bytes, giving more information about the tag.

```
union verification_type_info {
    Top_variable_info;
    Integer_variable_info;
    Float_variable_info;
    Long_variable_info;
    Double_variable_info;
    Null_variable_info;
    UninitializedThis_variable_info;
    Object_variable_info;
    Uninitialized_variable_info;
}
```

A verification type that specifies one location in the local variable array or in the operand stack is represented by the following items of the `verification_type_info` union:

- The `Top_variable_info` item indicates that the local variable has the verification type `top`.

```
Top_variable_info {
    u1 tag = ITEM_Top; /* 0 */
}
```

- The `Integer_variable_info` item indicates that the location has the verification type `int`.

```
Integer_variable_info {
    u1 tag = ITEM_Integer; /* 1 */
}
```

- The `Float_variable_info` item indicates that the location has the verification type `float`.

```
Float_variable_info {
    u1 tag = ITEM_Float; /* 2 */
}
```

- The `Null_variable_info` type indicates that the location has the verification type `null`.

```
Null_variable_info {
    u1 tag = ITEM_Null; /* 5 */
}
```

- The `UninitializedThis_variable_info` item indicates that the location has the verification type `uninitializedThis`.

```
UninitializedThis_variable_info {
    u1 tag = ITEM_UninitializedThis; /* 6 */
}
```

- The `Object_variable_info` item indicates that the location has the verification type which is the class represented by the `CONSTANT_Class_info` structure (§4.4.1) found in the `constant_pool` table at the index given by `cpool_index`.

```
Object_variable_info {
    u1 tag = ITEM_Object; /* 7 */
    u2 cpool_index;
}
```

- The `Uninitialized_variable_info` item indicates that the location has the verification type `uninitialized(Offset)`. The `Offset` item indicates the offset, in the code array of the `Code` attribute that contains this `StackMapTable` attribute, of the *new* instruction (*\$new*) that created the object being stored in the location.

```
Uninitialized_variable_info {
    u1 tag = ITEM_Uninitialized; /* 8 */
    u2 offset;
}
```

A verification type that specifies two locations in the local variable array or in the operand stack is represented by the following items of the `verification_type_info` union:

- The `Long_variable_info` item indicates that the first of two locations has the verification type `long`.

```
Long_variable_info {
    u1 tag = ITEM_Long; /* 4 */
}
```

- The `Double_variable_info` item indicates that the first of two locations has the verification type `double`.

```
Double_variable_info {
    u1 tag = ITEM_Double; /* 3 */
}
```

- The `Long_variable_info` and `Double_variable_info` items indicate the verification type of the second of two locations as follows:
 - If the first of the two locations is a local variable, then:
 - › It must not be the local variable with the highest index.
 - › The next higher numbered local variable has the verification type `top`.
 - If the first of the two locations is an operand stack entry, then:
 - › It must not be the topmost location of the operand stack.
 - › The next location closer to the top of the operand stack has the verification type `top`.

A stack map frame is represented by a discriminated union, `stack_map_frame`, which consists of a one-byte tag, indicating which item of the union is in use, followed by zero or more bytes, giving more information about the tag.

```
union stack_map_frame {
    same_frame;
    same_locals_1_stack_item_frame;
    same_locals_1_stack_item_frame_extended;
    chop_frame;
    same_frame_extended;
    append_frame;
    full_frame;
}
```

The tag indicates the *frame type* of the stack map frame:

- The frame type `same_frame` is represented by tags in the range [0-63]. This frame type indicates that the frame has exactly the same local variables as the previous

frame and that the operand stack is empty. The `offset_delta` value for the frame is the value of the tag item, `frame_type`.

```
same_frame {
    u1 frame_type = SAME; /* 0-63 */
}
```

- The frame type `same_locals_1_stack_item_frame` is represented by tags in the range [64, 127]. This frame type indicates that the frame has exactly the same local variables as the previous frame and that the operand stack has one entry. The `offset_delta` value for the frame is given by the formula `frame_type - 64`. The verification type of the one stack entry appears after the frame type.

```
same_locals_1_stack_item_frame {
    u1 frame_type = SAME_LOCALS_1_STACK_ITEM; /* 64-127 */
    verification_type_info stack[1];
}
```

- Tags in the range [128-246] are reserved for future use.
- The frame type `same_locals_1_stack_item_frame_extended` is represented by the tag 247. This frame type indicates that the frame has exactly the same local variables as the previous frame and that the operand stack has one entry. The `offset_delta` value for the frame is given explicitly, unlike in the frame type `same_locals_1_stack_item_frame`. The verification type of the one stack entry appears after `offset_delta`.

```
same_locals_1_stack_item_frame_extended {
    u1 frame_type = SAME_LOCALS_1_STACK_ITEM_EXTENDED; /* 247 */
    u2 offset_delta;
    verification_type_info stack[1];
}
```

- The frame type `chop_frame` is represented by tags in the range [248-250]. This frame type indicates that the frame has the same local variables as the previous frame except that the last k local variables are absent, and that the operand stack is empty. The value of k is given by the formula `251 - frame_type`. The `offset_delta` value for the frame is given explicitly.

```
chop_frame {
    u1 frame_type = CHOP; /* 248-250 */
    u2 offset_delta;
}
```

Assume the verification types of local variables in the previous frame are given by `locals`, an array structured as in the `full_frame` frame type. If `locals[M-1]` in the previous frame represented local variable x and `locals[M]` represented local variable y , then the effect of removing one local variable is

that `locals[M-1]` in the new frame represents local variable x and `locals[M]` is undefined.

It is an error if k is larger than the number of local variables in `locals` for the previous frame, that is, if the number of local variables in the new frame would be less than zero.

- The frame type `same_frame_extended` is represented by the tag 251. This frame type indicates that the frame has exactly the same local variables as the previous frame and that the operand stack is empty. The `offset_delta` value for the frame is given explicitly, unlike in the frame type `same_frame`.

```
same_frame_extended {
    u1 frame_type = SAME_FRAME_EXTENDED; /* 251 */
    u2 offset_delta;
}
```

- The frame type `append_frame` is represented by tags in the range [252-254]. This frame type indicates that the frame has the same locals as the previous frame except that k additional locals are defined, and that the operand stack is empty. The value of k is given by the formula `frame_type - 251`. The `offset_delta` value for the frame is given explicitly.

```
append_frame {
    u1 frame_type = APPEND; /* 252-254 */
    u2 offset_delta;
    verification_type_info locals[frame_type - 251];
}
```

The 0th entry in `locals` represents the verification type of the first additional local variable. If `locals[M]` represents local variable N , then:

- `locals[M+1]` represents local variable $N+1$ if `locals[M]` is one of `Top_variable_info`, `Integer_variable_info`, `Float_variable_info`, `Null_variable_info`, `UninitializedThis_variable_info`, `Object_variable_info`, or `Uninitialized_variable_info`; and
- `locals[M+1]` represents local variable $N+2$ if `locals[M]` is either `Long_variable_info` or `Double_variable_info`.

It is an error if, for any index i , `locals[i]` represents a local variable whose index is greater than the maximum number of local variables for the method.

- The frame type `full_frame` is represented by the tag 255. The `offset_delta` value for the frame is given explicitly.

```
full_frame {
    u1 frame_type = FULL_FRAME; /* 255 */
    u2 offset_delta;
    u2 number_of_locals;
    verification_type_info locals[number_of_locals];
    u2 number_of_stack_items;
    verification_type_info stack[number_of_stack_items];
}
```

The 0th entry in `locals` represents the verification type of local variable 0. If `locals[M]` represents local variable `N`, then:

- `locals[M+1]` represents local variable `N+1` if `locals[M]` is one of `Top_variable_info`, `Integer_variable_info`, `Float_variable_info`, `Null_variable_info`, `UninitializedThis_variable_info`, `Object_variable_info`, or `Uninitialized_variable_info`; and
- `locals[M+1]` represents local variable `N+2` if `locals[M]` is either `Long_variable_info` or `Double_variable_info`.

It is an error if, for any index `i`, `locals[i]` represents a local variable whose index is greater than the maximum number of local variables for the method.

The 0th entry in `stack` represents the verification type of the bottom of the operand stack, and subsequent entries in `stack` represent the verification types of stack entries closer to the top of the operand stack. We refer to the bottom of the operand stack as stack entry 0, and to subsequent entries of the operand stack as stack entry 1, 2, etc. If `stack[M]` represents stack entry `N`, then:

- `stack[M+1]` represents stack entry `N+1` if `stack[M]` is one of `Top_variable_info`, `Integer_variable_info`, `Float_variable_info`, `Null_variable_info`, `UninitializedThis_variable_info`, `Object_variable_info`, or `Uninitialized_variable_info`; and
- `stack[M+1]` represents stack entry `N+2` if `stack[M]` is either `Long_variable_info` or `Double_variable_info`.

It is an error if, for any index `i`, `stack[i]` represents a stack entry whose index is greater than the maximum operand stack size for the method.

4.7.5 The Exceptions Attribute

The Exceptions attribute is a variable-length attribute in the attributes table of a `method_info` structure (§4.6). The Exceptions attribute indicates which checked exceptions a method may throw.

There may be at most one Exceptions attribute in the attributes table of a `method_info` structure.

The Exceptions attribute has the following format:

```
Exceptions_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 number_of_exceptions;  
    u2 exception_index_table[number_of_exceptions];  
}
```

The items of the `Exceptions_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be the `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Exceptions".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`number_of_exceptions`

The value of the `number_of_exceptions` item indicates the number of entries in the `exception_index_table`.

`exception_index_table[]`

Each value in the `exception_index_table` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class type that this method is declared to throw.

A method should throw an exception only if at least one of the following three criteria is met:

- The exception is an instance of `RuntimeException` or one of its subclasses.
- The exception is an instance of `Error` or one of its subclasses.
- The exception is an instance of one of the exception classes specified in the `exception_index_table` just described, or one of their subclasses.

These requirements are not enforced in the Java Virtual Machine; they are enforced only at compile time.

4.7.6 The InnerClasses Attribute

The InnerClasses attribute is a variable-length attribute in the attributes table of a ClassFile structure (§4.1).

If the constant pool of a class or interface *C* contains at least one CONSTANT_Class_info entry (§4.4.1) which represents a class or interface that is not a member of a package, then there must be exactly one InnerClasses attribute in the attributes table of the ClassFile structure for *C*.

The InnerClasses attribute has the following format:

```
InnerClasses_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 number_of_classes;
    {    u2 inner_class_info_index;
        u2 outer_class_info_index;
        u2 inner_name_index;
        u2 inner_class_access_flags;
    } classes[number_of_classes];
}
```

The items of the InnerClasses_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "InnerClasses".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

number_of_classes

The value of the number_of_classes item indicates the number of entries in the classes array.

classes[]

Every CONSTANT_Class_info entry in the constant_pool table which represents a class or interface *C* that is not a package member must have exactly one corresponding entry in the classes array.

If a class or interface has members that are classes or interfaces, its `constant_pool` table (and hence its `InnerClasses` attribute) must refer to each such member (JLS §13.1), even if that member is not otherwise mentioned by the class.

In addition, the `constant_pool` table of every nested class and nested interface must refer to its enclosing class, so altogether, every nested class and nested interface will have `InnerClasses` information for each enclosing class and for each of its own nested classes and interfaces.

Each entry in the `classes` array contains the following four items:

`inner_class_info_index`

The value of the `inner_class_info_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure representing `C`.

`outer_class_info_index`

If `C` is not a member of a class or an interface - that is, if `C` is a top-level class or interface (JLS §7.6) or a local class (JLS §14.3) or an anonymous class (JLS §15.9.5) - then the value of the `outer_class_info_index` item must be zero.

Otherwise, the value of the `outer_class_info_index` item must be a valid index into the `constant_pool` table, and the entry at that index must be a `CONSTANT_Class_info` structure representing the class or interface of which `C` is a member. The value of the `outer_class_info_index` item must not equal the value of the `inner_class_info_index` item.

`inner_name_index`

If `C` is anonymous (JLS §15.9.5), the value of the `inner_name_index` item must be zero.

Otherwise, the value of the `inner_name_index` item must be a valid index into the `constant_pool` table, and the entry at that index must be a `CONSTANT_Utf8_info` structure that represents the original simple name of `C`, as given in the source code from which this class file was compiled.

`inner_class_access_flags`

The value of the `inner_class_access_flags` item is a mask of flags used to denote access permissions to and properties of class or interface `C` as declared in the source code from which this class file was compiled. It is used by a compiler to recover the original information when source code is not available. The flags are specified in Table 4.7.6-A.

Table 4.7.6-A. Nested class access and property flags

Flag Name	Value	Interpretation
ACC_PUBLIC	0x0001	Marked or implicitly <code>public</code> in source.
ACC_PRIVATE	0x0002	Marked <code>private</code> in source.
ACC_PROTECTED	0x0004	Marked <code>protected</code> in source.
ACC_STATIC	0x0008	Marked or implicitly <code>static</code> in source.
ACC_FINAL	0x0010	Marked or implicitly <code>final</code> in source.
ACC_INTERFACE	0x0200	Was an <code>interface</code> in source.
ACC_ABSTRACT	0x0400	Marked or implicitly <code>abstract</code> in source.
ACC_SYNTHETIC	0x1000	Declared synthetic; not present in the source code.
ACC_ANNOTATION	0x2000	Declared as an annotation interface.
ACC_ENUM	0x4000	Declared as an <code>enum</code> class.

All bits of the `inner_class_access_flags` item not assigned in Table 4.7.6-A are reserved for future use. They should be set to zero in generated class files and should be ignored by Java Virtual Machine implementations.

If a class file has a version number that is 51.0 or above, and has an `InnerClasses` attribute in its `attributes` table, then for all entries in the `classes` array of the `InnerClasses` attribute, the value of the `outer_class_info_index` item must be zero if the value of the `inner_name_index` item is zero.

Oracle's Java Virtual Machine implementation does not check the consistency of an `InnerClasses` attribute against a class file representing a class or interface referenced by the attribute.

4.7.7 The EnclosingMethod Attribute

The `EnclosingMethod` attribute is a fixed-length attribute in the `attributes` table of a `ClassFile` structure (§4.1). A class must have an `EnclosingMethod` attribute if and only if it represents a local class or an anonymous class (JLS §14.3, JLS §15.9.5).

There may be at most one `EnclosingMethod` attribute in the `attributes` table of a `ClassFile` structure.

The `EnclosingMethod` attribute has the following format:

```
EnclosingMethod_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 class_index;  
    u2 method_index;  
}
```

The items of the `EnclosingMethod_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "EnclosingMethod".

`attribute_length`

The value of the `attribute_length` item must be four.

`class_index`

The value of the `class_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing the innermost class that encloses the declaration of the current class.

`method_index`

If the current class is not immediately enclosed by a method or constructor, then the value of the `method_index` item must be zero.

In particular, `method_index` must be zero if the current class was immediately enclosed in source code by an instance initializer, static initializer, instance variable initializer, or class variable initializer. (The first two concern both local classes and anonymous classes, while the last two concern anonymous classes declared on the right hand side of a field assignment.)

Otherwise, the value of the `method_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_NameAndType_info` structure (§4.4.6) representing the name and type of a method in the class referenced by the `class_index` attribute above.

It is the responsibility of a Java compiler to ensure that the method identified via the `method_index` is indeed the closest lexically enclosing method of the class that contains this `EnclosingMethod` attribute.

4.7.8 The Synthetic Attribute

The Synthetic attribute is a fixed-length attribute in the attributes table of a `ClassFile`, `field_info`, or `method_info` structure (§4.1, §4.5, §4.6). A class member that does not appear in the source code must be marked using a Synthetic attribute, or else it must have its `ACC_SYNTHETIC` flag set. The only exceptions to this requirement are compiler-generated members which are not considered implementation artifacts, namely:

- an instance initialization method representing a default constructor of the Java programming language (§2.9.1)
- a class or interface initialization method (§2.9.2)
- the implicitly declared members of enum and record classes (JLS §8.9.3, JLS §8.10.3)

The Synthetic attribute was introduced in JDK 1.1 to support nested classes and interfaces.

It is a limitation of the class file format that only formal parameters and modules can be flagged as `ACC_MANDATED` (§4.7.24, §4.7.25) to indicate that, despite being compiler-generated, they are not considered implementation artifacts. There is no way to flag other compiler-generated constructs so that they too are not considered implementation artifacts (JLS §13.1). This limitation means that reflective APIs of the Java SE Platform may not accurately indicate the "mandated" status of such constructs.

The Synthetic attribute has the following format:

```
Synthetic_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
}
```

The items of the `Synthetic_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Synthetic".

`attribute_length`

The value of the `attribute_length` item must be zero.

4.7.9 The Signature Attribute

The Signature attribute is a fixed-length attribute in the attributes table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure (§4.1, §4.5, §4.6, §4.7.30). A Signature attribute stores a signature (§4.7.9.1) for a class, interface, constructor, method, field, or record component whose declaration in the Java programming language uses type variables or parameterized types. See *The Java Language Specification, Java SE 26 Edition* for details about these constructs.

There may be at most one Signature attribute in the attributes table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure.

The Signature attribute has the following format:

```
Signature_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 signature_index;  
}
```

The items of the `Signature_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Signature".

`attribute_length`

The value of the `attribute_length` item must be two.

`signature_index`

The value of the `signature_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a class signature if this Signature attribute is an attribute of a `ClassFile` structure; a method signature if this Signature attribute is an attribute of a `method_info` structure; or a field signature otherwise.

Oracle's Java Virtual Machine implementation does not check the well-formedness of Signature attributes during class loading or linking. Instead, Signature attributes are checked by methods of the Java SE Platform class libraries which expose generic signatures of classes, interfaces, constructors, methods, and fields. Examples include `getGenericSuperclass` in `Class` and `toGenericString` in `java.lang.reflect.Executable`.

4.7.9.1 Signatures

Signatures encode declarations written in the Java programming language that use types outside the type system of the Java Virtual Machine. They support reflection and debugging, as well as compilation when only class files are available.

A Java compiler must emit a signature for any class, interface, constructor, method, field, or record component whose declaration uses type variables or parameterized types. Specifically, a Java compiler must emit:

- A class signature for any class or interface declaration which is either generic, or has a parameterized type as a superclass or superinterface, or both.
- A method signature for any method or constructor declaration which is either generic, or has a type variable or parameterized type as the return type or a formal parameter type, or has a type variable in a throws clause, or any combination thereof.

If the throws clause of a method or constructor declaration does not involve type variables, then a compiler may treat the declaration as having no throws clause for the purpose of emitting a method signature.

- A field signature for any field, formal parameter, local variable, or record component declaration whose type uses a type variable or a parameterized type.

Signatures are specified using a grammar which follows the notation of §4.3.1. In addition to that notation:

- The syntax *[x]* on the right-hand side of a production denotes zero or one occurrences of *x*. That is, *x* is an *optional symbol*. The alternative which contains the optional symbol actually defines two alternatives: one that omits the optional symbol and one that includes it.
- A very long right-hand side may be continued on a second line by clearly indenting the second line.

The grammar includes the terminal symbol *Identifier* to denote the name of a type, field, method, formal parameter, local variable, or type variable, as generated by a Java compiler. Such a name must not contain any of the ASCII characters `. ; [ / < > :` (that is, the characters forbidden in method names (§4.2.2) and also colon) but may contain characters that must not appear in an identifier in the Java programming language (JLS §3.8).

Signatures rely on a hierarchy of nonterminals known as *type signatures*:

- A *Java type signature* represents either a reference type or a primitive type of the Java programming language.

JavaTypeSignature:
ReferenceTypeSignature
BaseType

The following production from §4.3.2 is repeated here for convenience:

BaseType:
(one of)
B C D F I J S Z

- A *reference type signature* represents a reference type of the Java programming language, that is, a class or interface type, a type variable, or an array type.

A *class type signature* represents a (possibly parameterized) class or interface type. A class type signature must be formulated such that it can be reliably

mapped to the binary name of the class it denotes, in internal form (§4.2.1), by erasing any type arguments and converting each . character to a \$ character.

A *type variable signature* represents a type variable.

An *array type signature* represents one dimension of an array type.

ReferenceTypeSignature:

ClassTypeSignature

TypeVariableSignature

ArrayTypeSignature

ClassTypeSignature:

⌞ [*PackageSpecifier*]

SimpleClassTypeSignature {*ClassTypeSignatureSuffix*} ;

PackageSpecifier:

Identifier / {*PackageSpecifier*}

SimpleClassTypeSignature:

Identifier [*TypeArguments*]

TypeArguments:

< *TypeArgument* {*TypeArgument*} >

TypeArgument:

[*WildcardIndicator*] *ReferenceTypeSignature*

*

WildcardIndicator:

+

-

ClassTypeSignatureSuffix:

. *SimpleClassTypeSignature*

TypeVariableSignature:

⋈ *Identifier* ;

ArrayTypeSignature:

[*JavaTypeSignature*

A *class signature* encodes type information about a (possibly generic) class or interface declaration. It describes any type parameters of the class or interface,

and lists its (possibly parameterized) direct superclass and direct superinterfaces, if any. A type parameter is described by its name, followed by any class bound and interface bounds.

ClassSignature:

[TypeParameters] SuperclassSignature {SuperinterfaceSignature}

TypeParameters:

< TypeParameter {TypeParameter} >

TypeParameter:

Identifier ClassBound {InterfaceBound}

ClassBound:

: [ReferenceTypeSignature]

InterfaceBound:

: ReferenceTypeSignature

SuperclassSignature:

ClassTypeSignature

SuperinterfaceSignature:

ClassTypeSignature

A *method signature* encodes type information about a (possibly generic) method declaration. It describes any type parameters of the method; the (possibly parameterized) types of any formal parameters; the (possibly parameterized) return type, if any; and the types of any exceptions declared in the method's throws clause.

MethodSignature:

[TypeParameters] ({JavaTypeSignature}) Result {ThrowsSignature}

Result:

JavaTypeSignature

VoidDescriptor

ThrowsSignature:

^ ClassTypeSignature

^ TypeVariableSignature

The following production from §4.3.3 is repeated here for convenience:

VoidDescriptor:

V

A method signature encoded by the `Signature` attribute may not correspond exactly to the method descriptor in the `method_info` structure (§4.3.3). In particular, there is no assurance that the number of formal parameter types in the method signature is the same as the number of parameter descriptors in the method descriptor. The numbers are the same for most methods, but certain constructors in the Java programming language have an implicitly declared parameter which a compiler represents with a parameter descriptor but may omit from the method signature. See the note in §4.7.18 for a similar situation involving parameter annotations.

A *field signature* encodes the (possibly parameterized) type of a field, formal parameter, local variable, or record component declaration.

FieldSignature:

ReferenceTypeSignature

4.7.10 The `SourceFile` Attribute

The `SourceFile` attribute is an optional fixed-length attribute in the attributes table of a `ClassFile` structure (§4.1).

There may be at most one `SourceFile` attribute in the attributes table of a `ClassFile` structure.

The `SourceFile` attribute has the following format:

```
SourceFile_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 sourcefile_index;
}
```

The items of the `SourceFile_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "SourceFile".

`attribute_length`

The value of the `attribute_length` item must be two.

sourcefile_index

The value of the `sourcefile_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing a string.

The string referenced by the `sourcefile_index` item will be interpreted as indicating the name of the source file from which this class file was compiled. It will not be interpreted as indicating the name of a directory containing the file or an absolute path name for the file; such platform-specific additional information must be supplied by the run-time interpreter or development tool at the time the file name is actually used.

4.7.11 The SourceDebugExtension Attribute

The `SourceDebugExtension` attribute is an optional attribute in the attributes table of a `ClassFile` structure (§4.1).

There may be at most one `SourceDebugExtension` attribute in the attributes table of a `ClassFile` structure.

The `SourceDebugExtension` attribute has the following format:

```
SourceDebugExtension_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u1 debug_extension[attribute_length];  
}
```

The items of the `SourceDebugExtension_attribute` structure are as follows:

attribute_name_index

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "SourceDebugExtension".

attribute_length

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

debug_extension[]

The `debug_extension` array holds extended debugging information which has no semantic effect on the Java Virtual Machine. The information is represented using a modified UTF-8 string (§4.4.7) with no terminating zero byte.

Note that the `debug_extension` array may denote a string longer than that which can be represented with an instance of class `String`.

4.7.12 The LineNumberTable Attribute

The LineNumberTable attribute is an optional variable-length attribute in the attributes table of a Code attribute (§4.7.3). It may be used by debuggers to determine which part of the code array corresponds to a given line number in the original source file.

If multiple LineNumberTable attributes are present in the attributes table of a Code attribute, then they may appear in any order.

There may be more than one LineNumberTable attribute *per line of a source file* in the attributes table of a Code attribute. That is, LineNumberTable attributes may together represent a given line of a source file, and need not be one-to-one with source lines.

The LineNumberTable attribute has the following format:

```
LineNumberTable_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 line_number_table_length;
    {    u2 start_pc;
        u2 line_number;
    } line_number_table[line_number_table_length];
}
```

The items of the LineNumberTable_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "LineNumberTable".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

line_number_table_length

The value of the line_number_table_length item indicates the number of entries in the line_number_table array.

line_number_table[]

Each entry in the line_number_table array indicates that the line number in the original source file changes at a given point in the code array. Each line_number_table entry must contain the following two items:

start_pc

The value of the `start_pc` item must be a valid index into the code array of this Code attribute. The item indicates the index into the code array at which the code for a new line in the original source file begins.

line_number

The value of the `line_number` item gives the corresponding line number in the original source file.

4.7.13 The `LocalVariableTable` Attribute

The `LocalVariableTable` attribute is an optional variable-length attribute in the attributes table of a Code attribute (§4.7.3). It may be used by debuggers to determine the value of a given local variable during the execution of a method.

If multiple `LocalVariableTable` attributes are present in the attributes table of a Code attribute, then they may appear in any order.

There may be no more than one `LocalVariableTable` attribute *per local variable* in the attributes table of a Code attribute.

The `LocalVariableTable` attribute has the following format:

```
LocalVariableTable_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 local_variable_table_length;
    {
        u2 start_pc;
        u2 length;
        u2 name_index;
        u2 descriptor_index;
        u2 index;
    } local_variable_table[local_variable_table_length];
}
```

The items of the `LocalVariableTable_attribute` structure are as follows:

attribute_name_index

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "LocalVariableTable".

attribute_length

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`local_variable_table_length`

The value of the `local_variable_table_length` item indicates the number of entries in the `local_variable_table` array.

`local_variable_table[]`

Each entry in the `local_variable_table` array indicates a range of code array offsets within which a local variable has a value, and indicates the index into the local variable array of the current frame at which that local variable can be found. Each entry must contain the following five items:

`start_pc, length`

The value of the `start_pc` item must be a valid index into the code array of this Code attribute and must be the index of the opcode of an instruction.

The value of `start_pc + length` must either be a valid index into the code array of this Code attribute and be the index of the opcode of an instruction, or it must be the first index beyond the end of that code array.

The `start_pc` and `length` items indicate that the given local variable has a value at indices into the code array in the interval `[start_pc, start_pc + length)`, that is, between `start_pc` inclusive and `start_pc + length` exclusive.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must contain a `CONSTANT_Utf8_info` structure representing a valid unqualified name denoting a local variable (§4.2.2).

`descriptor_index`

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must contain a `CONSTANT_Utf8_info` structure representing a field descriptor which encodes the type of a local variable in the source program (§4.3.2).

`index`

The value of the `index` item must be a valid index into the local variable array of the current frame. The given local variable is at `index` in the local variable array of the current frame.

If the given local variable is of type `double` or `long`, it occupies both `index` and `index + 1`.

4.7.14 The LocalVariableTypeTable Attribute

The LocalVariableTypeTable attribute is an optional variable-length attribute in the attributes table of a Code attribute (§4.7.3). It may be used by debuggers to determine the value of a given local variable during the execution of a method.

If multiple LocalVariableTypeTable attributes are present in the attributes table of a given Code attribute, then they may appear in any order.

There may be no more than one LocalVariableTypeTable attribute *per local variable* in the attributes table of a Code attribute.

The LocalVariableTypeTable attribute differs from the LocalVariableTable attribute (§4.7.13) in that it provides signature information rather than descriptor information. This difference is only significant for variables whose type uses a type variable or parameterized type. Such variables will appear in both tables, while variables of other types will appear only in LocalVariableTable.

The LocalVariableTypeTable attribute has the following format:

```
LocalVariableTypeTable_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 local_variable_type_table_length;
    {
        u2 start_pc;
        u2 length;
        u2 name_index;
        u2 signature_index;
        u2 index;
    } local_variable_type_table[local_variable_type_table_length];
}
```

The items of the LocalVariableTypeTable_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "LocalVariableTypeTable".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

local_variable_type_table_length

The value of the local_variable_type_table_length item indicates the number of entries in the local_variable_type_table array.

`local_variable_type_table[]`

Each entry in the `local_variable_type_table` array indicates a range of code array offsets within which a local variable has a value, and indicates the index into the local variable array of the current frame at which that local variable can be found. Each entry must contain the following five items:

`start_pc, length`

The value of the `start_pc` item must be a valid index into the code array of this Code attribute and must be the index of the opcode of an instruction.

The value of `start_pc + length` must either be a valid index into the code array of this Code attribute and be the index of the opcode of an instruction, or it must be the first index beyond the end of that code array.

The `start_pc` and `length` items indicate that the given local variable has a value at indices into the code array in the interval `[start_pc, start_pc + length)`, that is, between `start_pc` inclusive and `start_pc + length` exclusive.

`name_index`

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must contain a `CONSTANT_Utf8_info` structure representing a valid unqualified name denoting a local variable (§4.2.2).

`signature_index`

The value of the `signature_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must contain a `CONSTANT_Utf8_info` structure representing a field signature which encodes the type of a local variable in the source program (§4.7.9.1).

`index`

The value of the `index` item must be a valid index into the local variable array of the current frame. The given local variable is at `index` in the local variable array of the current frame.

If the given local variable is of type `double` or `long`, it occupies both `index` and `index + 1`.

4.7.15 The Deprecated Attribute

The Deprecated attribute is an optional fixed-length attribute in the attributes table of a `ClassFile`, `field_info`, or `method_info` structure (§4.1, §4.5, §4.6). A

class, interface, method, or field may be marked using a Deprecated attribute to indicate that the class, interface, method, or field has been superseded.

A run-time interpreter or tool that reads the class file format, such as a compiler, can use this marking to advise the user that a superseded class, interface, method, or field is being referred to. The presence of a Deprecated attribute does not alter the semantics of a class or interface.

The Deprecated attribute has the following format:

```
Deprecated_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
}
```

The items of the Deprecated_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "Deprecated".

attribute_length

The value of the attribute_length item must be zero.

4.7.16 The RuntimeVisibleAnnotations Attribute

The RuntimeVisibleAnnotations attribute is a variable-length attribute in the attributes table of a ClassFile, field_info, method_info, or record_component_info structure (§4.1, §4.5, §4.6, §4.7.30). The RuntimeVisibleAnnotations attribute stores run-time visible annotations on the declaration of the corresponding class, field, method, or record component.

There may be at most one RuntimeVisibleAnnotations attribute in the attributes table of a ClassFile, field_info, method_info, or record_component_info structure.

The RuntimeVisibleAnnotations attribute has the following format:

```
RuntimeVisibleAnnotations_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 num_annotations;
    annotation annotations[num_annotations];
}
```

The items of the `RuntimeVisibleAnnotations_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "RuntimeVisibleAnnotations".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`num_annotations`

The value of the `num_annotations` item gives the number of run-time visible annotations represented by the structure.

`annotations[]`

Each entry in the `annotations` table represents a single run-time visible annotation on a declaration. The annotation structure has the following format:

```
annotation {
    u2 type_index;
    u2 num_element_value_pairs;
    {   u2          element_name_index;
        element_value value;
    } element_value_pairs[num_element_value_pairs];
}
```

The items of the annotation structure are as follows:

`type_index`

The value of the `type_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a field descriptor (§4.3.2). The field descriptor denotes the type of the annotation represented by this annotation structure.

`num_element_value_pairs`

The value of the `num_element_value_pairs` item gives the number of element-value pairs of the annotation represented by this annotation structure.

`element_value_pairs[]`

Each value of the `element_value_pairs` table represents a single element-value pair in the annotation represented by this annotation structure. Each `element_value_pairs` entry contains the following two items:

`element_name_index`

The value of the `element_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7). The `constant_pool` entry denotes the name of the element of the element-value pair represented by this `element_value_pairs` entry.

In other words, the entry denotes an element of the annotation interface specified by `type_index`.

`value`

The value of the `value` item represents the value of the element-value pair represented by this `element_value_pairs` entry.

4.7.16.1 *The element_value structure*

The `element_value` structure is a discriminated union representing the value of an element-value pair. It has the following format:

```
element_value {
    u1 tag;
    union {
        u2 const_value_index;

        {    u2 type_name_index;
            u2 const_name_index;
        } enum_const_value;

        u2 class_info_index;

        annotation annotation_value;

        {    u2          num_values;
            element_value values[num_values];
        } array_value;
    } value;
}
```

The `tag` item uses a single ASCII character to indicate the type of the value of the element-value pair. This determines which item of the `value` union is in use. Table 4.7.16.1-A shows the valid characters for the `tag` item, the type indicated by

each character, and the item used in the value union for each character. The table's fourth column is used in the description below of one item of the value union.

Table 4.7.16.1-A. Interpretation of tag values as types

tag Item	Type	value Item	Constant Type
B	byte	const_value_index	CONSTANT_Integer
C	char	const_value_index	CONSTANT_Integer
D	double	const_value_index	CONSTANT_Double
F	float	const_value_index	CONSTANT_Float
I	int	const_value_index	CONSTANT_Integer
J	long	const_value_index	CONSTANT_Long
S	short	const_value_index	CONSTANT_Integer
Z	boolean	const_value_index	CONSTANT_Integer
s	String	const_value_index	CONSTANT_Utf8
e	Enum class	enum_const_value	<i>Not applicable</i>
c	Class	class_info_index	<i>Not applicable</i>
@	Annotation interface	annotation_value	<i>Not applicable</i>
[	Array type	array_value	<i>Not applicable</i>

The *value* item represents the value of an element-value pair. The item is a union, whose own items are as follows:

const_value_index

The `const_value_index` item denotes a constant of either a primitive type or the type `String` as the value of this element-value pair.

The value of the `const_value_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be of a type appropriate to the tag item, as specified in the fourth column of Table 4.7.16.1-A.

enum_const_value

The `enum_const_value` item denotes an enum constant as the value of this element-value pair.

The `enum_const_value` item consists of the following two items:

type_name_index

The value of the `type_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a field descriptor (§4.3.2). The `constant_pool` entry gives the internal form of the binary name of the type of the enum constant represented by this `element_value` structure (§4.2.1).

const_name_index

The value of the `const_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7). The `constant_pool` entry gives the simple name of the enum constant represented by this `element_value` structure.

class_info_index

The `class_info_index` item denotes a class literal as the value of this `element_value` pair.

The `class_info_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a return descriptor (§4.3.3). The return descriptor gives the type corresponding to the class literal represented by this `element_value` structure. Types correspond to class literals as follows:

- For a class literal `C.class`, where `C` is the name of a class, interface, or array type, the corresponding type is `C`. The return descriptor in the `constant_pool` will be a *ClassType* or an *ArrayType*.
- For a class literal `p.class`, where `p` is the name of a primitive type, the corresponding type is `p`. The return descriptor in the `constant_pool` will be a *BaseType* character.
- For a class literal `void.class`, the corresponding type is `void`. The return descriptor in the `constant_pool` will be `V`.

For example, the class literal `Object.class` corresponds to the type `Object`, so the `constant_pool` entry is `Ljava/lang/Object;`, whereas the class literal `int.class` corresponds to the type `int`, so the `constant_pool` entry is `I`.

The class literal `void.class` corresponds to `void`, so the `constant_pool` entry is `V`, whereas the class literal `Void.class` corresponds to the type `Void`, so the `constant_pool` entry is `Ljava/lang/Void;`.

`annotation_value`

The `annotation_value` item denotes a "nested" annotation as the value of this element-value pair.

The value of the `annotation_value` item is an annotation structure (§4.7.16) that gives the annotation represented by this `element_value` structure.

`array_value`

The `array_value` item denotes an array as the value of this element-value pair.

The `array_value` item consists of the following two items:

`num_values`

The value of the `num_values` item gives the number of elements in the array represented by this `element_value` structure.

`values[]`

Each value in the `values` table gives the corresponding element of the array represented by this `element_value` structure.

4.7.17 The `RuntimeInvisibleAnnotations` Attribute

The `RuntimeInvisibleAnnotations` attribute is a variable-length attribute in the `attributes` table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure (§4.1, §4.5, §4.6, §4.7.30). The `RuntimeInvisibleAnnotations` attribute stores run-time invisible annotations on the declaration of the corresponding class, method, field, or record component.

There may be at most one `RuntimeInvisibleAnnotations` attribute in the `attributes` table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure.

The `RuntimeInvisibleAnnotations` attribute is similar to the `RuntimeVisibleAnnotations` attribute (§4.7.16), except that the annotations represented by a `RuntimeInvisibleAnnotations` attribute must not be made available for return by reflective APIs, unless the Java Virtual Machine has been instructed to retain these annotations via some implementation-specific mechanism such as a command line flag. In the absence of such instructions, the Java Virtual Machine ignores this attribute.

The `RuntimeInvisibleAnnotations` attribute has the following format:

```
RuntimeInvisibleAnnotations_attribute {  
    u2      attribute_name_index;  
    u4      attribute_length;  
    u2      num_annotations;  
    annotation annotations[num_annotations];  
}
```

The items of the `RuntimeInvisibleAnnotations_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "RuntimeInvisibleAnnotations".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`num_annotations`

The value of the `num_annotations` item gives the number of run-time invisible annotations represented by the structure.

`annotations[]`

Each entry in the `annotations` table represents a single run-time invisible annotation on a declaration. The annotation structure is specified in §4.7.16.

4.7.18 The RuntimeVisibleParameterAnnotations Attribute

The `RuntimeVisibleParameterAnnotations` attribute is a variable-length attribute in the `attributes` table of the `method_info` structure (§4.6). The `RuntimeVisibleParameterAnnotations` attribute stores run-time visible annotations on the declarations of formal parameters of the corresponding method.

There may be at most one `RuntimeVisibleParameterAnnotations` attribute in the `attributes` table of a `method_info` structure.

The `RuntimeVisibleParameterAnnotations` attribute has the following format:

```

RuntimeVisibleParameterAnnotations_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u1 num_parameters;
    {    u2          num_annotations;
        annotation annotations[num_annotations];
    } parameter_annotations[num_parameters];
}

```

The items of the `RuntimeVisibleParameterAnnotations_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "RuntimeVisibleParameterAnnotations".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`num_parameters`

The value of the `num_parameters` item gives the number of run-time visible parameter annotations represented by this structure.

There is no assurance that this number is the same as the number of parameter descriptors in the method descriptor.

`parameter_annotations[]`

Each entry in the `parameter_annotations` table represents all of the run-time visible annotations on the declaration of a single formal parameter. Each `parameter_annotations` entry contains the following two items:

`num_annotations`

The value of the `num_annotations` item indicates the number of run-time visible annotations on the declaration of the formal parameter corresponding to the `parameter_annotations` entry.

`annotations[]`

Each entry in the `annotations` table represents a single run-time visible annotation on the declaration of the formal parameter corresponding to the `parameter_annotations` entry. The annotation structure is specified in §4.7.16.

The *i*'th entry in the `parameter_annotations` table may, but is not required to, correspond to the *i*'th parameter descriptor in the method descriptor (§4.3.3).

For example, a compiler may choose to create entries in the table corresponding only to those parameter descriptors which represent explicitly declared parameters in source code. In the Java programming language, a constructor of an inner class is specified to have an implicitly declared parameter before its explicitly declared parameters (JLS §8.8.1), so the corresponding `<init>` method in a class file has a parameter descriptor representing the implicitly declared parameter before any parameter descriptors representing explicitly declared parameters. If the first explicitly declared parameter is annotated in source code, then a compiler may create `parameter_annotations[0]` to store annotations corresponding to the *second* parameter descriptor.

4.7.19 The `RuntimeInvisibleParameterAnnotations` Attribute

The `RuntimeInvisibleParameterAnnotations` attribute is a variable-length attribute in the attributes table of a `method_info` structure (§4.6). The `RuntimeInvisibleParameterAnnotations` attribute stores run-time invisible annotations on the declarations of formal parameters of the corresponding method.

There may be at most one `RuntimeInvisibleParameterAnnotations` attribute in the attributes table of a `method_info` structure.

The `RuntimeInvisibleParameterAnnotations` attribute is similar to the `RuntimeVisibleParameterAnnotations` attribute (§4.7.18), except that the annotations represented by a `RuntimeInvisibleParameterAnnotations` attribute must not be made available for return by reflective APIs, unless the Java Virtual Machine has specifically been instructed to retain these annotations via some implementation-specific mechanism such as a command line flag. In the absence of such instructions, the Java Virtual Machine ignores this attribute.

The `RuntimeInvisibleParameterAnnotations` attribute has the following format:

```
RuntimeInvisibleParameterAnnotations_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u1 num_parameters;
    {   u2          num_annotations;
        annotation annotations[num_annotations];
    } parameter_annotations[num_parameters];
}
```

The items of the `RuntimeInvisibleParameterAnnotations_attribute` structure are as follows:

attribute_name_index

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "RuntimeInvisibleParameterAnnotations".

attribute_length

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

num_parameters

The value of the `num_parameters` item gives the number of run-time invisible parameter annotations represented by this structure.

There is no assurance that this number is the same as the number of parameter descriptors in the method descriptor.

parameter_annotations[]

Each entry in the `parameter_annotations` table represents all of the run-time invisible annotations on the declaration of a single formal parameter. Each `parameter_annotations` entry contains the following two items:

num_annotations

The value of the `num_annotations` item indicates the number of run-time invisible annotations on the declaration of the formal parameter corresponding to the `parameter_annotations` entry.

annotations[]

Each entry in the `annotations` table represents a single run-time invisible annotation on the declaration of the formal parameter corresponding to the `parameter_annotations` entry. The annotation structure is specified in §4.7.16.

The *i*'th entry in the `parameter_annotations` table may, but is not required to, correspond to the *i*'th parameter descriptor in the method descriptor (§4.3.3).

See the note in §4.7.18 for an example of when `parameter_annotations[0]` does not correspond to the first parameter descriptor in the method descriptor.

4.7.20 The RuntimeVisibleTypeAnnotations Attribute

The `RuntimeVisibleTypeAnnotations` attribute is an variable-length attribute in the `attributes` table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure, or `Code` attribute (§4.1, §4.5, §4.6, §4.7.30,

§4.7.3). The `RuntimeVisibleTypeAnnotations` attribute stores run-time visible annotations on types used in the declaration of the corresponding class, field, method, or record component, or in an expression in the corresponding method body. The `RuntimeVisibleTypeAnnotations` attribute also stores run-time visible annotations on type parameter declarations of generic classes, interfaces, methods, and constructors.

There may be at most one `RuntimeVisibleTypeAnnotations` attribute in the attributes table of a `ClassFile`, `field_info`, `method_info`, or `record_component_info` structure, or `Code` attribute.

An attributes table contains a `RuntimeVisibleTypeAnnotations` attribute only if types are annotated in kinds of declaration or expression that correspond to the parent structure or attribute of the attributes table.

For example, all annotations on types in the `implements` clause of a class declaration are recorded in the `RuntimeVisibleTypeAnnotations` attribute of the class's `ClassFile` structure. Meanwhile, all annotations on the type in a field declaration are recorded in the `RuntimeVisibleTypeAnnotations` attribute of the field's `field_info` structure.

The `RuntimeVisibleTypeAnnotations` attribute has the following format:

```
RuntimeVisibleTypeAnnotations_attribute {
    u2          attribute_name_index;
    u4          attribute_length;
    u2          num_annotations;
    type_annotation annotations[num_annotations];
}
```

The items of the `RuntimeVisibleTypeAnnotations_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing the string `"RuntimeVisibleTypeAnnotations"`.

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`num_annotations`

The value of the `num_annotations` item gives the number of run-time visible type annotations represented by the structure.

annotations[]

Each entry in the annotations table represents a single run-time visible annotation on a type used in a declaration or expression. The type_annotation structure has the following format:

```

type_annotation {
    u1 target_type;
    union {
        type_parameter_target;
        supertype_target;
        type_parameter_bound_target;
        empty_target;
        formal_parameter_target;
        throws_target;
        localvar_target;
        catch_target;
        offset_target;
        type_argument_target;
    } target_info;
    type_path target_path;
    u2      type_index;
    u2      num_element_value_pairs;
    {      u2      element_name_index;
        element_value value;
    } element_value_pairs[num_element_value_pairs];
}

```

The first three items - target_type, target_info, and target_path - specify the precise location of the annotated type. The last three items - type_index, num_element_value_pairs, and element_value_pairs[] - specify the annotation's own type and element-value pairs.

The items of the type_annotation structure are as follows:

target_type

The value of the target_type item denotes the kind of target on which the annotation appears. The various kinds of target correspond to the *type contexts* of the Java programming language where types are used in declarations and expressions (JLS §4.11).

The legal values of target_type are specified in Table 4.7.20-A and Table 4.7.20-B. Each value is a one-byte tag indicating which item of the target_info union follows the target_type item to give more information about the target.

The kinds of target in Table 4.7.20-A and Table 4.7.20-B correspond to the type contexts in JLS §4.11. Namely, target_type values 0x10-0x17 and 0x40-0x42

correspond to type contexts 1-11, while `target_type` values 0x43-0x4B correspond to type contexts 12-17.

The value of the `target_type` item determines whether the `type_annotation` structure appears in a `RuntimeVisibleTypeAnnotations` attribute in a `ClassFile` structure, a `field_info` structure, a `method_info` structure, or a `Code` attribute. Table 4.7.20-C gives the location of the `RuntimeVisibleTypeAnnotations` attribute for a `type_annotation` structure with each legal `target_type` value.

`target_info`

The value of the `target_info` item denotes precisely which type in a declaration or expression is annotated.

The items of the `target_info` union are specified in §4.7.20.1.

`target_path`

The value of the `target_path` item denotes precisely which part of the type indicated by `target_info` is annotated.

The format of the `type_path` structure is specified in §4.7.20.2.

`type_index`, `num_element_value_pairs`, `element_value_pairs[]`

The meaning of these items in the `type_annotation` structure is the same as their meaning in the `annotation` structure (§4.7.16).

Table 4.7.20-A. Interpretation of target_type values (Part 1)

Value	Kind of target	target_info item
0x00	type parameter declaration of generic class or interface	type_parameter_target
0x01	type parameter declaration of generic method or constructor	type_parameter_target
0x10	type in <code>extends</code> or <code>implements</code> clause of class declaration (including the direct superclass or direct superinterface of an anonymous class declaration), or in <code>extends</code> clause of interface declaration	supertype_target
0x11	type in bound of type parameter declaration of generic class or interface	type_parameter_bound_target
0x12	type in bound of type parameter declaration of generic method or constructor	type_parameter_bound_target
0x13	type in field or record component declaration	empty_target
0x14	return type of method, or type of newly constructed object	empty_target
0x15	receiver type of method or constructor	empty_target
0x16	type in formal parameter declaration of method, constructor, or lambda expression	formal_parameter_target
0x17	type in <code>throws</code> clause of method or constructor	throws_target

Table 4.7.20-B. Interpretation of target_type values (Part 2)

Value	Kind of target	target_info item
0x40	type in local variable declaration	localvar_target
0x41	type in resource variable declaration	localvar_target
0x42	type in exception parameter declaration	catch_target
0x43	type in <i>instanceof</i> expression	offset_target
0x44	type in <i>new</i> expression	offset_target
0x45	type in method reference expression using <i>::new</i>	offset_target
0x46	type in method reference expression using <i>::Identifier</i>	offset_target
0x47	type in cast expression	type_argument_target
0x48	type argument for generic constructor in <i>new</i> expression or explicit constructor invocation statement	type_argument_target
0x49	type argument for generic method in method invocation expression	type_argument_target
0x4A	type argument for generic constructor in method reference expression using <i>::new</i>	type_argument_target
0x4B	type argument for generic method in method reference expression using <i>::Identifier</i>	type_argument_target

Table 4.7.20-C. Location of enclosing attribute for `target_type` values

Value	Kind of target	Location
0x00	type parameter declaration of generic class or <code>ClassFile</code> interface	
0x01	type parameter declaration of generic method or <code>method_info</code> constructor	
0x10	type in <code>extends</code> clause of class or interface <code>ClassFile</code> declaration, or in <code>implements</code> clause of interface declaration	
0x11	type in bound of type parameter declaration of <code>ClassFile</code> generic class or interface	
0x12	type in bound of type parameter declaration of <code>method_info</code> generic method or constructor	
0x13	type in field or record component declaration	<code>field_info</code> , <code>record_component_info</code>
0x14	return type of method or constructor	<code>method_info</code>
0x15	receiver type of method or constructor	<code>method_info</code>
0x16	type in formal parameter declaration of method, <code>method_info</code> constructor, or lambda expression	
0x17	type in <code>throws</code> clause of method or constructor	<code>method_info</code>
0x40-0x4B	types in local variable declarations, resource variable declarations, exception parameter declarations, expressions	<code>Code</code>

4.7.20.1 *The `target_info` union*

The items of the `target_info` union (except for the first) specify precisely which type in a declaration or expression is annotated. The first item specifies not which type, but rather which declaration of a type parameter is annotated. The items are as follows:

- The `type_parameter_target` item indicates that an annotation appears on the declaration of the *i*'th type parameter of a generic class, generic interface, generic method, or generic constructor.

```
type_parameter_target {  
    u1 type_parameter_index;  
}
```

The value of the `type_parameter_index` item specifies which type parameter declaration is annotated. A `type_parameter_index` value of 0 specifies the first type parameter declaration.

- The `supertype_target` item indicates that an annotation appears on a type in the `extends` or `implements` clause of a class or interface declaration.

```
supertype_target {  
    u2 supertype_index;  
}
```

A `supertype_index` value of 65535 specifies that the annotation appears on the superclass in an `extends` clause of a class declaration.

Any other `supertype_index` value is an index into the `interfaces` array of the enclosing `ClassFile` structure, and specifies that the annotation appears on that superinterface in either the `implements` clause of a class declaration or the `extends` clause of an interface declaration.

- The `type_parameter_bound_target` item indicates that an annotation appears on the *i*'th bound of the *j*'th type parameter declaration of a generic class, interface, method, or constructor.

```
type_parameter_bound_target {  
    u1 type_parameter_index;  
    u1 bound_index;  
}
```

The value of the `type_parameter_index` item specifies which type parameter declaration has an annotated bound. A `type_parameter_index` value of 0 specifies the first type parameter declaration.

The value of the `bound_index` item specifies which bound of the type parameter declaration indicated by `type_parameter_index` is annotated. A `bound_index` value of 0 specifies the first bound of a type parameter declaration.

The `type_parameter_bound_target` item records that a bound is annotated, but does not record the type which constitutes the bound. The type may be found by inspecting the class signature or method signature stored in the appropriate `Signature` attribute.

- The `empty_target` item indicates that an annotation appears on either the type in a field declaration, the type in a record component declaration, the return type of a method, the type of a newly constructed object, or the receiver type of a method or constructor.

```
empty_target {  
}
```

Only one type appears in each of these locations, so there is no per-type information to represent in the `target_info` union.

- The `formal_parameter_target` item indicates that an annotation appears on the type in a formal parameter declaration of a method, constructor, or lambda expression.

```
formal_parameter_target {  
    u1 formal_parameter_index;  
}
```

The value of the `formal_parameter_index` item specifies which formal parameter declaration has an annotated type. A `formal_parameter_index` value of *i* may, but is not required to, correspond to the *i*'th parameter descriptor in the method descriptor (§4.3.3).

The `formal_parameter_target` item records that a formal parameter's type is annotated, but does not record the type itself. The type may be found by inspecting the method descriptor, although a `formal_parameter_index` value of 0 does not always indicate the first parameter descriptor in the method descriptor; see the note in §4.7.18 for a similar situation involving the `parameter_annotations` table.

- The `throws_target` item indicates that an annotation appears on the *i*'th type in the throws clause of a method or constructor declaration.

```
throws_target {  
    u2 throws_type_index;  
}
```

The value of the `throws_type_index` item is an index into the `exception_index_table` array of the `Exceptions` attribute of the `method_info` structure enclosing the `RuntimeVisibleTypeAnnotations` attribute.

- The `localvar_target` item indicates that an annotation appears on the type in a local variable declaration, including a variable declared as a resource in a try-with-resources statement.

```
localvar_target {  
    u2 table_length;  
    {  
        u2 start_pc;  
        u2 length;  
        u2 index;  
    } table[table_length];  
}
```

The value of the `table_length` item gives the number of entries in the `table` array. Each entry indicates a range of code array offsets within which a local variable has a value. It also indicates the index into the local variable array of the current frame at which that local variable can be found. Each entry contains the following three items:

`start_pc`, `length`

The given local variable has a value at indices into the code array in the interval `[start_pc, start_pc + length)`, that is, between `start_pc` inclusive and `start_pc + length` exclusive.

`index`

The given local variable must be at `index` in the local variable array of the current frame.

If the local variable at `index` is of type `double` or `long`, it occupies both `index` and `index + 1`.

A table is needed to fully specify the local variable whose type is annotated, because a single local variable may be represented with different local variable indices over multiple live ranges. The `start_pc`, `length`, and `index` items in each table entry specify the same information as a `LocalVariableTable` attribute.

The `localvar_target` item records that a local variable's type is annotated, but does not record the type itself. The type may be found by inspecting the appropriate `LocalVariableTable` attribute.

- The `catch_target` item indicates that an annotation appears on the *i*'th type in an exception parameter declaration.

```
catch_target {
    u2 exception_table_index;
}
```

The value of the `exception_table_index` item is an index into the `exception_table` array of the `Code` attribute enclosing the `RuntimeVisibleTypeAnnotations` attribute.

The possibility of more than one type in an exception parameter declaration arises from the multi-catch clause of the `try` statement, where the type of the exception parameter is a union of types (JLS §14.20). A compiler usually creates one `exception_table` entry for each type in the union, which allows the `catch_target` item to distinguish them. This preserves the correspondence between a type and its annotations.

- The `offset_target` item indicates that an annotation appears on either the type in an *instanceof* expression or a *new* expression, or the type before the `::` in a method reference expression.

```
offset_target {
    u2 offset;
}
```

The value of the `offset` item specifies the code array offset of either the bytecode instruction corresponding to the *instanceof* expression, the *new* bytecode instruction corresponding to the *new* expression, or the bytecode instruction corresponding to the method reference expression.

- The `type_argument_target` item indicates that an annotation appears either on the *i*'th type in a cast expression, or on the *i*'th type argument in the explicit type argument list for any of the following: a *new* expression, an explicit constructor invocation statement, a method invocation expression, or a method reference expression.

```
type_argument_target {
    u2 offset;
    u1 type_argument_index;
}
```

The value of the `offset` item specifies the code array offset of either the bytecode instruction corresponding to the cast expression, the *new* bytecode instruction corresponding to the *new* expression, the bytecode instruction corresponding to the explicit constructor invocation statement, the bytecode

instruction corresponding to the method invocation expression, or the bytecode instruction corresponding to the method reference expression.

For a cast expression, the value of the `type_argument_index` item specifies which type in the cast operator is annotated. A `type_argument_index` value of 0 specifies the first (or only) type in the cast operator.

The possibility of more than one type in a cast expression arises from a cast to an intersection type.

For an explicit type argument list, the value of the `type_argument_index` item specifies which type argument is annotated. A `type_argument_index` value of 0 specifies the first type argument.

4.7.20.2 *The type_path structure*

Wherever a type is used in a declaration or expression, the `type_path` structure identifies which part of the type is annotated. An annotation may appear on the type itself, but if the type is a reference type, then there are additional locations where an annotation may appear:

- If an array type `T[]` is used in a declaration or expression, then an annotation may appear on any component type of the array type, including the element type.
- If a nested type `T1.T2` is used in a declaration or expression, then an annotation may appear on the name of the innermost member type and any enclosing type for which a type annotation is admissible (JLS §9.7.4).
- If a parameterized type `T<A>` or `T<? extends A>` or `T<? super A>` is used in a declaration or expression, then an annotation may appear on any type argument or on the bound of any wildcard type argument.

For example, consider the different parts of `String[][]` that are annotated in:

```
@Foo String[][]    // Annotates the class type String
String @Foo [][]   // Annotates the array type String[][]
String[] @Foo []    // Annotates the array type String[]
```

or the different parts of the nested type `Outer.Middle.Inner` that are annotated in:

```
@Foo Outer.Middle.Inner
Outer.@Foo Middle.Inner
Outer.Middle.@Foo Inner
```

or the different parts of the parameterized types `Map<String, Object>` and `List<...>` that are annotated in:


```

@Foo Map<String,Object>
Map<@Foo String,Object>
Map<String,@Foo Object>

List<@Foo ? extends String>
List<? extends @Foo String>

```

The `type_path` structure has the following format:

```

type_path {
    u1 path_length;
    {    u1 type_path_kind;
        u1 type_argument_index;
    } path[path_length];
}

```

The value of the `path_length` item gives the number of entries in the `path` array:

- If the value of `path_length` is 0, and the type being annotated is a nested type, then the annotation applies to the outermost part of the type for which a type annotation is admissible.
- If the value of `path_length` is 0, and the type being annotated is not a nested type, then the annotation appears directly on the type itself.
- If the value of `path_length` is non-zero, then each entry in the `path` array represents an iterative, left-to-right step towards the precise location of the annotation in an array type, nested type, or parameterized type. (In an array type, the iteration visits the array type itself, then its component type, then the

component type of that component type, and so on, until the element type is reached.) Each entry contains the following two items:

`type_path_kind`

The legal values for the `type_path_kind` item are listed in Table 4.7.20.2-A.

Table 4.7.20.2-A. Interpretation of `type_path_kind` values

Value	Interpretation
0	Annotation is deeper in an array type
1	Annotation is deeper in a nested type
2	Annotation is on the bound of a wildcard type argument of a parameterized type
3	Annotation is on a type argument of a parameterized type

`type_argument_index`

If the value of the `type_path_kind` item is 0, 1, or 2, then the value of the `type_argument_index` item is 0.

If the value of the `type_path_kind` item is 3, then the value of the `type_argument_index` item specifies which type argument of a parameterized type is annotated, where 0 indicates the first type argument of a parameterized type.

Table 4.7.20.2-B. `type_path` structures for `@A Map<@B ? extends @C String, @D List<@E Object>>`

Annotation	path_length	path
@A	0	[]
@B	1	[{type_path_kind: 3; type_argument_index: 0}]
@C	2	[{type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 2; type_argument_index: 0}]
@D	1	[{type_path_kind: 3; type_argument_index: 1}]
@E	2	[{type_path_kind: 3; type_argument_index: 1}, {type_path_kind: 3; type_argument_index: 0}]

Table 4.7.20.2-C. type_path structures for @I String @F [] @G [] @H []

Annotation	path_length	path
@F	0	[]
@G	1	[{type_path_kind: 0; type_argument_index: 0}]
@H	2	[{type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]
@I	3	[{type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]

Table 4.7.20.2-D. type_path structures for @A List<@B Comparable<@F Object @C [] @D [] @E []>>

Annotation	path_length	path
@A	0	[]
@B	1	[{type_path_kind: 3; type_argument_index: 0}]
@C	2	[{type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}]
@D	3	[{type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]
@E	4	[{type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]
@F	5	[{type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]

Table 4.7.20.2-E. type_path structures for @A Outer . @B Middle . @C Inner

Assuming: <pre> class Outer { class Middle { class Inner {} } } </pre>		
Annotation	path_length	path
@A	0	[]
@B	1	[{type_path_kind: 1; type_argument_index: 0}]
@C	2	[{type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 1; type_argument_index: 0}]

Table 4.7.20.2-F. type_path structures for Outer . @A MiddleStatic . @B Inner

Assuming: <pre> class Outer { static class MiddleStatic { class Inner {} } } </pre>		
Annotation	path_length	path
@A	0	[]
@B	1	[{type_path_kind: 1; type_argument_index: 0}]
In the type Outer . MiddleStatic . Inner, type annotations on the simple name Outer are not admissible because the type name to its right, MiddleStatic, does not refer to an inner class of Outer.		

Table 4.7.20.2-G. type_path structures for Outer . MiddleStatic . @A InnerStatic

Assuming: <pre> class Outer { static class MiddleStatic { static class InnerStatic {} } } </pre>		
Annotation	path_length	path
@A	0	[]
In the type Outer . MiddleStatic . InnerStatic, type annotations on the simple name Outer are not admissible because the type name to its right, MiddleStatic, does not refer to an inner class of Outer. Similarly, type annotations on the simple name MiddleStatic are not admissible because the type name to its right, InnerStatic, does not refer to an inner class of MiddleStatic.		

Table 4.7.20.2-H. type_path structures for Outer . Middle<@A Foo . @B Bar> . Inner<@D String @C []>

Assuming: <pre> class Outer { class Middle<T> { class Inner<U> {} } } </pre>		
Annotation	path_length	path
@A	2	[{type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}]
@B	3	[{type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 1; type_argument_index: 0}]
@C	3	[{type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}]
@D	4	[{type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 1; type_argument_index: 0}, {type_path_kind: 3; type_argument_index: 0}, {type_path_kind: 0; type_argument_index: 0}]

4.7.21 The RuntimeInvisibleTypeAnnotations Attribute

The RuntimeInvisibleTypeAnnotations attribute is an variable-length attribute in the attributes table of a ClassFile, field_info, method_info, or record_component_info structure, or Code attribute (§4.1, §4.5, §4.6, §4.7.30, §4.7.3). The RuntimeInvisibleTypeAnnotations attribute stores run-time invisible annotations on types used in the corresponding declaration of a class, field, method, or record component, or in an expression in the corresponding method body. The RuntimeInvisibleTypeAnnotations attribute also stores annotations on type parameter declarations of generic classes, interfaces, methods, and constructors.

There may be at most one RuntimeInvisibleTypeAnnotations attribute in the attributes table of a ClassFile, field_info, method_info, or record_component_info structure, or Code attribute.

An attributes table contains a RuntimeInvisibleTypeAnnotations attribute only if types are annotated in kinds of declaration or expression that correspond to the parent structure or attribute of the attributes table.

The RuntimeInvisibleTypeAnnotations attribute has the following format:

```
RuntimeInvisibleTypeAnnotations_attribute {
    u2          attribute_name_index;
    u4          attribute_length;
    u2          num_annotations;
    type_annotation annotations[num_annotations];
}
```

The items of the RuntimeInvisibleTypeAnnotations_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure representing the string "RuntimeInvisibleTypeAnnotations".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

num_annotations

The value of the num_annotations item gives the number of run-time invisible type annotations represented by the structure.

annotations[]

Each entry in the annotations table represents a single run-time invisible annotation on a type used in a declaration or expression. The type_annotation structure is specified in §4.7.20.

4.7.22 The AnnotationDefault Attribute

The AnnotationDefault attribute is a variable-length attribute in the attributes table of certain method_info structures (§4.6), namely those representing elements of annotation interfaces (JLS §9.6.1). The AnnotationDefault attribute records the default value (JLS §9.6.2) for the element represented by the method_info structure.

There may be at most one AnnotationDefault attribute in the attributes table of a method_info structure which represents an element of an annotation interface.

The AnnotationDefault attribute has the following format:

```
AnnotationDefault_attribute {  
    u2          attribute_name_index;  
    u4          attribute_length;  
    element_value default_value;  
}
```

The items of the AnnotationDefault_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "AnnotationDefault".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

default_value

The default_value item represents the default value of the annotation interface element represented by the method_info structure enclosing this AnnotationDefault attribute.

4.7.23 The BootstrapMethods Attribute

The BootstrapMethods attribute is a variable-length attribute in the attributes table of a ClassFile structure (§4.1). The BootstrapMethods attribute records bootstrap methods used to produce dynamically-computed constants and dynamically-computed call sites (§4.4.10).

There must be exactly one BootstrapMethods attribute in the attributes table of a ClassFile structure if the constant_pool table of the ClassFile structure has at least one CONSTANT_Dynamic_info or CONSTANT_Invokedynamic_info entry.

There may be at most one BootstrapMethods attribute in the attributes table of a ClassFile structure.

The BootstrapMethods attribute has the following format:

```
BootstrapMethods_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 num_bootstrap_methods;
    {   u2 bootstrap_method_ref;
        u2 num_bootstrap_arguments;
        u2 bootstrap_arguments[num_bootstrap_arguments];
    } bootstrap_methods[num_bootstrap_methods];
}
```

The items of the BootstrapMethods_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "BootstrapMethods".

attribute_length

The value of the attribute_length item indicates the length of the attribute, excluding the initial six bytes.

num_bootstrap_methods

The value of the num_bootstrap_methods item determines the number of bootstrap method specifiers in the bootstrap_methods array.

bootstrap_methods[]

Each entry in the bootstrap_methods table contains an index to a CONSTANT_MethodHandle_info structure which specifies a bootstrap method, and a sequence (perhaps empty) of indexes to *static arguments* for the bootstrap method.

Each `bootstrap_methods` entry must contain the following three items:

`bootstrap_method_ref`

The value of the `bootstrap_method_ref` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_MethodHandle_info` structure (§4.4.8).

The method handle will be resolved during resolution of a dynamically-computed constant or call site (§5.4.3.6), and then invoked as if by invocation of `invokeWithArguments` in `java.lang.invoke.MethodHandle`. The method handle must be able to accept the array of arguments described in §5.4.3.6, or resolution will fail.

`num_bootstrap_arguments`

The value of the `num_bootstrap_arguments` item gives the number of items in the `bootstrap_arguments` array.

`bootstrap_arguments[]`

Each entry in the `bootstrap_arguments` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be loadable (§4.4).

4.7.24 The MethodParameters Attribute

The `MethodParameters` attribute is a variable-length attribute in the `attributes` table of a `method_info` structure (§4.6). A `MethodParameters` attribute records information about the formal parameters of a method, such as their names.

There may be at most one `MethodParameters` attribute in the `attributes` table of a `method_info` structure.

The `MethodParameters` attribute has the following format:

```
MethodParameters_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u1 parameters_count;
    {   u2 name_index;
        u2 access_flags;
    } parameters[parameters_count];
}
```

The items of the `MethodParameters_attribute` structure are as follows:

attribute_name_index

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "MethodParameters".

attribute_length

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

parameters_count

The value of the `parameters_count` item indicates the number of parameter descriptors in the method descriptor (§4.3.3) referenced by the `descriptor_index` of the attribute's enclosing `method_info` structure.

This is not a constraint which a Java Virtual Machine implementation must enforce during format checking (§4.8). The task of matching parameter descriptors in a method descriptor against the items in the `parameters` array below is done by the reflection libraries of the Java SE Platform.

parameters[]

Each entry in the `parameters` array contains the following pair of items:

name_index

The value of the `name_index` item must either be zero or a valid index into the `constant_pool` table.

If the value of the `name_index` item is zero, then this `parameters` element indicates a formal parameter with no name.

If the value of the `name_index` item is nonzero, the `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing a valid unqualified name denoting a formal parameter (§4.2.2).

access_flags

The value of the `access_flags` item is as follows:

`0x0010 (ACC_FINAL)`

Indicates that the formal parameter was declared `final`.

`0x1000 (ACC_SYNTHETIC)`

Indicates that the formal parameter was not explicitly or implicitly declared in source code, according to the specification of the language in which the source code was written (JLS §13.1). (The formal

parameter is an implementation artifact of the compiler which produced this class file.)

0x8000 (ACC_MANDATED)

Indicates that the formal parameter was implicitly declared in source code, according to the specification of the language in which the source code was written (JLS §13.1). (The formal parameter is mandated by a language specification, so all compilers for the language must emit it.)

The *i*'th entry in the `parameters` array corresponds to the *i*'th parameter descriptor in the enclosing method's descriptor. (The `parameters_count` item is one byte because a method descriptor is limited to 255 parameters.) Effectively, this means the `parameters` array stores information for all the parameters of the method. One could imagine other schemes, where entries in the `parameters` array specify their corresponding parameter descriptors, but it would unduly complicate the `MethodParameters` attribute.

The *i*'th entry in the `parameters` array may or may not correspond to the *i*'th type in the enclosing method's `Signature` attribute (if present), or to the *i*'th annotation in the enclosing method's parameter annotations.

4.7.25 The Module Attribute

The `Module` attribute is a variable-length attribute in the `attributes` table of a `ClassFile` structure (§4.1). The `Module` attribute indicates the modules required by a module; the packages exported and opened by a module; and the services used and provided by a module.

There may be at most one `Module` attribute in the `attributes` table of a `ClassFile` structure.

The `Module` attribute has the following format:

```

Module_attribute {
    u2 attribute_name_index;
    u4 attribute_length;

    u2 module_name_index;
    u2 module_flags;
    u2 module_version_index;

    u2 requires_count;
    {   u2 requires_index;
        u2 requires_flags;
        u2 requires_version_index;
    } requires[requires_count];

    u2 exports_count;
    {   u2 exports_index;
        u2 exports_flags;
        u2 exports_to_count;
        u2 exports_to_index[exports_to_count];
    } exports[exports_count];

    u2 opens_count;
    {   u2 opens_index;
        u2 opens_flags;
        u2 opens_to_count;
        u2 opens_to_index[opens_to_count];
    } opens[opens_count];

    u2 uses_count;
    u2 uses_index[uses_count];

    u2 provides_count;
    {   u2 provides_index;
        u2 provides_with_count;
        u2 provides_with_index[provides_with_count];
    } provides[provides_count];
}

```

The items of the `Module_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Module".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

module_name_index

The value of the `module_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Module_info` structure (§4.4.11) denoting the current module.

module_flags

The value of the `module_flags` item is as follows:

`0x0020` (`ACC_OPEN`)

Indicates that this module is open.

`0x1000` (`ACC_SYNTHETIC`)

Indicates that this module was not explicitly or implicitly declared.

`0x8000` (`ACC_MANDATED`)

Indicates that this module was implicitly declared.

module_version_index

The value of the `module_version_index` item must be either zero or a valid index into the `constant_pool` table. If the value of the item is zero, then no version information about the current module is present. If the value of the item is nonzero, then the `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing the version of the current module.

requires_count

The value of the `requires_count` item indicates the number of entries in the `requires` table.

If the current module is `java.base`, then `requires_count` must be zero.

If the current module is not `java.base`, then `requires_count` must be at least one.

requires[]

Each entry in the `requires` table specifies a dependence of the current module. The items in each entry are as follows:

requires_index

The value of the `requires_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Module_info` structure denoting a module on which the current module depends.

At most one entry in the `requires` table may specify a module of a given name with its `requires_index` item.

`requires_flags`

The value of the `requires_flags` item is as follows:

`0x0020 (ACC_TRANSITIVE)`

Indicates that any module which depends on the current module, implicitly declares a dependence on the module indicated by this entry.

`0x0040 (ACC_STATIC_PHASE)`

Indicates that this dependence is mandatory in the static phase, i.e., at compile time, but is optional in the dynamic phase, i.e., at run time.

`0x1000 (ACC_SYNTHETIC)`

Indicates that this dependence was not explicitly or implicitly declared in the source of the module declaration.

`0x8000 (ACC_MANDATED)`

Indicates that this dependence was implicitly declared in the source of the module declaration.

`requires_version_index`

The value of the `requires_version_index` item must be either zero or a valid index into the `constant_pool` table. If the value of the item is zero, then no version information about the dependence is present. If the value of the item is nonzero, then the `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure representing the version of the module specified by `requires_index`.

Unless the current module is `java.base`, exactly one entry in the `requires` table must have all of the following:

- A `requires_index` item that indicates `java.base`.
- A `requires_flags` item that has the `ACC_SYNTHETIC` flag not set. (The `ACC_MANDATED` and `ACC_TRANSITIVE` flags may be set.)
- If the class file version number is 54.0 or above, a `requires_flags` item that has the `ACC_STATIC_PHASE` flag not set.

`exports_count`

The value of the `exports_count` item indicates the number of entries in the `exports` table.

`exports[]`

Each entry in the `exports` table specifies a package exported by the current module, such that public and protected types in the package, and their

public and protected members, may be accessed from outside the current module, possibly from a limited set of "friend" modules.

The items in each entry are as follows:

`exports_index`

The value of the `exports_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Package_info` structure (§4.4.12) representing a package exported by the current module.

At most one entry in the `exports` table may specify a package of a given name with its `exports_index` item.

`exports_flags`

The value of the `exports_flags` item is as follows:

`0x1000 (ACC_SYNTHETIC)`

Indicates that this export was not explicitly or implicitly declared in the source of the module declaration.

`0x8000 (ACC_MANDATED)`

Indicates that this export was implicitly declared in the source of the module declaration.

`exports_to_count`

The value of the `exports_to_count` indicates the number of entries in the `exports_to_index` table.

If `exports_to_count` is zero, then this package is exported by the current module in an *unqualified* fashion; code in any other module may access the types and members in the package.

If `exports_to_count` is nonzero, then this package is exported by the current module in a *qualified* fashion; only code in the modules listed in the `exports_to_index` table may access the types and members in the package.

`exports_to_index[]`

The value of each entry in the `exports_to_index` table must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Module_info` structure denoting a module whose code can access the types and members in this exported package.

For each entry in the `exports` table, at most one entry in its `exports_to_index` table may specify a module of a given name.

`opens_count`

The value of the `opens_count` item indicates the number of entries in the `opens` table.

`opens_count` must be zero if the current module is open.

`opens[]`

Each entry in the `opens` table specifies a package opened by the current module, such that all types in the package, and all their members, may be accessed from outside the current module via the reflection libraries of the Java SE Platform, possibly from a limited set of "friend" modules.

The items in each entry are as follows:

`opens_index`

The value of the `opens_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Package_info` structure representing a package opened by the current module.

At most one entry in the `opens` table may specify a package of a given name with its `opens_index` item.

`opens_flags`

The value of the `opens_flags` item is as follows:

`0x1000 (ACC_SYNTHETIC)`

Indicates that this opening was not explicitly or implicitly declared in the source of the module declaration.

`0x8000 (ACC_MANDATED)`

Indicates that this opening was implicitly declared in the source of the module declaration.

`opens_to_count`

The value of the `opens_to_count` indicates the number of entries in the `opens_to_index` table.

If `opens_to_count` is zero, then this package is opened by the current module in an *unqualified* fashion; code in any other module may reflectively access the types and members in the package.

If `opens_to_count` is nonzero, then this package is opened by the current module in a *qualified* fashion; only code in the modules listed in the `opens_to_index` table may reflectively access the types and members in the package.

`opens_to_index[]`

The value of each entry in the `opens_to_index` table must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Module_info` structure denoting a module whose code can access the types and members in this opened package.

For each entry in the `opens` table, at most one entry in its `opens_to_index` table may specify a module of a given name.

`uses_count`

The value of the `uses_count` item indicates the number of entries in the `uses_index` table.

`uses_index[]`

The value of each entry in the `uses_index` table must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a service interface which the current module may discover via `java.util.ServiceLoader`.

At most one entry in the `uses_index` table may specify a service interface of a given name.

`provides_count`

The value of the `provides_count` item indicates the number of entries in the `provides` table.

`provides[]`

Each entry in the `provides` table represents a service implementation for a given service interface.

The items in each entry are as follows:

`provides_index`

The value of the `provides_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure representing a service interface for which the current module provides a service implementation.

At most one entry in the `provides` table may specify a service interface of a given name with its `provides_index` item.

`provides_with_count`

The value of the `provides_with_count` indicates the number of entries in the `provides_with_index` table.

`provides_with_count` must be nonzero.

`provides_with_index[]`

The value of each entry in the `provides_with_index` table must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure representing a service implementation for the service interface specified by `provides_index`.

For each entry in the `provides` table, at most one entry in its `provides_with_index` table may specify a service implementation of a given name.

4.7.26 The `ModulePackages` Attribute

The `ModulePackages` attribute is a variable-length attribute in the `attributes` table of a `ClassFile` structure (§4.1). The `ModulePackages` attribute indicates all the packages of a module that are exported or opened by the `Module` attribute, as well as all the packages of the service implementations recorded in the `Module` attribute. The `ModulePackages` attribute may also indicate packages in the module that are neither exported nor opened nor contain service implementations.

There may be at most one `ModulePackages` attribute in the `attributes` table of a `ClassFile` structure.

The `ModulePackages` attribute has the following format:

```
ModulePackages_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 package_count;
    u2 package_index[package_count];
}
```

The items of the `ModulePackages_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string `"ModulePackages"`.

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

package_count

The value of the package_count item indicates the number of entries in the package_index table.

package_index[]

The value of each entry in the package_index table must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Package_info structure (§4.4.12) representing a package in the current module.

At most one entry in the package_index table may specify a package of a given name.

4.7.27 The ModuleMainClass Attribute

The ModuleMainClass attribute is a fixed-length attribute in the attributes table of a ClassFile structure (§4.1). The ModuleMainClass attribute indicates the main class of a module.

There may be at most one ModuleMainClass attribute in the attributes table of a ClassFile structure.

The ModuleMainClass attribute has the following format:

```
ModuleMainClass_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 main_class_index;  
}
```

The items of the ModuleMainClass_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "ModuleMainClass".

attribute_length

The value of the attribute_length item must be two.

main_class_index

The value of the main_class_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a

CONSTANT_Class_info structure (§4.4.1) representing the main class of the current module.

4.7.28 The NestHost Attribute

The NestHost attribute is a fixed-length attribute in the attributes table of a ClassFile structure. The NestHost attribute records the nest host of the nest to which the current class or interface claims to belong (§5.4.4).

There may be at most one NestHost attribute in the attributes table of a ClassFile structure.

The NestHost attribute has the following format:

```
NestHost_attribute {  
    u2 attribute_name_index;  
    u4 attribute_length;  
    u2 host_class_index;  
}
```

The items of the NestHost_attribute structure are as follows:

attribute_name_index

The value of the attribute_name_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Utf8_info structure (§4.4.7) representing the string "NestHost".

attribute_length

The value of the attribute_length item must be two.

host_class_index

The value of the host_class_index item must be a valid index into the constant_pool table. The constant_pool entry at that index must be a CONSTANT_Class_info structure (§4.4.1) representing a class or interface which is the nest host for the current class or interface.

If the nest host cannot be loaded, or is not in the same run-time package as the current class or interface, or does not authorize nest membership for the current class or interface, then an error may occur during access control (§5.4.4).

4.7.29 The NestMembers Attribute

The NestMembers attribute is a variable-length attribute in the attributes table of a ClassFile structure (§4.1). The NestMembers attribute records the classes and

interfaces that are authorized to claim membership in the nest hosted by the current class or interface (§5.4.4).

There may be at most one `NestMembers` attribute in the attributes table of a `ClassFile` structure.

The attributes table of a `ClassFile` structure must not contain both a `NestMembers` attribute and a `NestHost` attribute.

This rule prevents a nest host from claiming membership in a different nest. It is implicitly a member of the nest that it hosts.

The `NestMembers` attribute has the following format:

```
NestMembers_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 number_of_classes;
    u2 classes[number_of_classes];
}
```

The items of the `NestMembers_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "NestMembers".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`number_of_classes`

The value of the `number_of_classes` item indicates the number of entries in the `classes` array.

`classes[]`

Each value in the `classes` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class or interface which is a member of the nest hosted by the current class or interface.

The `classes` array is consulted by access control (§5.4.4). It should consist of references to other classes and interfaces that are in the same run-time package and have `NestHost` attributes which reference the current class or interface. Array items that do not meet these criteria are ignored by access control.

4.7.30 The Record Attribute

The Record attribute is a variable-length attribute in the attributes table of a `ClassFile` structure (§4.1). The Record attribute indicates that the current class is a record class (JLS §8.10), and stores information about the record components of the record class (JLS §8.10.1).

There may be at most one Record attribute in the attributes table of a `ClassFile` structure.

The Record attribute has the following format:

```
Record_attribute {
    u2          attribute_name_index;
    u4          attribute_length;
    u2          components_count;
    record_component_info components[components_count];
}
```

The items of the `Record_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "Record".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`components_count`

The value of the `components_count` item indicates the number of entries in the components table.

`components[]`

Each entry in the components table specifies a record component of the current class, in the order the record components were declared. The `record_component_info` structure has the following format:

```
record_component_info {
    u2          name_index;
    u2          descriptor_index;
    u2          attributes_count;
    attribute_info attributes[attributes_count];
}
```

The items of the `record_component_info` structure are as follows:

name_index

The value of the `name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a valid unqualified name denoting the record component (§4.2.2).

descriptor_index

The value of the `descriptor_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing a field descriptor which encodes the type of the record component (§4.3.2).

attributes_count

The value of the `attributes_count` item indicates the number of additional attributes of this record component.

attributes[]

Each value of the `attributes` table must be an `attribute_info` structure (§4.7).

A record component can have any number of optional attributes associated with it.

The attributes defined by this specification as appearing in the `attributes` table of a `record_component_info` structure are listed in Table 4.7-C.

The rules concerning attributes defined to appear in the `attributes` table of a `record_component_info` structure are given in §4.7.

The rules concerning non-predefined attributes in the `attributes` table of a `record_component_info` structure are given in §4.7.1.

4.7.31 The PermittedSubclasses Attribute

The `PermittedSubclasses` attribute is a variable-length attribute in the `attributes` table of a `ClassFile` structure (§4.1). The `PermittedSubclasses` attribute records the classes and interfaces that are authorized to directly extend or implement the current class or interface (§5.3.5).

The Java programming language uses the modifier `sealed` to indicate a class or interface that limits its direct subclasses or direct subinterfaces. One might suppose that this modifier would correspond to an `ACC_SEALED` flag in a `class` file, since the related modifier `final` corresponds to the `ACC_FINAL` flag. In fact, a `sealed` class or interface is indicated in a `class` file by the presence of the `PermittedSubclasses` attribute.

There may be at most one `PermittedSubclasses` attribute in the `attributes` table of a `ClassFile` structure whose `access_flags` item does not have the `ACC_FINAL` flag set.

There must be no `PermittedSubclasses` attribute in the `attributes` table of a `ClassFile` structure whose `access_flags` item has the `ACC_FINAL` flag set.

`sealed` is distinct from `final`: a `sealed` class has a list of authorized subclasses, which a `final` class has no subclasses. Thus, a `ClassFile` structure may have a `PermittedSubclasses` attribute, or have its `ACC_FINAL` flag set, but not both.

The `PermittedSubclasses` attribute has the following format:

```
PermittedSubclasses_attribute {
    u2 attribute_name_index;
    u4 attribute_length;
    u2 number_of_classes;
    u2 classes[number_of_classes];
}
```

The items of the `PermittedSubclasses_attribute` structure are as follows:

`attribute_name_index`

The value of the `attribute_name_index` item must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Utf8_info` structure (§4.4.7) representing the string "`PermittedSubclasses`".

`attribute_length`

The value of the `attribute_length` item indicates the length of the attribute, excluding the initial six bytes.

`number_of_classes`

The value of the `number_of_classes` item indicates the number of entries in the `classes` array.

`classes[]`

Each value in the `classes` array must be a valid index into the `constant_pool` table. The `constant_pool` entry at that index must be a `CONSTANT_Class_info` structure (§4.4.1) representing a class or interface which is authorized to directly extend or implement the current class or interface.

The `classes` array is consulted when a class or interface is created that attempts to directly extend or implement the current class or interface (§5.3.5). Array items that represent classes or interfaces which do not attempt to directly extend or implement the current class or interface are ignored.

4.8 Format Checking

When a prospective class file is loaded by the Java Virtual Machine (§5.3), the Java Virtual Machine first ensures that the file has the basic format of a class file (§4.1). This process is known as *format checking*. The checks are as follows:

- The first four bytes must contain the right magic number.
- All predefined attributes (§4.7) must be of the proper length, except for StackMapTable, RuntimeVisibleAnnotations, RuntimeInvisibleAnnotations, RuntimeVisibleParameterAnnotations, RuntimeInvisibleParameterAnnotations, RuntimeVisibleTypeAnnotations, RuntimeInvisibleTypeAnnotations, and AnnotationDefault.
- The class file must not be truncated or have extra bytes at the end.
- The constant pool must satisfy the constraints documented throughout §4.4.

For example, each `CONSTANT_Class_info` structure in the constant pool must contain in its `name_index` item a valid constant pool index for a `CONSTANT_Utf8_info` structure.

- All field references and method references in the constant pool must have valid names, valid classes, and valid descriptors (§4.3).

Format checking does not ensure that the given field or method actually exists in the given class, nor that the descriptors given refer to real classes. Format checking ensures only that these items are well formed. More detailed checking is performed when the bytecodes themselves are verified, and during resolution.

These checks for basic class file integrity are necessary for any interpretation of the class file contents. Format checking is distinct from bytecode verification, although historically they have been confused because both are a form of integrity check.

4.9 Constraints on Java Virtual Machine Code

The code for a method, instance initialization method (§2.9.1), or class or interface initialization method (§2.9.2) is stored in the code array of the `Code` attribute of a `method_info` structure of a class file (§4.7.3). This section describes the constraints associated with the contents of the `Code_attribute` structure.

4.9.1 Static Constraints

The *static constraints* on a `class` file are those defining the well-formedness of the file. These constraints have been given in the previous sections, except for static constraints on the code in the `class` file. The static constraints on the code in a `class` file specify how Java Virtual Machine instructions must be laid out in the code array and what the operands of individual instructions must be.

The static constraints on the instructions in the code array are as follows:

- Only instances of the instructions documented in §6.5 may appear in the code array. Instances of instructions using the reserved opcodes (§6.2) or any opcodes not documented in this specification must not appear in the code array.

If the `class` file version number is 51.0 or above, then instances of instructions using the *jsr*, *jsr_w*, or *ret* opcodes must not appear in the code array.

- The opcode of the first instruction in the code array begins at index 0.
- For each instruction in the code array except the last, the index of the opcode of the next instruction equals the index of the opcode of the current instruction plus the length of that instruction, including all its operands.

The *wide* instruction is treated like any other instruction for these purposes; the opcode specifying the operation that a *wide* instruction is to modify is treated as one of the operands of that *wide* instruction. That opcode must never be directly reachable by the computation.

- The last byte of the last instruction in the code array must be the byte at index `code_length - 1`.

The static constraints on the operands of instructions in the code array are as follows:

- The target of each jump and branch instruction (*jsr*, *jsr_w*, *goto*, *goto_w*, *ifeq*, *ifne*, *ifle*, *iflt*, *ifge*, *ifgt*, *ifnull*, *ifnonnull*, *if_icmpeq*, *if_icmpne*, *if_icmple*, *if_icmplt*, *if_icmpge*, *if_icmpgt*, *if_acmpeq*, *if_acmpne*) must be the opcode of an instruction within this method.

The target of a jump or branch instruction must never be the opcode used to specify the operation to be modified by a *wide* instruction; a jump or branch target may be the *wide* instruction itself.

- Each target, including the default, of each *tableswitch* instruction must be the opcode of an instruction within this method.

Each *tableswitch* instruction must have a number of entries in its jump table that is consistent with the value of its *low* and *high* jump table operands, and its *low* value must be less than or equal to its *high* value.

No target of a *tableswitch* instruction may be the opcode used to specify the operation to be modified by a *wide* instruction; a *tableswitch* target may be a *wide* instruction itself.

- Each target, including the default, of each *lookupswitch* instruction must be the opcode of an instruction within this method.

Each *lookupswitch* instruction must have a number of *match-offset* pairs that is consistent with the value of its *npairs* operand. The *match-offset* pairs must be sorted in increasing numerical order by signed match value.

No target of a *lookupswitch* instruction may be the opcode used to specify the operation to be modified by a *wide* instruction; a *lookupswitch* target may be a *wide* instruction itself.

- The operands of each *ldc* instruction and each *ldc_w* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be loadable (§4.4), and not any of the following:

- An entry of kind `CONSTANT_Long` or `CONSTANT_Double`.
- An entry of kind `CONSTANT_Dynamic` that references a `CONSTANT_NameAndType_info` structure which indicates a descriptor of `J` (denoting long) or `D` (denoting double).

- The operands of each *ldc2_w* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be loadable, and in particular one of the following:

- An entry of kind `CONSTANT_Long` or `CONSTANT_Double`.
- An entry of kind `CONSTANT_Dynamic` that references a `CONSTANT_NameAndType_info` structure which indicates a descriptor of `J` (denoting long) or `D` (denoting double).

The subsequent constant pool index must also be a valid index into the constant pool, and the constant pool entry at that index must not be used.

- The operands of each *getfield*, *putfield*, *getstatic*, and *putstatic* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be of kind `CONSTANT_Fieldref`.

- The *indexbyte* operands of each *invokevirtual* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be of kind `CONSTANT_Methodref`.
- The *indexbyte* operands of each *invokespecial* and *invokestatic* instruction must represent a valid index into the `constant_pool` table. If the class file version number is less than 52.0, the constant pool entry referenced by that index must be of kind `CONSTANT_Methodref`; if the class file version number is 52.0 or above, the constant pool entry referenced by that index must be of kind `CONSTANT_Methodref` or `CONSTANT_InterfaceMethodref`.
- The *indexbyte* operands of each *invokeinterface* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be of kind `CONSTANT_InterfaceMethodref`.

The value of the *count* operand of each *invokeinterface* instruction must reflect the number of local variables necessary to store the arguments to be passed to the interface method, as implied by the descriptor of the `CONSTANT_NameAndType_info` structure referenced by the `CONSTANT_InterfaceMethodref` constant pool entry.

The fourth operand byte of each *invokeinterface* instruction must have the value zero.

- The *indexbyte* operands of each *invokedynamic* instruction must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be of kind `CONSTANT_Invokedynamic`.

The third and fourth operand bytes of each *invokedynamic* instruction must have the value zero.

- Only the *invokespecial* instruction is allowed to invoke an instance initialization method (§2.9.1).

No other method whose name begins with the character '`<`' (`'\u003c'`) may be called by the method invocation instructions. In particular, the class or interface initialization method specially named `<clinit>` is never called explicitly from Java Virtual Machine instructions, but only implicitly by the Java Virtual Machine itself.

- The operands of each *instanceof*, *checkcast*, *new*, and *anewarray* instruction, and the *indexbyte* operands of each *multianewarray* instruction, must represent a valid index into the `constant_pool` table. The constant pool entry referenced by that index must be of kind `CONSTANT_Class`.

- No *new* instruction may reference a constant pool entry of kind `CONSTANT_Class` that represents an array type (§4.3.2). The *new* instruction cannot be used to create an array.
- No *anewarray* instruction may be used to create an array of more than 255 dimensions.
- A *multianewarray* instruction must be used only to create an array of a type that has at least as many dimensions as the value of its *dimensions* operand. That is, while a *multianewarray* instruction is not required to create all of the dimensions of the array type referenced by its *indexbyte* operands, it must not attempt to create more dimensions than are in the array type.

The *dimensions* operand of each *multianewarray* instruction must not be zero.

- The *atype* operand of each *newarray* instruction must take one of the values `T_BOOLEAN` (4), `T_CHAR` (5), `T_FLOAT` (6), `T_DOUBLE` (7), `T_BYTE` (8), `T_SHORT` (9), `T_INT` (10), or `T_LONG` (11).
- The *index* operand of each *iload*, *fload*, *aload*, *istore*, *fstore*, *astore*, *iinc*, and *ret* instruction must be a non-negative integer no greater than `max_locals - 1`.

The implicit index of each *iload*_*<n>*, *fload*_*<n>*, *aload*_*<n>*, *istore*_*<n>*, *fstore*_*<n>*, and *astore*_*<n>* instruction must be no greater than `max_locals - 1`.

- The *index* operand of each *lload*, *dload*, *lstore*, and *dstore* instruction must be no greater than `max_locals - 2`.

The implicit index of each *lload*_*<n>*, *dload*_*<n>*, *lstore*_*<n>*, and *dstore*_*<n>* instruction must be no greater than `max_locals - 2`.

- The *indexbyte* operands of each *wide* instruction modifying an *iload*, *fload*, *aload*, *istore*, *fstore*, *astore*, *iinc*, or *ret* instruction must represent a non-negative integer no greater than `max_locals - 1`.

The *indexbyte* operands of each *wide* instruction modifying an *lload*, *dload*, *lstore*, or *dstore* instruction must represent a non-negative integer no greater than `max_locals - 2`.

4.9.2 Structural Constraints

The structural constraints on the code array specify constraints on relationships between Java Virtual Machine instructions. The structural constraints are as follows:

- Each instruction must only be executed with the appropriate type and number of arguments in the operand stack and local variable array, regardless of the execution path that leads to its invocation.

As noted in §2.3.4 and §2.11.1, the Java Virtual Machine implicitly converts values of types `boolean`, `byte`, `short`, and `char` to type `int`, allowing instructions expecting values of type `int` to operate on them.

- If an instruction can be executed along several different execution paths, the operand stack must have the same depth (§2.6.2) prior to the execution of the instruction, regardless of the path taken.
- At no point during execution can the operand stack grow to a depth greater than that implied by the `max_stack` item.
- At no point during execution can more values be popped from the operand stack than it contains.
- At no point during execution can the order of the local variable pair holding a value of type `long` or `double` be reversed or the pair split up. At no point can the local variables of such a pair be operated on individually.
- No local variable (or local variable pair, in the case of a value of type `long` or `double`) can be accessed before it is assigned a value.
- Each *invokespecial* instruction must name one of the following:
 - an instance initialization method (§2.9.1)
 - a method in the current class or interface
 - a method in a superclass of the current class
 - a method in a direct superinterface of the current class or interface
 - a method in `Object`

If an *invokespecial* instruction names an instance initialization method, then the target reference on the operand stack must be an uninitialized class instance.

An instance initialization method must never be invoked on an initialized class instance. In addition:

- If the target reference on the operand stack is *an uninitialized class instance for the current class*, then *invokespecial* must name an instance initialization method from the current class or its direct superclass.
- If the target reference on the operand stack is *a class instance created by an earlier new instruction*, then *invokespecial* must name an instance initialization method from the class of that class instance.
- In either case, if the instruction is covered by an exception handler, then any copy of the target reference stored in a local variable must be treated as unusable by the exception handler code.

If an *invokespecial* instruction names a method which is not an instance initialization method, then the target reference on the operand stack must be a class instance whose type is assignment compatible with the current class (JLS §5.2).

The general rule for *invokespecial* is that the class or interface named by *invokespecial* must be "above" the caller class or interface, while the receiver object targeted by *invokespecial* must be "at" or "below" the caller class or interface. The latter clause is especially important: a class or interface can only perform *invokespecial* on its own objects. See §*invokespecial* for an explanation of how the latter clause is implemented in Prolog.

- Each instance initialization method, except for the instance initialization method derived from the constructor of class `Object`, must call either another instance initialization method of `this` or an instance initialization method of its direct superclass `super` before its instance members are *accessed*, and before the calling instance initialization method returns.

However, instance fields of `this` that are declared in the current class may be *assigned* by *putfield* before calling any instance initialization method.

- When any instance method is invoked or when any instance variable is accessed, the class instance that contains the instance method or instance variable must already be initialized.
- There must never be an uninitialized class instance on the operand stack or in a local variable when a *jsr* or *jsr_w* instruction is executed.
- The type of every class instance that is the target of a method invocation instruction (that is, the type of the target reference on the operand stack) must be assignment compatible with the class or interface type specified in the instruction.

- The types of the arguments to each method invocation must be method invocation compatible with the method descriptor (JLS §5.3, §4.3.3).
- Each return instruction must match its method's return type:
 - If the method returns a `boolean`, `byte`, `char`, `short`, or `int`, only the *ireturn* instruction may be used.
 - If the method returns a `float`, `long`, or `double`, only an *freturn*, *lreturn*, or *dreturn* instruction, respectively, may be used.
 - If the method returns a reference type, only an *areturn* instruction may be used, and the type of the returned value must be assignment compatible with the return descriptor of the method (§4.3.3).
 - All instance initialization methods, class or interface initialization methods, and methods declared to return `void` must use only the *return* instruction.
- The type of every class instance accessed by a *getfield* instruction or modified by a *putfield* instruction (that is, the type of the target reference on the operand stack) must be assignment compatible with the class type specified in the instruction.
- The type of every value stored by a *putfield* or *putstatic* instruction must be compatible with the descriptor of the field (§4.3.2) of the class instance or class being stored into:
 - If the descriptor type is `boolean`, `byte`, `char`, `short`, or `int`, then the value must be an `int`.
 - If the descriptor type is `float`, `long`, or `double`, then the value must be a `float`, `long`, or `double`, respectively.
 - If the descriptor type is a reference type, then the value must be of a type that is assignment compatible with the descriptor type.
- The type of every value stored into an array by an *aastore* instruction must be a reference type.

The component type of the array being stored into by the *aastore* instruction must also be a reference type.
- Each *athrow* instruction must throw only values that are instances of class `Throwable` or of subclasses of `Throwable`.

Each class mentioned in a `catch_type` item of the `exception_table` array of the method's `Code_attribute` structure must be `Throwable` or a subclass of `Throwable`.

- If *getfield* or *putfield* is used to access a protected field declared in a superclass that is a member of a different run-time package than the current class, then the type of the class instance being accessed (that is, the type of the target reference on the operand stack) must be assignment compatible with the current class.

If *invokevirtual* or *invokespecial* is used to access a protected method declared in a superclass that is a member of a different run-time package than the current class, then the type of the class instance being accessed (that is, the type of the target reference on the operand stack) must be assignment compatible with the current class.

- Execution never falls off the bottom of the code array.
- No return address (a value of type `returnAddress`) may be loaded from a local variable.
- The instruction following each *jsr* or *jsr_w* instruction may be returned to only by a single *ret* instruction.
- No *jsr* or *jsr_w* instruction that is returned to may be used to recursively call a subroutine if that subroutine is already present in the subroutine call chain. (Subroutines can be nested when using try-finally constructs from within a finally clause.)
- Each instance of type `returnAddress` can be returned to at most once.

If a *ret* instruction returns to a point in the subroutine call chain above the *ret* instruction corresponding to a given instance of type `returnAddress`, then that instance can never be used as a return address.

4.10 Verification of class Files

Even though a compiler for the Java programming language must only produce class files that satisfy all the static and structural constraints in the previous sections, the Java Virtual Machine has no guarantee that any file it is asked to load was generated by that compiler or is properly formed. Applications such as web browsers do not download source code, which they then compile; these applications download already-compiled class files. The browser needs to determine whether the class file was produced by a trustworthy compiler or by an adversary attempting to exploit the Java Virtual Machine.

An additional problem with compile-time checking is version skew. A user may have successfully compiled a class, say `PurchaseStockOptions`, to be a subclass of `TradingClass`. But the definition of `TradingClass` might have changed since the time

the class was compiled in a way that is not compatible with pre-existing binaries. Methods might have been deleted or had their return types or modifiers changed. Fields might have changed types or changed from instance variables to class variables. The access modifiers of a method or variable may have changed from `public` to `private`. For a discussion of these issues, see Chapter 13, "Binary Compatibility," in *The Java Language Specification, Java SE 26 Edition*.

Because of these potential problems, the Java Virtual Machine needs to verify for itself that the desired constraints are satisfied by the `class` files it attempts to incorporate. A Java Virtual Machine implementation verifies that each `class` file satisfies the necessary constraints at linking time (§5.4).

Link-time verification enhances the performance of the run-time interpreter. Expensive checks that would otherwise have to be performed to verify constraints at run time for each interpreted instruction can be eliminated. The Java Virtual Machine can assume that these checks have already been performed. For example, the Java Virtual Machine will already know the following:

- There are no operand stack overflows or underflows.
- All local variable uses and stores are valid.
- The arguments to all the Java Virtual Machine instructions are of valid types.

There are two strategies that Java Virtual Machine implementations may use for verification:

- Verification by type checking must be used to verify `class` files whose version number is greater than or equal to 50.0.
- Verification by type inference must be supported by all Java Virtual Machine implementations, except those conforming to the Java ME CLDC and Java Card profiles, in order to verify `class` files whose version number is less than 50.0.

Verification on Java Virtual Machine implementations supporting the Java ME CLDC and Java Card profiles is governed by their respective specifications.

4.10.1 Verification by Type Checking

A `class` file whose version number is 50.0 or above (§4.1) must be verified using the type checking rules given in this section.

If, and only if, a `class` file's version number equals 50.0, then if the type checking fails, a Java Virtual Machine implementation may choose to attempt to perform verification by type inference (§4.10.2).

This is a pragmatic adjustment, designed to ease the transition to the new verification discipline. Many tools that manipulate `class` files may alter the bytecodes of a method in a manner that requires adjustment of the method's stack map frames. If a tool does not make the necessary adjustments to the stack map frames, type checking may fail even though the bytecode is in principle valid (and would consequently verify under the old type inference scheme). To allow implementors time to adapt their tools, Java Virtual Machine implementations may fall back to the older verification discipline, but only for a limited time.

In cases where type checking fails but type inference is invoked and succeeds, a certain performance penalty is expected. Such a penalty is unavoidable. It also should serve as a signal to tool vendors that their output needs to be adjusted, and provides vendors with additional incentive to make these adjustments.

In summary, failover to verification by type inference supports both the gradual addition of stack map frames to the Java SE Platform (if they are not present in a version 50.0 `class` file, failover is allowed) and the gradual removal of the `jsr` and `jsr_w` instructions from the Java SE Platform (if they are present in a version 50.0 `class` file, failover is allowed).

If a Java Virtual Machine implementation ever attempts to perform verification by type inference on version 50.0 class files, it must do so in all cases where verification by type checking fails.

This means that a Java Virtual Machine implementation cannot choose to resort to type inference in one case and not in another. It must either reject `class` files that do not verify via type checking, or else consistently failover to the type inferencing verifier whenever type checking fails.

The type checker enforces type rules that are specified by means of Prolog clauses. English language text is used to describe the type rules in an informal way, while the Prolog clauses provide a formal specification.

The type checker requires a list of stack map frames for each method with a `Code` attribute (§4.7.3). A list of stack map frames is given by the `StackMapTable` attribute (§4.7.4) of a `Code` attribute. The intent is that a stack map frame must appear at the beginning of each basic block in a method. The stack map frame specifies the verification type of each operand stack entry and of each local variable at the start of each basic block. The type checker reads the stack map frames for each method with a `Code` attribute and uses these maps to generate a proof of the type safety of the instructions in the `Code` attribute.

The Prolog predicate `methodIsTypeSafe` (§4.10.1.5) determines whether a method with a `Code` attribute is type safe. If a method with a `Code` attribute is not type safe, verification throws a `VerifyError` to indicate that the class file is malformed.

If, for a given class, every method with a `Code` attribute passes the `methodIsTypeSafe` check, the class file has type checked successfully and bytecode verification has completed successfully.

The rest of this section explains the process of type checking in detail:

- First, we give Prolog predicates for core Java Virtual Machine artifacts like classes and methods (§4.10.1.1).
- Second, we specify the type system known to the type checker (§4.10.1.2).
- Third, we specify the Prolog representation of instructions and stack map frames (§4.10.1.3, §4.10.1.4).
- Fourth, we specify how a method is type checked (§4.10.1.5, §4.10.1.6).
- Fifth, we discuss type checking issues common to all load and store instructions (§4.10.1.7), and also issues of access to protected members (§4.10.1.8).
- Finally, we specify the rules to type check each instruction (§4.10.1.9).

4.10.1.1 *Accessors for Java Virtual Machine Artifacts*

We stipulate the existence of 17 Prolog predicates ("accessors") that have certain expected behavior but whose formal definitions are not given in this specification. The Prolog term `Class`, as used here, represents a binary class or interface that has been successfully loaded (§5.3); the Prolog term `Method` represents a method of a `Class`; and the Prolog term `Loader` represents a class loader of a `Class`. This specification does not mandate a precise structure for these entities.

`classClassName(Class, ClassName)`

Extracts the name, `ClassName`, of the class `Class`.

`classIsInterface(Class)`

True iff the class, `Class`, is an interface.

`classSuperClassName(Class, SuperClassName)`

Extracts the name, `SuperClassName`, of the superclass of class `Class`.

`classInterfaceNames(Class, InterfaceNames)`

Extracts a list, `InterfaceNames`, of the names of the direct superinterfaces of the class `Class`.

`classDeclaresMember(Class, MemberName, MemberDescriptor)`

Asserts that a class, `Class`, declares a field or method with name `MemberName` and descriptor `MemberDescriptor`. This assertion does not consider members declared in the superclasses or superinterfaces of `Class`.

`classDefiningLoader(Class, Loader)`

Extracts the defining class loader, `Loader`, of the class `Class`.

`isBootstrapLoader(Loader)`

True iff the class loader `Loader` is the bootstrap class loader.

`loadedClass(Name, InitiatingLoader, ClassDefinition)`

True iff there exists a class named `Name` whose representation (in accordance with this specification) when loaded by the class loader `InitiatingLoader` is `ClassDefinition`.

`methodName(Method, Name)`

Extracts the name, `Name`, of the method `Method`.

`methodAccessFlags(Method, AccessFlags)`

Extracts the access flags, `AccessFlags`, of the method `Method`.

`methodDescriptor(Method, Descriptor)`

Extracts the descriptor, `Descriptor`, of the method `Method`.

`isProtected(MemberClass, MemberName, MemberDescriptor)`

True iff there is a member named `MemberName` with descriptor `MemberDescriptor` in the class `MemberClass` and it is protected.

`isNotProtected(MemberClass, MemberName, MemberDescriptor)`

True iff there is a member named `MemberName` with descriptor `MemberDescriptor` in the class `MemberClass` and it is not protected.

`parseFieldDescriptor(Descriptor, Type)`

Converts a field descriptor, `Descriptor`, into the corresponding verification type `Type` (§4.10.1.2).

The verification type derived from descriptor types `byte`, `short`, `boolean`, and `char`, when not used as array component types, is `int`.

`parseMethodDescriptor(Descriptor, ArgTypeList, ReturnType)`

Converts a method descriptor, `Descriptor`, into a list of verification types, `ArgTypeList`, corresponding to the method argument types, and a verification type, `ReturnType`, corresponding to the return type.

The verification type derived from descriptor types `byte`, `short`, `boolean`, and `char`, when not used as array component types, is `int`. A void return is represented with the special symbol `void`.

`parseCodeAttribute(Class, Method, FrameSize, MaxStack, ParsedCode, Handlers, StackMap)`

Extracts the instruction stream, `ParsedCode`, of the method `Method` in `Class`, as well as the maximum operand stack size, `MaxStack`, the maximal number of local variables, `FrameSize`, the exception handlers, `Handlers`, and the stack map `StackMap`.

The representation of the instruction stream and stack map attribute must be as specified in §4.10.1.3 and §4.10.1.4.

`samePackageName(Class1, Class2)`

True iff the package names of `Class1` and `Class2` are the same.

The above accessors are used to define `loadedSuperClasses`, which produces a list of a class's superclasses by recurring on each class's direct superclass until `Object`, which has no superclass, is reached.

```
loadedSuperClasses(Class, [ Superclass | Rest ]) :-
    classSuperClassName(Class, SuperclassName),
    classDefiningLoader(Class, L),
    loadedClass(SuperclassName, L, Superclass),
    loadedSuperClasses(Superclass, Rest).

loadedSuperClasses(Class, []) :-
    \+ classSuperClassName(Class, _).
```

When type checking a method's body, it is convenient to access information about the method. For this purpose, we define an *environment*, a six-tuple consisting of:

- a class
- a method
- the declared return type of the method (or `void`)
- the instructions in a method
- the maximal size of the operand stack
- a list of exception handlers

We specify accessors to extract information from the environment.

```

allInstructions(Environment, Instructions) :-
    Environment = environment(_Class, _Method, _ReturnType,
                              Instructions, _, _).

exceptionHandlers(Environment, Handlers) :-
    Environment = environment(_Class, _Method, _ReturnType,
                              _Instructions, _, Handlers).

maxOperandStackLength(Environment, MaxStack) :-
    Environment = environment(_Class, _Method, _ReturnType,
                              _Instructions, MaxStack, _Handlers).

currentClassLoader(Environment, Loader) :-
    Environment = environment(Class, _Method, _ReturnType,
                              _Instructions, _, _),
    classDefiningLoader(Class, Loader).

thisClass(Environment, Class) :-
    Environment = environment(Class, _Method, _ReturnType,
                              _Instructions, _, _).

thisType(Environment, class(ClassName, L)) :-
    Environment = environment(Class, _Method, _ReturnType,
                              _Instructions, _, _),
    classDefiningLoader(Class, L),
    classClassName(Class, ClassName).

thisMethodReturnType(Environment, ReturnType) :-
    Environment = environment(_Class, _Method, ReturnType,
                              _Instructions, _, _).

offsetStackFrame(Environment, Offset, StackFrame) :-
    allInstructions(Environment, Instructions),
    member(stackMap(Offset, StackFrame), Instructions).

```

Finally, we specify a general predicate used throughout the type rules:

```

notMember(_, []).
notMember(X, [A | More]) :- X \= A, notMember(X, More).

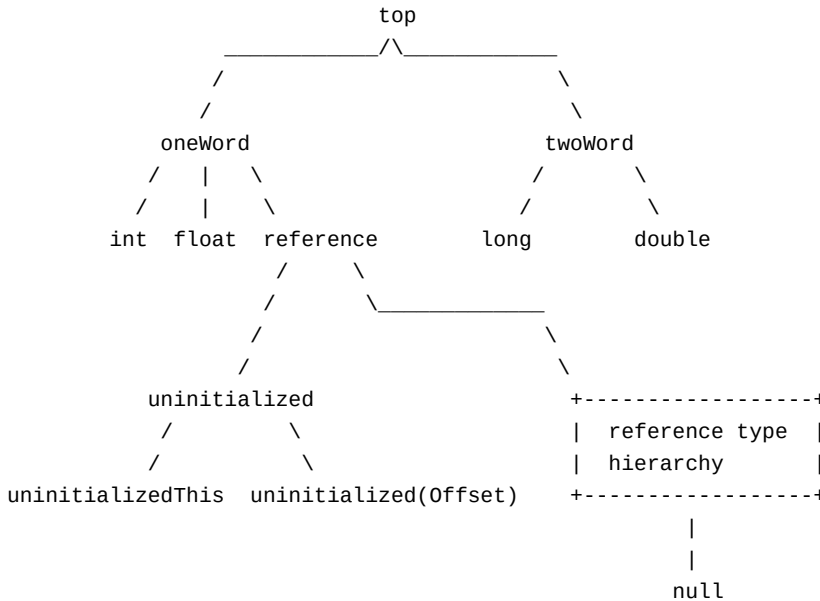
```

The principle guiding the determination as to which accessors are stipulated and which are fully specified is that we do not want to over-specify the representation of the class file. Providing specific accessors to the `Class` or `Method` term would force us to completely specify the format for a Prolog term representing the class file.

4.10.1.2 Verification Type System

The type checker enforces a type system based upon a hierarchy of *verification types*, illustrated below.

Verification type hierarchy:



Most verification types have a direct correspondence with the primitive and reference types described in §2.3 and §2.4, and represented by field descriptors in Table 4.3-A:

- The primitive types `double`, `float`, `int`, and `long` (field descriptors `D`, `F`, `I`, `J`) each correspond to the verification type of the same name.
- The primitive types `byte`, `char`, `short`, and `boolean` (field descriptors `B`, `C`, `S`, `Z`) all correspond to the verification type `int`.
- Class and interface types (field descriptors beginning `L`) correspond to verification types that use the functor `class`. The verification type `class(N, L)` represents the type of the class or interface whose binary name is `N` as loaded by the loader `L`. Note that `L` is an initiating loader (§5.3) of the class represented by `class(N, L)` and may, or may not, be the class's defining loader.

For example, the class type `Object` would be represented as `class('java/lang/Object', L)`, where the defining loader of class `java/lang/Object`, as loaded by `L`, is the bootstrap loader.

- Array types (field descriptors beginning `[]`) correspond to verification types that use the functor `arrayOf`. Note that the primitive types `byte`, `char`, `short`, and `boolean` do not correspond to verification types, but an array type whose element type is `byte`, `char`, `short`, or `boolean` *does* correspond to a verification type; such verification types support the *baload*, *bastore*, *caload*, *castore*, *saload*, *sastore*, and *newarray* instructions.
 - The verification type `arrayOf( $\tau$ )` represents the array type whose component type is the verification type τ .
 - The verification type `arrayOf(byte)` represents the array type whose component type is `byte`.
 - The verification type `arrayOf(char)` represents the array type whose component type is `char`.
 - The verification type `arrayOf(short)` represents the array type whose component type is `short`.
 - The verification type `arrayOf(boolean)` represents the array type whose component type is `boolean`.

For example, the array types `int[]` and `Object[]` would be represented by the verification types `arrayOf(int)` and `arrayOf(class('java/lang/Object', L))` respectively. The array types `byte[]` and `boolean[][]` would be represented by the verification types `arrayOf(byte)` and `arrayOf(arrayOf(boolean))` respectively.

The remaining verification types are described as follows:

- The verification types `top`, `oneword`, `twoword`, and `reference` denote abstract unions of other types, as illustrated above, and are represented in Prolog as atoms.
- The verification types `uninitialized`, `uninitializedThis`, and `uninitialized(Offset)` describe references to objects created with *new* that have not yet been initialized (§2.9.1). The types `uninitialized` and `uninitializedThis` are represented as atoms. The type `uninitialized(Offset)` is represented by applying the functor `uninitialized` to an argument representing the numerical value of the `Offset`.
- The verification type `null` describes the result of the *aconst_null* instruction, and is represented in Prolog as an atom.

The subtyping rules for verification types are as follows.

Subtyping is reflexive.

`isAssignable(X, X).`

The type `top` is a supertype of all other types.

```
isAssignable(oneWord, top).
isAssignable(twoWord, top).
```

A type is a subtype of some other type, `X`, if it has a supertype that is a subtype of `X`.

```
isAssignable(int, X)      :- isAssignable(oneWord, X).
isAssignable(float, X)   :- isAssignable(oneWord, X).
isAssignable(long, X)    :- isAssignable(twoWord, X).
isAssignable(double, X)  :- isAssignable(twoWord, X).

isAssignable(reference, X) :- isAssignable(oneWord, X).
isAssignable(class(_, _), X) :- isAssignable(reference, X).
isAssignable(arrayOf(_, X) :- isAssignable(reference, X).

isAssignable(null, X) :- isAssignable(reference, X).

isAssignable(uninitialized, X)      :- isAssignable(reference, X).
isAssignable(uninitializedThis, X) :- isAssignable(uninitialized, X).
isAssignable(uninitialized(_, X) :- isAssignable(uninitialized, X).
```

The type `null` is a subtype of all reference types.

```
isAssignable(null, class(_, _)).
isAssignable(null, arrayOf(_)).
```

These subtype rules are not necessarily the most obvious formulation of subtyping. There is a clear split between subtyping rules among reference types and rules for the remaining verification types. The split allows us to state general subtyping relations between reference types and other verification types. These relations hold independently of a reference type's position in the type hierarchy, and help to prevent excessive class loading by a Java Virtual Machine implementation. For example, we do not want to start climbing the superclass hierarchy in response to a query of the form `class(foo, L) <: twoWord`.

We also have a rule that says subtyping is reflexive, so it is always trivially possible to assign between two identical types—both in the case of simple types, like from `int` to `int`, and more complex types, like from `arrayOf(int)` to `arrayOf(int)` or from `class('C', Loader)` to `class('C', Loader)`. Together, the above rules cover all subtyping relationships except for between two distinct reference types.

The `iswideningReference` predicate is used to determine if an assignment from one reference type to another, different, reference type is allowed.

```
isAssignable(From, To) :- iswideningReference(From, To).
```

The `iswideningReference` predicate is not reflexive: widening only occurs when converting between distinct reference types.

The verifier allows any class or interface type to be widened to any interface type.

```
iswideningReference(class(_, _), class(To, L)) :-
    loadedClass(To, L, ToClass),
    classIsInterface(ToClass).
```

This approach is less strict than the Java programming language, which will not allow an assignment to an interface unless the referenced object is statically known to implement the interface. The Java Virtual Machine instead uses a run-time check to ensure that invocations of interface methods actually operate on objects that implement the interface (*\$invokeinterface*). There is no requirement that a reference stored by a variable of an interface type refers to an object that actually implements that interface.

A class type can be widened to another class type if the second type refers to the loaded class of one of the first type's superclasses, or if they both refer to the same class with different initiating loaders.

```
iswideningReference(class(From, L1), class(To, L2)) :-
    From \= To,
    loadedClass(From, L1, FromClass),
    loadedClass(To, L2, ToClass),
    loadedSuperclasses(FromClass, Supers),
    member(ToClass, Supers).
```

```
iswideningReference(class(ClassName, L1), class(ClassName, L2)) :-
    L1 \= L2,
    loadedClass(ClassName, L1, Class),
    loadedClass(ClassName, L2, Class).
```

If two class types *S* and *T* have the same name and the same loader, there is no need to "widen" from one to the other, as *isAssignable* is already true, according to the earlier rule.

Array types are subtypes of *Object*, *Cloneable*, and *java.io.Serializable*. Subtyping between arrays of reference type is covariant.

```
iswideningReference(arrayOf(_), class(ClassName, L)) :-
    (ClassName = 'java/lang/Object' ;
     ClassName = 'java/lang/Cloneable' ;
     ClassName = 'java/io/Serializable')
    loadedClass(ClassName, L, LoadedClass),
    classDefiningLoader(LoadedClass, BL),
    isBootstrapLoader(BL).
```

```
iswideningReference(arrayOf(X), arrayOf(Y)) :-
    iswideningReference(X, Y).
```

4.10.1.3 Instruction Representation

Individual bytecode instructions are represented in Prolog as terms whose functor is the name of the instruction and whose arguments are its parsed operands.

For example, an *aload* instruction is represented as the term `aload(N)`, which includes the index `N` that is the operand of the instruction.

The instructions as a whole are represented as a list of terms of the form:

```
instruction(Offset, AnInstruction)
```

For example, `instruction(21, aload(1))`.

The order of instructions in this list must be the same as in the class file.

Some instructions have operands that refer to entries in the `constant_pool` table. Such entries are represented as functor applications of the form:

- `class(Name, Loader)` or `arrayOf(ComponentType)` for a constant pool entry that is a `CONSTANT_Class_info` structure (§4.4.1).

These are verification types, as described in §4.10.1.2.

If the `name_index` item of the structure gives the name of a class or interface, `Name` is that name, and `Loader` is the defining loader of the class or interface containing the constant pool.

If the `name_index` item of the structure gives an array type, `ComponentType` is the array component type.

- `field(FieldClassType, FieldName, FieldDescriptor)` for a constant pool entry that is a `CONSTANT_Fieldref_info` structure (§4.4.2).

`FieldClassType` is the verification type of the class, interface, or array type referenced by the `class_index` item in the structure. `FieldName` and `FieldDescriptor` correspond to the name and field descriptor referenced by the `name_and_type_index` item of the structure.

- `method(MethodClassType, MethodName, MethodDescriptor)` for a constant pool entry that is a `CONSTANT_Methodref_info` structure (§4.4.2).

`MethodClassType` is the verification type of the class, interface, or array type referenced by the `class_index` item of the structure. `MethodName` and `MethodDescriptor` correspond to the name and method descriptor referenced by the `name_and_type_index` item of the structure.

- `imethod(MethodClassType, MethodName, MethodDescriptor)` for a constant pool entry that is a `CONSTANT_InterfaceMethodref_info` structure (§4.4.2).

`MethodClassType` is the verification type of the class, interface, or array type referenced by the `class_index` item of the structure. `MethodName` and `MethodDescriptor` correspond to the name and method descriptor referenced by the `name_and_type_index` item of the structure.

- `string(Value)` for a constant pool entry that is a `CONSTANT_String_info` structure (§4.4.3).

`Value` is the string referenced by the `string_index` item of the structure.

- `int(Value)` for a constant pool entry that is a `CONSTANT_Integer_info` structure (§4.4.4).

`Value` is the `int` constant represented by the `bytes` item of the structure.

- `float(Value)` for a constant pool entry that is a `CONSTANT_Float_info` structure (§4.4.4).

`Value` is the `float` constant represented by the `bytes` item of the structure.

- `long(Value)` for a constant pool entry that is a `CONSTANT_Long_info` structure (§4.4.5).

`Value` is the `long` constant represented by the `high_bytes` and `low_bytes` items of the structure.

- `double(Value)` for a constant pool entry that is a `CONSTANT_Double_info` structure (§4.4.5).

`Value` is the `double` constant represented by the `high_bytes` and `low_bytes` items of the structure.

- `methodHandle(Kind, Reference)` for a constant pool entry that is a `CONSTANT_MethodHandle_info` structure (§4.4.8).

`Kind` is the value of the `reference_kind` item of the structure. `Reference` is the value of the `reference_index` item of the structure.

- `methodType(MethodDescriptor)` for a constant pool entry that is a `CONSTANT_MethodType_info` structure (§4.4.9).

`MethodDescriptor` is the method descriptor referenced by the `descriptor_index` item of the structure.

- `dconstant(ConstantName, FieldDescriptor)` for a constant pool entry that is a `CONSTANT_Dynamic_info` structure (§4.4.10).

`ConstantName` and `FieldDescriptor` correspond to the name and field descriptor referenced by the `name_and_type_index` item of the structure. (The `bootstrap_method_attr_index` item is irrelevant to verification.)

- `dmethod(CallSiteName, MethodDescriptor)` for a constant pool entry that is a `CONSTANT_InvokeDynamic_info` structure (§4.4.10).

`CallSiteName` and `MethodDescriptor` correspond to the name and method descriptor referenced by the `name_and_type_index` item of the structure. (The `bootstrap_method_attr_index` item is irrelevant to verification.)

For example, a *getfield* instruction whose operand refers to a constant pool entry representing a field `foo` of type `F` in class `Bar` would be represented as `getfield(field(class('Bar', L), 'foo', 'F'))`, where `L` is the class loader of the class containing the instruction. An *ldc* instruction for loading the `int` constant `91` would be represented as `ldc(int(91))`.

4.10.1.4 Stack Map Frames and Type Transitions

Stack map frames are represented in Prolog as a list of terms of the form:

```
stackMap(Offset, TypeState)
```

where:

- `Offset` is an integer indicating the bytecode offset at which the stack map frame applies (§4.7.4).

The order of bytecode offsets in this list must be the same as in the class file.

- `TypeState` is the expected incoming type state for the instruction at `Offset`.

A *type state* is a mapping from locations in the operand stack and local variables of a method to verification types. It has the form:

```
frame(Locals, OperandStack, Flags)
```

where:

- `Locals` is a list of verification types, such that the *i*'th element of the list (with 0-based indexing) represents the type of local variable *i*.

Types of size 2 (long and double) are represented by two local variables (§2.6.1), with the first local variable being the type itself and the second local variable being `top` (§4.10.1.7).

- `operandStack` is a list of verification types, such that the first element of the list represents the type of the top of the operand stack, and the types of stack entries below the top follow in the list in the appropriate order.

Types of size 2 (long and double) are represented by two stack entries, with the first entry being top and the second entry being the type itself.

For example, a stack with a double value, an int value, and a long value is represented in a type state as a stack with five entries: top and double entries for the double value, an int entry for the int value, and top and long entries for the long value. Accordingly, `OperandStack` is the list `[top, double, int, top, long]`.

- `Flags` is a list which may either be empty or have the single element `flagThisUninit`.

If any local variable in `Locals` has the type `uninitializedThis`, then `Flags` has the single element `flagThisUninit`, otherwise `Flags` is an empty list.

`flagThisUninit` is used in constructors to mark type states where initialization of `this` has not yet been completed. In such type states, it is illegal to return from the method.

Subtyping of verification types is extended pointwise to type states. The local variable array of a method has a fixed length by construction (see `methodInitialStackFrame` in §4.10.1.5), but the operand stack grows and shrinks, so we require an explicit check on the length of the operand stacks whose assignability is desired for subtyping.

```
frameIsAssignable(frame(Locals1, StackMap1, Flags1),
                  frame(Locals2, StackMap2, Flags2)) :-
    length(StackMap1, StackMapLength),
    length(StackMap2, StackMapLength),
    maplist(isAssignable, Locals1, Locals2),
    maplist(isAssignable, StackMap1, StackMap2),
    subset(Flags1, Flags2).
```

Most of the type rules for individual instructions (§4.10.1.9) depend on the notion of a valid *type transition*. A type transition is *valid* if one can pop a list of expected types off the incoming type state's operand stack and replace them with an expected result type, resulting in a new type state where the length of the operand stack does not exceed its declared maximum size.

```

validTypeTransition(Environment, ExpectedTypesOnStack, ResultType,
                    frame(Locals, InputOperandStack, Flags),
                    frame(Locals, NextOperandStack, Flags)) :-
    popMatchingList(InputOperandStack, ExpectedTypesOnStack,
                    InterimOperandStack),
    pushOperandStack(InterimOperandStack, ResultType, NextOperandStack),
    operandStackHasLegalLength(Environment, NextOperandStack).

```

Pop a list of types off the stack.

```

popMatchingList(OperandStack, [], OperandStack).
popMatchingList(OperandStack, [P | Rest], NewOperandStack) :-
    popMatchingType(OperandStack, P, TempOperandStack, _ActualType),
    popMatchingList(TempOperandStack, Rest, NewOperandStack).

```

Pop an individual type off the stack. The exact behavior depends on the stack contents. If the logical top of the stack is some subtype of the specified type, Type, then pop it. If a type occupies two stack entries, then the logical top of the stack is really the type just below the top, and the top of the stack is the unusable type top.

```

popMatchingType([ActualType | OperandStack],
                Type, OperandStack, ActualType) :-
    sizeOf(Type, 1),
    isAssignable(ActualType, Type).

popMatchingType([top, ActualType | OperandStack],
                Type, OperandStack, ActualType) :-
    sizeOf(Type, 2),
    isAssignable(ActualType, Type).

sizeOf(X, 2) :- isAssignable(X, twoWord).
sizeOf(X, 1) :- isAssignable(X, oneWord).
sizeOf(top, 1).

```

Push a logical type onto the stack. The exact behavior varies with the size of the type. If the pushed type is of size 1, we just push it onto the stack. If the pushed type is of size 2, we push it, and then push top.

```

pushOperandStack(OperandStack, 'void', OperandStack).
pushOperandStack(OperandStack, Type, [Type | OperandStack]) :-
    sizeOf(Type, 1).
pushOperandStack(OperandStack, Type, [top, Type | OperandStack]) :-
    sizeOf(Type, 2).

```

The length of the operand stack must not exceed the declared maximum size.


```

operandStackHasLegalLength(Environment, OperandStack) :-
    length(OperandStack, Length),
    maxOperandStackLength(Environment, MaxStack),
    Length <= MaxStack.

```

The *dup* instructions pop expected types off the incoming type state's operand stack and replace them with predefined result types, resulting in a new type state. However, these instructions are not defined in terms of type transitions because there is no need to match types by means of the subtyping relation. Instead, the *dup* instructions manipulate the operand stack entirely in terms of the *category* of types on the stack (§2.11.1).

Category 1 types occupy a single stack entry. Popping a logical type of category 1, *Type*, off the stack is possible if the top of the stack is *Type* and *Type* is not top (otherwise it could denote the upper half of a category 2 type). The result is the incoming stack, with the top entry popped off.

```

popCategory1([Type | Rest], Type, Rest) :-
    Type \= top,
    sizeof(Type, 1).

```

Category 2 types occupy two stack entries. Popping a logical type of category 2, *Type*, off the stack is possible if the top of the stack is *type top*, and the entry directly below it is *Type*. The result is the incoming stack, with the top two entries popped off.

```

popCategory2([top, Type | Rest], Type, Rest) :-
    sizeof(Type, 2).

```

The *dup* instructions push a list of types onto the stack in essentially the same way as when a type is pushed for a valid type transition.

```

canSafelyPush(Environment, InputOperandStack, Type, OutputOperandStack) :-
    pushOperandStack(InputOperandStack, Type, OutputOperandStack),
    operandStackHasLegalLength(Environment, OutputOperandStack).

canSafelyPushList(Environment, InputOperandStack, Types,
    OutputOperandStack) :-
    canPushList(InputOperandStack, Types, OutputOperandStack),
    operandStackHasLegalLength(Environment, OutputOperandStack).

canPushList(InputOperandStack, [], InputOperandStack).
canPushList(InputOperandStack, [Type | Rest], OutputOperandStack) :-
    pushOperandStack(InputOperandStack, Type, InterimOperandStack),
    canPushList(InterimOperandStack, Rest, OutputOperandStack).

```

Many of the type rules for individual instructions use the following clause to easily pop a list of types off the stack.

```
canPop(frame(Locals, OperandStack, Flags), Types,
        frame(Locals, PoppedOperandStack, Flags)) :-
    popMatchingList(OperandStack, Types, PoppedOperandStack).
```

Finally, certain array instructions (*\$aload*, *\$arraylength*, *\$baload*, *\$bastore*) peek at types on the operand stack in order to check they are array types. The following clause accesses the *i*'th element of the operand stack from a type state.

```
nth1OperandStackIs(i, frame(_Locals, OperandStack, _Flags), Element) :-
    nth1(i, OperandStack, Element).
```

4.10.1.5 Type Checking Methods

A method with a Code attribute is type safe if it is possible to merge the code and the stack map frames into a single stream such that each stack map frame precedes the instruction it corresponds to, and the merged stream is type correct. The method's exception handlers, if any, must also be legal.

```
methodIsTypeSafe(Class, Method) :-
    parseCodeAttribute(Class, Method, FrameSize, MaxStack,
                       ParsedCode, Handlers, StackMap),
    mergeStackMapAndCode(StackMap, ParsedCode, MergedCode),
    methodInitialStackFrame(Class, Method, FrameSize, StackFrame, ReturnType),
    Environment = environment(Class, Method, ReturnType, MergedCode,
                              MaxStack, Handlers),
    handlersAreLegal(Environment),
    mergedCodeIsTypeSafe(Environment, MergedCode, StackFrame).
```

Let us consider exception handlers first.

An exception handler is represented by a functor application of the form:

```
handler(Start, End, Target, ClassName)
```

whose arguments are, respectively, the start and end of the range of instructions covered by the handler, the first instruction of the handler code, and the name of the exception class that this handler is designed to handle.

An exception handler is *legal* if its start (*Start*) is less than its end (*End*), there exists an instruction whose offset is equal to *Start*, there exists an instruction whose offset equals *End*, and the handler's exception class is assignable to the class

Throwable. The exception class of a handler is Throwable if the handler's class entry is 0, otherwise it is the class named in the handler.

```

handlersAreLegal(Environment) :-
    exceptionHandlers(Environment, Handlers),
    checklist(handlerIsLegal(Environment), Handlers).

handlerIsLegal(Environment, Handler) :-
    Handler = handler(Start, End, Target, _),
    Start < End,
    allInstructions(Environment, Instructions),
    member(instruction(Start, _), Instructions),
    offsetStackFrame(Environment, Target, _),
    instructionsIncludeEnd(Instructions, End),
    currentClassLoader(Environment, CurrentLoader),
    handlerExceptionClass(Handler, ExceptionClass, CurrentLoader),
    isBootstrapLoader(BL),
    isAssignable(ExceptionClass, class('java/lang/Throwable', BL)).

instructionsIncludeEnd(Instructions, End) :-
    member(instruction(End, _), Instructions).
instructionsIncludeEnd(Instructions, End) :-
    member(endOfCode(End), Instructions).

handlerExceptionClass(handler(_, _, _, 0),
    class('java/lang/Throwable', BL), _) :-
    isBootstrapLoader(BL).

handlerExceptionClass(handler(_, _, _, Name),
    class(Name, L), L) :-
    Name \= 0.

```

Let us now turn to the stream of instructions and stack map frames.

Merging instructions and stack map frames into a single stream involves four cases:

- Merging an empty StackMap and a list of instructions yields the original list of instructions.

```
mergeStackMapAndCode([], CodeList, CodeList).
```

- Given a list of stack map frames beginning with the type state for the instruction at Offset, and a list of instructions beginning at Offset, the merged list is the head of the stack map frame list, followed by the head of the instruction list, followed by the merge of the tails of the two lists.

```

mergeStackMapAndCode([stackMap(Offset, Map) | RestMap],
    [instruction(Offset, Parse) | RestCode],
    [stackMap(Offset, Map),
        instruction(Offset, Parse) | RestMerge]) :-
    mergeStackMapAndCode(RestMap, RestCode, RestMerge).

```

- Otherwise, given a list of stack map frames beginning with the type state for the instruction at `OffsetM`, and a list of instructions beginning at `OffsetP`, then, if `OffsetP < OffsetM`, the merged list consists of the head of the instruction list, followed by the merge of the stack map frame list and the tail of the instruction list.

```

mergeStackMapAndCode([stackMap(OffsetM, Map) | RestMap],
    [instruction(OffsetP, Parse) | RestCode],
    [instruction(OffsetP, Parse) | RestMerge]) :-
    OffsetP < OffsetM,
    mergeStackMapAndCode([stackMap(OffsetM, Map) | RestMap],
        RestCode, RestMerge).

```

- Otherwise, the merge of the two lists is undefined. Since the instruction list has monotonically increasing offsets, the merge of the two lists is not defined unless every stack map frame offset has a corresponding instruction offset and the stack map frames are in monotonically increasing order.

To determine if the merged stream for a method is type correct, we first infer the method's initial type state.

The initial type state of a method consists of an empty operand stack and local variable types derived from the type of this and the arguments, as well as the appropriate flag, depending on whether this is an `<init>` method.

```

methodInitialStackFrame(Class, Method, FrameSize, frame(Locals, [], Flags),
    ReturnType):-
    methodDescriptor(Method, Descriptor),
    parseMethodDescriptor(Descriptor, RawArgs, ReturnType),
    expandTypeList(RawArgs, Args),
    methodInitialThisType(Class, Method, ThisList),
    flags(ThisList, Flags),
    append(ThisList, Args, ThisArgs),
    expandToLength(ThisArgs, FrameSize, top, Locals).

```

Given a list of types, the following clause produces a list where every type of size 2 has been substituted by two entries: one for itself, and one `top` entry. The result then corresponds to the representation of the list as 32-bit words in the Java Virtual Machine.

```

expandTypeList([], []).
expandTypeList([Item | List], [Item | Result]) :-
    sizeof(Item, 1),
    expandTypeList(List, Result).
expandTypeList([Item | List], [Item, top | Result]) :-
    sizeof(Item, 2),
    expandTypeList(List, Result).

flags([uninitializedThis], [flagThisUninit]).
flags(X, []) :- X \= [uninitializedThis].

expandToLength(List, Size, _Filler, List) :-
    length(List, Size).
expandToLength(List, Size, Filler, Result) :-
    length(List, ListLength),
    ListLength < Size,
    Delta is Size - ListLength,
    length(Extra, Delta),
    checklist(=(Filler), Extra),
    append(List, Extra, Result).

```

For the initial type state of an instance method, we compute the type of this and put it in a list. The type of this in an <init> method of a class with a superclass (that is, of any class except Object) is `uninitializedThis`; the type of this in any other instance method, including an <init> method of Object, is `class(N, L)` where N is the name of the class containing the method and L is its defining class loader.

For the initial type state of a static method, this is irrelevant, so the list is empty.

```

methodInitialThisType(_Class, Method, []) :-
    methodAccessFlags(Method, AccessFlags),
    member(static, AccessFlags),
    methodName(Method, MethodName),
    MethodName \= '<init>'.

methodInitialThisType(Class, Method, [This]) :-
    methodAccessFlags(Method, AccessFlags),
    notMember(static, AccessFlags),
    instanceMethodInitialThisType(Class, Method, This).

instanceMethodInitialThisType(Class, Method, uninitializedThis) :-
    isSubclassInstanceInit(Class, Method).

instanceMethodInitialThisType(Class, Method, class(ClassName, L)) :-
    \+ isSubclassInstanceInit(Class, Method),
    classClassName(Class, ClassName),
    classDefiningLoader(Class, L).

isSubclassInstanceInit(Class, Method) :-
    methodName(Method, '<init>'),
    classSuperClassName(Class, _).

```

4.10.1.6 Type Checking Code Streams

Given an incoming type state, the `mergedCodeIsTypeSafe` predicate checks that a merged stream of instructions and stack map frames is type correct:

- If we have a stack map frame and an incoming type state, the type state must be assignable to the one in the stack map frame. We may then proceed to type check the rest of the stream with the type state given in the stack map frame.

```

mergedCodeIsTypeSafe(Environment, [stackMap(Offset, MapFrame) | MoreCode],
                        frame(Locals, OperandStack, Flags)) :-
    frameIsAssignable(frame(Locals, OperandStack, Flags), MapFrame),
    mergedCodeIsTypeSafe(Environment, MoreCode, MapFrame).

```

- A merged code stream is type safe relative to an incoming type state T if it begins with an instruction I that is type safe relative to T , and I *satisfies* its exception

handlers (see below), and the tail of the stream is type safe given the type state following that execution of `I`.

`NextStackFrame` indicates what falls through to the following instruction. For an unconditional branch instruction, it will have the special value `afterGoto`. `ExceptionStackFrame` indicates what is passed to exception handlers.

```
mergedCodeIsTypeSafe(Environment, [instruction(Offset, Parse) | MoreCode],
                      frame(Locals, OperandStack, Flags)) :-
    instructionIsTypeSafe(Parse, Environment, Offset,
                          frame(Locals, OperandStack, Flags),
                          NextStackFrame, ExceptionStackFrame),
    instructionSatisfiesHandlers(Environment, Offset, ExceptionStackFrame),
    mergedCodeIsTypeSafe(Environment, MoreCode, NextStackFrame).
```

- After an unconditional branch (indicated by an incoming type state of `afterGoto`), if we have a stack map frame giving the type state for the following instructions, we can proceed and type check them using the type state provided by the stack map frame.

```
mergedCodeIsTypeSafe(Environment, [stackMap(Offset, MapFrame) | MoreCode],
                      afterGoto) :-
    mergedCodeIsTypeSafe(Environment, MoreCode, MapFrame).
```

- It is illegal to have code after an unconditional branch without a stack map frame being provided for it.

```
mergedCodeIsTypeSafe(_Environment, [instruction(_, _) | _MoreCode],
                      afterGoto) :-
    write_ln('No stack frame after unconditional branch'),
    fail.
```

- If we have an unconditional branch at the end of the code, stop.

```
mergedCodeIsTypeSafe(_Environment, [endOfCode(Offset)],
                      afterGoto).
```

Branching to a target is type safe if the target has an associated stack frame, `Frame`, and the current stack frame, `StackFrame`, is assignable to `Frame`.

```
targetIsTypeSafe(Environment, StackFrame, Target) :-
    offsetStackFrame(Environment, Target, Frame),
    frameIsAssignable(StackFrame, Frame).
```

An instruction *satisfies its exception handlers* if it satisfies every exception handler that is applicable to the instruction.

```
instructionSatisfiesHandlers(Environment, Offset, ExceptionStackFrame) :-
    exceptionHandlers(Environment, Handlers),
    sublist(isApplicableHandler(Offset), Handlers, ApplicableHandlers),
    checklist(instructionSatisfiesHandler(Environment, ExceptionStackFrame),
        ApplicableHandlers).
```

An exception handler is *applicable* to an instruction if the offset of the instruction is greater or equal to the start of the handler's range and less than the end of the handler's range.

```
isApplicableHandler(Offset, handler(Start, End, _Target, _ClassName)) :-
    Offset >= Start,
    Offset < End.
```

An instruction *satisfies* an exception handler if the instructions's outgoing type state is `ExcStackFrame`, and the handler's target (the initial instruction of the handler code) is type safe assuming an incoming type state τ . The type state τ is derived from `ExcStackFrame` by replacing the operand stack with a stack whose sole element is the handler's exception class.

```
instructionSatisfiesHandler(Environment, ExcStackFrame, Handler) :-
    Handler = handler(_, _, Target, _),
    currentClassLoader(Environment, CurrentLoader),
    handlerExceptionClass(Handler, ExceptionClass, CurrentLoader),
    /* The stack consists of just the exception. */
    ExcStackFrame = frame(Locals, _, Flags),
    TrueExcStackFrame = frame(Locals, [ ExceptionClass ], Flags),
    operandStackHasLegalLength(Environment, TrueExcStackFrame),
    targetIsTypeSafe(Environment, TrueExcStackFrame, Target).
```

4.10.1.7 Type Checking Load and Store Instructions

All load instructions are variations on a common pattern, varying the type of the value that the instruction loads.

Loading a value of type `Type` from local variable `Index` is type safe, if the type of that local variable is `ActualType`, `ActualType` is assignable to `Type`, and pushing `ActualType` onto the incoming operand stack is a valid type transition (§4.10.1.4) that yields a new type state `NextStackFrame`. After execution of the load instruction, the type state will be `NextStackFrame`.


```

loadIsTypeSafe(Environment, Index, Type, StackFrame, NextStackFrame) :-
    StackFrame = frame(Locals, _OperandStack, _Flags),
    nth0(Index, Locals, ActualType),
    isAssignable(ActualType, Type),
    validTypeTransition(Environment, [], ActualType, StackFrame,
                        NextStackFrame).

```

All store instructions are variations on a common pattern, varying the type of the value that the instruction stores.

In general, a store instruction is type safe if the local variable it references is of a type that is a supertype of *Type*, and the top of the operand stack is of a subtype of *Type*, where *Type* is the type the instruction is designed to store.

More precisely, the store is type safe if one can pop a type *ActualType* that "matches" *Type* (that is, is a subtype of *Type*) off the operand stack (§4.10.1.4), and then legally assign that type the local variable L_{Index} .

```

storeIsTypeSafe(_Environment, Index, Type,
                frame(Locals, OperandStack, Flags),
                frame(NextLocals, NextOperandStack, Flags)) :-
    popMatchingType(OperandStack, Type, NextOperandStack, ActualType),
    modifyLocalVariable(Index, ActualType, Locals, NextLocals).

```

Given local variables *Locals*, modifying *Index* to have type *Type* results in the local variable list *NewLocals*. The modifications are somewhat involved, because some values (and their corresponding types) occupy two local variables. Hence, modifying L_N may require modifying L_{N+1} (because the type will occupy both the *N* and *N+1* slots) or L_{N-1} (because local *N* used to be the upper half of the two word value/type starting at local *N-1*, and so local *N-1* must be invalidated), or both. This is described further below. We start at L_0 and count up.

```

modifyLocalVariable(Index, Type, Locals, NewLocals) :-
    modifyLocalVariable(0, Index, Type, Locals, NewLocals).

```

Given *LocalsRest*, the suffix of the local variable list starting at index *I*, modifying local variable *Index* to have type *Type* results in the local variable list suffix *NextLocalsRest*.

If $I < Index-1$, just copy the input to the output and recurse forward. If $I = Index-1$, the type of local *I* may change. This can occur if L_I has a type of size 2. Once we set L_{I+1} to the new type (and the corresponding value), the type/value of L_I will be invalidated, as its upper half will be trashed. Then we recurse forward.

```

modifyLocalVariable(I, Index, Type,
                    [Locals1 | LocalsRest],
                    [Locals1 | NextLocalsRest] ) :-
    I < Index - 1,
    I1 is I + 1,
    modifyLocalVariable(I1, Index, Type, LocalsRest, NextLocalsRest).

modifyLocalVariable(I, Index, Type,
                    [Locals1 | LocalsRest],
                    [NextLocals1 | NextLocalsRest] ) :-
    I := Index - 1,
    modifyPreIndexVariable(Locals1, NextLocals1),
    modifyLocalVariable(Index, Index, Type, LocalsRest, NextLocalsRest).

```

When we find the variable, and it only occupies one word, we change it to Type and we're done. When we find the variable, and it occupies two words, we change its type to Type and the next word to top.

```

modifyLocalVariable(Index, Index, Type,
                    [_ | LocalsRest], [Type | LocalsRest]) :-
    sizeof(Type, 1).

modifyLocalVariable(Index, Index, Type,
                    [_ , _ | LocalsRest], [Type, top | LocalsRest]) :-
    sizeof(Type, 2).

```

We refer to a local whose index immediately precedes a local whose type will be modified as a *pre-index variable*. The future type of a pre-index variable of type InputType is Result. If the type, Type, of the pre-index local is of size 1, it doesn't change. If the type of the pre-index local, Type, is 2, we need to mark the lower half of its two word value as unusable, by setting its type to top.

```

modifyPreIndexVariable(Type, Type) :- sizeof(Type, 1).
modifyPreIndexVariable(Type, top) :- sizeof(Type, 2).

```

4.10.1.8 Type Checking for protected Members

All instructions that access members must contend with the rules concerning protected members. This section describes the protected check that corresponds to JLS §6.6.2.1.

The protected check applies only to protected members of superclasses of the current class. protected members in other classes will be caught by the access checking done at resolution (§5.4.4). There are four cases:

- If the referenced type is not a class type with the same name as a superclass, it cannot be a superclass, and so it can safely be ignored.

```
passesProtectedCheck(Environment, class(MemberClassName, _), MemberName,
                    MemberDescriptor, StackFrame) :-
    thisClass(Environment, CurrentClass),
    \+ hasSuperclassWithName(CurrentClass, MemberClassName).
```

```
passesProtectedCheck(Environment, arrayOf(_), MemberName,
                    MemberDescriptor, StackFrame).
```

```
hasSuperclassWithName(Class, SuperclassName) :-
    loadedSuperclasses(Class, Supers),
    member(Super, Supers),
    classClassName(Super, SuperclassName).
```

- If the MemberClassName is the same as the name of a superclass, the class being resolved may indeed be a superclass. In this case, if no superclass named MemberClassName in a different run-time package has a protected member named MemberName with descriptor MemberDescriptor, the protected check does not apply.

This is because the actual class being resolved will either be one of these superclasses, in which case we know that it is either in the same run-time package, and the access is legal; or the member in question is not protected and the check does not apply; or it will be a subclass, in which case the check would succeed anyway; or it will be some other class in the same run-time package, in which case the access is legal and the check need not take place; or the verifier need not flag this as a problem, since it will be caught anyway because resolution will per force fail.

```
passesProtectedCheck(Environment, class(MemberClassName, _), MemberName,
                    MemberDescriptor, StackFrame) :-
    thisClass(Environment, CurrentClass),
    hasSuperclassWithName(CurrentClass, MemberClassName),
    loadedSuperclasses(CurrentClass, Supers),
    classesInOtherPkgWithProtectedMember(
        CurrentClass, MemberName, MemberDescriptor, MemberClassName,
        Supers, []).
```

- If there does exist a protected superclass member in a different run-time package, then load MemberClassName; if the member in question is not

protected, the check does not apply. (Using a superclass member that is not protected is trivially correct.)

```
passesProtectedCheck(Environment, class(MemberClassName, Loader),
                     MemberName, MemberDescriptor, StackFrame) :-
    thisClass(Environment, CurrentClass),
    hasSuperclassWithName(CurrentClass, MemberClassName),
    loadedSuperclasses(CurrentClass, Supers),
    classesInOtherPkgWithProtectedMember(
        CurrentClass, MemberName, MemberDescriptor, MemberClassName,
        Supers, List),
    List \= [],
    loadedClass(MemberClassName, Loader, ReferencedClass),
    isNotProtected(ReferencedClass, MemberName, MemberDescriptor).
```

- Otherwise, use of a member of an object of type Target requires that Target be assignable to the type of the current class.

```
passesProtectedCheck(Environment, class(MemberClassName, Loader),
                     MemberName, MemberDescriptor,
                     frame(_Locals, [Target | _Rest], _Flags)) :-
    thisClass(Environment, CurrentClass),
    hasSuperclassWithName(CurrentClass, MemberClassName),
    loadedSuperclasses(CurrentClass, Supers),
    classesInOtherPkgWithProtectedMember(
        CurrentClass, MemberName, MemberDescriptor, MemberClassName,
        Supers, List),
    List \= [],
    loadedClass(MemberClassName, Loader, ReferencedClass),
    isProtected(ReferencedClass, MemberName, MemberDescriptor),
    thisType(Environment, ThisType),
    isAssignable(Target, ThisType).
```

The predicate `classesInOtherPkgWithProtectedMember(Class, MemberName, MemberDescriptor, MemberClassName, Supers, List)` is true if `List` is the set of classes in `Supers` with name `MemberClassName` that are in a different run-time package than `Class` which declare a protected member named `MemberName` with descriptor `MemberDescriptor`.

```

classesInOtherPkgWithProtectedMember( _, _, _, _, [], []).

classesInOtherPkgWithProtectedMember(Class, MemberName,
                                     MemberDescriptor, MemberClassName,
                                     [Super | Tail], [Super | T]) :-
    classClassName(Super, MemberClassName),
    \+ sameRuntimePackage(Class, Super),
    isProtected(Super, MemberName, MemberDescriptor),
    classesInOtherPkgWithProtectedMember(
        Class, MemberName, MemberDescriptor, MemberClassName, Tail, T).

classesInOtherPkgWithProtectedMember(Class, MemberName,
                                     MemberDescriptor, MemberClassName,
                                     [Super | Tail], T) :-
    classClassName(Super, MemberClassName),
    \+ sameRuntimePackage(Class, Super),
    isNotProtected(Super, MemberName, MemberDescriptor),
    classesInOtherPkgWithProtectedMember(
        Class, MemberName, MemberDescriptor, MemberClassName, Tail, T).

classesInOtherPkgWithProtectedMember(Class, MemberName,
                                     MemberDescriptor, MemberClassName,
                                     [Super | Tail], T) :-
    classClassName(Super, MemberClassName),
    sameRuntimePackage(Class, Super),
    classesInOtherPkgWithProtectedMember(
        Class, MemberName, MemberDescriptor, MemberClassName, Tail, T).

classesInOtherPkgWithProtectedMember(Class, MemberName,
                                     MemberDescriptor, MemberClassName,
                                     [Super | Tail], T) :-
    classClassName(Super, SuperClassName),
    SuperClassName \= MemberClassName,
    classesInOtherPkgWithProtectedMember(
        Class, MemberName, MemberDescriptor, MemberClassName, Tail, T).

sameRuntimePackage(Class1, Class2) :-
    classDefiningLoader(Class1, L),
    classDefiningLoader(Class2, L),
    samePackageName(Class1, Class2).

```

4.10.1.9 Type Checking Instructions

In general, the type rule for an instruction is given relative to an environment `Environment` that defines the class and method in which the instruction occurs (§4.10.1.1), and the offset `Offset` within the method at which the instruction occurs. The rule states that if the incoming type state `StackFrame` fulfills certain requirements, then:

- The instruction is type safe.
- It is provable that the type state after the instruction completes normally has a particular form given by `NextStackFrame`, and that the type state after the instruction completes abruptly is given by `ExceptionStackFrame`.

The type state after an instruction completes abruptly is the same as the incoming type state, except that the operand stack is empty.

```
exceptionStackFrame(StackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, _OperandStack, Flags),
    ExceptionStackFrame = frame(Locals, [], Flags).
```

Many instructions have type rules that are completely isomorphic to the rules for other instructions. If an instruction `b1` is isomorphic to another instruction `b2`, then the type rule for `b1` is the same as the type rule for `b2`.

```
instructionIsTypeSafe(Instruction, Environment, Offset, StackFrame,
    NextStackFrame, ExceptionStackFrame) :-
    instructionHasEquivalentTypeRule(Instruction, IsomorphicInstruction),
    instructionIsTypeSafe(IsomorphicInstruction, Environment, Offset,
        StackFrame, NextStackFrame,
        ExceptionStackFrame).
```

The English language description of each rule is intended to be readable, intuitive, and concise. As such, the description avoids repeating all the contextual assumptions given above. In particular:

- The description does not explicitly mention the environment.
- When the description speaks of the operand stack or local variables in the following, it is referring to the operand stack and local variable components of a type state: either the incoming type state or the outgoing one.
- The type state after the instruction completes abruptly is almost always identical to the incoming type state. The description only discusses the type state after the instruction completes abruptly when that is not the case.

- The description speaks of popping and pushing types onto the operand stack, and does not explicitly discuss issues of stack underflow or overflow. The description assumes these operations can be completed successfully, but the Prolog clauses for operand stack manipulation ensure that the necessary checks are made.
- The description discusses only the manipulation of logical types. In practice, some types take more than one word. The description abstracts from these representation details, but the Prolog clauses that manipulate data do not.

Any ambiguities can be resolved by referring to the formal Prolog clauses.

aaload***aaload***

An *aaload* instruction is type safe iff one can validly replace types matching `int` and an array type with component type `ComponentType` where `ComponentType` is a subtype of `Object`, with `ComponentType` yielding the outgoing type state.

```
instructionIsTypeSafe(aaload, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    nth10operandStackIs(2, StackFrame, ArrayType),
    arrayComponentType(ArrayType, ComponentType),
    isBootstrapLoader(BL),
    validTypeTransition(Environment,
                        [int, arrayOf(class('java/lang/Object', BL))],
                        ComponentType, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The component type of an array of `x` is `x`. We define the component type of `null` to be `null`.

```
arrayComponentType(arrayOf(X), X).
arrayComponentType(null, null).
```


aastore***aastore***

An *aastore* instruction is type safe iff one can validly pop types matching `Object`, `int`, and an array of `Object` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(aastore, _Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    isBootstrapLoader(BL),
    canPop(StackFrame,
           [class('java/lang/Object', BL),
            int,
            arrayOf(class('java/lang/Object', BL))],
           NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

aconst_null***aconst_null***

An *aconst_null* instruction is type safe if one can validly push the type `null` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(aconst_null, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [], null, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

aload, aload_<n>***aload, aload_<n>***

An *aload* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a load instruction with operand *Index* and type reference is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(aload(Index), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    loadIsTypeSafe(Environment, Index, reference, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *aload_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *aload* instruction is type safe.

```
instructionHasEquivalentTypeRule(aload_0, aload(0)).
instructionHasEquivalentTypeRule(aload_1, aload(1)).
instructionHasEquivalentTypeRule(aload_2, aload(2)).
instructionHasEquivalentTypeRule(aload_3, aload(3)).
```

anewarray***anewarray***

An *anewarray* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting a class, interface, or array type, and one can legally replace a type matching int on the incoming operand stack with an array with component type CP yielding the outgoing type state.

```
instructionIsTypeSafe(anewarray(CP), Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    (CP = class(_, _) ; CP = arrayOf(_)),  
    validTypeTransition(Environment, [int], arrayOf(CP),  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

areturn***areturn***

An *areturn* instruction is type safe iff the enclosing method has a declared return type, `ReturnType`, that is a reference type, and one can validly pop a type matching `ReturnType` off the incoming operand stack.

```
instructionIsTypeSafe(areturn, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, ReturnType),  
    isAssignable(ReturnType, reference),  
    canPop(StackFrame, [ReturnType], _PoppedStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

arraylength***arraylength***

An *arraylength* instruction is type safe iff one can validly replace an array type on the incoming operand stack with the type `int` yielding the outgoing type state.

```
instructionIsTypeSafe(arraylength, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    nth10operandStackIs(1, StackFrame, ArrayType),  
    arrayComponentType(ArrayType, _),  
    validTypeTransition(Environment, [top], int, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

astore, astore_<n>***astore, astore_<n>***

An *astore* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a store instruction with operand *Index* and type reference is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(astore(Index), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    storeIsTypeSafe(Environment, Index, reference, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *astore_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *astore* instruction is type safe.

```
instructionHasEquivalentTypeRule(astore_0, astore(0)).
instructionHasEquivalentTypeRule(astore_1, astore(1)).
instructionHasEquivalentTypeRule(astore_2, astore(2)).
instructionHasEquivalentTypeRule(astore_3, astore(3)).
```

athrow***athrow***

An *athrow* instruction is type safe iff the top of the operand stack matches Throwable.

```
instructionIsTypeSafe(athrow, _Environment, _Offset, StackFrame,
                      afterGoto, ExceptionStackFrame) :-
    isBootstrapLoader(BL),
    canPop(StackFrame, [class('java/lang/Throwable', BL)], _PoppedStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


baload***baload***

A *baload* instruction is type safe iff one can validly replace types matching `int` and a small array type on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(baload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :  
    nth10operandStackIs(2, StackFrame, ArrayType),  
    isSmallArray(ArrayType),  
    validTypeTransition(Environment, [int, top], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An array type is a *small array type* if it is an array of `byte`, an array of `boolean`, or a subtype thereof (`null`).

```
isSmallArray(arrayOf(byte)).  
isSmallArray(arrayOf(boolean)).  
isSmallArray(null).
```

bastore***bastore***

A *bastore* instruction is type safe iff one can validly pop types matching `int`, `int` and a small array type off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(bastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    nth10operandStackIs(3, StackFrame, ArrayType),  
    isSmallArray(ArrayType),  
    canPop(StackFrame, [int, int, top], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

bipush***bipush***

A *bipush* instruction is type safe iff the equivalent *sipush* instruction is type safe.

```
instructionHasEquivalentTypeRule(bipush(Value), sipush(Value)).
```

caload***caload***

A *caload* instruction is type safe iff one can validly replace types matching `int` and `array of char` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(caload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(char)], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

castore***castore***

A *castore* instruction is type safe iff one can validly pop types matching `int`, `int` and array of `char` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(castore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [int, int, arrayOf(char)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

checkcast***checkcast***

A *checkcast* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting either a class or an array, and one can validly replace the type object on top of the incoming operand stack with the type denoted by CP yielding the outgoing type state.

```
instructionIsTypeSafe(checkcast(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    (CP = class(_, _) ; CP = arrayOf(_)),
    isBootstrapLoader(BL),
    validTypeTransition(Environment, [class('java/lang/Object', BL)], CP,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

d2f, d2i, d2l***d2f, d2i, d2l***

A *d2f* instruction is type safe if one can validly pop double off the incoming operand stack and replace it with float, yielding the outgoing type state.

```
instructionIsTypeSafe(d2f, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [double], float,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *d2i* instruction is type safe if one can validly pop double off the incoming operand stack and replace it with int, yielding the outgoing type state.

```
instructionIsTypeSafe(d2i, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [double], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *d2l* instruction is type safe if one can validly pop double off the incoming operand stack and replace it with long, yielding the outgoing type state.

```
instructionIsTypeSafe(d2l, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [double], long,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dadd***dadd***

A *dadd* instruction is type safe iff one can validly replace types matching `double` and `double` on the incoming operand stack with `double` yielding the outgoing type state.

```
instructionIsTypeSafe(dadd, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [double, double], double,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


daload***daload***

A *daload* instruction is type safe iff one can validly replace types matching `int` and `array of double` on the incoming operand stack with `double` yielding the outgoing type state.

```
instructionIsTypeSafe(daload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(double)], double,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dastore***dastore***

A *dastore* instruction is type safe iff one can validly pop types matching double, int and array of double off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(dastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [double, int, arrayOf(double)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dcmp<op>***dcmp<op>***

A *dcmpg* instruction is type safe iff one can validly replace types matching `double` and `double` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(dcmpg, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [double, double], int,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dcmpl* instruction is type safe iff the equivalent *dcmpg* instruction is type safe.

```
instructionHasEquivalentTypeRule(dcmpl, dcmpg).
```

dconst_<d>***dconst_<d>***

A *dconst_0* instruction is type safe if one can validly push the type double onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(dconst_0, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [], double, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dconst_1* instruction is type safe iff the equivalent *dconst_0* instruction is type safe.

```
instructionHasEquivalentTypeRule(dconst_1, dconst_0).
```

ddiv***ddiv***

A *ddiv* instruction is type safe iff the equivalent *dadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ddiv, dadd).
```

dload, dload_<n>***dload, dload_<n>***

A *dload* instruction with operand Index is type safe and yields an outgoing type state NextStackFrame, if a load instruction with operand Index and type double is type safe and yields an outgoing type state NextStackFrame.

```
instructionIsTypeSafe(dload(Index), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    loadIsTypeSafe(Environment, Index, double, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *dload_<n>*, for $0 \leq n \leq 3$, are typesafe iff the equivalent *dload* instruction is type safe.

```
instructionHasEquivalentTypeRule(dload_0, dload(0)).
instructionHasEquivalentTypeRule(dload_1, dload(1)).
instructionHasEquivalentTypeRule(dload_2, dload(2)).
instructionHasEquivalentTypeRule(dload_3, dload(3)).
```

dmul***dmul***

A *dmul* instruction is type safe iff the equivalent *dadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(dmul, dadd).
```

dneg***dneg***

A *dneg* instruction is type safe iff there is a type matching double on the incoming operand stack. The *dneg* instruction does not alter the type state.

```
instructionIsTypeSafe(dneg, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [double], double,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


drem***drem***

A *drem* instruction is type safe iff the equivalent *dadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(drem, dadd).
```

dreturn***dreturn***

A *dreturn* instruction is type safe if the enclosing method has a declared return type of `double`, and one can validly pop a type matching `double` off the incoming operand stack.

```
instructionIsTypeSafe(dreturn, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, double),  
    canPop(StackFrame, [double], _PoppedStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dstore, dstore_<n>***dstore, dstore_<n>***

A *dstore* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a store instruction with operand *Index* and type *double* is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(dstore(Index), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    storeIsTypeSafe(Environment, Index, double, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *dstore_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *dstore* instruction is type safe.

```
instructionHasEquivalentTypeRule(dstore_0, dstore(0)).
instructionHasEquivalentTypeRule(dstore_1, dstore(1)).
instructionHasEquivalentTypeRule(dstore_2, dstore(2)).
instructionHasEquivalentTypeRule(dstore_3, dstore(3)).
```

dsub***dsub***

A *dsub* instruction is type safe iff the equivalent *dadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(dsub, dadd).
```

dup***dup***

A *dup* instruction is type safe iff one can validly replace a category 1 type, Type, with the types Type, Type, yielding the outgoing type state.

```
instructionIsTypeSafe(dup, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    popCategory1(InputOperandStack, Type, _),
    canSafelyPush(Environment, InputOperandStack, Type, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dup_x1***dup_x1***

A *dup_x1* instruction is type safe iff one can validly replace two category 1 types, Type1, and Type2, on the incoming operand stack with the types Type1, Type2, Type1, yielding the outgoing type state.

```
instructionIsTypeSafe(dup_x1, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Rest),
    canSafelyPushList(Environment, Rest, [Type1, Type2, Type1],
                      OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

dup_x2***dup_x2***

A *dup_x2* instruction is type safe iff it is a *type safe form* of the *dup_x2* instruction.

```
instructionIsTypeSafe(dup_x2, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    dup_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dup_x2* instruction is a *type safe form* of the *dup_x2* instruction iff it is a *type safe form 1 dup_x2* instruction or a *type safe form 2 dup_x2* instruction.

```
dup_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup_x2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

```
dup_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup_x2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

A *dup_x2* instruction is a *type safe form 1 dup_x2* instruction iff one can validly replace three category 1 types, Type1, Type2, Type3 on the incoming operand stack with the types Type1, Type2, Type3, Type1, yielding the outgoing type state.

```
dup_x2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Stack2),
    popCategory1(Stack2, Type3, Rest),
    canSafelyPushList(Environment, Rest, [Type1, Type3, Type2, Type1],
                      OutputOperandStack).
```

A *dup_x2* instruction is a *type safe form 2 dup_x2* instruction iff one can validly replace a category 1 type, Type1, and a category 2 type, Type2, on the incoming operand stack with the types Type1, Type2, Type1, yielding the outgoing type state.

```
dup_x2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory2(Stack1, Type2, Rest),
    canSafelyPushList(Environment, Rest, [Type1, Type2, Type1],
                      OutputOperandStack).
```

dup2***dup2***

A *dup2* instruction is type safe iff it is a *type safe form* of the *dup2* instruction.

```
instructionIsTypeSafe(dup2, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    dup2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dup2* instruction is a *type safe form* of the *dup2* instruction iff it is a *type safe form 1 dup2* instruction or a *type safe form 2 dup2* instruction.

```
dup2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

```
dup2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

A *dup2* instruction is a *type safe form 1 dup2* instruction iff one can validly replace two category 1 types, Type1 and Type2 on the incoming operand stack with the types Type1, Type2, Type1, Type2, yielding the outgoing type state.

```
dup2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, TempStack),
    popCategory1(TempStack, Type2, _),
    canSafelyPushList(Environment, InputOperandStack, [Type2, Type1],
                      OutputOperandStack).
```

A *dup2* instruction is a *type safe form 2 dup2* instruction iff one can validly replace a category 2 type, Type on the incoming operand stack with the types Type, Type, yielding the outgoing type state.

```
dup2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory2(InputOperandStack, Type, _),
    canSafelyPush(Environment, InputOperandStack, Type, OutputOperandStack).
```


dup2_x1***dup2_x1***

A *dup2_x1* instruction is type safe iff it is a *type safe form* of the *dup2_x1* instruction.

```
instructionIsTypeSafe(dup2_x1, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    dup2_x1FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dup2_x1* instruction is a *type safe form* of the *dup2_x1* instruction iff it is a *type safe form 1 dup2_x1* instruction or a *type safe form 2 dup2_x1* instruction.

```
dup2_x1FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x1Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).

dup2_x1FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x1Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

A *dup2_x1* instruction is a *type safe form 1 dup2_x1* instruction iff one can validly replace three category 1 types, Type1, Type2, Type3, on the incoming operand stack with the types Type1, Type2, Type3, Type1, Type2, yielding the outgoing type state.

```
dup2_x1Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Stack2),
    popCategory1(Stack2, Type3, Rest),
    canSafelyPushList(Environment, Rest, [Type2, Type1, Type3, Type2, Type1],
                      OutputOperandStack).
```

A *dup2_x1* instruction is a *type safe form 2 dup2_x1* instruction iff one can validly replace a category 2 type, Type1, and a category 1 type, Type2, on the incoming operand stack with the types Type1, Type2, Type1, yielding the outgoing type state.

```
dup2_x1Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory2(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Rest),
    canSafelyPushList(Environment, Rest, [Type1, Type2, Type1],
                      OutputOperandStack).
```

dup2_x2***dup2_x2***

A *dup2_x2* instruction is type safe iff it is a *type safe form* of the *dup2_x2* instruction.

```
instructionIsTypeSafe(dup2_x2, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    dup2_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *dup2_x2* instruction is a *type safe form* of the *dup2_x2* instruction iff one of the following holds:

- it is a *type safe form 1 dup2_x2* instruction.
- it is a *type safe form 2 dup2_x2* instruction.
- it is a *type safe form 3 dup2_x2* instruction.
- it is a *type safe form 4 dup2_x2* instruction.

```
dup2_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

```
dup2_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

```
dup2_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x2Form3IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

```
dup2_x2FormIsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    dup2_x2Form4IsTypeSafe(Environment, InputOperandStack, OutputOperandStack).
```

A *dup2_x2* instruction is a *type safe form 1 dup2_x2* instruction iff one can validly replace four category 1 types, Type1, Type2, Type3, Type4, on the incoming operand stack with the types Type1, Type2, Type3, Type4, Type1, Type2, yielding the outgoing type state.

```

dup2_x2Form1IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Stack2),
    popCategory1(Stack2, Type3, Stack3),
    popCategory1(Stack3, Type4, Rest),
    canSafelyPushList(Environment, Rest,
        [Type2, Type1, Type4, Type3, Type2, Type1],
        OutputOperandStack).

```

A *dup2_x2* instruction is a *type safe form 2 dup2_x2* instruction iff one can validly replace a category 2 type, Type1, and two category 1 types, Type2, Type3, on the incoming operand stack with the types Type1, Type2, Type3, Type1, yielding the outgoing type state.

```

dup2_x2Form2IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory2(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Stack2),
    popCategory1(Stack2, Type3, Rest),
    canSafelyPushList(Environment, Rest,
        [Type1, Type3, Type2, Type1],
        OutputOperandStack).

```

A *dup2_x2* instruction is a *type safe form 3 dup2_x2* instruction iff one can validly replace two category 1 types, Type1, Type2, and a category 2 type, Type3, on the incoming operand stack with the types Type1, Type2, Type3, Type1, Type2, yielding the outgoing type state.

```

dup2_x2Form3IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory1(InputOperandStack, Type1, Stack1),
    popCategory1(Stack1, Type2, Stack2),
    popCategory2(Stack2, Type3, Rest),
    canSafelyPushList(Environment, Rest,
        [Type2, Type1, Type3, Type2, Type1],
        OutputOperandStack).

```

A *dup2_x2* instruction is a *type safe form 4 dup2_x2* instruction iff one can validly replace two category 2 types, Type1, Type2, on the incoming operand stack with the types Type1, Type2, Type1, yielding the outgoing type state.

```

dup2_x2Form4IsTypeSafe(Environment, InputOperandStack, OutputOperandStack) :-
    popCategory2(InputOperandStack, Type1, Stack1),
    popCategory2(Stack1, Type2, Rest),
    canSafelyPushList(Environment, Rest, [Type1, Type2, Type1],
        OutputOperandStack).

```

f2d, f2i, f2l***f2d, f2i, f2l***

An *f2d* instruction is type safe if one can validly pop float off the incoming operand stack and replace it with double, yielding the outgoing type state.

```
instructionIsTypeSafe(f2d, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [float], double,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *f2i* instruction is type safe if one can validly pop float off the incoming operand stack and replace it with int, yielding the outgoing type state.

```
instructionIsTypeSafe(f2i, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [float], int,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *f2l* instruction is type safe if one can validly pop float off the incoming operand stack and replace it with long, yielding the outgoing type state.

```
instructionIsTypeSafe(f2l, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [float], long,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

fadd***fadd***

An *fadd* instruction is type safe iff one can validly replace types matching `float` and `float` on the incoming operand stack with `float` yielding the outgoing type state.

```
instructionIsTypeSafe(fadd, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [float, float], float,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

faload***faload***

An *faload* instruction is type safe iff one can validly replace types matching `int` and `array of float` on the incoming operand stack with `float` yielding the outgoing type state.

```
instructionIsTypeSafe(faload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(float)], float,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

fastore***fastore***

An *fastore* instruction is type safe iff one can validly pop types matching `float`, `int` and array of `float` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(fastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [float, int, arrayOf(float)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

fcmp<op>***fcmp<op>***

An *fcmpg* instruction is type safe iff one can validly replace types matching `float` and `float` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(fcmpg, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [float, float], int,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *fcmpl* instruction is type safe iff the equivalent *fcmpg* instruction is type safe.

```
instructionHasEquivalentTypeRule(fcmpl, fcmpg).
```


fconst_<f>***fconst_<f>***

An *fconst_0* instruction is type safe if one can validly push the type `float` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(fconst_0, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [], float, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The rules for the other variants of *fconst* are equivalent.

```
instructionHasEquivalentTypeRule(fconst_1, fconst_0).
instructionHasEquivalentTypeRule(fconst_2, fconst_0).
```

fdiv***fdiv***

An *fdiv* instruction is type safe iff the equivalent *fadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(fdiv, fadd).
```

fload, fload_<n>***fload, fload_<n>***

An *fload* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a load instruction with operand *Index* and type *float* is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(fload(Index), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    loadIsTypeSafe(Environment, Index, float, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *fload_<n>*, for $0 \leq n \leq 3$, are typesafe iff the equivalent *fload* instruction is type safe.

```
instructionHasEquivalentTypeRule(fload_0, fload(0)).
instructionHasEquivalentTypeRule(fload_1, fload(1)).
instructionHasEquivalentTypeRule(fload_2, fload(2)).
instructionHasEquivalentTypeRule(fload_3, fload(3)).
```

fmul***fmul***

An *fmul* instruction is type safe iff the equivalent *fadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(fmul, fadd).
```

fneg***fneg***

An *fneg* instruction is type safe iff there is a type matching `float` on the incoming operand stack. The *fneg* instruction does not alter the type state.

```
instructionIsTypeSafe(fneg, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [float], float,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

frem***frem***

An *frem* instruction is type safe iff the equivalent *fadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(frem, fadd).
```

freturn***freturn***

An *freturn* instruction is type safe if the enclosing method has a declared return type of `float`, and one can validly pop a type matching `float` off the incoming operand stack.

```
instructionIsTypeSafe(freturn, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, float),  
    canPop(StackFrame, [float], _PoppedStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

fstore, fstore_<n>***fstore, fstore_<n>***

An *fstore* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a store instruction with operand *Index* and type *float* is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(fstore(Index), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    storeIsTypeSafe(Environment, Index, float, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *fstore_<n>*, for $0 \leq n \leq 3$, are typesafe iff the equivalent *fstore* instruction is type safe.

```
instructionHasEquivalentTypeRule(fstore_0, fstore(0)).
instructionHasEquivalentTypeRule(fstore_1, fstore(1)).
instructionHasEquivalentTypeRule(fstore_2, fstore(2)).
instructionHasEquivalentTypeRule(fstore_3, fstore(3)).
```


fsub***fsub***

An *fsub* instruction is type safe iff the equivalent *fadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(fsub, fadd).
```

getfield***getfield***

A *getfield* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting a field whose declared type is `FieldType` that is a member of a type `FieldClassType`, and one can validly replace `FieldClassType` with `FieldType` on the incoming operand stack yielding the outgoing type state. protected fields are subject to additional checks (§4.10.1.8).

```
instructionIsTypeSafe(getfield(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    CP = field(FieldClassType, FieldName, FieldDescriptor),
    parseFieldDescriptor(FieldDescriptor, FieldType),
    passesProtectedCheck(Environment, FieldClassType, FieldName,
                        FieldDescriptor, StackFrame),
    validTypeTransition(Environment, [FieldClassType], FieldType,
                      StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

getstatic***getstatic***

A *getstatic* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting a field whose declared type is `FieldType`, and one can validly push `FieldType` on the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(getstatic(CP), Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    CP = field(_FieldClassType, _FieldName, FieldDescriptor),  
    parseFieldDescriptor(FieldDescriptor, FieldType),  
    validTypeTransition(Environment, [], FieldType,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

goto, goto_w***goto, goto_w***

A *goto* instruction is type safe iff its target operand is a valid branch target.

```
instructionIsTypeSafe(goto(Target), Environment, _Offset, StackFrame,  
                     afterGoto, ExceptionStackFrame) :-  
    targetTypeIsTypeSafe(Environment, StackFrame, Target),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *goto_w* instruction is type safe iff the equivalent *goto* instruction is type safe.

```
instructionHasEquivalentTypeRule(goto_w(Target), goto(Target)).
```

i2b, i2c, i2d, i2f, i2l, i2s***i2b, i2c, i2d, i2f, i2l, i2s***

An *i2b* instruction is type safe iff the equivalent *ineg* instruction is type safe.

```
instructionHasEquivalentTypeRule(i2b, neg).
```

An *i2c* instruction is type safe iff the equivalent *ineg* instruction is type safe.

```
instructionHasEquivalentTypeRule(i2c, neg).
```

An *i2d* instruction is type safe if one can validly pop int off the incoming operand stack and replace it with double, yielding the outgoing type state.

```
instructionIsTypeSafe(i2d, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [int], double,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *i2f* instruction is type safe if one can validly pop int off the incoming operand stack and replace it with float, yielding the outgoing type state.

```
instructionIsTypeSafe(i2f, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [int], float,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *i2l* instruction is type safe if one can validly pop int off the incoming operand stack and replace it with long, yielding the outgoing type state.

```
instructionIsTypeSafe(i2l, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [int], long,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *i2s* instruction is type safe iff the equivalent *ineg* instruction is type safe.

```
instructionHasEquivalentTypeRule(i2s, neg).
```

iadd***iadd***

An *iadd* instruction is type safe iff one can validly replace types matching `int` and `int` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(iadd, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, int], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

iaload***iaload***

An *iaload* instruction is type safe iff one can validly replace types matching `int` and `array of int` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(iaload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(int)], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

iand***iand***

An *iand* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(iand, iadd).
```


iastore***iastore***

An *iastore* instruction is type safe iff one can validly pop types matching `int`, `int` and array of `int` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(iastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [int, int, arrayOf(int)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

iconst_<i>***iconst_<i>***

An *iconst_m1* instruction is type safe if one can validly push the type `int` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(iconst_m1, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [], int, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The rules for the other variants of *iconst* are equivalent.

```
instructionHasEquivalentTypeRule(iconst_0, iconst_m1).
instructionHasEquivalentTypeRule(iconst_1, iconst_m1).
instructionHasEquivalentTypeRule(iconst_2, iconst_m1).
instructionHasEquivalentTypeRule(iconst_3, iconst_m1).
instructionHasEquivalentTypeRule(iconst_4, iconst_m1).
instructionHasEquivalentTypeRule(iconst_5, iconst_m1).
```

idiv***idiv***

An *idiv* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(idiv, iadd).
```

if_acmp<cond>***if_acmp<cond>***

An *if_acmpeq* instruction is type safe iff one can validly pop types matching reference and reference on the incoming operand stack yielding the outgoing type state `NextStackFrame`, and the operand of the instruction, `Target`, is a valid branch target assuming an incoming type state of `NextStackFrame`.

```
instructionIsTypeSafe(if_acmpeq(Target), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    canPop(StackFrame, [reference, reference], NextStackFrame),
    targetIsTypeSafe(Environment, NextStackFrame, Target),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The rule for *if_acmpne* is identical.

```
instructionHasEquivalentTypeRule(if_acmpne(Target), if_acmpeq(Target)).
```

if_icmp<cond>***if_icmp<cond>***

An *if_icmpeq* instruction is type safe iff one can validly pop types matching `int` and `int` on the incoming operand stack yielding the outgoing type state `NextStackFrame`, and the operand of the instruction, `Target`, is a valid branch target assuming an incoming type state of `NextStackFrame`.

```
instructionIsTypeSafe(if_icmpeq(Target), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    canPop(StackFrame, [int, int], NextStackFrame),
    targetIsTypeSafe(Environment, NextStackFrame, Target),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The rules for all other variants of the *if_icmp<cond>* instruction are identical.

```
instructionHasEquivalentTypeRule(if_icmpge(Target), if_icmpeq(Target)).
instructionHasEquivalentTypeRule(if_icmpgt(Target), if_icmpeq(Target)).
instructionHasEquivalentTypeRule(if_icmple(Target), if_icmpeq(Target)).
instructionHasEquivalentTypeRule(if_icmplt(Target), if_icmpeq(Target)).
instructionHasEquivalentTypeRule(if_icmpne(Target), if_icmpeq(Target)).
```

if<cond>***if<cond>***

An *ifeq* instruction is type safe iff one can validly pop a type matching `int` off the incoming operand stack yielding the outgoing type state `NextStackFrame`, and the operand of the instruction, `Target`, is a valid branch target assuming an incoming type state of `NextStackFrame`.

```
instructionIsTypeSafe(ifeq(Target), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    canPop(StackFrame, [int], NextStackFrame),
    targetIsTypeSafe(Environment, NextStackFrame, Target),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The rules for all other variations of the *if<cond>* instruction are identical.

```
instructionHasEquivalentTypeRule(ifge(Target), ifeq(Target)).
instructionHasEquivalentTypeRule(ifgt(Target), ifeq(Target)).
instructionHasEquivalentTypeRule(ifle(Target), ifeq(Target)).
instructionHasEquivalentTypeRule(iflt(Target), ifeq(Target)).
instructionHasEquivalentTypeRule(ifne(Target), ifeq(Target)).
```

ifnonnull, ifnull***ifnonnull, ifnull***

An *ifnonnull* instruction is type safe iff one can validly pop a type matching reference off the incoming operand stack yielding the outgoing type state `NextStackFrame`, and the operand of the instruction, `Target`, is a valid branch target assuming an incoming type state of `NextStackFrame`.

```
instructionIsTypeSafe(ifnonnull(Target), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    canPop(StackFrame, [reference], NextStackFrame),
    targetTypeIsTypeSafe(Environment, NextStackFrame, Target),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *ifnull* instruction is type safe iff the equivalent *ifnonnull* instruction is type safe.

```
instructionHasEquivalentTypeRule(ifnull(Target), ifnonnull(Target)).
```

iinc***iinc***

An *iinc* instruction with first operand Index is type safe iff L_{Index} has type int. The *iinc* instruction does not change the type state.

```
instructionIsTypeSafe(iinc(Index, _Value), _Environment, _Offset,
                     StackFrame, StackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, _OperandStack, _Flags),
    nth0(Index, Locals, int),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


iload, iload_<n>***iload, iload_<n>***

An *iload* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a load instruction with operand *Index* and type *int* is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(iload(Index), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    loadIsTypeSafe(Environment, Index, int, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *iload_<n>*, for $0 \leq n \leq 3$, are typesafe iff the equivalent *iload* instruction is type safe.

```
instructionHasEquivalentTypeRule(iload_0, iload(0)).
instructionHasEquivalentTypeRule(iload_1, iload(1)).
instructionHasEquivalentTypeRule(iload_2, iload(2)).
instructionHasEquivalentTypeRule(iload_3, iload(3)).
```

imul***imul***

An *imul* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(imul, iadd).
```

ineg***ineg***

An *ineg* instruction is type safe iff there is a type matching `int` on the incoming operand stack. The *ineg* instruction does not alter the type state.

```
instructionIsTypeSafe(ineg, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int], int, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

instanceof***instanceof***

An *instanceof* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting either a class or an array, and one can validly replace the type Object on top of the incoming operand stack with type int yielding the outgoing type state.

```
instructionIsTypeSafe(instanceof(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    (CP = class(_, _) ; CP = arrayOf(_)),
    isBootstrapLoader(BL),
    validTypeTransition(Environment, [class('java/lang/Object', BL)], int,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

invokedynamic***invokedynamic***

An *invokedynamic* instruction is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting an dynamic call site with name CallSiteName with descriptor Descriptor.
- CallSiteName is not <init>.
- CallSiteName is not <clinit>.
- One can validly replace types matching the argument types given in Descriptor on the incoming operand stack with the return type given in Descriptor, yielding the outgoing type state.

```
instructionIsTypeSafe(invokedynamic(CP,0,0), Environment, _Offset,
                      StackFrame, NextStackFrame, ExceptionStackFrame) :-
    CP = dmethod(CallSiteName, Descriptor),
    CallSiteName \= '<init>',
    CallSiteName \= '<clinit>',
    parseMethodDescriptor(Descriptor, OperandArgList, ReturnType),
    reverse(OperandArgList, StackArgList),
    validTypeTransition(Environment, StackArgList, ReturnType,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

invokeinterface

invokeinterface

An *invokeinterface* instruction is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting an interface method with descriptor Descriptor that is a member of a type MethodClassType.
- MethodName is not <init>.
- MethodName is not <clinit>.
- Its second operand, Count, is a valid count operand (see below).
- One can validly replace types matching MethodClassType and the argument types given in Descriptor on the incoming operand stack with the return type given in Descriptor, yielding the outgoing type state.

```
instructionIsTypeSafe(invokeinterface(CP, Count, 0), Environment, _Offset,
                        StackFrame, NextStackFrame, ExceptionStackFrame) :-
    CP = imethod(MethodClassType, MethodName, Descriptor),
    MethodName \= '<init>',
    MethodName \= '<clinit>',
    parseMethodDescriptor(Descriptor, OperandArgList, ReturnType),
    reverse([MethodClassType | OperandArgList], StackArgList),
    canPop(StackFrame, StackArgList, TempFrame),
    validTypeTransition(Environment, [], ReturnType,
                        TempFrame, NextStackFrame),
    countIsValid(Count, StackFrame, TempFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The Count operand of an *invokeinterface* instruction is valid if it equals the size of the arguments to the instruction. This is equal to the difference between the size of InputFrame and OutputFrame.

```
countIsValid(Count, InputFrame, OutputFrame) :-
    InputFrame = frame(_Locals1, OperandStack1, _Flags1),
    OutputFrame = frame(_Locals2, OperandStack2, _Flags2),
    length(OperandStack1, Length1),
    length(OperandStack2, Length2),
    Count ::= Length1 - Length2.
```

invokespecial***invokespecial***

An *invokespecial* instruction is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting a method named `MethodName` with descriptor `Descriptor` that is a member of a type `MethodClassType`.
- Either:
 - `MethodName` is not `<init>`.
 - `MethodName` is not `<clinit>`.
 - `MethodClassType` is the current class, a superclass of the current class, or a direct superinterface of the current class.
 - One can validly replace types matching the current class and the argument types given in `Descriptor` on the incoming operand stack with the return type given in `Descriptor`, yielding the outgoing type state.

```

instructionIsTypeSafe(invokespecial(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    (CP = method(MethodClassType, MethodName, Descriptor) ;
     CP = imethod(MethodClassType, MethodName, Descriptor)),
    MethodName \= '<init>',
    MethodName \= '<clinit>',
    validSpecialMethodClassType(Environment, MethodClassType),
    parseMethodDescriptor(Descriptor, OperandArgList, ReturnType),
    thisType(Environment, ThisType),
    reverse([ThisType | OperandArgList], StackArgList),
    validTypeTransition(Environment, StackArgList, ReturnType,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).

validSpecialMethodClassType(Environment, class(MethodClassName, _)) :-
    thisClass(Environment, ThisClass),
    classClassName(ThisClass, MethodClassName).

validSpecialMethodClassType(Environment, class(MethodClassName, L)) :-
    thisClass(Environment, ThisClass),
    loadedSuperclasses(ThisClass, Supers),
    member(Super, Supers),
    classClassName(Super, MethodClassName),
    loadedClass(MethodClassName, L, Super).

validSpecialMethodClassType(Environment, class(MethodClassName, _)) :-
    thisClass(Environment, ThisClass),
    classInterfaceNames(ThisClass, InterfaceNames),
    member(MethodClassName, InterfaceNames).

```

The `validSpecialMethodClassType` clause enforces the structural constraint that *invokespecial*, for other than an instance initialization method, must name a method in the current class/interface or a superclass/superinterface.

The `validTypeTransition` clause enforces the structural constraint that *invokespecial*, for other than an instance initialization method, targets a receiver object of the current class or deeper. To see why, consider that `StackArgList` simulates the list of types on the operand stack expected by the method, starting with the current class (the class performing *invokespecial*). The actual types on the operand stack are in `StackFrame`. The effect of `validTypeTransition` is to pop the first type from the operand stack in `StackFrame` and check it is a subtype of the first term of `StackArgList`, namely the current class. Thus, the actual receiver type is compatible with the current class.

A sharp-eyed reader might notice that enforcing this structural constraint supercedes the structural constraint pertaining to *invokespecial* of a protected method. Thus,

the Prolog code above makes no reference to `passesProtectedCheck` (§4.10.1.8), whereas the Prolog code for *invokespecial* of an instance initialization method uses `passesProtectedCheck` to ensure the actual receiver type is compatible with the current class when certain protected instance initialization methods are named.

- Or:
 - `MethodName` is `<init>`.
 - `Descriptor` specifies a void return type.
 - One can validly pop types matching the argument types given in `Descriptor` and `uninitializedThis` off the incoming operand stack, yielding `OperandStack`.
 - `MethodClassType` is the current class or the direct superclass of the current class.
 - The outgoing type state is derived from the incoming type state by first replacing the incoming operand stack with `OperandStack` and then replacing all instances of `uninitializedThis` with the type of the current class.

```

instructionIsTypeSafe(invokespecial(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    (CP = method(MethodClassType, '<init>', Descriptor) ;
     CP = imethod(MethodClassType, '<init>', Descriptor)),
    parseMethodDescriptor(Descriptor, OperandArgList, void),
    reverse([uninitializedThis | OperandArgList], StackArgList),
    canPop(StackFrame, StackArgList, frame(Locals, OperandStack, Flags)),
    validThisInitClassType(Environment, MethodClassType),
    thisType(Environment, This),
    substitute(uninitializedThis, This, OperandStack, NextOperandStack),
    substitute(uninitializedThis, This, Locals, NextLocals),
    substitute(uninitializedThis, top, Locals, ExceptionLocals),
    NextStackFrame = frame(NextLocals, NextOperandStack, []),
    ExceptionStackFrame = frame(ExceptionLocals, [], Flags).

validThisInitClassType(Environment, class(MethodClassName, _)) :-
    thisClass(Environment, ThisClass),
    classClassName(ThisClass, MethodClassName).

validThisInitClassType(Environment, class(MethodClassName, _)) :-
    thisClass(Environment, ThisClass),
    classSuperClassName(ThisClass, MethodClassName).

substitute(_Old, _New, [], []).
substitute(Old, New, [Old | FromRest], [New | ToRest]) :-
    substitute(Old, New, FromRest, ToRest).
substitute(Old, New, [From1 | FromRest], [From1 | ToRest]) :-
    From1 \= Old,
    substitute(Old, New, FromRest, ToRest).

```

- Or:
 - MethodName is <init>.
 - Descriptor specifies a void return type.
 - One can validly pop types matching the argument types given in Descriptor and an uninitialized type, uninitialized(Address), off the incoming operand stack, yielding OperandStack.
 - The instruction at Address created a new MethodClassType.
 - The outgoing type state is derived from the incoming type state by first replacing the incoming operand stack with OperandStack and then replacing all instances of uninitialized(Address) with MethodClassType.
 - If the method is protected, the usage conforms to the special rules governing access to protected members (§4.10.1.8).

```
instructionIsTypeSafe(invokespecial(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    (CP = method(MethodClassType, '<init>', Descriptor) ;
     CP = imethod(MethodClassType, '<init>', Descriptor)),
    parseMethodDescriptor(Descriptor, OperandArgList, void),
    reverse([uninitialized(Address) | OperandArgList], StackArgList),
    canPop(StackFrame, StackArgList, frame(Locals, OperandStack, Flags)),
    allInstructions(Environment, Instructions),
    member(instruction(Address, new(MethodClassType)), Instructions),
    substitute(uninitialized(Address), MethodClassType,
               OperandStack, NextOperandStack),
    substitute(uninitialized(Address), MethodClassType, Locals, NextLocals),
    substitute(uninitialized(Address), top, Locals, ExceptionLocals),
    NextStackFrame = frame(NextLocals, NextOperandStack, Flags),
    ExceptionStackFrame = frame(ExceptionLocals, [], Flags),
    passesProtectedCheck(Environment, MethodClassType, '<init>',
                          Descriptor, NextStackFrame).
```

The rule for *invokespecial* of an <init> method is the sole motivation for passing back a distinct exception stack frame. The concern is that when initializing an object, the *invokespecial* invocation could fail, leaving the object in a partially initialized, permanently unusable state. To prevent repeated initialization attempts after an object fails to initialize the first time, an exception handler must consider any references to the object stored in local variables to have type top rather than uninitializedThis or uninitialized(Offset).

In the special case of initializing the current object (that is, when invoking <init> for type uninitializedThis), the original frame typically holds an uninitialized object in local

variable 0 and has flag `flagThisUninit`. Normal termination of *invokespecial* initializes the uninitialized object and turns off the `flagThisUninit` flag. But if the *invokespecial* invocation throws an exception, the exception frame contains the broken object (with type `top`) and the `flagThisUninit` flag (the old flag). There is no way to perform a return instruction given that type state, so the handler would have to throw another exception (or loop forever). In fact, it's not even possible to express a handler with that type state, because there is no way for a stack frame, as expressed by the `StackMapTable` attribute (§4.7.4), to carry `flagThisUninit` without any accompanying use of type `uninitializedThis`.

If not for these special constraints on object initialization, the local variable types and flags of the exception stack frame would always be the same as the local variable types and flags of the input stack frame.

invokestatic***invokestatic***

An *invokestatic* instruction is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting a method named *MethodName* with descriptor *Descriptor*.
- *MethodName* is not *<init>*.
- *MethodName* is not *<clinit>*.
- One can validly replace types matching the argument types given in *Descriptor* on the incoming operand stack with the return type given in *Descriptor*, yielding the outgoing type state.

```
instructionIsTypeSafe(invokestatic(CP, Environment, _Offset, StackFrame,
                                NextStackFrame, ExceptionStackFrame) :-
    (CP = method(_MethodClassType, MethodName, Descriptor) ;
     CP = imethod(_MethodClassType, MethodName, Descriptor)),
    MethodName \= '<init>',
    MethodName \= '<clinit>',
    parseMethodDescriptor(Descriptor, OperandArgList, ReturnType),
    reverse(OperandArgList, StackArgList),
    validTypeTransition(Environment, StackArgList, ReturnType,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

invokevirtual***invokevirtual***

An *invokevirtual* instruction is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting a method named `MethodName` with descriptor `Descriptor` that is a member of a type `MethodClassType`.
- `MethodName` is not `<init>`.
- `MethodName` is not `<clinit>`.
- One can validly replace types matching `MethodClassType` and the argument types given in `Descriptor` on the incoming operand stack with the return type given in `Descriptor`, yielding the outgoing type state.
- If the method is protected, the usage conforms to the special rules governing access to protected members (§4.10.1.8).

```
instructionIsTypeSafe(invokevirtual(CP, Environment, _Offset, StackFrame,
                                   NextStackFrame, ExceptionStackFrame) :-
    CP = method(MethodClassType, MethodName, Descriptor),
    MethodName \= '<init>',
    MethodName \= '<clinit>',
    parseMethodDescriptor(Descriptor, OperandArgList, ReturnType),
    reverse(OperandArgList, ArgList),
    reverse([MethodClassType | OperandArgList], StackArgList),
    validTypeTransition(Environment, StackArgList, ReturnType,
                        StackFrame, NextStackFrame),
    canPop(StackFrame, ArgList, PoppedFrame),
    passesProtectedCheck(Environment, MethodClassType, MethodName,
                        Descriptor, PoppedFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

ior, irem***ior, irem***

An *ior* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ior, iadd).
```

An *irem* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(irem, iadd).
```

ireturn***ireturn***

An *ireturn* instruction is type safe if the enclosing method has a declared return type of `int`, and one can validly pop a type matching `int` off the incoming operand stack.

```
instructionIsTypeSafe(ireturn, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, int),  
    canPop(StackFrame, [int], _PoppedStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


ishl, ishr, iushr***ishl, ishr, iushr***

An *ishl* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ishl, iadd).
```

An *ishr* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ishr, iadd).
```

An *iushr* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(iushr, iadd).
```

istore, istore_<n>***istore, istore_<n>***

An *istore* instruction with operand Index is type safe and yields an outgoing type state NextStackFrame, if a store instruction with operand Index and type int is type safe and yields an outgoing type state NextStackFrame.

```
instructionIsTypeSafe(istore(Index), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    storeIsTypeSafe(Environment, Index, int, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *istore_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *istore* instruction is type safe.

```
instructionHasEquivalentTypeRule(istore_0, istore(0)).
instructionHasEquivalentTypeRule(istore_1, istore(1)).
instructionHasEquivalentTypeRule(istore_2, istore(2)).
instructionHasEquivalentTypeRule(istore_3, istore(3)).
```

isub, ixor***isub, ixor***

An *isub* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(isub, iadd).
```

An *ixor* instruction is type safe iff the equivalent *iadd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ixor, iadd).
```

l2d, l2f, l2i***l2d, l2f, l2i***

An *l2d* instruction is type safe if one can validly pop long off the incoming operand stack and replace it with double, yielding the outgoing type state.

```
instructionIsTypeSafe(l2d, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [long], double,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *l2f* instruction is type safe if one can validly pop long off the incoming operand stack and replace it with float, yielding the outgoing type state.

```
instructionIsTypeSafe(l2f, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [long], float,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *l2i* instruction is type safe if one can validly pop long off the incoming operand stack and replace it with int, yielding the outgoing type state.

```
instructionIsTypeSafe(l2i, Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [long], int,
                        StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

ladd***ladd***

An *ladd* instruction is type safe iff one can validly replace types matching long and long on the incoming operand stack with long yielding the outgoing type state.

```
instructionIsTypeSafe(ladd, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [long, long], long,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

laload***laload***

An *laload* instruction is type safe iff one can validly replace types matching `int` and `array of long` on the incoming operand stack with `long` yielding the outgoing type state.

```
instructionIsTypeSafe(laload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(long)], long,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

land***land***

An *land* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(land, ladd).
```

lastore***lastore***

An *lastore* instruction is type safe iff one can validly pop types matching long, int and array of long off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(lastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [long, int, arrayOf(long)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


lcmp***lcmp***

A *lcmp* instruction is type safe iff one can validly replace types matching long and long on the incoming operand stack with int yielding the outgoing type state.

```
instructionIsTypeSafe(lcmp, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [long, long], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

lconst_<l>***lconst_<l>***

An *lconst_0* instruction is type safe if one can validly push the type `long` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(lconst_0, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [], long, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *lconst_1* instruction is type safe iff the equivalent *lconst_0* instruction is type safe.

```
instructionHasEquivalentTypeRule(lconst_1, lconst_0).
```

ldc, ldc_w, ldc2_w***ldc, ldc_w, ldc2_w***

An *ldc* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting an entity of type Type, where Type is loadable (§4.4), but not long or double, and one can validly push Type onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(ldc(CP), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    loadableConstant(CP, Type),
    Type \= long,
    Type \= double,
    validTypeTransition(Environment, [], Type, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

```
loadableConstant(CP, Type) :-
    member([CP, Type], [
        [int(_), int],
        [float(_), float],
        [long(_), long],
        [double(_), double]
    ]).
```

```
loadableConstant(CP, Type) :-
    isBootstrapLoader(BL),
    member([CP, Type], [
        [class(_,_), class('java/lang/Class', BL)],
        [arrayOf(_), class('java/lang/Class', BL)],
        [string(_), class('java/lang/String', BL)],
        [methodHandle(_,_), class('java/lang/invoke/MethodHandle', BL)],
        [methodType(_,_), class('java/lang/invoke/MethodType', BL)]
    ]).
```

```
loadableConstant(CP, Type) :-
    CP = dconstant(_, FieldDescriptor),
    parseFieldDescriptor(FieldDescriptor, Type).
```

An *ldc_w* instruction is type safe iff the equivalent *ldc* instruction is type safe.

```
instructionHasEquivalentTypeRule(ldc_w(CP), ldc(CP))
```

An *ldc2_w* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting an entity of type Type, where Type is either long or double, and

one can validly push `Type` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(ldc2_w(CP), Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    loadableConstant(CP, Type),  
    (Type = long ; Type = double),  
    validTypeTransition(Environment, [], Type, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

ldiv***ldiv***

An *ldiv* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(ldiv, ladd).
```

lload, lload_<n>***lload, lload_<n>***

An *lload* instruction with operand *Index* is type safe and yields an outgoing type state *NextStackFrame*, if a *load* instruction with operand *Index* and type *long* is type safe and yields an outgoing type state *NextStackFrame*.

```
instructionIsTypeSafe(lload(Index), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    loadIsTypeSafe(Environment, Index, long, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *lload_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *lload* instruction is type safe.

```
instructionHasEquivalentTypeRule(lload_0, lload(0)).
instructionHasEquivalentTypeRule(lload_1, lload(1)).
instructionHasEquivalentTypeRule(lload_2, lload(2)).
instructionHasEquivalentTypeRule(lload_3, lload(3)).
```

lmul***lmul***

An *lmul* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(lmul, ladd).
```

lneg***lneg***

An *lneg* instruction is type safe iff there is a type matching long on the incoming operand stack. The *lneg* instruction does not alter the type state.

```
instructionIsTypeSafe(lneg, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [long], long,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


lookupswitch

lookupswitch

A *lookupswitch* instruction is type safe if its keys are sorted, one can validly pop `int` off the incoming operand stack yielding a new type state `BranchStackFrame`, and all of the instruction's targets are valid branch targets assuming `BranchStackFrame` as their incoming type state.

```
instructionIsTypeSafe(lookupswitch(Targets, Keys), Environment, _, StackFrame,
                      afterGoto, ExceptionStackFrame) :-
    sort(Keys, Keys),
    canPop(StackFrame, [int], BranchStackFrame),
    checklist(targetIsTypeSafe(Environment, BranchStackFrame), Targets),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

lor, lrem***lor, lrem***

A *lor* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(lor, ladd).
```

An *lrem* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(lrem, ladd).
```

lreturn***lreturn***

An *lreturn* instruction is type safe if the enclosing method has a declared return type of `long`, and one can validly pop a type matching `long` off the incoming operand stack.

```
instructionIsTypeSafe(lreturn, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, long),  
    canPop(StackFrame, [long], _PoppedStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

lshl, lshr, lushr***lshl, lshr, lushr***

An *lshl* instruction is type safe if one can validly replace the types `int` and `long` on the incoming operand stack with the type `long` yielding the outgoing type state.

```
instructionIsTypeSafe(lshl, Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    validTypeTransition(Environment, [int, long], long,
                       StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

An *lshr* instruction is type safe iff the equivalent *lshl* instruction is type safe.

```
instructionHasEquivalentTypeRule(lshr, lshl).
```

An *lushr* instruction is type safe iff the equivalent *lshl* instruction is type safe.

```
instructionHasEquivalentTypeRule(lushr, lshl).
```

lstore, lstore_<n>***lstore, lstore_<n>***

An *lstore* instruction with operand Index is type safe and yields an outgoing type state NextStackFrame, if a store instruction with operand Index and type long is type safe and yields an outgoing type state NextStackFrame.

```
instructionIsTypeSafe(lstore(Index), Environment, _Offset, StackFrame,
                     NextStackFrame, ExceptionStackFrame) :-
    storeIsTypeSafe(Environment, Index, long, StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The instructions *lstore_<n>*, for $0 \leq n \leq 3$, are type safe iff the equivalent *lstore* instruction is type safe.

```
instructionHasEquivalentTypeRule(lstore_0, lstore(0)).
instructionHasEquivalentTypeRule(lstore_1, lstore(1)).
instructionHasEquivalentTypeRule(lstore_2, lstore(2)).
instructionHasEquivalentTypeRule(lstore_3, lstore(3)).
```

lsub, lxor***lsub, lxor***

An *lsub* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(lsub, ladd).
```

An *lxor* instruction is type safe iff the equivalent *ladd* instruction is type safe.

```
instructionHasEquivalentTypeRule(lxor, ladd).
```

monitorenter, monitorexit monitorenter, monitorexit

A *monitorenter* instruction is type safe iff one can validly pop a type matching reference off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(monitorenter, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [reference], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *monitorexit* instruction is type safe iff the equivalent *monitorenter* instruction is type safe.

```
instructionHasEquivalentTypeRule(monitorexit, monitorenter).
```

multianewarray***multianewarray***

A *multianewarray* instruction with operands CP and Dim is type safe iff CP refers to a constant pool entry denoting an array type whose dimension is greater or equal to Dim, Dim is strictly positive, and one can validly replace Dim int types on the incoming operand stack with the type denoted by CP yielding the outgoing type state.

```
instructionIsTypeSafe(multianewarray(CP, Dim), Environment, _Offset,
                      StackFrame, NextStackFrame, ExceptionStackFrame) :-
    CP = arrayOf(_),
    arrayDimensions(CP, Dimensions),
    Dimensions >= Dim,
    Dim > 0,
    /* Make a list of Dim ints */
    findall(int, between(1, Dim, _), IntList),
    validTypeTransition(Environment, IntList, CP,
                      StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The dimension of an array type whose component type is also an array type is one more than the dimension of its component type.

```
arrayDimensions(arrayOf(X), XDimensions + 1) :-
    arrayDimensions(X, XDimensions).

arrayDimensions(Type, 0) :-
    Type \= arrayOf(_).
```


new***new***

A *new* instruction with operand CP at offset Offset is type safe iff CP refers to a constant pool entry denoting a class or interface type, the type uninitialized(Offset) does not appear in the incoming operand stack, and one can validly push uninitialized(Offset) onto the incoming operand stack and replace uninitialized(Offset) with top in the incoming local variables yielding the outgoing type state.

```
instructionIsTypeSafe(new(CP), Environment, Offset, StackFrame,
                        NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, OperandStack, Flags),
    CP = class(_, _),
    NewItem = uninitialized(Offset),
    notMember(NewItem, OperandStack),
    substitute(NewItem, top, Locals, NewLocals),
    validTypeTransition(Environment, [], NewItem,
                        frame(NewLocals, OperandStack, Flags),
                        NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The substitute predicate is defined in the rule for *invokespecial* (§*invokespecial*).

newarray***newarray***

A *newarray* instruction with operand *TypeCode* is type safe iff *TypeCode* corresponds to the primitive type *ElementType*, and one can validly replace the type *int* on the incoming operand stack with the type 'array of *ElementType*', yielding the outgoing type state.

```
instructionIsTypeSafe(newarray(TypeCode), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    primitiveArrayInfo(TypeCode, _TypeChar, ElementType, _VerifierType),
    validTypeTransition(Environment, [int], arrayOf(ElementType),
                      StackFrame, NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

The correspondence between type codes and primitive types is specified by the following predicate:

```
primitiveArrayInfo(4, 0'Z, boolean, int).
primitiveArrayInfo(5, 0'C, char, int).
primitiveArrayInfo(6, 0'F, float, float).
primitiveArrayInfo(7, 0'D, double, double).
primitiveArrayInfo(8, 0'B, byte, int).
primitiveArrayInfo(9, 0'S, short, int).
primitiveArrayInfo(10, 0'I, int, int).
primitiveArrayInfo(11, 0'J, long, long).
```

nop***nop***

A *nop* instruction is always type safe. The *nop* instruction does not affect the type state.

```
instructionIsTypeSafe(nop, _Environment, _Offset, StackFrame,  
                      StackFrame, ExceptionStackFrame) :-  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

pop, pop2***pop, pop2***

A *pop* instruction is type safe iff one can validly pop a category 1 type off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(pop, _Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, [Type | Rest], Flags),
    popCategory1([Type | Rest], Type, Rest),
    NextStackFrame = frame(Locals, Rest, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *pop2* instruction is type safe iff it is a *type safe form* of the *pop2* instruction.

```
instructionIsTypeSafe(pop2, _Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(Locals, InputOperandStack, Flags),
    pop2SomeFormIsTypeSafe(InputOperandStack, OutputOperandStack),
    NextStackFrame = frame(Locals, OutputOperandStack, Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

A *pop2* instruction is a *type safe form* of the *pop2* instruction iff it is a *type safe form 1 pop2* instruction or a *type safe form 2 pop2* instruction.

```
pop2SomeFormIsTypeSafe(InputOperandStack, OutputOperandStack) :-
    pop2Form1IsTypeSafe(InputOperandStack, OutputOperandStack).
```

```
pop2SomeFormIsTypeSafe(InputOperandStack, OutputOperandStack) :-
    pop2Form2IsTypeSafe(InputOperandStack, OutputOperandStack).
```

A *pop2* instruction is a *type safe form 1 pop2* instruction iff one can validly pop two types of size 1 off the incoming operand stack yielding the outgoing type state.

```
pop2Form1IsTypeSafe([Type1, Type2 | Rest], Rest) :-
    popCategory1([Type1 | Rest], Type1, Rest),
    popCategory1([Type2 | Rest], Type2, Rest).
```

A *pop2* instruction is a *type safe form 2 pop2* instruction iff one can validly pop a type of size 2 off the incoming operand stack yielding the outgoing type state.

```
pop2Form2IsTypeSafe([top, Type | Rest], Rest) :-
    popCategory2([top, Type | Rest], Type, Rest).
```

putfield***putfield***

A *putfield* instruction with operand CP is type safe iff all of the following are true:

- Its first operand, CP, refers to a constant pool entry denoting a field whose declared type is `FieldType` that is a member of a type `FieldClassType`.
- Either:
 - One can validly pop types matching `FieldType` and `FieldClassType` off the incoming operand stack yielding the outgoing type state.
 - protected fields are subject to additional checks (§4.10.1.8).

```
instructionIsTypeSafe(putfield(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    CP = field(FieldClassType, FieldName, FieldDescriptor),
    parseFieldDescriptor(FieldDescriptor, FieldType),
    canPop(StackFrame, [FieldType], PoppedFrame),
    passesProtectedCheck(Environment, FieldClassType, FieldName,
                        FieldDescriptor, PoppedFrame),
    canPop(StackFrame, [FieldType, FieldClassType], NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

- Or:
 - If the instruction occurs in an instance initialization method of the class named by `FieldClassType` and assigns to a field declared by the class, then one can validly pop types matching `FieldType` and `uninitializedThis` off the incoming operand stack yielding the outgoing type state. This allows instance fields of this that are declared in the current class to be assigned prior to complete initialization of this.

```
instructionIsTypeSafe(putfield(CP), Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    CP = field(FieldClassType, FieldName, FieldDescriptor),
    thisType(Environment, FieldClassType),
    Environment = environment(CurrentClass, CurrentMethod, _, _, _, _),
    methodName(CurrentMethod, '<init>'),
    classDeclaresMember(CurrentClass, FieldName, FieldDescriptor),
    parseFieldDescriptor(FieldDescriptor, FieldType),
    canPop(StackFrame, [FieldType, uninitializedThis], NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

putstatic***putstatic***

A *putstatic* instruction with operand CP is type safe iff CP refers to a constant pool entry denoting a field whose declared type is `FieldType`, and one can validly pop a type matching `FieldType` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(putstatic(CP), _Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    CP = field(_FieldType, _FieldName, FieldDescriptor),
    parseFieldDescriptor(FieldDescriptor, FieldType),
    canPop(StackFrame, [FieldType], NextStackFrame),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

return***return***

A *return* instruction is type safe if the enclosing method declares a void return type, and either:

- The enclosing method is not an <init> method, or
- this has already been completely initialized at the point where the instruction occurs.

```
instructionIsTypeSafe(return, Environment, _Offset, StackFrame,  
                      afterGoto, ExceptionStackFrame) :-  
    thisMethodReturnType(Environment, void),  
    StackFrame = frame(_Locals, _OperandStack, Flags),  
    notMember(flagThisUninit, Flags),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

saload***saload***

An *saload* instruction is type safe iff one can validly replace types matching `int` and `array of short` on the incoming operand stack with `int` yielding the outgoing type state.

```
instructionIsTypeSafe(saload, Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [int, arrayOf(short)], int,  
                        StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```


sastore***sastore***

An *sastore* instruction is type safe iff one can validly pop types matching `int`, `int`, and array of `short` off the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(sastore, _Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    canPop(StackFrame, [int, int, arrayOf(short)], NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

sipush***sipush***

An *sipush* instruction is type safe iff one can validly push the type `int` onto the incoming operand stack yielding the outgoing type state.

```
instructionIsTypeSafe(sipush(_Value), Environment, _Offset, StackFrame,  
                      NextStackFrame, ExceptionStackFrame) :-  
    validTypeTransition(Environment, [], int, StackFrame, NextStackFrame),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

swap***swap***

A *swap* instruction is type safe iff one can validly replace two category 1 types, Type1 and Type2, on the incoming operand stack with the types Type2 and Type1 yielding the outgoing type state.

```
instructionIsTypeSafe(swap, _Environment, _Offset, StackFrame,
                      NextStackFrame, ExceptionStackFrame) :-
    StackFrame = frame(_Locals, [Type1, Type2 | Rest], _Flags),
    popCategory1([Type1 | Rest], Type1, Rest),
    popCategory1([Type2 | Rest], Type2, Rest),
    NextStackFrame = frame(_Locals, [Type2, Type1 | Rest], _Flags),
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

tableswitch***tableswitch***

A *tableswitch* instruction is type safe if its keys are sorted, one can validly pop int off the incoming operand stack yielding a new type state BranchStackFrame, and all of the instruction's targets are valid branch targets assuming BranchStackFrame as their incoming type state.

```
instructionIsTypeSafe(tableswitch(Targets, Keys), Environment, _Offset,  
                      StackFrame, afterGoto, ExceptionStackFrame) :-  
    sort(Keys, Keys),  
    canPop(StackFrame, [int], BranchStackFrame),  
    checklist(targetIsTypeSafe(Environment, BranchStackFrame), Targets),  
    exceptionStackFrame(StackFrame, ExceptionStackFrame).
```

wide***wide***

The *wide* instructions follow the same rules as the instructions they widen.

```
instructionHasEquivalentTypeRule(wide(WidenedInstruction),  
                                WidenedInstruction).
```

4.10.2 Verification by Type Inference

A class file that does not contain a `StackMapTable` attribute (which necessarily has a version number of 49.0 or below) must be verified using type inference.

4.10.2.1 The Process of Verification by Type Inference

During linking, the verifier checks the code array of the `Code` attribute for each method of the class file by performing data-flow analysis on each method. The verifier ensures that at any given point in the program, no matter what code path is taken to reach that point, all of the following are true:

- The operand stack is always the same size and contains the same types of values.
- No local variable is accessed unless it is known to contain a value of an appropriate type.
- Methods are invoked with the appropriate arguments.
- Fields are assigned only using values of appropriate types.
- All opcodes have appropriately typed arguments on the operand stack and in the local variable array.

For efficiency reasons, certain tests that could in principle be performed by the verifier are delayed until the first time the code for the method is actually invoked. In so doing, the verifier avoids loading class files unless it has to.

For example, if a method invokes another method that returns an instance of class *A*, and that instance is assigned only to a field of the same type, the verifier does not bother to check if the class *A* actually exists. However, if it is assigned to a field of the type *B*, the definitions of both *A* and *B* must be loaded in to ensure that *A* is a subclass of *B*.

4.10.2.2 The Bytecode Verifier

The code for each method is verified independently. First, the bytes that make up the code are broken up into a sequence of instructions, and the index into the code array of the start of each instruction is placed in an array. The verifier then goes through the code a second time and parses the instructions. During this pass a data structure is built to hold information about each Java Virtual Machine instruction in the method. The operands, if any, of each instruction are checked to make sure they are valid. For instance:

- Branches must be within the bounds of the code array for the method.
- The targets of all control-flow instructions are each the start of an instruction. In the case of a *wide* instruction, the *wide* opcode is considered the start of the

instruction, and the opcode giving the operation modified by that *wide* instruction is not considered to start an instruction. Branches into the middle of an instruction are disallowed.

- No instruction can access or modify a local variable at an index greater than or equal to the number of local variables that its method indicates it allocates.
- All references to the constant pool must be to an entry of the appropriate type. (For example, the instruction *getfield* must reference a field.)
- The code does not end in the middle of an instruction.
- Execution cannot fall off the end of the code.
- For each exception handler, the starting and ending point of code protected by the handler must be at the beginning of an instruction or, in the case of the ending point, immediately past the end of the code. The starting point must be before the ending point. The exception handler code must start at a valid instruction, and it must not start at an opcode being modified by the *wide* instruction.

For each instruction of the method, the verifier records the contents of the operand stack and the contents of the local variable array prior to the execution of that instruction. For the operand stack, it needs to know the stack height and the type of each value on it. For each local variable, it needs to know either the type of the contents of that local variable or that the local variable contains an unusable or unknown value (it might be uninitialized). The bytecode verifier does not need to distinguish between the integral types (e.g., byte, short, char) when determining the value types on the operand stack.

Next, a data-flow analyzer is initialized. For the first instruction of the method, the local variables that represent parameters initially contain values of the types indicated by the method's type descriptor; the operand stack is empty. All other local variables contain an illegal value. For the other instructions, which have not been examined yet, no information is available regarding the operand stack or local variables.

Finally, the data-flow analyzer is run. For each instruction, a "changed" bit indicates whether this instruction needs to be looked at. Initially, the "changed" bit is set only for the first instruction. The data-flow analyzer executes the following loop:

1. Select a Java Virtual Machine instruction whose "changed" bit is set. If no instruction remains whose "changed" bit is set, the method has successfully been verified. Otherwise, turn off the "changed" bit of the selected instruction.

2. Model the effect of the instruction on the operand stack and local variable array by doing the following:
 - If the instruction uses values from the operand stack, ensure that there are a sufficient number of values on the stack and that the top values on the stack are of an appropriate type. Otherwise, verification fails.
 - If the instruction uses a local variable, ensure that the specified local variable contains a value of the appropriate type. Otherwise, verification fails.
 - If the instruction pushes values onto the operand stack, ensure that there is sufficient room on the operand stack for the new values. Add the indicated types to the top of the modeled operand stack.
 - If the instruction modifies a local variable, record that the local variable now contains the new type.
3. Determine the instructions that can follow the current instruction. Successor instructions can be one of the following:
 - The next instruction, if the current instruction is not an unconditional control transfer instruction (for instance, *goto*, *return*, or *athrow*). Verification fails if it is possible to "fall off" the last instruction of the method.
 - The target(s) of a conditional or unconditional branch or switch.
 - Any exception handlers for this instruction.
4. Merge the state of the operand stack and local variable array at the end of the execution of the current instruction into each of the successor instructions, as follows:
 - If this is the first time the successor instruction has been visited, record that the operand stack and local variable values calculated in step 2 are the state of the operand stack and local variable array prior to executing the successor instruction. Set the "changed" bit for the successor instruction.
 - If the successor instruction has been seen before, merge the operand stack and local variable values calculated in step 2 into the values already there. Set the "changed" bit if there is any modification to the values.

In the special case of control transfer to an exception handler:

- Record that a single object, of the exception type indicated by the exception handler, is the state of the operand stack prior to executing the successor

instruction. There must be sufficient room on the operand stack for this single value, as if an instruction had pushed it.

- Record that the local variable values from immediately before step 2 are the state of the local variable array prior to executing the successor instruction. The local variable values calculated in step 2 are irrelevant.

5. Continue at step 1.

To merge two operand stacks, the number of values on each stack must be identical. The types of values on the stacks must also be identical, except that differently typed reference values may appear at corresponding places on the two stacks. In this case, the merged operand stack contains a reference type representing the first common superclass, superinterface, or array supertype of the two types. Such a reference type always exists because the type `Object` is a superclass of all class, interface, and array types. If the operand stacks cannot be merged, verification of the method fails.

To merge two local variable array states, corresponding pairs of local variables are compared. The value of the merged local variable is computed using the rules above, except that the corresponding values are permitted to be different primitive types. In that case, the verifier records that the merged local variable contains an unusable value.

If the data-flow analyzer runs on a method without reporting a verification failure, then the method has been successfully verified by the `class` file verifier.

Certain instructions and data types complicate the data-flow analyzer. We now examine each of these in more detail.

4.10.2.3 *Values of Types `long` and `double`*

Values of the `long` and `double` types are treated specially by the verification process.

Whenever a value of type `long` or `double` is moved into a local variable at index n , index $n+1$ is specially marked to indicate that it has been reserved by the value at index n and must not be used as a local variable index. Any value previously at index $n+1$ becomes unusable.

Whenever a value is moved to a local variable at index n , the index $n-1$ is examined to see if it is the index of a value of type `long` or `double`. If so, the local variable at index $n-1$ is changed to indicate that it now contains an unusable value. Since the local variable at index n has been overwritten, the local variable at index $n-1$ cannot represent a value of type `long` or `double`.

Dealing with values of types `long` or `double` on the operand stack is simpler; the verifier treats them as single values on the stack. For example, the verification code for the *dadd* opcode (add two double values) checks that the top two items on the stack are both of type `double`. When calculating operand stack length, values of type `long` and `double` have length two.

Untyped instructions that manipulate the operand stack must treat values of type `long` and `double` as atomic (indivisible). For example, the verifier reports a failure if the top value on the stack is a `double` and it encounters an instruction such as *pop* or *dup*. The instructions *pop2* or *dup2* must be used instead.

4.10.2.4 Instance Initialization Methods and Newly Created Objects

Creating a new class instance is a multistep process. The statement:

```
...
new MyClass(i, j, k);
...
```

can be implemented by the following:

```
...
new #1          // Allocate uninitialized space for MyClass
dup             // Duplicate object on the operand stack
iload_1         // Push i
iload_2         // Push j
iload_3         // Push k
invokespecial #5 // Invoke MyClass.<init>
...
```

This instruction sequence leaves the newly created and initialized object on top of the operand stack. (Additional examples of compilation to the instruction set of the Java Virtual Machine are given in §3 (*Compiling for the Java Virtual Machine*).)

The instance initialization method (§2.9.1) for class `MyClass` sees the new uninitialized object as its `this` argument in local variable 0. Before that method invokes another instance initialization method of `MyClass` or its direct superclass on this, the only operation the method can perform on this is assigning fields declared within `MyClass`.

When doing dataflow analysis on instance methods, the verifier initializes local variable 0 to contain an object of the current class, or, for instance initialization methods, local variable 0 contains a special type indicating an uninitialized object. After an appropriate instance initialization method is invoked (from the current class or its direct superclass) on this object, all occurrences of this special type on the verifier's model of the operand stack and in the local variable array are

replaced by the current class type. The verifier rejects code that uses the new object before it has been initialized or that initializes the object more than once. In addition, it ensures that every normal return of the method has invoked an instance initialization method either in the class of this method or in the direct superclass.

Similarly, a special type is created and pushed on the verifier's model of the operand stack as the result of the Java Virtual Machine instruction *new*. The special type indicates the instruction by which the class instance was created and the type of the uninitialized class instance created. When an instance initialization method declared in the class of the uninitialized class instance is invoked on that class instance, all occurrences of the special type are replaced by the intended type of the class instance. This change in type may propagate to subsequent instructions as the dataflow analysis proceeds.

The instruction number needs to be stored as part of the special type, as there may be multiple not-yet-initialized instances of a class in existence on the operand stack at one time. For example, the Java Virtual Machine instruction sequence that implements:

```
new InputStream(new Foo(), new InputStream("foo"))
```

may have two uninitialized instances of `InputStream` on the operand stack at once. When an instance initialization method is invoked on a class instance, only those occurrences of the special type on the operand stack or in the local variable array that are the same object as the class instance are replaced.

4.10.2.5 *Exceptions and finally*

To implement the `try-finally` construct, a compiler for the Java programming language that generates class files with version number 50.0 or below may use the exception-handling facilities together with two special instructions: *jsr* ("jump to subroutine") and *ret* ("return from subroutine"). The `finally` clause is compiled as a subroutine within the Java Virtual Machine code for its method, much like the code for an exception handler. When a *jsr* instruction that invokes the subroutine is executed, it pushes its return address, the address of the instruction after the *jsr* that is being executed, onto the operand stack as a value of type `returnAddress`. The code for the subroutine stores the return address in a local variable. At the end of the subroutine, a *ret* instruction fetches the return address from the local variable and transfers control to the instruction at the return address.

Control can be transferred to the `finally` clause (the `finally` subroutine can be invoked) in several different ways. If the `try` clause completes normally, the `finally` subroutine is invoked via a *jsr* instruction before evaluating the next

expression. A *break* or *continue* inside the *try* clause that transfers control outside the *try* clause executes a *jsr* to the code for the *finally* clause first. If the *try* clause executes a *return*, the compiled code does the following:

1. Saves the return value (if any) in a local variable.
2. Executes a *jsr* to the code for the *finally* clause.
3. Upon return from the *finally* clause, returns the value saved in the local variable.

The compiler sets up a special exception handler, which catches any exception thrown by the *try* clause. If an exception is thrown in the *try* clause, this exception handler does the following:

1. Saves the exception in a local variable.
2. Executes a *jsr* to the *finally* clause.
3. Upon return from the *finally* clause, rethrows the exception.

For more information about the implementation of the *try-finally* construct, see §3.13.

The code for the *finally* clause presents a special problem to the verifier. Usually, if a particular instruction can be reached via multiple paths and a particular local variable contains incompatible values through those multiple paths, then the local variable becomes unusable. However, a *finally* clause might be called from several different places, yielding several different circumstances:

- The invocation from the exception handler may have a certain local variable that contains an exception.
- The invocation to implement *return* may have some local variable that contains the return value.
- The invocation from the bottom of the *try* clause may have an indeterminate value in that same local variable.

The code for the *finally* clause itself might pass verification, but after completing the updating all the successors of the *ret* instruction, the verifier would note that the local variable that the exception handler expects to hold an exception, or that the return code expects to hold a return value, now contains an indeterminate value.

Verifying code that contains a *finally* clause is complicated. The basic idea is the following:

- Each instruction keeps track of the list of *jsr* targets needed to reach that instruction. For most code, this list is empty. For instructions inside code for the

`finally` clause, it is of length one. For multiply nested `finally` code (extremely rare!), it may be longer than one.

- For each instruction and each *jsr* needed to reach that instruction, a bit vector is maintained of all local variables accessed or modified since the execution of the *jsr* instruction.
- When executing the *ret* instruction, which implements a return from a subroutine, there must be only one possible subroutine from which the instruction can be returning. Two different subroutines cannot "merge" their execution to a single *ret* instruction.
- To perform the data-flow analysis on a *ret* instruction, a special procedure is used. Since the verifier knows the subroutine from which the instruction must be returning, it can find all the *jsr* instructions that call the subroutine and merge the state of the operand stack and local variable array at the time of the *ret* instruction into the operand stack and local variable array of the instructions following the *jsr*. Merging uses a special set of values for local variables:
 - For any local variable that the bit vector (constructed above) indicates has been accessed or modified by the subroutine, use the type of the local variable at the time of the *ret*.
 - For other local variables, use the type of the local variable before the *jsr* instruction.

4.11 Limitations of the Java Virtual Machine

The following limitations of the Java Virtual Machine are implicit in the class file format:

- The per-class or per-interface constant pool is limited to 65535 entries by the 16-bit `constant_pool_count` field of the `ClassFile` structure (§4.1). This acts as an internal limit on the total complexity of a single class or interface.
- The number of fields that may be declared by a class or interface is limited to 65535 by the size of the `fields_count` item of the `ClassFile` structure (§4.1).

Note that the value of the `fields_count` item of the `ClassFile` structure does not include fields that are inherited from superclasses or superinterfaces.

- The number of methods that may be declared by a class or interface is limited to 65535 by the size of the `methods_count` item of the `ClassFile` structure (§4.1).

Note that the value of the `methods_count` item of the `ClassFile` structure does not include methods that are inherited from superclasses or superinterfaces.

- The number of direct superinterfaces of a class or interface is limited to 65535 by the size of the `interfaces_count` item of the `ClassFile` structure (§4.1).
- The greatest number of local variables in the local variables array of a frame created upon invocation of a method (§2.6) is limited to 65535 by the size of the `max_locals` item of the `Code` attribute (§4.7.3) giving the code of the method, and by the 16-bit local variable indexing of the Java Virtual Machine instruction set.

Note that values of type `long` and `double` are each considered to reserve two local variables and contribute two units toward the `max_locals` value, so use of local variables of those types further reduces this limit.

- The size of an operand stack in a frame (§2.6) is limited to 65535 values by the `max_stack` field of the `Code` attribute (§4.7.3).

Note that values of type `long` and `double` are each considered to contribute two units toward the `max_stack` value, so use of values of these types on the operand stack further reduces this limit.

- The number of method parameters is limited to 255 by the definition of a method descriptor (§4.3.3), where the limit includes one unit for this in the case of instance or interface method invocations.

Note that a method descriptor is defined in terms of a notion of method parameter length in which a parameter of type `long` or `double` contributes two units to the length, so parameters of these types further reduce the limit.

- The length of field and method names, field and method descriptors, and other constant string values (including those referenced by `ConstantValue` (§4.7.2) attributes) is limited to 65535 characters by the 16-bit unsigned `length` item of the `CONSTANT_Utf8_info` structure (§4.4.7).

Note that the limit is on the number of bytes in the encoding and not on the number of encoded characters. UTF-8 encodes some characters using two or three bytes. Thus, strings incorporating multibyte characters are further constrained.

- The number of dimensions in an array is limited to 255 by the size of the *dimensions* opcode of the *multianewarray* instruction and by the constraints

imposed on the *multianewarray*, *anewarray*, and *newarray* instructions (§4.9.1, §4.9.2).

Loading, Linking, and Initializing

THE Java Virtual Machine dynamically loads, links and initializes classes and interfaces. Loading is the process of finding the binary representation of a class or interface type with a particular name and *creating* a class or interface from that binary representation. Linking is the process of taking a class or interface and combining it into the run-time state of the Java Virtual Machine so that it can be executed. Initialization of a class or interface consists of executing the class or interface initialization method `<clinit>` (§2.9.2).

In this chapter, §5.1 describes how the Java Virtual Machine derives symbolic references from the binary representation of a class or interface. §5.2 explains how the processes of loading, linking, and initialization are first initiated by the Java Virtual Machine. §5.3 specifies how binary representations of classes and interfaces are loaded by class loaders and how classes and interfaces are created. Linking is described in §5.4. §5.5 details how classes and interfaces are initialized. §5.6 introduces the notion of binding native methods. Finally, §5.7 describes when the Java Virtual Machine terminates.

5.1 The Run-Time Constant Pool

The Java Virtual Machine maintains a run-time constant pool for each class and interface (§2.5.5). This data structure serves many of the purposes of the symbol table of a conventional programming language implementation. The `constant_pool` table in the binary representation of a class or interface (§4.4) is used to construct the run-time constant pool upon class or interface creation (§5.3).

There are two kinds of entry in the run-time constant pool: symbolic references, which may later be resolved (§5.4.3), and static constants, which require no further processing.

The symbolic references in the run-time constant pool are derived from entries in the `constant_pool` table in accordance with the structure of each entry:

- A symbolic reference to a class or interface is derived from a `CONSTANT_Class_info` structure (§4.4.1). Such a reference gives the name of the class or interface in the following form:
 - For a nonarray class or an interface, the name is the binary name (§4.2.1) of the class or interface.
 - For an array class of n dimensions, the name begins with n occurrences of the ASCII `[` character followed by a representation of the element type:
 - › If the element type is a primitive type, it is represented by the corresponding field descriptor (§4.3.2).
 - › Otherwise, if the element type is a reference type, it is represented by the ASCII `L` character followed by the binary name of the element type followed by the ASCII `;` character.

Whenever this chapter refers to the name of a class or interface, the name should be understood to be in the form above. (This is also the form returned by the `Class.getName` method.)

- A symbolic reference to a field of a class or an interface is derived from a `CONSTANT_Fieldref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the field, as well as a symbolic reference to the class or interface in which the field is to be found.
- A symbolic reference to a method of a class is derived from a `CONSTANT_Methodref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the method, as well as a symbolic reference to the class in which the method is to be found.
- A symbolic reference to a method of an interface is derived from a `CONSTANT_InterfaceMethodref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the interface method, as well as a symbolic reference to the interface in which the method is to be found.
- A symbolic reference to a method handle is derived from a `CONSTANT_MethodHandle_info` structure (§4.4.8). Such a reference gives a symbolic reference to a field of a class or interface, or a method of a class, or a method of an interface, depending on the kind of the method handle.

- A symbolic reference to a method type is derived from a `CONSTANT_MethodType_info` structure (§4.4.9). Such a reference gives a method descriptor (§4.3.3).
- A symbolic reference to a *dynamically-computed constant* is derived from a `CONSTANT_Dynamic_info` structure (§4.4.10). Such a reference gives:
 - a symbolic reference to a method handle, which will be invoked to compute the constant's value;
 - a sequence of symbolic references and static constants, which will serve as *static arguments* when the method handle is invoked;
 - an unqualified name and a field descriptor.
- A symbolic reference to a *dynamically-computed call site* is derived from a `CONSTANT_InvokeDynamic_info` structure (§4.4.10). Such a reference gives:
 - a symbolic reference to a method handle, which will be invoked in the course of an *invokedynamic* instruction (§*invokedynamic*) to compute an instance of `java.lang.invoke.CallSite`;
 - a sequence of symbolic references and static constants, which will serve as *static arguments* when the method handle is invoked;
 - an unqualified name and a method descriptor.

The static constants in the run-time constant pool are also derived from entries in the `constant_pool` table in accordance with the structure of each entry:

- A string constant is a reference to an instance of class `String`, and is derived from a `CONSTANT_String_info` structure (§4.4.3). To derive a string constant, the Java Virtual Machine examines the sequence of code points given by the `CONSTANT_String_info` structure:
 - If the method `String.intern` has previously been invoked on an instance of class `String` containing a sequence of Unicode code points identical to that given by the `CONSTANT_String_info` structure, then the string constant is a reference to that same instance of class `String`.
 - Otherwise, a new instance of class `String` is created containing the sequence of Unicode code points given by the `CONSTANT_String_info` structure. The string constant is a reference to the new instance. Finally, the method `String.intern` is invoked on the new instance.

- Numeric constants are derived from `CONSTANT_Integer_info`, `CONSTANT_Float_info`, `CONSTANT_Long_info`, and `CONSTANT_Double_info` structures (§4.4.4, §4.4.5).

Note that `CONSTANT_Float_info` structures represent values in IEEE 754 single format and `CONSTANT_Double_info` structures represent values in IEEE 754 double format. The numeric constants derived from these structures must thus be values that can be represented using IEEE 754 single and double formats, respectively.

The remaining structures in the `constant_pool` table - the descriptive structures `CONSTANT_NameAndType_info`, `CONSTANT_Module_info`, and `CONSTANT_Package_info`, and the foundational structure `CONSTANT_Utf8_info` - are only used indirectly when constructing the run-time constant pool. No entries in the run-time constant pool correspond directly to these structures.

Some entries in the run-time constant pool are *loadable*, which means:

- They may be pushed onto the stack by the *ldc* family of instructions (*\$ldc*, *\$ldc_w*, *\$ldc2_w*).
- They may be static arguments to bootstrap methods for dynamically-computed constants and call sites (§5.4.3.6).

An entry in the run-time constant pool is loadable if it is derived from an entry in the `constant_pool` table that is loadable (see Table 4.4-C). Accordingly, the following entries in the run-time constant pool are loadable:

- Symbolic references to classes and interfaces
- Symbolic references to method handles
- Symbolic references to method types
- Symbolic references to dynamically-computed constants
- Static constants

5.2 Java Virtual Machine Startup

The Java Virtual Machine starts up by creating an initial class or interface using the bootstrap class loader (§5.3.1) or a user-defined class loader (§5.3.2). The Java Virtual Machine then links the initial class or interface, initializes it, creates an instance of it (if necessary), and invokes a main method as specified in (JLS §12.1.4). The invocation of this method drives all further execution. Execution

of the Java Virtual Machine instructions of the instance initialization method (if needed) and main method of the initial class or interface may cause linking (and consequently creation) of additional classes and interfaces, as well as invocation of additional methods.

The initial class or interface is specified in an implementation-dependent manner. For example, the initial class or interface could be provided as a command line argument. Alternatively, the implementation of the Java Virtual Machine could itself provide an initial class that sets up a class loader which in turn loads an application. Other choices of the initial class or interface are possible so long as they are consistent with the specification given in the previous paragraph.

In contrast, the selection and invocation of a main method of the initial class or interface proceeds according to the implementation-independent rules given in (JLS §12.1.4).

5.3 Creation and Loading

Creation of a class or interface *c* denoted by the name *n* consists of the construction of an implementation-specific internal representation of *c* in the method area of the Java Virtual Machine (§2.5.4).

Class or interface creation is triggered by another class or interface *D*, whose run-time constant pool symbolically references *c* by means of the name *n* (§5.4.3.1). If *n* does not denote an array class, then the Java Virtual Machine relies on a *class loader* to locate a binary representation for a class or interface called *n* (§4.1). Once a class loader has located a binary representation, it relies in turn on the Java Virtual Machine to derive the class or interface *c* from the binary representation, and then to create *c* in the method area. Array classes do not have an external binary representation; they are created by the Java Virtual Machine via a different process.

Class or interface creation may also be triggered by *D* invoking methods in certain Java SE Platform class libraries (§2.12) such as reflection.

There are two kinds of class loaders: the bootstrap class loader supplied by the Java Virtual Machine, and user-defined class loaders. Every user-defined class loader is an instance of a subclass of the abstract class `ClassLoader`. Applications employ user-defined class loaders in order to extend the manner in which the Java Virtual Machine dynamically creates classes. User-defined class loaders can be used to create classes that originate from user-defined sources. For example, a class

could be downloaded across a network, generated on the fly, or extracted from an encrypted file.

When the Java Virtual Machine asks a class loader L to locate a binary representation for a class or interface called N , L *loads* the class or interface C denoted by N . L may load C directly, by locating a binary representation and asking the Java Virtual Machine to derive and create C from the binary representation. Alternatively, L may load C indirectly, by delegating to another class loader which loads C directly or indirectly.

If L loads C directly, we say that L *defines* C or, equivalently, that L is the *defining loader* of C .

Whether L loads C directly or indirectly, we say that L *initiates* loading of C , or, equivalently, that L is an *initiating loader* of C .

Due to class loader delegation, the loader L_1 that initiates loading at the Java Virtual Machine's request may not be the same as the loader L_2 that completes loading by defining the class or interface. In this case, we say that each of L_1 and L_2 *initiates* loading of C , or, equivalently, that each of L_1 and L_2 is an *initiating loader* of C . Any loaders in a delegation chain between L_1 and L_2 are not considered to be initiating loaders of C .

We will sometimes represent a class or interface using the following notation, instead of using an identifier like C or D :

- $\langle N, L_d \rangle$ - where N denotes the name of the class or interface and L_d denotes the defining loader of the class or interface.
- N^{L_i} - where N denotes the name of the class or interface and L_i denotes an initiating loader of the class or interface.

It should be clear that *loading* a class or interface is a joint effort between the Java Virtual Machine and a class loader (or multiple class loaders, if delegation occurs). The ultimate outcome of loading is that the Java Virtual Machine creates a class or interface in its method area, so it is often convenient to say that a class or interface is *loaded and thereby created*.

The complex back-and-forth nature of loading, combined with the ability of user-defined class loaders to exhibit arbitrary behavior, means that exceptions can be thrown *after* the Java Virtual Machine has created a class or interface but *before* every class loader participating in loading has completed. This specification accounts for such exceptions in what is often referred to *the process of loading and creating a class or interface*.

The Java Virtual Machine uses one of three procedures to create a class or interface C denoted by the name N in the run-time constant pool of a class or interface D :

- If N denotes either a nonarray class or an interface, and D was defined by the bootstrap class loader, then the bootstrap class loader initiates loading of C (§5.3.1).
- If N denotes either a nonarray class or an interface, and D was defined by a user-defined class loader, then that same user-defined class loader initiates loading of C (§5.3.2).
- If N denotes an array class, then the Java Virtual Machine creates an array class C denoted by N , in association with the defining loader of D (§5.3.3).

Although the defining loader of D is relevant in the course of creating an array class, it is not used to load and thereby create the array class.

If an error occurs during loading of a class or interface - either when a class loader is locating a binary representation, or when the Java Virtual Machine is deriving and creating a class from it - then the error must be thrown at a point in the program that (directly or indirectly) uses the class or interface being loaded.

A well-behaved class loader should maintain three properties:

- Given the same name, a good class loader should always return the same `Class` object.
- If a class loader L_1 delegates loading of a class C to another loader L_2 , then for any type T that occurs as the direct superclass or a direct superinterface of C , or as the type of a field in C , or as the type of a formal parameter of a method or constructor in C , or as a return type of a method in C , L_1 and L_2 should return the same `Class` object.
- If a user-defined classloader prefetches binary representations of classes and interfaces, or loads a group of related classes together, then it must reflect loading errors only at points in the program where they could have arisen without prefetching or group loading.

After creation, a class or interface is determined not by its name alone, but by a pair: its binary name (§4.2.1) and its defining loader. Each such class or interface belongs to a single *run-time package*. The run-time package of a class or interface is determined by the package name and the defining loader of the class or interface.

5.3.1 Loading Using the Bootstrap Class Loader

The process of loading and creating the nonarray class or interface C denoted by N using the bootstrap class loader is as follows.

First, the Java Virtual Machine determines whether the bootstrap class loader has already been recorded as an initiating loader of a class or interface denoted by N . If so, this class or interface is C , and no class loading or creation is necessary.

Otherwise, the Java Virtual Machine passes the argument *N* to an invocation of a method on the bootstrap class loader. To load *C*, the bootstrap class loader locates a purported representation of *C* in a platform-dependent manner, then asks the Java Virtual Machine to derive a class or interface *C* denoted by *N* from the purported representation using the bootstrap class loader, and then to create *C*, via the algorithm of §5.3.5.

Typically, a class or interface will be represented using a file in a hierarchical file system, and the name of the class or interface will be encoded in the pathname of the file to aid in locating it.

If no purported representation of *C* is found, the bootstrap class loader throws a `ClassNotFoundException`. The process of loading and creating *C* then fails with a `NoClassDefFoundError` whose cause is the `ClassNotFoundException`.

If a purported representation of *C* is found, but deriving *C* from the purported representation fails, then the process of loading and creating *C* fails for the same reason.

Otherwise, the process of loading and creating *C* succeeds.

5.3.2 Loading Using a User-defined Class Loader

The process of loading and creating the nonarray class or interface *C* denoted by *N* using a user-defined class loader *L* is as follows.

First, the Java Virtual Machine determines whether *L* has already been recorded as an initiating loader of a class or interface denoted by *N*. If so, this class or interface is *C*, and no class loading or creation is necessary.

Otherwise, the Java Virtual Machine invokes the `loadClass` method of class `ClassLoader` on *L*, passing the name *N* of a class or interface. *L* must perform one of the following two operations to load and thereby create a class or interface *C*:

1. The class loader *L* can load *C* directly. This is accomplished by obtaining an array of bytes that purports to represent *C* as a `ClassFile` structure (§4.1), and then invoking the method `defineClass` of class `ClassLoader`. Invoking `defineClass` causes the Java Virtual Machine to derive a class or interface *C* denoted by *N* from the array of bytes using *L*, and then to create *C*, via the algorithm of §5.3.5. *L* should use the result of `defineClass` as the result of `loadClass`.
2. The class loader *L* can load *C* indirectly, by delegating the loading of *C* to some other class loader *L'*. This is accomplished by passing the argument *N* to an invocation of a method on *L'* (typically the `loadClass` method of

class `ClassLoader`). L should use the result of that method as the result of `loadClass`.

The following rules apply regardless of which operation is performed:

- If a class loader cannot find a purported representation of a class or interface denoted by N , it must throw a `ClassNotFoundException`. The process of loading and creating C then fails with a `NoClassDefFoundError` whose cause is the `ClassNotFoundException`.
- If a class loader finds a purported representation of C , but deriving C from the purported representation fails, then the process of loading and creating C fails for the same reason.
- If a class loader throws an exception other than a `ClassNotFoundException`, then the process of loading and creating C fails for the same reason.

If the invocation of `loadClass` on L has a result, then:

- If the result is `null`, or the result is a class or interface with a name other than N , then the result is discarded, and the process of loading and creation fails with a `NoClassDefFoundError`.
- Otherwise, the result is the created class or interface C . The Java Virtual Machine records that L is an initiating loader of C (§5.3.4). The process of loading and creating C succeeds.

Since JDK 1.1, Oracle's Java Virtual Machine implementation has invoked the one-argument `loadClass` method on a class loader to cause it to load a class or interface. The argument to `loadClass` is the name of the class or interface to be loaded. There is also a two-argument version of the `loadClass` method, where the second argument is a `boolean` that indicates whether the class or interface is to be linked or not. Only the two-argument version was supplied in JDK 1.0.2, and Oracle's Java Virtual Machine implementation relied on it to link the loaded class or interface. From JDK 1.1 onward, Oracle's Java Virtual Machine implementation links the class or interface directly, without relying on the class loader.

5.3.3 Creating Array Classes

The following steps are used to create the array class C denoted by the name N in association with the class loader L . L may be either the bootstrap class loader or a user-defined class loader.

First, the Java Virtual Machine determines whether L has already been recorded as an initiating loader of an array class with the same component type as N . If so, this class is C , and no array class creation is necessary.

Otherwise, the following steps are performed to create C :

1. If the component type is a reference type, the algorithm of this section (§5.3) is applied recursively using L in order to load and thereby create the component type of c .
2. The Java Virtual Machine creates a new array class with the indicated component type and number of dimensions.

If the component type is a reference type, the Java Virtual Machine marks c to have the defining loader of the component type as its defining loader. Otherwise, the Java Virtual Machine marks c to have the bootstrap class loader as its defining loader.

In any case, the Java Virtual Machine then records that L is an initiating loader for c (§5.3.4).

If the component type is a reference type, the accessibility of the array class is determined by the accessibility of its component type (§5.4.4). Otherwise, the array class is accessible to all classes and interfaces.

5.3.4 Loading Constraints

Ensuring type safe linkage in the presence of class loaders requires special care. It is possible that when two different class loaders initiate loading of a class or interface denoted by N , the name N may denote a different class or interface in each loader.

When a class or interface $C = \langle N_1, L_1 \rangle$ makes a symbolic reference to a field or method of another class or interface $D = \langle N_2, L_2 \rangle$, the symbolic reference includes a descriptor specifying the type of the field, or the return and argument types of the method. It is essential that any class or interface name N mentioned in the field or method descriptor (§4.3.2, §4.3.3) denote the same class or interface when loaded by L_1 and when loaded by L_2 .

To ensure this, the Java Virtual Machine imposes *loading constraints* of the form $N^{L_1} = N^{L_2}$ during preparation (§5.4.2) and resolution (§5.4.3). To enforce these constraints, the Java Virtual Machine will, at certain prescribed times (see §5.3.1, §5.3.2, §5.3.3, and §5.3.5), record that a particular loader is an initiating loader of a particular class. After recording that a loader is an initiating loader of a class, the Java Virtual Machine must immediately check to see if any loading constraints are violated. If so, the record is retracted, the Java Virtual Machine throws a `LinkageError`, and the loading operation that caused the recording to take place fails.

Similarly, after imposing a loading constraint (see §5.4.2, §5.4.3.2, §5.4.3.3, and §5.4.3.4), the Java Virtual Machine must immediately check to see if any loading

constraints are violated. If so, the newly imposed loading constraint is retracted, the Java Virtual Machine throws a `LinkageError`, and the operation that caused the constraint to be imposed (either resolution or preparation, as the case may be) fails.

The situations described here are the only times at which the Java Virtual Machine checks whether any loading constraints have been violated. A loading constraint is violated if, and only if, all the following four conditions hold:

- There exists a loader L such that L has been recorded by the Java Virtual Machine as an initiating loader of a class C named N .
- There exists a loader L' such that L' has been recorded by the Java Virtual Machine as an initiating loader of a class C' named N .
- The equivalence relation defined by the (reflexive, transitive closure of the) set of imposed constraints implies $N^L = N^{L'}$.
- $C \neq C'$.

A full discussion of class loaders and type safety is beyond the scope of this specification. For a more comprehensive discussion, readers are referred to *Dynamic Class Loading in the Java Virtual Machine* by Sheng Liang and Gilad Bracha (*Proceedings of the 1998 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications*).

5.3.5 Deriving a Class from a class File Representation

Class loaders require the cooperation of the Java Virtual Machine to derive and create a class or interface from a binary representation provided by the loader (§5.3.1, §5.3.2). The following steps are used to derive a class or interface C denoted by N from a purported representation in class file format when requested by a class loader L .

1. First, the Java Virtual Machine determines whether the request by class loader L to derive a class or interface denoted by N is permitted.

If L has already been recorded as an initiating loader of a class or interface denoted by N , derivation throws a `LinkageError`.

If the Java Virtual Machine is already in the process of deriving a class or interface denoted by N as requested by class loader L , derivation throws a `ClassCircularityError`.

2. Next, the Java Virtual Machine attempts to parse the purported representation. The purported representation may not in fact be a valid representation of *c*, so derivation must detect the following problems:

- If the purported representation is not a `ClassFile` structure (§4.1, §4.8), derivation throws a `ClassFormatError`.
- Otherwise, if the purported representation is not of a supported major or minor version (§4.1), derivation throws an `UnsupportedClassVersionError`.

`UnsupportedClassVersionError`, a subclass of `ClassFormatError`, was introduced in JDK 1.2 to enable easy identification of a `ClassFormatError` caused by an attempt to load a class whose representation uses an unsupported version of the class file format. In JDK 1.1 and earlier, an instance of `NoClassDefFoundError` or `ClassFormatError` was thrown in case of an unsupported version, depending on whether the class was being loaded by the system class loader or a user-defined class loader.

- Otherwise, if the purported representation does not actually represent a class or interface named *N*, derivation throws a `NoClassDefFoundError`.

This occurs when the purported representation has either a `this_class` item which specifies a name other than *N*, or an `access_flags` item which has the `ACC_MODULE` flag set.

3. If *c* has a direct superclass, the symbolic reference from *c* to its direct superclass is resolved using the algorithm of §5.4.3.1, with *L* acting as the defining loader

of *c*. Note that if *c* is an interface it must have *Object* as its direct superclass, which must already have been loaded. Only *Object* has no direct superclass.

Any exception that can be thrown as a result of failure of class or interface resolution can be thrown as a result of derivation. In addition, derivation must detect the following problems:

- If the class or interface named as the direct superclass of *c* is in fact an interface or a final class, derivation throws an *IncompatibleClassChangeError*.
- Otherwise, if the class named as the direct superclass of *c* has a *PermittedSubclasses* attribute (§4.7.31) and any of the following is true, derivation throws an *IncompatibleClassChangeError*:
 - The superclass is in a different run-time module than *c* (§5.3.6).
 - *c* does not have its *ACC_PUBLIC* flag set (§4.1) and the superclass is in a different run-time package than *c* (§5.3).
 - No entry in the *classes* array of the superclass's *PermittedSubclasses* attribute refers to a class or interface with the name *N*.
- Otherwise, if *c* is a class and some instance method declared in *c* can override (§5.4.5) a final instance method declared in a superclass of *c*, derivation throws an *IncompatibleClassChangeError*.

4. If *c* has any direct superinterfaces, the symbolic references from *c* to its direct superinterfaces are resolved using the algorithm of §5.4.3.1, with *L* acting as the defining loader of *c*.

Any exception that can be thrown as a result of failure of class or interface resolution can be thrown as a result of derivation. In addition, derivation must detect the following problems:

- If any class or interface named as a direct superinterface of *c* is not in fact an interface, derivation throws an `IncompatibleClassChangeError`.
- Otherwise, for each direct superinterface named by *c*, if the superinterface has a `PermittedSubclasses` attribute (§4.7.31) and any of the following is true, derivation throws an `IncompatibleClassChangeError`:
 - The superinterface is in a different run-time module than *c*.
 - *c* does not have its `ACC_PUBLIC` flag set (§4.1) and the superinterface is in a different run-time package than *c*.
 - No entry in the `classes` array of the superinterface's `PermittedSubclasses` attribute refers to a class or interface with the name *N*.

If no exception is thrown in steps 1-4, then derivation of the class or interface *c* succeeds. The Java Virtual Machine marks *c* to have *L* as its defining loader, records that *L* is an initiating loader of *c* (§5.3.4), and creates *c* in the method area (§2.5.4).

When derivation succeeds, the process of loading and creating *c* is not complete until every class loader that was involved in loading *c* (directly or indirectly) returns *c* as its result. Depending on the behavior of user-defined class loaders, the process of loading and creating *c* may yet fail (§5.3.2).

If an exception is thrown in steps 1-4, then derivation of the class or interface *c* fails with that exception.

Requests to derive a class or interface may be made concurrently by class loader code executing in multiple threads, but the derivation process is sequential. The Java Virtual Machine implementation ensures that only one request by a given class loader to derive a class or interface of a given name is processed at a time, while all other such requests wait until the first request is complete.

As specified by the derivation process, if the first request is successful, no subsequent requests will be permitted. The `ClassLoader` API provides mechanisms to synchronize derivation requests and cache successful results so that redundant derivation requests do not occur.

5.3.6 Modules and Layers

The Java Virtual Machine supports the organization of classes and interfaces into modules. The membership of a class or interface *C* in a module *M* is used to control access to *C* from classes and interfaces in modules other than *M* (§5.4.4).

Module membership is defined in terms of run-time packages (§5.3). A program determines the names of the packages in each module, and the class loaders that will create the classes and interfaces of the named packages; it then specifies the packages and class loaders to an invocation of the `defineModules` method of the class `ModuleLayer`. Invoking `defineModules` causes the Java Virtual Machine to create new *run-time modules* that are associated with the run-time packages of the class loaders.

Every run-time module indicates the run-time packages that it *exports*, which influences access to the `public` classes and interfaces in those run-time packages. Every run-time module also indicates the other run-time modules that it *reads*, which influences access by its own code to the `public` types and interfaces in those run-time modules.

We say that *a class is in a run-time module* iff the class's run-time package is associated (or will be associated, if the class is actually created) with that run-time module.

A class created by a class loader is in exactly one run-time package and therefore exactly one run-time module, because the Java Virtual Machine does not support a run-time package being associated with (or more evocatively, "split across") multiple run-time modules.

A run-time module is implicitly bound to exactly one class loader, by the semantics of `defineModules`. On the other hand, a class loader may create classes in more than one run-time module, because the Java Virtual Machine does not require all the run-time packages of a class loader to be associated with the same run-time module.

In other words, the relationship between class loaders and run-time modules need not be 1:1. For a given set of modules to be loaded, if a program can determine that the names of the packages in each module are found only in that module, then the program may specify only one class loader to the invocation of `defineModules`. This class loader will create classes across multiple run-time modules.

Every run-time module created by `defineModules` is part of a *layer*. A layer represents a set of class loaders that jointly serve to create classes in a set of run-time modules. There are two kinds of layers: the boot layer supplied by the Java Virtual Machine, and user-defined layers. The boot layer is created at Java Virtual Machine

startup in an implementation-dependent manner. It associates the standard run-time module `java.base` with standard run-time packages defined by the bootstrap class loader, such as `java.lang`. User-defined layers are created by programs in order to construct sets of run-time modules that depend on `java.base` and other standard run-time modules.

A run-time module is implicitly part of exactly one layer, by the semantics of `defineModules`. However, a class loader may create classes in the run-time modules of different layers, because the same class loader may be specified to multiple invocations of `defineModules`. Access control is governed by a class's run-time module, not by the class loader which created the class or by the layer(s) which the class loader serves.

The set of class loaders specified for a layer, and the set of run-time modules which are part of a layer, are immutable after the layer is created. However, the `ModuleLayer` class affords programs a degree of dynamic control over the relationships between the run-time modules in a user-defined layer.

If a user-defined layer contains more than one class loader, then any delegation between the class loaders is the responsibility of the program that created the layer. The Java Virtual Machine does not check that the layer's class loaders delegate to each other in accordance with how the layer's run-time modules read each other. Moreover, if the layer's run-time modules are modified via the `ModuleLayer` class to read additional run-time modules, then the Java Virtual Machine does not check that the layer's class loaders are modified by some out-of-band mechanism to delegate in a corresponding fashion.

There are similarities and differences between class loaders and layers. On the one hand, a layer is similar to a class loader in that each may delegate to, respectively, one or more parent layers or class loaders that created, respectively, modules or classes at an earlier time. That is, the set of modules specified to a layer may depend on modules not specified to the layer, and instead specified previously to one or more parent layers. On the other hand, a layer may be used to create new modules only once, whereas a class loader may be used to create new classes or interfaces at any time via multiple invocations of the `defineClass` method.

It is possible for a class loader to define a class or interface in a run-time package that was not associated with a run-time module by any of the layers which the class loader serves. This may occur if the run-time package embodies a named package that was not specified to `defineModules`, or if the class or interface has a simple binary name (§4.2.1) and thus is a member of a run-time package that embodies an unnamed package (JLS §7.4.2). In either case, the class or interface is treated as a member of a special run-time module which is implicitly bound to the class loader. This special run-time module is known as the *unnamed module* of the

class loader. The run-time package of the class or interface is associated with the unnamed module of the class loader. There are special rules for unnamed modules, designed to maximize their interoperability with other run-time modules, as follows:

- A class loader's unnamed module is distinct from all other run-time modules bound to the same class loader.
- A class loader's unnamed module is distinct from all run-time modules (including unnamed modules) bound to other class loaders.
- Every unnamed module reads every run-time module.
- Every unnamed module exports, to every run-time module, every run-time package associated with itself.

5.4 Linking

Linking a class or interface involves verifying and preparing that class or interface, its direct superclass, its direct superinterfaces, and its element type (if it is an array type), if necessary. Linking also involves resolution of symbolic references in the class or interface, though not necessarily at the same time as the class or interface is verified and prepared.

This specification allows an implementation flexibility as to when linking activities (and, because of recursion, loading) take place, provided that all of the following properties are maintained:

- A class or interface is completely loaded before it is linked.
- A class or interface is completely verified and prepared before it is initialized.
- Errors detected during linkage are thrown at a point in the program where some action is taken by the program that might, directly or indirectly, require linkage to the class or interface involved in the error.
- A symbolic reference to a dynamically-computed constant is not resolved until either (i) an *ldc*, *ldc_w*, or *ldc2_w* instruction that refers to it is executed, or (ii) a bootstrap method that refers to it as a static argument is invoked.

A symbolic reference to a dynamically-computed call site is not resolved until a bootstrap method that refers to it as a static argument is invoked.

For example, a Java Virtual Machine implementation may choose a "lazy" linkage strategy, where each symbolic reference in a class or interface (other than the symbolic references above) is resolved individually when it is used. Alternatively,

an implementation may choose an "eager" linkage strategy, where all symbolic references are resolved at once when the class or interface is being verified. This means that the resolution process may continue, in some implementations, after a class or interface has been initialized. Whichever strategy is followed, any error detected during resolution must be thrown at a point in the program that (directly or indirectly) uses a symbolic reference to the class or interface.

Because linking involves the allocation of new data structures, it may fail with an `OutOfMemoryError`.

5.4.1 Verification

Verification (§4.10) ensures that the binary representation of a class or interface is structurally correct (§4.9). Verification may cause additional classes and interfaces to be loaded (§5.3) but need not cause them to be verified or prepared.

If the binary representation of a class or interface does not satisfy the static or structural constraints listed in §4.9, then a `VerifyError` must be thrown at the point in the program that caused the class or interface to be verified.

If an attempt by the Java Virtual Machine to verify a class or interface fails because an error is thrown that is an instance of `LinkageError` (or a subclass), then subsequent attempts to verify the class or interface always fail with the same error that was thrown as a result of the initial verification attempt.

5.4.2 Preparation

Preparation involves creating the static fields for a class or interface and initializing such fields to their default values (§2.3, §2.4). This does not require the execution of any Java Virtual Machine code; explicit initializers for static fields are executed as part of initialization (§5.5), not preparation.

During preparation of a class or interface c , the Java Virtual Machine also imposes loading constraints (§5.3.4):

1. Let L_1 be the defining loader of c . For each instance method m declared in c that can override (§5.4.5) an instance method declared in a superclass or superinterface $D = \langle N_2, L_2 \rangle$, for each class or interface name N mentioned by the descriptor of m (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$.
2. For each instance method m declared in a superinterface $I = \langle N_3, L_3 \rangle$ of c , if c does not itself declare an instance method that can override m , then a method is selected (§5.4.6) with respect to c and the method m in I . Let $D = \langle N_2, L_2 \rangle$

be the class or interface that declares the selected method. For each class or interface name N mentioned by the descriptor of m , the Java Virtual Machine imposes the loading constraint $N^{L-2} = N^{L-3}$.

Preparation may occur at any time following creation but must be completed prior to initialization.

5.4.3 Resolution

Many Java Virtual Machine instructions - *anewarray*, *checkcast*, *getfield*, *getstatic*, *instanceof*, *invokedynamic*, *invokeinterface*, *invokespecial*, *invokestatic*, *invokevirtual*, *ldc*, *ldc_w*, *ldc2_w*, *multianewarray*, *new*, *putfield*, and *putstatic* - rely on symbolic references in the run-time constant pool. Execution of any of these instructions requires *resolution* of the symbolic reference.

Resolution is the process of dynamically determining one or more concrete values from a symbolic reference in the run-time constant pool. Initially, all symbolic references in the run-time constant pool are unresolved.

Resolution of an unresolved symbolic reference to (i) a class or interface, (ii) a field, (iii) a method, (iv) a method type, (v) a method handle, or (vi) a dynamically-computed constant, proceeds in accordance with the rules given in §5.4.3.1 through §5.4.3.5. In the first three of those sections, the class or interface in whose run-time constant pool the symbolic reference appears is labeled D . Then:

- If no error occurs during resolution of the symbolic reference, then resolution succeeds.

Subsequent attempts to resolve the symbolic reference always succeed trivially and result in the same entity produced by the initial resolution. If the symbolic reference is to a dynamically-computed constant, the bootstrap method is not re-executed for these subsequent attempts.

- If an error occurs during resolution of the symbolic reference, then it is either (i) an instance of `IncompatibleClassChangeError` (or a subclass); (ii) an instance of `Error` (or a subclass) that arose from resolution or invocation of a bootstrap method; or (iii) an instance of `LinkageError` (or a subclass) that arose because class loading failed or a loader constraint was violated. The error must be thrown at a point in the program that (directly or indirectly) uses the symbolic reference.

Subsequent attempts to resolve the symbolic reference always fail with the same error that was thrown as a result of the initial resolution attempt. If the symbolic reference is to a dynamically-computed constant, the bootstrap method is not re-executed for these subsequent attempts.

Because errors occurring on an initial attempt at resolution are thrown again on subsequent attempts, a class in one module that attempts to access, via resolution of a symbolic reference in its run-time constant pool, an unexported public type in a different module will always receive the same error indicating an inaccessible type (§5.4.4), *even if the Java SE Platform API is used to dynamically export the public type's package at some time after the class's first attempt.*

Resolution of an unresolved symbolic reference to a dynamically-computed call site proceeds in accordance with the rules given in §5.4.3.6. Then:

- If no error occurs during resolution of the symbolic reference, then resolution succeeds *solely for the instruction in the class file that required resolution*. This instruction necessarily has an opcode of *invokedynamic*.

Subsequent attempts to resolve the symbolic reference *by that instruction in the class file* always succeed trivially and result in the same entity produced by the initial resolution. The bootstrap method is not re-executed for these subsequent attempts.

The symbolic reference is still unresolved for all other instructions in the class file, of any opcode, which indicate the same entry in the run-time constant pool as the *invokedynamic* instruction above.

- If an error occurs during resolution of the symbolic reference, then it is either (i) an instance of `IncompatibleClassChangeError` (or a subclass); (ii) an instance of `Error` (or a subclass) that arose from resolution or invocation of a bootstrap method; or (iii) an instance of `LinkageError` (or a subclass) that arose because class loading failed or a loader constraint was violated. The error must be thrown at a point in the program that (directly or indirectly) uses the symbolic reference.

Subsequent attempts *by the same instruction in the class file* to resolve the symbolic reference always fail with the same error that was thrown as a result of the initial resolution attempt. The bootstrap method is not re-executed for these subsequent attempts.

The symbolic reference is still unresolved for all other instructions in the class file, of any opcode, which indicate the same entry in the run-time constant pool as the *invokedynamic* instruction above.

Certain of the instructions above require additional linking checks when resolving symbolic references. For instance, in order for a *getfield* instruction to successfully resolve the symbolic reference to the field on which it operates, it must not only complete the field resolution steps given in §5.4.3.2 but also check that the field is not static. If it is a static field, a linking exception must be thrown.

Linking exceptions generated by checks that are specific to the execution of a particular Java Virtual Machine instruction are given in the description of that instruction and are not covered in this general discussion of resolution. Note that such exceptions, although described as part of the execution of Java Virtual Machine instructions rather than resolution, are still properly considered failures of resolution.

5.4.3.1 *Class and Interface Resolution*

To resolve an unresolved symbolic reference from D to a class or interface C denoted by N , the following steps are performed:

1. The defining loader of D is used to load and thereby create a class or interface denoted by N . This class or interface is C . The details of the process are given in §5.3.

Any exception that can be thrown as a result of failure to load and thereby create C can thus be thrown as a result of failure of class and interface resolution.

2. If C is an array class and its element type is a reference type, then a symbolic reference to the class or interface representing the element type is resolved by invoking the algorithm in §5.4.3.1 recursively.
3. Finally, access control is applied for the access from D to C (§5.4.4).

If steps 1 and 2 succeed but step 3 fails, C is still valid and usable. Nevertheless, resolution fails, and D is prohibited from accessing C .

5.4.3.2 *Field Resolution*

To resolve an unresolved symbolic reference from D to a field in a class or interface C , the symbolic reference to C given by the field reference must first be resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of a class or interface reference can be thrown as a result of failure of field resolution. If the reference to C can be successfully resolved, an exception relating to the failure of resolution of the field reference itself can be thrown.

When resolving a field reference, field resolution first attempts to look up the referenced field in C and its superclasses:

1. If C declares a field with the name and descriptor specified by the field reference, field lookup succeeds. The declared field is the result of the field lookup.
2. Otherwise, field lookup is applied recursively to the direct superinterfaces of the specified class or interface C .

3. Otherwise, if C has a superclass S , field lookup is applied recursively to S .
4. Otherwise, field lookup fails.

Then, the result of field resolution is determined:

- If field lookup failed, field resolution throws a `NoSuchFieldError`.
- Otherwise, field lookup succeeded. Access control is applied for the access from D to the field which is the result of field lookup (§5.4.4). Then:
 - If access control failed, field resolution fails for the same reason.
 - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

Let $\langle E, L_1 \rangle$ be the class or interface in which the referenced field is actually declared. Let L_2 be the defining loader of D .

For any class or interface name N mentioned by the descriptor of the referenced field (§4.3.2), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

If imposing this constraint results in any loading constraints being violated, then field resolution fails. Otherwise, field resolution succeeds.

5.4.3.3 *Method Resolution*

To resolve an unresolved symbolic reference from D to a method in a class C , the symbolic reference to C given by the method reference is first resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of a class reference can be thrown as a result of failure of method resolution. If the reference to C can be successfully resolved, exceptions relating to the resolution of the method reference itself can be thrown.

When resolving a method reference:

1. If C is an interface, method resolution throws an `IncompatibleClassChangeError`.
2. Otherwise, method resolution attempts to locate the referenced method in C and its superclasses:
 - If C declares exactly one method with the name specified by the method reference, and the declaration is a signature polymorphic method (§2.9.3), then method lookup succeeds. The descriptor specified by the method

reference is resolved, as if by resolution of an unresolved symbolic reference to a method type (§5.4.3.5).

The resolved method is the signature polymorphic method declaration. It is not necessary for *C* to declare a method with the descriptor specified by the method reference.

- Otherwise, if *C* declares a method with the name and descriptor specified by the method reference, method lookup succeeds.
 - Otherwise, if *C* has a superclass, step 2 of method resolution is recursively invoked on the direct superclass of *C*.
3. Otherwise, method resolution attempts to locate the referenced method in the superinterfaces of the specified class *C*:
- If the *maximally-specific superinterface methods* of *C* for the name and descriptor specified by the method reference include exactly one method that does not have its ACC_ABSTRACT flag set, then this method is chosen and method lookup succeeds.
 - Otherwise, if any superinterface of *C* declares a method with the name and descriptor specified by the method reference that has neither its ACC_PRIVATE flag nor its ACC_STATIC flag set, one of these is arbitrarily chosen and method lookup succeeds.
 - Otherwise, method lookup fails.

A *maximally-specific superinterface method* of a class or interface *C* for a particular method name and descriptor is any method for which all of the following are true:

- The method is declared in a superinterface (direct or indirect) of *C*.
- The method is declared with the specified name and descriptor.
- The method has neither its ACC_PRIVATE flag nor its ACC_STATIC flag set.
- Where the method is declared in interface *I*, there exists no other maximally-specific superinterface method of *C* with the specified name and descriptor that is declared in a subinterface of *I*.

The result of method resolution is determined as follows:

- If method lookup failed, method resolution throws a `NoSuchMethodError`.

- Otherwise, method lookup succeeded. Access control is applied for the access from D to the method which is the result of method lookup (§5.4.4). Then:
 - If access control failed, method resolution fails for the same reason.
 - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

Let $\langle E, L_1 \rangle$ be the class or interface in which the referenced method m is actually declared. Let L_2 be the defining loader of D .

For each class or interface name N mentioned by the descriptor of the referenced method (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

If imposing these constraints results in any loading constraints being violated, then method resolution fails. Otherwise, method resolution succeeds.

When resolution searches for a method in the class's superinterfaces, the best outcome is to identify a maximally-specific non-abstract method. It is possible that this method will be chosen by method selection, so it is desirable to add class loader constraints for it.

Otherwise, the result is nondeterministic. This is not new: *The Java® Virtual Machine Specification* has never identified exactly which method is chosen, and how "ties" should be broken. Prior to Java SE 8, this was mostly an unobservable distinction. However, beginning with Java SE 8, the set of interface methods is more heterogenous, so care must be taken to avoid problems with nondeterministic behavior. Thus:

- Superinterface methods that are `private` and `static` are ignored by resolution. This is consistent with the Java programming language, where such interface methods are not inherited.
- Any behavior controlled by the resolved method should not depend on whether the method is abstract or not.

Note that if the result of resolution is an abstract method, the referenced class C may be non-abstract. Requiring C to be abstract would conflict with the nondeterministic choice of superinterface methods. Instead, resolution assumes that the run time class of the invoked object has a concrete implementation of the method.

5.4.3.4 Interface Method Resolution

To resolve an unresolved symbolic reference from D to an interface method in an interface C , the symbolic reference to C given by the interface method reference is first resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of an interface reference can be thrown as a result of failure of interface method resolution. If the reference to C can be successfully resolved,

exceptions relating to the resolution of the interface method reference itself can be thrown.

When resolving an interface method reference:

1. If *c* is not an interface, interface method resolution throws an `IncompatibleClassChangeError`.
2. Otherwise, if *c* declares a method with the name and descriptor specified by the interface method reference, method lookup succeeds.
3. Otherwise, if the class object declares a method with the name and descriptor specified by the interface method reference, which has its `ACC_PUBLIC` flag set and does not have its `ACC_STATIC` flag set, method lookup succeeds.
4. Otherwise, if the maximally-specific superinterface methods (§5.4.3.3) of *c* for the name and descriptor specified by the method reference include exactly one method that does not have its `ACC_ABSTRACT` flag set, then this method is chosen and method lookup succeeds.
5. Otherwise, if any superinterface of *c* declares a method with the name and descriptor specified by the method reference that has neither its `ACC_PRIVATE` flag nor its `ACC_STATIC` flag set, one of these is arbitrarily chosen and method lookup succeeds.
6. Otherwise, method lookup fails.

The result of interface method resolution is determined as follows:

- If method lookup failed, interface method resolution throws a `NoSuchMethodError`.

- Otherwise, method lookup succeeded. Access control is applied for the access from D to the method which is the result of method lookup (§5.4.4). Then:
 - If access control failed, interface method resolution fails for the same reason.
 - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

Let $\langle E, L_1 \rangle$ be the class or interface in which the referenced interface method m is actually declared. Let L_2 be the defining loader of D .

For each class or interface name N mentioned by the descriptor of the referenced method (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

If imposing these constraints results in any loading constraints being violated, then interface method resolution fails. Otherwise, interface method resolution succeeds.

Access control is necessary because interface method resolution may pick a `private` method of interface `C`. (Prior to Java SE 8, the result of interface method resolution could be a non-`public` method of class `Object` or a `static` method of class `Object`; such results were not consistent with the inheritance model of the Java programming language, and are disallowed in Java SE 8 and above.)

5.4.3.5 *Method Type and Method Handle Resolution*

To resolve an unresolved symbolic reference to a method type, it is as if resolution occurs of unresolved symbolic references to the classes and interfaces (§5.4.3.1) whose names are mentioned by the method descriptor (§4.3.3), in the order in which they are mentioned.

Any exception that can be thrown as a result of failure of resolution of a reference to a class or interface can thus be thrown as a result of failure of method type resolution.

The result of successful method type resolution is a reference to an instance of `java.lang.invoke.MethodType` which represents the method descriptor.

Method type resolution occurs regardless of whether the run-time constant pool actually contains symbolic references to classes and interfaces indicated in the method descriptor. Also, the resolution is deemed to occur on *unresolved* symbolic references, so a failure to resolve one method type will not necessarily lead to a later failure to resolve another method type with the same textual method descriptor, if suitable classes and interfaces can be loaded by the later time.

Resolution of an unresolved symbolic reference to a method handle is more complicated. Each method handle resolved by the Java Virtual Machine has an equivalent instruction sequence called its *bytecode behavior*, indicated by the method handle's *kind*. The integer values and descriptions of the nine kinds of method handle are given in Table 5.4.3.5-A.

Symbolic references by an instruction sequence to fields or methods are indicated by $C.x:T$, where x and T are the name and descriptor (§4.3.2, §4.3.3) of the field or method, and C is the class or interface in which the field or method is to be found.

Table 5.4.3.5-A. Bytecode Behaviors for Method Handles

Kind	Description	Interpretation
1	REF_getField	getfield $C.f:T$
2	REF_getStatic	getstatic $C.f:T$
3	REF_putField	putfield $C.f:T$
4	REF_putStatic	putstatic $C.f:T$
5	REF_invokeVirtual	invokevirtual $C.m:(A^*)T$
6	REF_invokeStatic	invokestatic $C.m:(A^*)T$
7	REF_invokeSpecial	invokespecial $C.m:(A^*)T$
8	REF_newInvokeSpecial	new C ; dup; invokespecial $C.<init>:(A^*)V$
9	REF_invokeInterface	invokeinterface $C.m:(A^*)T$

Let MH be the symbolic reference to a method handle (§5.1) being resolved. Also:

- Let R be the symbolic reference to a field or method given by MH .

For example, R is a symbolic reference to $C.f$ for bytecode behavior of kind 1, and a symbolic reference to $C.<init>$ for bytecode behavior of kind 8.

- Let C be the class, interface, or array type referenced by R .
- Let T be the type given by the field descriptor of R , or the return type given by the method descriptor of R . Let A^* be the sequence (perhaps empty) of parameter types given by the method descriptor of R .

To resolve MH , all symbolic references to classes, interfaces, fields, and methods in MH 's bytecode behavior are resolved, using the following three steps:

1. R is resolved. This occurs as if by field resolution (§5.4.3.2) when MH 's bytecode behavior is kind 1, 2, 3, or 4, and as if by method resolution (§5.4.3.3) when MH 's

bytecode behavior is kind 5, 6, 7, or 8, and as if by interface method resolution (§5.4.3.4) when *MH*'s bytecode behavior is kind 9.

2. The following constraints apply to the result of resolving *R*. These constraints correspond to those that would be enforced during verification or execution of the instruction sequence for the relevant bytecode behavior.
 - If *MH*'s bytecode behavior is kind 7 (`REF_invokeSpecial`), then *C* must be the current class or interface, a superclass of the current class, a direct superinterface of the current class or interface, or `Object`.
 - If *MH*'s bytecode behavior is kind 8 (`REF_newInvokeSpecial`), then *R* must resolve to an instance initialization method declared in class *C*.
 - If *R* resolves to a protected member, then the following rules apply depending on the kind of *MH*'s bytecode behavior:
 - For kinds 1, 3, and 5 (`REF_getField`, `REF_putField`, and `REF_invokeVirtual`): If *C.f* or *C.m* resolved to a protected field or method, and *C* is in a different run-time package than the current class, then *C* must be a subclass of the current class.
 - For kind 8 (`REF_newInvokeSpecial`): If *C* . `<init>` resolved to a protected method, then *C* must be declared in the same run-time package as the current class.
 - *R* must resolve to a static or non-static member depending on the kind of *MH*'s bytecode behavior:
 - For kinds 1, 3, 5, 7, and 9 (`REF_getField`, `REF_putField`, `REF_invokeVirtual`, `REF_invokeSpecial`, and `REF_invokeInterface`): *C.f* or *C.m* must resolve to a non-static field or method.
 - For kinds 2, 4, and 6 (`REF_getStatic`, `REF_putStatic`, and `REF_invokeStatic`): *C.f* or *C.m* must resolve to a static field or method.

3. A reference to an instance of `java.lang.invoke.MethodType` is obtained as if by resolution of an unresolved symbolic reference to a method type that contains the method descriptor specified in Table 5.4.3.5-B for the kind of *MH*.

It is as if the symbolic reference to a method handle contains a symbolic reference to the method type that the resolved method handle will eventually have. The detailed structure of the method type is obtained by inspecting Table 5.4.3.5-B.

Table 5.4.3.5-B. Method Descriptors for Method Handles

Kind	Description	Method descriptor
1	REF_getField	(C)T
2	REF_getStatic	()T
3	REF_putField	(C,T)V
4	REF_putStatic	(T)V
5	REF_invokeVirtual	(C,A*)T
6	REF_invokeStatic	(A*)T
7	REF_invokeSpecial	(C,A*)T
8	REF_newInvokeSpecial	(A*)C
9	REF_invokeInterface	(C,A*)T

In steps 1 and 3, any exception that can be thrown as a result of failure of resolution of a symbolic reference to a class, interface, field, or method can be thrown as a result of failure of method handle resolution. In step 2, any failure due to the specified constraints causes a failure of method handle resolution due to an `IllegalAccessException`.

The intent is that resolving a method handle can be done in exactly the same circumstances that the Java Virtual Machine would successfully verify and resolve the symbolic references in the bytecode behavior. In particular, method handles to `private`, `protected`, and `static` members can be created in exactly those classes for which the corresponding normal accesses are legal.

The result of successful method handle resolution is a reference to an instance of `java.lang.invoke.MethodHandle` which represents the method handle *MH*.

The type descriptor of this `java.lang.invoke.MethodHandle` instance is the `java.lang.invoke.MethodType` instance produced in the third step of method handle resolution above.

The type descriptor of a method handle is such that a valid call to `invokeExact` in `java.lang.invoke.MethodHandle` on the method handle has exactly the same stack effects as the bytecode behavior. Calling this method handle on a valid set of arguments has exactly the same effect and returns the same result (if any) as the corresponding bytecode behavior.

If the method referenced by *R* has the `ACC_VARARGS` flag set (§4.6), then the `java.lang.invoke.MethodHandle` instance is a variable arity method handle; otherwise, it is a fixed arity method handle.

A variable arity method handle performs argument list boxing (JLS §15.12.4.2) when invoked via `invoke`, while its behavior with respect to `invokeExact` is as if the `ACC_VARARGS` flag were not set.

Method handle resolution throws an `IncompatibleClassChangeError` if the method referenced by *R* has the `ACC_VARARGS` flag set and either *A** is an empty sequence or the last parameter type in *A** is not an array type. That is, creation of a variable arity method handle fails.

An implementation of the Java Virtual Machine is not required to intern method types or method handles. That is, two distinct symbolic references to method types or method handles which are structurally identical might not resolve to the same instance of `java.lang.invoke.MethodType` or `java.lang.invoke.MethodHandle` respectively.

The `java.lang.invoke.MethodHandles` class in the Java SE Platform API allows creation of method handles with no bytecode behavior. Their behavior is defined by the method of `java.lang.invoke.MethodHandles` that creates them. For example, a method handle may, when invoked, first apply transformations to its argument values, then supply the transformed values to the invocation of another method handle, then apply a transformation to the value returned from that invocation, then return the transformed value as its own result.

5.4.3.6 *Dynamically-Computed Constant and Call Site Resolution*

To resolve an unresolved symbolic reference *R* to a dynamically-computed constant or call site, there are three tasks. First, *R* is examined to determine which code will serve as its *bootstrap method*, and which arguments will be passed to that code. Second, the arguments are packaged into an array and the bootstrap method is invoked. Third, the result of the bootstrap method is validated, and used as the result of resolution.

The first task involves the following steps:

1. R gives a symbolic reference to a *bootstrap method handle*. The bootstrap method handle is resolved (§5.4.3.5) to obtain a reference to an instance of `java.lang.invoke.MethodHandle`.

Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a method handle can be thrown in this step.

If R is a symbolic reference to a dynamically-computed constant, then let D be the type descriptor of the bootstrap method handle. (That is, D is a reference to an instance of `java.lang.invoke.MethodType`.) The first parameter type indicated by D must be `java.lang.invoke.MethodHandles.Lookup`, or else resolution fails with a `BootstrapMethodError`. For historical reasons, the bootstrap method handle for a dynamically-computed call site is not similarly constrained.

2. If R is a symbolic reference to a dynamically-computed constant, then it gives a field descriptor.

If the field descriptor indicates a primitive type, then a reference to the pre-defined `Class` object representing that type is obtained (see the method `isPrimitive` in class `Class`).

Otherwise, the field descriptor indicates a class or interface type, or an array type. A reference to the `Class` object representing the type indicated by the field descriptor is obtained, as if by resolution of an unresolved symbolic reference to a class or interface (§5.4.3.1) whose name corresponds to the type indicated by the field descriptor.

Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a class or interface can be thrown in this step.

3. If R is a symbolic reference to a dynamically-computed call site, then it gives a method descriptor.

A reference to an instance of `java.lang.invoke.MethodType` is obtained, as if by resolution of an unresolved symbolic reference to a method type (§5.4.3.5) with the same parameter and return types as the method descriptor.

Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a method type can be thrown in this step.

4. R gives zero or more *static arguments*, which communicate application-specific metadata to the bootstrap method. Each static argument A is resolved, in the order given by R , as follows:

- If A is a string constant, then a reference to its instance of class `String` is obtained.

- If *A* is a numeric constant, then a reference to an object representing the number is obtained by the following procedure:
 - a. Let *v* be the value of the numeric constant, and let *τ* be a field descriptor which corresponds to the type of the numeric constant.
 - b. Let *MH* be a method handle produced as if by invocation of the `identity` method of `java.lang.invoke.MethodHandles` with an argument representing the class `Object`.
 - c. A reference to an object is obtained as if by the invocation `MH.invoke(v)` with method descriptor `(T)Ljava/lang/Object;`.
- If *A* is a symbolic reference to a dynamically-computed constant with a field descriptor indicating a primitive type *T*, then *A* is resolved, producing a primitive value *v*. Given *v* and *T*, a reference is obtained to an object encoding *v* according to the procedure specified above for numeric constants.
- If *A* is any other kind of symbolic reference, then the result is the result of resolving *A*.

Among the symbolic references in the run-time constant pool, symbolic references to dynamically-computed constants are special because they are derived from `constant_pool` entries that can syntactically refer to themselves via the `BootstrapMethods` attribute (§4.7.23). However, the Java Virtual Machine does not support resolving a symbolic reference to a dynamically-computed constant that depends on itself (that is, as a static argument to its own bootstrap method). Accordingly, when both *R* and *A* are symbolic references to dynamically-computed constants, if *A* is the same as *R* or *A* gives a static argument that (directly or indirectly) references *R*, then resolution fails with a `StackOverflowError` at the point where re-resolution of *R* would be required.

Unlike class initialization (§5.5), where cycles are allowed between uninitialized classes, resolution does not allow cycles in symbolic references to dynamically-computed constants. If an implementation of resolution makes recursive use of a stack, then a `StackOverflowError` will occur naturally. If not, the implementation is required to detect the cycle rather than, say, looping infinitely or returning a default value for the dynamically-computed constant.

A similar cycle may arise if the body of a bootstrap method makes reference to a dynamically-computed constant currently being resolved. This has always been possible for *invokedynamic* bootstraps, and does not require special treatment in

resolution; the recursive `invokeWithArguments` calls will naturally lead to a `StackOverflowError`.

Any exception that can be thrown as a result of failure of resolution of a symbolic reference can be thrown in this step.

The second task, to invoke the bootstrap method handle, involves the following steps:

1. An array is allocated with component type `Object` and length $n+3$, where n is the number of static arguments given by R ($n \geq 0$).

The zeroth component of the array is set to a reference to an instance of `java.lang.invoke.MethodHandles.Lookup` for the class in which R occurs, produced as if by invocation of the `lookup` method of `java.lang.invoke.MethodHandles`.

The first component of the array is set to a reference to an instance of `String` that denotes N , the unqualified name given by R .

The second component of the array is set to the reference to an instance of `Class` or `java.lang.invoke.MethodType` that was obtained earlier for the field descriptor or method descriptor given by R .

Subsequent components of the array are set to the references that were obtained earlier from resolving R 's static arguments, if any. The references appear in the array in the same order as the corresponding static arguments are given by R .

A Java Virtual Machine implementation may be able to skip allocation of the array and, without any change in observable behavior, pass the arguments directly to the bootstrap method.

2. The bootstrap method handle is invoked, as if by the invocation `BMH.invokeWithArguments(args)`, where `BMH` is the bootstrap method handle and `args` is the array allocated above.

Due to the behavior of the `invokeWithArguments` method of `java.lang.invoke.MethodHandle`, the type descriptor of the bootstrap method handle need not exactly match the run-time types of the arguments. For example, the second parameter type of the bootstrap method handle (corresponding to the unqualified name given in the first component of the array above) could be `Object` instead of `String`. If the bootstrap method handle is variable arity, then some or all of the arguments may be collected into a trailing array parameter.

The invocation occurs within a thread that is attempting resolution of this symbolic reference. If there are several such threads, the bootstrap method

handle may be invoked concurrently. Bootstrap methods which access global application data should take the usual precautions against race conditions.

If the invocation fails by throwing an instance of `Error` or a subclass of `Error`, resolution fails with that exception.

If the invocation fails by throwing an exception that is not an instance of `Error` or a subclass of `Error`, resolution fails with a `BootstrapMethodError` whose cause is the thrown exception.

If several threads concurrently invoke the bootstrap method handle for this symbolic reference, the Java Virtual Machine chooses the result of one invocation and installs it visibly to all threads. Any other bootstrap methods executing for this symbolic reference are allowed to complete, but their results are ignored.

The third task, to validate the reference, *o*, produced by invocation of the bootstrap method handle, is as follows:

- If *R* is a symbolic reference to a dynamically-computed constant, then *o* is converted to type *T*, the type indicated by the field descriptor given by *R*.

o's conversion occurs as if by the invocation `MH.invoke(o)` with method descriptor `(Ljava/lang/Object;)T`, where *MH* is a method handle produced as if by invocation of the identity method of `java.lang.invoke.MethodHandles` with an argument representing the class `Object`.

The result of *o*'s conversion is the result of resolution.

If the conversion fails by throwing a `NullPointerException` or a `ClassCastException`, resolution fails with a `BootstrapMethodError`.

- If *R* is a symbolic reference to a dynamically-computed call site, then *o* is the result of resolution if it has all of the following properties:
 - *o* is not `null`.
 - *o* is an instance of `java.lang.invoke.CallSite` or a subclass of `java.lang.invoke.CallSite`.
 - The type of the `java.lang.invoke.CallSite` is semantically equal to the method descriptor given by *R*.

If *o* does not have these properties, resolution fails with a `BootstrapMethodError`.

Many of the steps above perform computations "as if by invocation" of certain methods. In each case, the invocation behavior is given in detail by the

specifications for *invokestatic* and *invokevirtual*. The invocation occurs in the thread and from the class that is attempting resolution of the symbolic reference *R*. However, no corresponding method references are required to appear in the run-time constant pool, no particular method's operand stack is necessarily used, and the value of the *max_stack* item of any method's Code attribute is not enforced for the invocation.

5.4.4 Access Control

Access control is applied during resolution (§5.4.3) to ensure that a reference to a class, interface, field, or method is permitted. Access control succeeds if a specified class, interface, field, or method is *accessible* to the referring class or interface.

A class or interface *C* is accessible to a class or interface *D* if and only if one of the following is true:

- *C* is public, and a member of the same run-time module as *D* (§5.3.6).
- *C* is public, and a member of a different run-time module than *D*, and *C*'s run-time module is read by *D*'s run-time module, and *C*'s run-time module exports *C*'s run-time package to *D*'s run-time module.
- *C* is not public, and *C* and *D* are members of the same run-time package.

If *C* is not accessible to *D*, then access control throws an `IllegalArgumentException`. Otherwise, access control succeeds.

A field or method *R* is accessible to a class or interface *D* if and only if any of the following is true:

- *R* is public.
- *R* is protected and is declared in a class *C*, and *D* is either a subclass of *C* or *C* itself.

Furthermore, if *R* is not static, then the symbolic reference to *R* must contain a symbolic reference to a class *T*, such that *T* is either a subclass of *D*, a superclass of *D*, or *D* itself.

During verification of *D*, it was required that, even if *T* is a superclass of *D*, the target reference of a protected field access or method invocation must be an instance of *D* or a subclass of *D* (§4.10.1.8).

- *R* is either protected or has default access (that is, neither public nor protected nor private), and is declared by a class in the same run-time package as *D*.

- R is private and is declared by a class or interface C that belongs to the same nest as D , according to the nestmate test below.

If R is not accessible to D , then access control throws an `IllegalAccessException`. Otherwise, access control succeeds.

A *nest* is a set of classes and interfaces that allow mutual access to their private members. One of the classes or interfaces is the *nest host*. It enumerates the classes and interfaces which belong to the nest, using the `NestMembers` attribute (§4.7.29). Each of them in turn designates it as the nest host, using the `NestHost` attribute (§4.7.28). A class or interface which lacks a `NestHost` attribute belongs to the nest hosted by itself; if it also lacks a `NestMembers` attribute, then this nest is a singleton consisting only of the class or interface itself.

The Java Virtual Machine determines the nest to which a given class or interface belongs (that is, the nest host designated by the class or interface) as part of access control, rather than when the class or interface is loaded. Certain methods of the Java SE Platform API may determine the nest to which a given class or interface belongs prior to access control, in which case the Java Virtual Machine respects that prior determination during access control.

To determine whether a class or interface C belongs to the same nest as a class or interface D , the *nestmate test* is applied. C and D belong to the same nest if and only if the nestmate test succeeds. The nestmate test is as follows:

- If C and D are the same class or interface, then the nestmate test succeeds.
- Otherwise, the following steps are performed, in order:
 1. Let H be the nest host of D , if the nest host of D has previously been determined. If the nest host of D has *not* previously been determined, then it is determined using the algorithm below, yielding H .
 2. Let H' be the nest host of C , if the nest host of C has previously been determined. If the nest host of C has *not* previously been determined, then it is determined using the algorithm below, yielding H' .
 3. H and H' are compared. If H and H' are the same class or interface, then the nestmate test succeeds. Otherwise, the nestmate test fails.

The nest host of a class or interface M is determined as follows:

- If M lacks a `NestHost` attribute, then M is its own nest host.

- Otherwise, M has a `NestHost` attribute, and its `host_class_index` item is used as an index into the run-time constant pool of M . The symbolic reference at that index is resolved (§5.4.3.1).

If resolution of the symbolic reference fails, then M is its own nest host. Any exception thrown as a result of failure of class or interface resolution is *not* rethrown.

Otherwise, resolution of the symbolic reference succeeds. Let H be the resolved class or interface. The nest host of M is determined by the following rules:

- If any of the following is true, then M is its own nest host:
 - › H is not in the same run-time package as M .
 - › H lacks a `NestMembers` attribute.
 - › H has a `NestMembers` attribute, but there is no entry in its `classes` array that refers to a class or interface with the name N , where N is the name of M .
- Otherwise, H is the nest host of M .

5.4.5 Method Overriding

An instance method m_C *can override* another instance method m_A iff all of the following are true:

- m_C has the same name and descriptor as m_A .
- m_C is not marked `ACC_PRIVATE`.
- One of the following is true:
 - m_A is marked `ACC_PUBLIC`.
 - m_A is marked `ACC_PROTECTED`.
 - m_A is marked neither `ACC_PUBLIC` nor `ACC_PROTECTED` nor `ACC_PRIVATE`, and either (a) the declaration of m_A appears in the same run-time package as the declaration of m_C , or (b) if m_A is declared in a class A and m_C is declared in a class C , then there exists a method m_B declared in a class B such that C is a subclass of B and B is a subclass of A and m_C can override m_B and m_B can override m_A .

Part (b) of the final case allows for "transitive overriding" of methods with default access. For example, given the following class declarations in a package P :

```
public class A          {          void m() {} }
public class B extends A { public void m() {} }
public class C extends B {          void m() {} }
```

and the following class declaration in a different package:

```
public class D extends P.C { void m() {} }
```

then:

- B.m can override A.m.
- C.m can override B.m and A.m.
- D.m can override B.m and, transitively, A.m, but it cannot override C.m.

5.4.6 Method Selection

During execution of an *invokeinterface* or *invokevirtual* instruction, a method is *selected* with respect to (i) the run-time type of the object on the stack, and (ii) a method that was previously *resolved* by the instruction. The rules to select a method with respect to a class or interface *C* and a method m_R are as follows:

1. If m_R is marked ACC_PRIVATE, then it is the selected method.
2. Otherwise, the selected method is determined by the following lookup procedure:
 - If *C* contains a declaration of an instance method *m* that can override m_R (§5.4.5), then *m* is the selected method.
 - Otherwise, if *C* has a superclass, a search for a declaration of an instance method that can override m_R is performed, starting with the direct superclass of *C* and continuing with the direct superclass of that class, and so forth, until a method is found or no further superclasses exist. If a method is found, it is the selected method.
 - Otherwise, the maximally-specific superinterface methods of *C* are determined (§5.4.3.3). If exactly one matches m_R 's name and descriptor and is not abstract, then it is the selected method.

Any maximally-specific superinterface method selected in this step can override m_R ; there is no need to check this explicitly.

While *C* will typically be a class, it may be an interface when these rules are applied during preparation (§5.4.2).

5.5 Initialization

Initialization of a class or interface involves assigning any `ConstantValue` attribute values to its static fields and executing any declared class or interface initialization method (§2.9.2).

A class or interface *c* may be initialized only as a result of:

- The execution of any one of the Java Virtual Machine instructions *new*, *getstatic*, *putstatic*, or *invokestatic* that references *c* (§*new*, §*getstatic*, §*putstatic*, §*invokestatic*).

Upon execution of a *new* instruction, the class to be initialized is the class referenced by the instruction.

Upon execution of a *getstatic*, *putstatic*, or *invokestatic* instruction, the class or interface to be initialized is the class or interface that declares the resolved field or method.

- The first invocation of a `java.lang.invoke.MethodHandle` instance which was the result of method handle resolution (§5.4.3.5) for a method handle of kind 2 (`REF_getStatic`), 4 (`REF_putStatic`), 6 (`REF_invokeStatic`), or 8 (`REF_newInvokeSpecial`).

This implies that the class of a bootstrap method is initialized when the bootstrap method is invoked for an *invokedynamic* instruction (§*invokedynamic*), as part of the continuing resolution of the call site specifier.

- Invocation of certain reflective methods in the class library (§2.12), for example, in class `Class` or in package `java.lang.reflect`.
- If *c* is a class, the initialization of one of its subclasses.
- If *c* is an interface that declares a non-abstract, non-static method, the initialization of a class that implements *c* directly or indirectly.
- Its designation as the initial class or interface at Java Virtual Machine startup (§5.2).

Prior to initialization, a class or interface must be linked, that is, verified, prepared, and optionally resolved.

Because the Java Virtual Machine is multithreaded, initialization of a class or interface requires careful synchronization, since some other thread may be trying to initialize the same class or interface at the same time. There is also the possibility that initialization of a class or interface may be requested recursively as part of the initialization of that class or interface. The implementation of the Java

Virtual Machine is responsible for taking care of synchronization and recursive initialization by using the following procedure. It assumes that the class or interface has already been verified and prepared, and that the class or interface contains state that indicates one of four situations:

- This class or interface is verified and prepared but not initialized.
- This class or interface is being initialized by some particular thread.
- This class or interface is fully initialized and ready for use.
- This class or interface is in an erroneous state, perhaps because initialization was attempted and failed.

The precise form of the initialization state is left to the discretion of the JVM implementation.

For each class or interface *C*, there is a unique initialization lock *LC*. The mapping from *C* to *LC* is also left to the discretion of the Java Virtual Machine implementation. For example, *LC* could be the `Class` object for *C*, or the monitor associated with that `Class` object. The procedure for initializing *C* is then as follows:

1. Synchronize on the initialization lock, *LC*, for *C*. This involves waiting until the current thread can acquire *LC*.
2. If the initialization state of *C* indicates that initialization is in progress for *C* by some other thread, then release *LC* and block the current thread until informed that the in-progress initialization has completed, at which time repeat this procedure.

Thread interrupt status is unaffected by execution of the initialization procedure.

3. If the initialization state of *C* indicates that initialization is in progress for *C* by the current thread, then this must be a recursive request for initialization. Release *LC* and complete normally.
4. If the initialization state of *C* indicates that *C* has already been initialized, then no further action is required. Release *LC* and complete normally.
5. If the initialization state of *C* is in an erroneous state, then initialization is not possible. Release *LC* and throw a `NoClassDefFoundError`.

6. Otherwise, record the fact that initialization of *C* is in progress by the current thread, and release *LC*.

Then, initialize each static field of *C* with the constant value in its `ConstantValue` attribute (§4.7.2), in the order the fields appear in the `ClassFile` structure.

7. Next, if *C* is a class rather than an interface, then let *SC* be its superclass and let *SI*₁, ..., *SI*_{*n*} be all superinterfaces of *C* (whether direct or indirect) that declare at least one non-abstract, non-static method. The order of superinterfaces is given by a recursive enumeration over the superinterface hierarchy of each interface directly implemented by *C*. For each interface *I* directly implemented by *C* (in the order of the `interfaces` array of *C*), the enumeration recurs on *I*'s superinterfaces (in the order of the `interfaces` array of *I*) before returning *I*.

For each *S* in the list [*SC*, *SI*₁, ..., *SI*_{*n*}], if *S* has not yet been initialized, then recursively perform this entire procedure for *S*. If necessary, verify and prepare *S* first.

If the initialization of *S* completes abruptly because of a thrown exception, then acquire *LC*, label *C* as erroneous, notify all waiting threads, release *LC*, and complete abruptly, throwing the same exception that resulted from initializing *S*.

8. Next, determine whether assertions are enabled for *C* by querying its defining loader.
9. Next, if *C* declares a class or interface initialization method, execute that method.
10. If the execution of the class or interface initialization method completes normally, or if *C* declares no class or interface initialization method, then acquire *LC*, label *C* as fully initialized, notify all waiting threads, release *LC*, and complete this procedure normally.
11. Otherwise, the class or interface initialization method must have completed abruptly by throwing some exception *E*. If the class of *E* is not `Error` or one of its subclasses, then create a new instance of the class `ExceptionInInitializerError` with *E* as the argument, and use this object in place of *E* in the following step. If a new instance of `ExceptionInInitializerError` cannot be created because an `OutOfMemoryError` occurs, then use an `OutOfMemoryError` object in place of *E* in the following step.

12. Acquire *LC*, label *c* as erroneous, notify all waiting threads, release *LC*, and complete this procedure abruptly with reason *E* or its replacement as determined in the previous step.

A Java Virtual Machine implementation may optimize this procedure by eliding the lock acquisition in step 1 (and release in step 4/5) when it can determine that the initialization of the class has already completed, provided that, in terms of the Java memory model, all *happens-before* orderings (JLS §17.4.5) that would exist if the lock were acquired, still exist when the optimization is performed.

5.6 Binding Native Method Implementations

Binding is the process by which a function written in a language other than the Java programming language and implementing a native method is integrated into the Java Virtual Machine so that it can be executed. Although this process is traditionally referred to as linking, the term binding is used in the specification to avoid confusion with linking of classes or interfaces by the Java Virtual Machine.

5.7 Java Virtual Machine Termination

The Java Virtual Machine executes code in threads (§2.5). A thread is either a non-daemon thread, a daemon thread, or a shutdown hook.

Readers are referred to the API specifications of `Thread` and `Runtime` for details of how threads obtain daemon status, and how shutdown hooks are registered.

A thread *terminates* if either (i) its `run` method completes normally, or (ii) its `run` method completes abruptly and the relevant uncaught exception handler (§2.10) completes normally or abruptly. With no code left to run, the thread has completed execution and therefore has no current method (§2.5.1).

The Java Virtual Machine *terminates* when one of the following situations has occurred:

1. A thread invoked `System.exit` or `Runtime.exit`, and all of the shutdown hooks which consequently were started by the Java Virtual Machine, if any, have terminated.
2. A thread invoked `Runtime.halt`. (No shutdown hooks are started in this situation.)

3. The Java Virtual Machine implementation recognized an external event as requesting termination of the Java Virtual Machine, and all of the shutdown hooks which consequently were started by the Java Virtual Machine, if any, have terminated.

The nature of the event is outside the scope of this specification, but is necessarily something that a Java Virtual Machine implementation can handle reliably. An example is receiving a signal from the operating system.

4. An external event occurred that the Java Virtual Machine implementation cannot handle. (No shutdown hooks are started in this situation.)

The nature of the event is outside the scope of this specification, but is necessarily something that a Java Virtual Machine implementation cannot recognize or recover from in any way. Examples include a fatal error occurring in the process running the implementation, or power being removed from the computer running the implementation.

Upon Java Virtual Machine termination, any daemon or non-daemon thread that has not yet terminated will execute no further Java code. The current method of the thread does not complete normally or abruptly.

If the Java Virtual Machine terminates because a thread invoked `Runtime.halt` *while shutdown hooks were running*, then, in addition to daemon and non-daemon threads, any shutdown hook that has not yet terminated will execute no further Java code.

Native applications can use the JNI Invocation API to create and destroy the Java Virtual Machine in such a way that a Java program, having started execution in the `main` method of an initial class (JLS §12.1), exits when all of its non-daemon threads have terminated (JLS §12.8). The Java Virtual Machine does not terminate "automatically" when the last non-daemon thread terminates.

The Java Virtual Machine Instruction Set

A Java Virtual Machine instruction consists of an opcode specifying the operation to be performed, followed by zero or more operands embodying values to be operated upon. This chapter gives details about the format of each Java Virtual Machine instruction and the operation it performs.

6.1 Assumptions: The Meaning of "Must"

The description of each instruction is always given in the context of Java Virtual Machine code that satisfies the static and structural constraints of §4 (*The class File Format*). In the description of individual Java Virtual Machine instructions, we frequently state that some situation "must" or "must not" be the case: "The *value2* must be of type `int`." The constraints of §4 (*The class File Format*) guarantee that all such expectations will in fact be met. If some constraint (a "must" or "must not") in an instruction description is not satisfied at run time, the behavior of the Java Virtual Machine is undefined.

The Java Virtual Machine checks that Java Virtual Machine code satisfies the static and structural constraints at link time using a `class` file verifier (§4.10). Thus, the Java Virtual Machine will only attempt to execute code from valid `class` files. Performing verification at link time is attractive in that the checks are performed just once, substantially reducing the amount of work that must be done at run time. Other implementation strategies are possible, provided that they comply with *The Java Language Specification, Java SE 26 Edition* and *The Java Virtual Machine Specification, Java SE 26 Edition*.

6.2 Reserved Opcodes

In addition to the opcodes of the instructions specified later in this chapter, which are used in `class` files (§4 (*The class File Format*)), three opcodes are reserved for internal use by a Java Virtual Machine implementation. If the instruction set of the Java Virtual Machine is extended in the future, these reserved opcodes are guaranteed not to be used.

Two of the reserved opcodes, numbers 254 (0xfe) and 255 (0xff), have the mnemonics *impdep1* and *impdep2*, respectively. These instructions are intended to provide "back doors" or traps to implementation-specific functionality implemented in software and hardware, respectively. The third reserved opcode, number 202 (0xca), has the mnemonic *breakpoint* and is intended to be used by debuggers to implement breakpoints.

Although these opcodes have been reserved, they may be used only inside a Java Virtual Machine implementation. They cannot appear in valid `class` files. Tools such as debuggers or JIT code generators (§2.13) that might directly interact with Java Virtual Machine code that has been already loaded and executed may encounter these opcodes. Such tools should attempt to behave gracefully if they encounter any of these reserved instructions.

6.3 Virtual Machine Errors

A Java Virtual Machine implementation throws an object that is an instance of a subclass of the class `VirtualMachineError` when an internal error or resource limitation prevents it from implementing the semantics described in this chapter. This specification cannot predict where internal errors or resource limitations may be encountered and does not mandate precisely when they can be reported. Thus, any of the `VirtualMachineError` subclasses defined below may be thrown at any time during the operation of the Java Virtual Machine:

- **InternalError:** An internal error has occurred in the Java Virtual Machine implementation because of a fault in the software implementing the virtual machine, a fault in the underlying host system software, or a fault in the hardware. This error is delivered asynchronously (§2.10) when it is detected and may occur at any point in a program.
- **OutOfMemoryError:** The Java Virtual Machine implementation has run out of either virtual or physical memory, and the automatic storage manager was unable to reclaim enough memory to satisfy an object creation request.

- `StackOverflowError`: The Java Virtual Machine implementation has run out of stack space for a thread, typically because the thread is doing an unbounded number of recursive invocations as a result of a fault in the executing program.
- `UnknownError`: An exception or error has occurred, but the Java Virtual Machine implementation is unable to report the actual exception or error.

6.4 Format of Instruction Descriptions

Java Virtual Machine instructions are represented in this chapter by entries of the form shown below, in alphabetical order and each beginning on a new page.

mnemonic

mnemonic

Operation

Short description of the instruction

Format	<i>mnemonic</i>
	<i>operand1</i>
	<i>operand2</i>
	...

Forms

mnemonic = opcode

Operand

..., *value1*, *value2* →

Stack

..., *value3*

Description

A longer description detailing constraints on operand stack contents or constant pool entries, the operation performed, the type of the results, etc.

Linking

If any linking exceptions may be thrown by the execution of this instruction, they are set off one to a line, in the order in which they must be thrown.

Exceptions

Run-time

If any run-time exceptions can be thrown by the execution of an instruction, they are set off one to a line, in the order in which they must be thrown.

Exceptions

Other than the linking and run-time exceptions, if any, listed for an instruction, that instruction must not throw any run-time exceptions except for instances of `VirtualMachineError` or its subclasses.

Notes

Comments not strictly part of the specification of an instruction are set aside as notes at the end of the description.

Each cell in the instruction format diagram represents a single 8-bit byte. The instruction's *mnemonic* is its name. Its opcode is its numeric representation and is given in both decimal and hexadecimal forms. Only the numeric representation is actually present in the Java Virtual Machine code in a `class` file.

Keep in mind that there are "operands" generated at compile time and embedded within Java Virtual Machine instructions, as well as "operands" calculated at run time and supplied on the operand stack. Although they are supplied from several different areas, all these operands represent the same thing: values to be operated upon by the Java Virtual Machine instruction being executed. By implicitly taking many of its operands from its operand stack, rather than representing them explicitly in its compiled code as additional operand bytes, register numbers, etc., the Java Virtual Machine's code stays compact.

Some instructions are presented as members of a family of related instructions sharing a single description, format, and operand stack diagram. As such, a family of instructions includes several opcodes and opcode mnemonics; only the family mnemonic appears in the instruction format diagram, and a separate forms line lists all member mnemonics and opcodes. For example, the Forms line for the *lconst_<l>* family of instructions, giving mnemonic and opcode information for the two instructions in that family (*lconst_0* and *lconst_1*), is

lconst_0 = 9 (0x9)

lconst_1 = 10 (0xa)

In the description of the Java Virtual Machine instructions, the effect of an instruction's execution on the operand stack (§2.6.2) of the current frame (§2.6) is represented textually, with the stack growing from left to right and each value represented separately. Thus,

..., *value1*, *value2* →

..., *result*

shows an operation that begins by having *value2* on top of the operand stack with *value1* just beneath it. As a result of the execution of the instruction, *value1* and *value2* are popped from the operand stack and replaced by *result* value, which has been calculated by the instruction. The remainder of the operand stack, represented by an ellipsis (...), is unaffected by the instruction's execution.

Values of types `long` and `double` are represented by a single entry on the operand stack.

In the First Edition of *The Java® Virtual Machine Specification*, values on the operand stack of types `long` and `double` were each represented in the stack diagram by two entries.

6.5 Instructions

aaload***aaload***

Operation	Load reference from array	
Format	<table border="1"><tr><td><i>aaload</i></td></tr></table>	<i>aaload</i>
<i>aaload</i>		
Forms	<i>aaload</i> = 50 (0x32)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type reference. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The reference <i>value</i> in the component of the array at <i>index</i> is retrieved and pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>aaload</i> throws a <code>NullPointerException</code> . Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>aaload</i> instruction throws an <code>ArrayIndexOutOfBoundsException</code> .	

aastore***aastore***

Operation Store into reference array

Format

<i>aastore</i>

Forms *aastore* = 83 (0x53)

Operand Stack ..., *arrayref*, *index*, *value* →
...

Description The *arrayref* must be of type reference and must refer to an array whose components are of type reference. The *index* must be of type int, and *value* must be of type reference. The *arrayref*, *index*, and *value* are popped from the operand stack.

If *value* is null, then *value* is stored as the component of the array at *index*.

Otherwise, *value* is non-null. If *value* is a value of the component type of the array referenced by *arrayref*, then *value* is stored as the component of the array at *index*.

Whether *value* is a value of the array component type is determined according to the rules given for §*checkcast*.

Run-time If *arrayref* is null, *aastore* throws a NullPointerException.

Exceptions Otherwise, if *index* is not within the bounds of the array referenced by *arrayref*, the *aastore* instruction throws an ArrayIndexOutOfBoundsException.

Otherwise, if the non-null *value* is not a value of the array component type, *aastore* throws an ArrayStoreException.

aconst_null***aconst_null***

Operation Push null

Format

<i>aconst_null</i>

Forms *aconst_null* = 1 (0x1)

Operand ... →

Stack ..., null

Description Push the null object reference onto the operand stack.

Notes The Java Virtual Machine does not mandate a concrete value for null.

aload

aload

Operation Load reference from local variable

Format	<i>aload</i>
	<i>index</i>

Forms *aload* = 25 (0x19)

Operand ... →

Stack ..., *objectref*

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The local variable at *index* must contain a reference. The *objectref* in the local variable at *index* is pushed onto the operand stack.

Notes The *aload* instruction cannot be used to load a value of type `returnAddress` from a local variable onto the operand stack. This asymmetry with the *astore* instruction (§*astore*) is intentional.

 The *aload* opcode can be used in conjunction with the *wide* instruction (§*wide*) to access a local variable using a two-byte unsigned index.

aload_<n>***aload_<n>***

Operation Load reference from local variable

Format

<i>aload_<n></i>

Forms *aload_0* = 42 (0x2a)
 aload_1 = 43 (0x2b)
 aload_2 = 44 (0x2c)
 aload_3 = 45 (0x2d)

Operand ... →
Stack ..., *objectref*

Description The <n> must be an index into the local variable array of the current frame (§2.6). The local variable at <n> must contain a reference. The *objectref* in the local variable at <n> is pushed onto the operand stack.

Notes An *aload_<n>* instruction cannot be used to load a value of type *returnAddress* from a local variable onto the operand stack. This asymmetry with the corresponding *astore_<n>* instruction (§*astore_<n>*) is intentional.

Each of the *aload_<n>* instructions is the same as *aload* with an *index* of <n>, except that the operand <n> is implicit.

anewarray

anewarray

Operation Create new array of reference

Format	<i>anewarray</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms anewarray = 189 (0xbd)

Operand ..., *count* →

Stack ..., *arrayref*

Description The *count* must be of type `int`. It is popped off the operand stack. The *count* represents the number of components of the array to be created. The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The run-time constant pool entry at the index must be a symbolic reference to a class, array, or interface type. The named class, array, or interface type is resolved (§5.4.3.1). A new array with components of that type, of length *count*, is allocated from the garbage-collected heap, and a reference *arrayref* to this new array object is pushed onto the operand stack. All components of the new array are initialized to `null`, the default value for reference types (§2.4).

Linking Exceptions During resolution of the symbolic reference to the class, array, or interface type, any of the exceptions documented in §5.4.3.1 can be thrown.

Run-time Exceptions Otherwise, if *count* is less than zero, the *anewarray* instruction throws a `NegativeArraySizeException`.

Notes The *anewarray* instruction is used to create a single dimension of an array of object references or part of a multidimensional array.

areturn***areturn***

Operation Return reference from method

Format

<i>areturn</i>

Forms *areturn* = 176 (0xb0)

Operand Stack ..., *objectref* →
[empty]

Description The *objectref* must be of type reference and must refer to an object of a type that is assignment compatible (JLS §5.2) with the type represented by the return descriptor (§4.3.3) of the current method. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread. If no exception is thrown, *objectref* is popped from the operand stack of the current frame (§2.6) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The interpreter then reinstates the frame of the invoker and returns control to the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *areturn* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *areturn* throws an `IllegalMonitorStateException`.

arraylength***arraylength***

Operation	Get length of array
Format	<i>arraylength</i>
Forms	<i>arraylength</i> = 190 (0xbe)
Operand Stack	..., <i>arrayref</i> → ..., <i>length</i>
Description	The <i>arrayref</i> must be of type reference and must refer to an array. It is popped from the operand stack. The <i>length</i> of the array it references is determined. That <i>length</i> is pushed onto the operand stack as an int.
Run-time Exceptions	If the <i>arrayref</i> is null, the <i>arraylength</i> instruction throws a NullPointerException.

astore

astore

Operation Store reference into local variable

Format	<i>astore</i>
	<i>index</i>

Forms *astore* = 58 (0x3a)

Operand Stack ..., *objectref* →
 ...

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The *objectref* on the top of the operand stack must be of type `returnAddress` or of type `reference`. It is popped from the operand stack, and the value of the local variable at *index* is set to *objectref*.

Notes The *astore* instruction is used with an *objectref* of type `returnAddress` when implementing the `finally` clause of the Java programming language (§3.13).

 The *aload* instruction (§*aload*) cannot be used to load a value of type `returnAddress` from a local variable onto the operand stack. This asymmetry with the *astore* instruction is intentional.

 The *astore* opcode can be used in conjunction with the *wide* instruction (§*wide*) to access a local variable using a two-byte unsigned index.

astore_<n>***astore_<n>***

Operation Store reference into local variable

Format

<i>astore_<n></i>

Forms *astore_0* = 75 (0x4b)
 astore_1 = 76 (0x4c)
 astore_2 = 77 (0x4d)
 astore_3 = 78 (0x4e)

Operand Stack ..., *objectref* →
 ...

Description The *<n>* must be an index into the local variable array of the current frame (§2.6). The *objectref* on the top of the operand stack must be of type `returnAddress` or of type `reference`. It is popped from the operand stack, and the value of the local variable at *<n>* is set to *objectref*.

Notes An *astore_<n>* instruction is used with an *objectref* of type `returnAddress` when implementing the `finally` clauses of the Java programming language (§3.13).

An *aload_<n>* instruction (§*aload_<n>*) cannot be used to load a value of type `returnAddress` from a local variable onto the operand stack. This asymmetry with the corresponding *astore_<n>* instruction is intentional.

Each of the *astore_<n>* instructions is the same as *astore* with an *index* of *<n>*, except that the operand *<n>* is implicit.

athrow***athrow***

Operation Throw exception or error

Format

<i>athrow</i>

Forms *athrow* = 191 (0xbf)

Operand Stack ..., *objectref* →
 objectref

Description The *objectref* must be of type reference and must refer to an object that is an instance of class `Throwable` or of a subclass of `Throwable`. It is popped from the operand stack. The *objectref* is then thrown by searching the current method (§2.6) for the first exception handler that matches the class of *objectref*, as given by the algorithm in §2.10.

If an exception handler that matches *objectref* is found, it contains the location of the code intended to handle this exception. The pc register is reset to that location, the operand stack of the current frame is cleared, *objectref* is pushed back onto the operand stack, and execution continues.

If no matching exception handler is found in the current frame, that frame is popped. If the current frame represents an invocation of a synchronized method, the monitor entered or reentered on invocation of the method is exited as if by execution of a *monitorexit* instruction (§*monitorexit*). Finally, the frame of its invoker is reinstated, if such a frame exists, and the *objectref* is rethrown. If no such frame exists, the current thread exits.

Run-time Exceptions If *objectref* is null, *athrow* throws a `NullPointerException` instead of *objectref*.

Otherwise, if the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the method of the current frame is a synchronized method and the current thread is not the owner of the monitor

entered or reentered on invocation of the method, *athrow* throws an `IllegalMonitorStateException` instead of the object previously being thrown. This can happen, for example, if an abruptly completing synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *athrow* throws an `IllegalMonitorStateException` instead of the object previously being thrown.

Notes

The operand stack diagram for the *athrow* instruction may be misleading: If a handler for this exception is matched in the current method, the *athrow* instruction discards all the values on the operand stack, then pushes the thrown object onto the operand stack. However, if no handler is matched in the current method and the exception is thrown farther up the method invocation chain, then the operand stack of the method (if any) that handles the exception is cleared and *objectref* is pushed onto that empty operand stack. All intervening frames from the method that threw the exception up to, but not including, the method that handles the exception are discarded.

baload***baload***

Operation Load byte or boolean from array

Format

<i>baload</i>

Forms *baload* = 51 (0x33)

Operand ..., *arrayref*, *index* →

Stack ..., *value*

Description The *arrayref* must be of type reference and must refer to an array whose components are of type byte or of type boolean. The *index* must be of type int. Both *arrayref* and *index* are popped from the operand stack. The byte *value* in the component of the array at *index* is retrieved, sign-extended to an int *value*, and pushed onto the top of the operand stack.

Run-time If *arrayref* is null, *baload* throws a `NullPointerException`.

Exceptions Otherwise, if *index* is not within the bounds of the array referenced by *arrayref*, the *baload* instruction throws an `ArrayIndexOutOfBoundsException`.

Notes The *baload* instruction is used to load values from both byte and boolean arrays. In Oracle's Java Virtual Machine implementation, boolean arrays - that is, arrays of type `T_BOOLEAN` (§2.2, §*newarray*) - are implemented as arrays of 8-bit values. Other implementations may implement packed boolean arrays; the *baload* instruction of such implementations must be used to access those arrays.

bastore***bastore***

Operation	Store into byte or boolean array
Format	<i>bastore</i>
Forms	<i>bastore</i> = 84 (0x54)
Operand Stack	..., <i>arrayref</i> , <i>index</i> , <i>value</i> → ...
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type byte or of type boolean. The <i>index</i> and the <i>value</i> must both be of type int. The <i>arrayref</i>, <i>index</i>, and <i>value</i> are popped from the operand stack. If the <i>arrayref</i> refers to an array whose components are of type byte, then the int <i>value</i> is truncated to a byte and stored as the component of the array indexed by <i>index</i>. If the <i>arrayref</i> refers to an array whose components are of type boolean, then the int <i>value</i> is narrowed by taking the bitwise AND of <i>value</i> and 1; the result is stored as the component of the array indexed by <i>index</i>.
Run-time Exceptions	If <i>arrayref</i> is null, <i>bastore</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i>, the <i>bastore</i> instruction throws an <code>ArrayIndexOutOfBoundsException</code>.
Notes	The <i>bastore</i> instruction is used to store values into both byte and boolean arrays. In Oracle's Java Virtual Machine implementation, boolean arrays - that is, arrays of type <code>T_BOOLEAN</code> (§2.2, §newarray) - are implemented as arrays of 8-bit values. Other implementations may implement packed boolean arrays; in such implementations the <i>bastore</i> instruction must be able to store boolean values into packed boolean arrays as well as byte values into byte arrays.

bipush

bipush

Operation	Push byte		
Format	<table><tr><td><i>bipush</i></td></tr><tr><td><i>byte</i></td></tr></table>	<i>bipush</i>	<i>byte</i>
<i>bipush</i>			
<i>byte</i>			
Forms	<i>bipush</i> = 16 (0x10)		
Operand	... →		
Stack	..., <i>value</i>		
Description	The immediate <i>byte</i> is sign-extended to an int <i>value</i> . That <i>value</i> is pushed onto the operand stack.		

caload***caload***

Operation	Load char from array	
Format	<table border="1"><tr><td><i>caload</i></td></tr></table>	<i>caload</i>
<i>caload</i>		
Forms	<i>caload</i> = 52 (0x34)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type char. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The component of the array at <i>index</i> is retrieved and zero-extended to an int <i>value</i> . That <i>value</i> is pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>caload</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>caload</i> instruction throws an ArrayIndexOutOfBoundsException.	

castore***castore***

Operation Store into char array

Format

<i>castore</i>

Forms

castore = 85 (0x55)

Operand

..., *arrayref*, *index*, *value* →

Stack

...

Description

The *arrayref* must be of type reference and must refer to an array whose components are of type char. The *index* and the *value* must both be of type int. The *arrayref*, *index*, and *value* are popped from the operand stack. The int *value* is truncated to a char and stored as the component of the array indexed by *index*.

Run-time

If *arrayref* is null, *castore* throws a `NullPointerException`.

Exceptions

Otherwise, if *index* is not within the bounds of the array referenced by *arrayref*, the *castore* instruction throws an `ArrayIndexOutOfBoundsException`.

checkcast

checkcast

Operation

Check whether object is of given type

Format	<i>checkcast</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms

checkcast = 192 (0xc0)

Operand

..., *objectref* →

Stack

..., *objectref*

Description

The *objectref* must be of type reference. The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The run-time

constant pool entry at the index must be a symbolic reference to a class, array, or interface type.

If *objectref* is `null`, then the operand stack is unchanged.

Otherwise, the named class, array, or interface type is resolved (§5.4.3.1). If *objectref* is a value of the type given by the resolved class, array, or interface type, the operand stack is unchanged.

The following rules are used to determine whether a non-`null` reference to an object is a value of a reference type, τ .

- If the reference is to an instance of a class C , then:
 - If τ is the type of a class D , then the reference is a value of τ if C is D or a subclass of D ;
 - If τ is the type of an interface I , then the reference is a value of τ if C implements I .
- If the reference is to an array with component type SC , then:
 - If τ is a class type, then the reference is a value of τ if τ is `Object`;
 - If τ is an interface type, then the reference is a value of τ if τ is `Cloneable` or `java.io.Serializable`, as defined by the bootstrap class loader (§5.3);
 - If τ is an array type $TC[]$, that is, an array of components of type TC , then the reference is a value of τ if one of the following is true:
 - › TC and SC are the same type;
 - › TC is the class type `Object`;
 - › TC is a class or interface type, SC is a class or interface type, and the class or interface named by SC extends or implements the class or interface named by TC ;
 - › SC is an array type $SCC[]$, and (applying these rules recursively) a reference to an array with component type SCC is a value of type TC .

Linking Exceptions

During resolution of the symbolic reference to the class, array, or interface type, any of the exceptions documented in §5.4.3.1 can be thrown.

Run-time Exception Otherwise, if *objectref* is not null and is not a value of the type given by the resolved class, array, or interface type, the *checkcast* instruction throws a `ClassCastException`.

Notes The *checkcast* instruction is very similar to the *instanceof* instruction (§*instanceof*). It differs in its treatment of null, its behavior when its test fails (*checkcast* throws an exception, *instanceof* pushes a result code), and its effect on the operand stack.

d2f***d2f***

Operation	Convert double to float	
Format	<table border="1"><tr><td><i>d2f</i></td></tr></table>	<i>d2f</i>
<i>d2f</i>		
Forms	<i>d2f</i> = 144 (0x90)	
Operand Stack	..., <i>value</i> → ..., <i>result</i>	
Description	The <i>value</i> on the top of the operand stack must be of type double. It is popped from the operand stack and converted to a float <i>result</i> using the round to nearest rounding policy (§2.8). The <i>result</i> is pushed onto the operand stack. A finite <i>value</i> too small to be represented as a float is converted to a zero of the same sign; a finite <i>value</i> too large to be represented as a float is converted to an infinity of the same sign. A double NaN is converted to a float NaN.	
Notes	The <i>d2f</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> and may also lose precision.	

d2i***d2i***

Operation	Convert double to int
Format	<i>d2i</i>
Forms	<i>d2i</i> = 142 (0x8e)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type double. It is popped from the operand stack and converted to an int <i>result</i>. The <i>result</i> is pushed onto the operand stack: • If the <i>value</i> is NaN, the result of the conversion is an int 0. • Otherwise, if the <i>value</i> is not an infinity, it is rounded to an integer value <i>v</i> using the round toward zero rounding policy (§2.8). If this integer value <i>v</i> can be represented as an int, then the result is the int value <i>v</i>. • Otherwise, either the <i>value</i> must be too small (a negative value of large magnitude or negative infinity), and the result is the smallest representable value of type int, or the <i>value</i> must be too large (a positive value of large magnitude or positive infinity), and the result is the largest representable value of type int.
Notes	The <i>d2i</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> and may also lose precision.

d2l***d2l***

Operation Convert double to long

Format

<i>d2l</i>

Forms *d2l* = 143 (0x8f)

Operand ..., *value* →

Stack ..., *result*

Description The *value* on the top of the operand stack must be of type double. It is popped from the operand stack and converted to a long. The *result* is pushed onto the operand stack:

- If the *value* is NaN, the result of the conversion is a long 0.
- Otherwise, if the *value* is not an infinity, it is rounded to an integer value *v* using the round toward zero rounding policy (§2.8). If this integer value *v* can be represented as a long, then the result is the long value *v*.
- Otherwise, either the *value* must be too small (a negative value of large magnitude or negative infinity), and the result is the smallest representable value of type long, or the *value* must be too large (a positive value of large magnitude or positive infinity), and the result is the largest representable value of type long.

Notes The *d2l* instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of *value* and may also lose precision.

dadd***dadd*****Operation** Add double**Format**

<i>dadd</i>

Forms *dadd* = 99 (0x63)**Operand Stack** ..., *value1*, *value2* →
..., *result***Description** Both *value1* and *value2* must be of type double. The values are popped from the operand stack. The double *result* is *value1* + *value2*. The *result* is pushed onto the operand stack.

The result of a *dadd* instruction is governed by the rules of IEEE 754 arithmetic:

- If either *value1* or *value2* is NaN, the result is NaN.
- The sum of two infinities of opposite sign is NaN.
- The sum of two infinities of the same sign is the infinity of that sign.
- The sum of an infinity and any finite value is equal to the infinity.
- The sum of two zeroes of opposite sign is positive zero.
- The sum of two zeroes of the same sign is the zero of that sign.
- The sum of a zero and a nonzero finite value is equal to the nonzero value.
- The sum of two nonzero finite values of the same magnitude and opposite sign is positive zero.
- In the remaining cases, where neither operand is an infinity, a zero, or NaN and the values have the same sign or have different magnitudes, the sum is computed and rounded to the nearest representable value using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a double,

we say the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a double, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of a *dadd* instruction never throws a run-time exception.

daload***daload***

Operation	Load double from array	
Format	<table border="1"><tr><td><i>daload</i></td></tr></table>	<i>daload</i>
<i>daload</i>		
Forms	<i>daload</i> = 49 (0x31)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type double. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The double <i>value</i> in the component of the array at <i>index</i> is retrieved and pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>daload</i> throws a <code>NullPointerException</code> . Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>daload</i> instruction throws an <code>ArrayIndexOutOfBoundsException</code> .	

dastore***dastore***

Operation	Store into double array
Format	<i>dastore</i>
Forms	<i>dastore</i> = 82 (0x52)
Operand Stack	..., <i>arrayref</i> , <i>index</i> , <i>value</i> → ...
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type double. The <i>index</i> must be of type int, and <i>value</i> must be of type double. The <i>arrayref</i> , <i>index</i> , and <i>value</i> are popped from the operand stack. The double <i>value</i> is stored as the component of the array indexed by <i>index</i> .
Run-time Exceptions	If <i>arrayref</i> is null, <i>dastore</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>dastore</i> instruction throws an ArrayIndexOutOfBoundsException.

dcmp<op>***dcmp<op>***

Operation	Compare double
Format	<i>dcmp<op></i>
Forms	<i>dcmpg</i> = 152 (0x98) <i>dcmpl</i> = 151 (0x97)
Operand Stack	..., <i>value1</i> , <i>value2</i> → ..., <i>result</i>
Description	Both <i>value1</i> and <i>value2</i> must be of type double. The values are popped from the operand stack and a floating-point comparison is performed: • If <i>value1</i> is greater than <i>value2</i>, the int value 1 is pushed onto the operand stack. • Otherwise, if <i>value1</i> is equal to <i>value2</i>, the int value 0 is pushed onto the operand stack. • Otherwise, if <i>value1</i> is less than <i>value2</i>, the int value -1 is pushed onto the operand stack. • Otherwise, at least one of <i>value1</i> or <i>value2</i> is NaN. The <i>dcmpg</i> instruction pushes the int value 1 onto the operand stack and the <i>dcmpl</i> instruction pushes the int value -1 onto the operand stack. Floating-point comparison is performed in accordance with IEEE 754. All values other than NaN are ordered, with negative infinity less than all finite values and positive infinity greater than all finite values. Positive zero and negative zero are considered equal.
Notes	The <i>dcmpg</i> and <i>dcmpl</i> instructions differ only in their treatment of a comparison involving NaN. NaN is unordered, so any double comparison fails if either or both of its operands are NaN. With both <i>dcmpg</i> and <i>dcmpl</i> available, any double comparison may be compiled to push the same <i>result</i> onto the operand stack

whether the comparison fails on non-NaN values or fails because it encountered a NaN. For more information, see §3.5.

dconst_<d>***dconst_<d>***

Operation	Push double
Format	<i>dconst_<d></i>
Forms	<i>dconst_0</i> = 14 (0xe) <i>dconst_1</i> = 15 (0xf)
Operand	... →
Stack	..., <d>
Description	Push the double constant <d> (0.0 or 1.0) onto the operand stack.

ddiv

ddiv

Operation Divide double

Format

<i>ddiv</i>

Forms *ddiv* = 111 (0x6f)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type double. The values are popped from the operand stack. The double *result* is *value1* / *value2*. The *result* is pushed onto the operand stack.

The result of a *ddiv* instruction is governed by the rules of IEEE 754 arithmetic:

- If either *value1* or *value2* is NaN, the result is NaN.
- If neither *value1* nor *value2* is NaN, the sign of the result is positive if both values have the same sign, negative if the values have different signs.
- Division of an infinity by an infinity results in NaN.
- Division of an infinity by a finite value results in a signed infinity, with the sign-producing rule just given.
- Division of a finite value by an infinity results in a signed zero, with the sign-producing rule just given.
- Division of a zero by a zero results in NaN; division of zero by any other finite value results in a signed zero, with the sign-producing rule just given.
- Division of a nonzero finite value by a zero results in a signed infinity, with the sign-producing rule just given.
- In the remaining cases, where neither operand is an infinity, a zero, or NaN, the quotient is computed and rounded to the nearest double using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a double,

we say the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a double, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, division by zero, or loss of precision may occur, execution of a *ddiv* instruction never throws a run-time exception.

dload

dload

Operation Load double from local variable

Format	<i>dload</i>
	<i>index</i>

Forms *dload* = 24 (0x18)

Operand ... →
Stack ..., *value*

Description The *index* is an unsigned byte. Both *index* and *index*+1 must be indices into the local variable array of the current frame (§2.6). The local variable at *index* must contain a double. The *value* of the local variable at *index* is pushed onto the operand stack.

Notes The *dload* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

dload_<n>***dload_<n>***

Operation Load double from local variable

Format

<i>dload_<n></i>

Forms *dload_0* = 38 (0x26)
dload_1 = 39 (0x27)
dload_2 = 40 (0x28)
dload_3 = 41 (0x29)

Operand ... →
Stack ..., *value*

Description Both <*n*> and <*n*>+1 must be indices into the local variable array of the current frame (§2.6). The local variable at <*n*> must contain a double. The *value* of the local variable at <*n*> is pushed onto the operand stack.

Notes Each of the *dload_<n>* instructions is the same as *dload* with an *index* of <*n*>, except that the operand <*n*> is implicit.

dmul***dmul***

Operation Multiply double

Format

<i>dmul</i>

Forms *dmul* = 107 (0x6b)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type double. The values are popped from the operand stack. The double *result* is *value1* * *value2*. The *result* is pushed onto the operand stack.

The result of a *dmul* instruction is governed by the rules of IEEE 754 arithmetic:

- If either *value1* or *value2* is NaN, the result is NaN.
- If neither *value1* nor *value2* is NaN, the sign of the result is positive if both values have the same sign and negative if the values have different signs.
- Multiplication of an infinity by a zero results in NaN.
- Multiplication of an infinity by a finite value results in a signed infinity, with the sign-producing rule just given.
- In the remaining cases, where neither an infinity nor NaN is involved, the product is computed and rounded to the nearest representable value using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a double, we say the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a double, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of a *dmul* instruction never throws a run-time exception.

dneg***dneg*****Operation** Negate double**Format**

<i>dneg</i>

Forms *dneg* = 119 (0x77)**Operand** ..., *value* →**Stack** ..., *result***Description** The value must be of type double. It is popped from the operand stack. The double *result* is the arithmetic negation of *value*. The *result* is pushed onto the operand stack.

For double values, negation is not the same as subtraction from zero. If x is $+0.0$, then $0.0 - x$ equals $+0.0$, but $-x$ equals -0.0 . Unary minus merely inverts the sign of a double.

Special cases of interest:

- If the operand is NaN, the result is NaN (recall that NaN has no sign).

The Java Virtual Machine has not adopted the stronger requirement from the 2019 version of the IEEE 754 Standard that negation inverts the sign bit for all inputs, including NaN.

- If the operand is an infinity, the result is the infinity of opposite sign.
- If the operand is a zero, the result is the zero of opposite sign.

drem***drem***

Operation Remainder double

Format

<i>drem</i>

Forms *drem* = 115 (0x73)

Operand ..., *value1*, *value2* →

Stack ..., *result*

Description Both *value1* and *value2* must be of type double. The values are popped from the operand stack. The double *result* is calculated and pushed onto the operand stack.

The result of a *drem* instruction is not the same as the result of the remainder operation defined by IEEE 754, due to the choice of rounding policy in the Java Virtual Machine (§2.8). The IEEE 754 remainder operation computes the remainder from a rounding division, not a truncating division, and so its behavior is *not* analogous to that of the usual integer remainder operator. Instead, the Java Virtual Machine defines *drem* to behave in a manner analogous to that of the integer remainder instructions *irem* and *lrem*, with an implied division using the round toward

zero rounding policy; this may be compared with the C library function `fmod`.

The result of a *drem* instruction is governed by the following rules, which match IEEE 754 arithmetic except for how the implied division is computed:

- If either *value1* or *value2* is NaN, the result is NaN.
- If neither *value1* nor *value2* is NaN, the sign of the result equals the sign of the dividend.
- If the dividend is an infinity or the divisor is a zero or both, the result is NaN.
- If the dividend is finite and the divisor is an infinity, the result equals the dividend.
- If the dividend is a zero and the divisor is finite, the result equals the dividend.
- In the remaining cases, where neither operand is an infinity, a zero, or NaN, the floating-point remainder *result* from a dividend *value1* and a divisor *value2* is defined by the mathematical relation $result = value1 - (value2 * q)$, where *q* is an integer that is negative only if *value1* / *value2* is negative, and positive only if *value1* / *value2* is positive, and whose magnitude is as large as possible without exceeding the magnitude of the true mathematical quotient of *value1* and *value2*.

Despite the fact that division by zero may occur, evaluation of a *drem* instruction never throws a run-time exception. Overflow, underflow, or loss of precision cannot occur.

Notes

The IEEE 754 remainder operation may be computed by the library routine `Math.IEEEremainder` or `StrictMath.IEEEremainder`.

dreturn***dreturn***

Operation Return double from method

Format

<i>dreturn</i>

Forms *dreturn* = 175 (0xaf)

Operand ..., *value* →

Stack [empty]

Description The current method must have return type double. The *value* must be of type double. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§monitorexit) in the current thread. If no exception is thrown, *value* is popped from the operand stack of the current frame (§2.6) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The interpreter then returns control to the invoker of the method, reinstating the frame of the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *dreturn* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *dreturn* throws an `IllegalMonitorStateException`.

dstore

dstore

Operation Store double into local variable

Format	<i>dstore</i>
	<i>index</i>

Forms *dstore* = 57 (0x39)

Operand Stack ..., *value* →
 ...

Description The *index* is an unsigned byte. Both *index* and *index*+1 must be indices into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type double. It is popped from the operand stack. The local variables at *index* and *index*+1 are set to *value*.

Notes The *dstore* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

dstore_<n>***dstore_<n>***

Operation Store double into local variable

Format

<i>dstore_<n></i>

Forms

dstore_0 = 71 (0x47)

dstore_1 = 72 (0x48)

dstore_2 = 73 (0x49)

dstore_3 = 74 (0x4a)

Operand

..., *value* →

Stack

...

Description

Both *<n>* and *<n>+1* must be indices into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type double. It is popped from the operand stack. The local variables at *<n>* and *<n>+1* are set to *value*.

Notes

Each of the *dstore_<n>* instructions is the same as *dstore* with an *index* of *<n>*, except that the operand *<n>* is implicit.

dsub***dsub***

Operation Subtract double

Format

<i>dsub</i>

Forms *dsub* = 103 (0x67)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type double. The values are popped from the operand stack. The double *result* is *value1* - *value2*. The *result* is pushed onto the operand stack.

For double subtraction, it is always the case that $a - b$ produces the same result as $a + (-b)$. However, for the *dsub* instruction, subtraction from zero is not the same as negation, because if x is $+0.0$, then $0.0 - x$ equals $+0.0$, but $-x$ equals -0.0 .

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of a *dsub* instruction never throws a run-time exception.

dup***dup***

Operation Duplicate the top operand stack value

Format

<i>dup</i>

Forms *dup* = 89 (0x59)

Operand ..., *value* →

Stack ..., *value*, *value*

Description Duplicate the top value on the operand stack and push the duplicated value onto the operand stack.

The *dup* instruction must not be used unless *value* is a value of a category 1 computational type (§2.11.1).

dup_x1

dup_x1

Operation Duplicate the top operand stack value and insert two values down

Format

<i>dup_x1</i>

Forms *dup_x1* = 90 (0x5a)

Operand Stack ..., *value2*, *value1* →
..., *value1*, *value2*, *value1*

Description Duplicate the top value on the operand stack and insert the duplicated value two values down in the operand stack.

The *dup_x1* instruction must not be used unless both *value1* and *value2* are values of a category 1 computational type (§2.11.1).

dup_x2

dup_x2

Operation	Duplicate the top operand stack value and insert two or three values down
Format	<i>dup_x2</i>
Forms	<i>dup_x2</i> = 91 (0x5b)
Operand Stack	Form 1: ..., <i>value3</i>, <i>value2</i>, <i>value1</i> → ..., <i>value1</i>, <i>value3</i>, <i>value2</i>, <i>value1</i> where <i>value1</i>, <i>value2</i>, and <i>value3</i> are all values of a category 1 computational type (§2.11.1). Form 2: ..., <i>value2</i>, <i>value1</i> → ..., <i>value1</i>, <i>value2</i>, <i>value1</i> where <i>value1</i> is a value of a category 1 computational type and <i>value2</i> is a value of a category 2 computational type (§2.11.1).
Description	Duplicate the top value on the operand stack and insert the duplicated value two or three values down in the operand stack.

dup2***dup2***

Operation Duplicate the top one or two operand stack values

Format

<i>dup2</i>

Forms

dup2 = 92 (0x5c)

**Operand
Stack**

Form 1:

..., *value2*, *value1* →

..., *value2*, *value1*, *value2*, *value1*

where both *value1* and *value2* are values of a category 1 computational type (§2.11.1).

Form 2:

..., *value* →

..., *value*, *value*

where *value* is a value of a category 2 computational type (§2.11.1).

Description

Duplicate the top one or two values on the operand stack and push the duplicated value or values back onto the operand stack in the original order.

dup2_x1

dup2_x1

Operation	Duplicate the top one or two operand stack values and insert two or three values down
Format	<i>dup2_x1</i>
Forms	<i>dup2_x1</i> = 93 (0x5d)
Operand Stack	Form 1: ..., <i>value3</i>, <i>value2</i>, <i>value1</i> → ..., <i>value2</i>, <i>value1</i>, <i>value3</i>, <i>value2</i>, <i>value1</i> where <i>value1</i>, <i>value2</i>, and <i>value3</i> are all values of a category 1 computational type (§2.11.1). Form 2: ..., <i>value2</i>, <i>value1</i> → ..., <i>value1</i>, <i>value2</i>, <i>value1</i> where <i>value1</i> is a value of a category 2 computational type and <i>value2</i> is a value of a category 1 computational type (§2.11.1).
Description	Duplicate the top one or two values on the operand stack and insert the duplicated values, in the original order, one value beneath the original value or values in the operand stack.

dup2_x2***dup2_x2***

Operation Duplicate the top one or two operand stack values and insert two, three, or four values down

Format

<i>dup2_x2</i>

Forms *dup2_x2* = 94 (0x5e)

Operand Stack Form 1:
 ..., *value4*, *value3*, *value2*, *value1* →
 ..., *value2*, *value1*, *value4*, *value3*, *value2*, *value1*
 where *value1*, *value2*, *value3*, and *value4* are all values of a category 1 computational type (§2.11.1).

Form 2:
 ..., *value3*, *value2*, *value1* →
 ..., *value1*, *value3*, *value2*, *value1*
 where *value1* is a value of a category 2 computational type and *value2* and *value3* are both values of a category 1 computational type (§2.11.1).

Form 3:
 ..., *value3*, *value2*, *value1* →
 ..., *value2*, *value1*, *value3*, *value2*, *value1*
 where *value1* and *value2* are both values of a category 1 computational type and *value3* is a value of a category 2 computational type (§2.11.1).

Form 4:
 ..., *value2*, *value1* →
 ..., *value1*, *value2*, *value1*
 where *value1* and *value2* are both values of a category 2 computational type (§2.11.1).

Description Duplicate the top one or two values on the operand stack and insert the duplicated values, in the original order, into the operand stack.

f2d***f2d***

Operation	Convert <code>float</code> to <code>double</code>
Format	<i>f2d</i>
Forms	<i>f2d</i> = 141 (0x8d)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>float</code> . It is popped from the operand stack and converted to a <code>double</code> <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>f2d</i> instruction performs a widening primitive conversion (JLS §5.1.2).

f2i***f2i***

Operation	Convert float to int
Format	<i>f2i</i>
Forms	<i>f2i</i> = 139 (0x8b)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>float</code>. It is popped from the operand stack and converted to an <code>int</code> <i>result</i>. The <i>result</i> is pushed onto the operand stack: • If the <i>value</i> is NaN, the <i>result</i> of the conversion is an <code>int</code> 0. • Otherwise, if the <i>value</i> is not an infinity, it is rounded to an integer value <i>v</i> using the round toward zero rounding policy (§2.8). If this integer value <i>v</i> can be represented as an <code>int</code>, then the <i>result</i> is the <code>int</code> value <i>v</i>. • Otherwise, either the <i>value</i> must be too small (a negative value of large magnitude or negative infinity), and the <i>result</i> is the smallest representable value of type <code>int</code>, or the <i>value</i> must be too large (a positive value of large magnitude or positive infinity), and the <i>result</i> is the largest representable value of type <code>int</code>.
Notes	The <i>f2i</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> and may also lose precision.

f2l***f2l***

Operation	Convert <code>float</code> to <code>long</code>
Format	<i>f2l</i>
Forms	<i>f2l</i> = 140 (0x8c)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>float</code>. It is popped from the operand stack and converted to a <code>long</code> <i>result</i>. The <i>result</i> is pushed onto the operand stack: • If the <i>value</i> is NaN, the result of the conversion is a <code>long</code> 0. • Otherwise, if the <i>value</i> is not an infinity, it is rounded to an integer value <i>v</i> using the round toward zero rounding policy (§2.8). If this integer value <i>v</i> can be represented as a <code>long</code>, then the <i>result</i> is the <code>long</code> value <i>v</i>. • Otherwise, either the <i>value</i> must be too small (a negative value of large magnitude or negative infinity), and the <i>result</i> is the smallest representable value of type <code>long</code>, or the <i>value</i> must be too large (a positive value of large magnitude or positive infinity), and the <i>result</i> is the largest representable value of type <code>long</code>.
Notes	The <i>f2l</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> and may also lose precision.

fadd***fadd***

Operation	Add float
Format	<i>fadd</i>
Forms	<i>fadd</i> = 98 (0x62)
Operand Stack	..., <i>value1</i> , <i>value2</i> → ..., <i>result</i>
Description	Both <i>value1</i> and <i>value2</i> must be of type float. The values are popped from the operand stack. The float <i>result</i> is <i>value1</i> + <i>value2</i>. The <i>result</i> is pushed onto the operand stack. The result of an <i>fadd</i> instruction is governed by the rules of IEEE 754 arithmetic: • If either <i>value1</i> or <i>value2</i> is NaN, the result is NaN. • The sum of two infinities of opposite sign is NaN. • The sum of two infinities of the same sign is the infinity of that sign. • The sum of an infinity and any finite value is equal to the infinity. • The sum of two zeroes of opposite sign is positive zero. • The sum of two zeroes of the same sign is the zero of that sign. • The sum of a zero and a nonzero finite value is equal to the nonzero value. • The sum of two nonzero finite values of the same magnitude and opposite sign is positive zero. • In the remaining cases, where neither operand is an infinity, a zero, or NaN and the values have the same sign or have different magnitudes, the sum is computed and rounded to the nearest representable value using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a float,

we say the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a float, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of an *fadd* instruction never throws a run-time exception.

faload***faload***

Operation	Load float from array	
Format	<table border="1"><tr><td><i>faload</i></td></tr></table>	<i>faload</i>
<i>faload</i>		
Forms	<i>faload</i> = 48 (0x30)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type float. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The float value in the component of the array at <i>index</i> is retrieved and pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>faload</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>faload</i> instruction throws an ArrayIndexOutOfBoundsException.	

fastore***fastore***

Operation	Store into float array	
Format	<table border="1"><tr><td><i>fastore</i></td></tr></table>	<i>fastore</i>
<i>fastore</i>		
Forms	<i>fastore</i> = 81 (0x51)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> , <i>value</i> → ...	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type float. The <i>index</i> must be of type int, and the <i>value</i> must be of type float. The <i>arrayref</i> , <i>index</i> , and <i>value</i> are popped from the operand stack. The float <i>value</i> is stored as the component of the array indexed by <i>index</i> .	
Run-time Exceptions	If <i>arrayref</i> is null, <i>fastore</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>fastore</i> instruction throws an ArrayIndexOutOfBoundsException.	

fcmp<op>***fcmp<op>***

Operation	Compare float
Format	<i>fcmp<op></i>
Forms	<i>fcmpg</i> = 150 (0x96) <i>fcmpl</i> = 149 (0x95)
Operand Stack	..., <i>value1</i> , <i>value2</i> → ..., <i>result</i>
Description	Both <i>value1</i> and <i>value2</i> must be of type float. The values are popped from the operand stack and a floating-point comparison is performed: • If <i>value1</i> is greater than <i>value2</i>, the int value 1 is pushed onto the operand stack. • Otherwise, if <i>value1</i> is equal to <i>value2</i>, the int value 0 is pushed onto the operand stack. • Otherwise, if <i>value1</i> is less than <i>value2</i>, the int value -1 is pushed onto the operand stack. • Otherwise, at least one of <i>value1</i> or <i>value2</i> is NaN. The <i>fcmpg</i> instruction pushes the int value 1 onto the operand stack and the <i>fcmpl</i> instruction pushes the int value -1 onto the operand stack. Floating-point comparison is performed in accordance with IEEE 754. All values other than NaN are ordered, with negative infinity less than all finite values and positive infinity greater than all finite values. Positive zero and negative zero are considered equal.
Notes	The <i>fcmpg</i> and <i>fcmpl</i> instructions differ only in their treatment of a comparison involving NaN. NaN is unordered, so any float comparison fails if either or both of its operands are NaN. With both <i>fcmpg</i> and <i>fcmpl</i> available, any float comparison may be compiled to push the same <i>result</i> onto the operand stack

whether the comparison fails on non-NaN values or fails because it encountered a NaN. For more information, see §3.5.

fconst_<f>

fconst_<f>

Operation	Push float
Format	fconst_<f>
Forms	fconst_0 = 11 (0xb) fconst_1 = 12 (0xc) fconst_2 = 13 (0xd)
Operand	... →
Stack	..., <f>
Description	Push the float constant <f> (0.0, 1.0, or 2.0) onto the operand stack.

fdiv

fdiv

Operation	Divide float
Format	fdiv
Forms	fdiv = 110 (0x6e)
Operand Stack	..., value1, value2 → ..., result
Description	Both <i>value1</i> and <i>value2</i> must be of type <code>float</code>. The values are popped from the operand stack. The <code>float</code> <i>result</i> is <i>value1</i> / <i>value2</i>. The <i>result</i> is pushed onto the operand stack. The result of an <i>fdiv</i> instruction is governed by the rules of IEEE 754 arithmetic: • If either <i>value1</i> or <i>value2</i> is NaN, the result is NaN. • If neither <i>value1</i> nor <i>value2</i> is NaN, the sign of the result is positive if both values have the same sign, negative if the values have different signs. • Division of an infinity by an infinity results in NaN. • Division of an infinity by a finite value results in a signed infinity, with the sign-producing rule just given. • Division of a finite value by an infinity results in a signed zero, with the sign-producing rule just given. • Division of a zero by a zero results in NaN; division of zero by any other finite value results in a signed zero, with the sign-producing rule just given. • Division of a nonzero finite value by a zero results in a signed infinity, with the sign-producing rule just given. • In the remaining cases, where neither operand is an infinity, a zero, or NaN, the quotient is computed and rounded to the nearest <code>float</code> using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a <code>float</code>, we say the

operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a `float`, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, division by zero, or loss of precision may occur, execution of an *fdiv* instruction never throws a run-time exception.

fload

fload

Operation Load `fload` from local variable

Format	<i>fload</i>
	<i>index</i>

Forms *fload* = 23 (0x17)

Operand ... →
Stack ..., *value*

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The local variable at *index* must contain a `fload`. The *value* of the local variable at *index* is pushed onto the operand stack.

Notes The *fload* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

fload_<n>***fload_<n>***

Operation Load `fload` from local variable

Format

<i>fload_<n></i>

Forms

fload_0 = 34 (0x22)

fload_1 = 35 (0x23)

fload_2 = 36 (0x24)

fload_3 = 37 (0x25)

Operand

... →

Stack

..., *value*

Description

The *<n>* must be an index into the local variable array of the current frame (§2.6). The local variable at *<n>* must contain a `fload`. The *value* of the local variable at *<n>* is pushed onto the operand stack.

Notes

Each of the *fload_<n>* instructions is the same as *fload* with an *index* of *<n>*, except that the operand *<n>* is implicit.

fmul***fmul*****Operation** Multiply float**Format**

<i>fmul</i>

Forms *fmul* = 106 (0x6a)**Operand Stack** ..., *value1*, *value2* →
..., *result***Description** Both *value1* and *value2* must be of type float. The values are popped from the operand stack. The float *result* is *value1* * *value2*. The *result* is pushed onto the operand stack.The result of an *fmul* instruction is governed by the rules of IEEE 754 arithmetic:

- If either *value1* or *value2* is NaN, the result is NaN.
- If neither *value1* nor *value2* is NaN, the sign of the result is positive if both values have the same sign, and negative if the values have different signs.
- Multiplication of an infinity by a zero results in NaN.
- Multiplication of an infinity by a finite value results in a signed infinity, with the sign-producing rule just given.
- In the remaining cases, where neither an infinity nor NaN is involved, the product is computed and rounded to the nearest representable value using the round to nearest rounding policy (§2.8). If the magnitude is too large to represent as a float, we say the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent as a float, we say the operation underflows; the result is then a zero of appropriate sign.

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of an *fmul* instruction never throws a run-time exception.

fneg***fneg*****Operation** Negate float**Format**

<i>fneg</i>

Forms *fneg* = 118 (0x76)**Operand** ..., *value* →**Stack** ..., *result***Description** The *value* must be of type float. It is popped from the operand stack. The float *result* is the arithmetic negation of *value*. The *result* is pushed onto the operand stack.

For float values, negation is not the same as subtraction from zero. If x is $+0.0$, then $0.0 - x$ equals $+0.0$, but $-x$ equals -0.0 . Unary minus merely inverts the sign of a float.

Special cases of interest:

- If the operand is NaN, the result is NaN (recall that NaN has no sign).

The Java Virtual Machine has not adopted the stronger requirement from the 2019 version of the IEEE 754 Standard that negation inverts the sign bit for all inputs, including NaN.

- If the operand is an infinity, the result is the infinity of opposite sign.
- If the operand is a zero, the result is the zero of opposite sign.

frem***frem*****Operation** Remainder float**Format**

<i>frem</i>

Forms *frem* = 114 (0x72)**Operand Stack** ..., *value1*, *value2* →
 ..., *result***Description** Both *value1* and *value2* must be of type float. The values are popped from the operand stack. The float *result* is calculated and pushed onto the operand stack.

The result of an *frem* instruction is not the same as the result of the remainder operation defined by IEEE 754, due to the choice of rounding policy in the Java Virtual Machine (§2.8). The IEEE 754 remainder operation computes the remainder from a rounding division, not a truncating division, and so its behavior is *not* analogous to that of the usual integer remainder operator. Instead, the Java Virtual Machine defines *frem* to behave in a manner analogous to that of the integer remainder instructions *irem* and *lrem*, with an implied division using the round toward

zero rounding policy; this may be compared with the C library function `fmod`.

The result of an *frem* instruction is governed by the following rules, which match IEEE 754 arithmetic except for how the implied division is computed:

- If either *value1* or *value2* is NaN, the result is NaN.
- If neither *value1* nor *value2* is NaN, the sign of the result equals the sign of the dividend.
- If the dividend is an infinity or the divisor is a zero or both, the result is NaN.
- If the dividend is finite and the divisor is an infinity, the result equals the dividend.
- If the dividend is a zero and the divisor is finite, the result equals the dividend.
- In the remaining cases, where neither operand is an infinity, a zero, or NaN, the floating-point remainder *result* from a dividend *value1* and a divisor *value2* is defined by the mathematical relation $result = value1 - (value2 * q)$, where *q* is an integer that is negative only if *value1* / *value2* is negative, and positive only if *value1* / *value2* is positive, and whose magnitude is as large as possible without exceeding the magnitude of the true mathematical quotient of *value1* and *value2*.

Despite the fact that division by zero may occur, evaluation of an *frem* instruction never throws a run-time exception. Overflow, underflow, or loss of precision cannot occur.

Notes

The IEEE 754 remainder operation may be computed by the library routine `Math.IEEEremainder` or `StrictMath.IEEEremainder`.

freturn***freturn***

Operation Return float from method

Format

<i>freturn</i>

Forms *freturn* = 174 (0xae)

Operand Stack ..., *value* →
[empty]

Description The current method must have return type float. The *value* must be of type float. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§monitorexit) in the current thread. If no exception is thrown, *value* is popped from the operand stack of the current frame (§2.6) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The interpreter then returns control to the invoker of the method, reinstating the frame of the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *freturn* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *freturn* throws an `IllegalMonitorStateException`.

fstore

fstore

Operation Store `float` into local variable

Format	<i>fstore</i>
	<i>index</i>

Forms *fstore* = 56 (0x38)

Operand Stack ..., *value* →
 ...

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type `float`. It is popped from the operand stack, and the value of the local variable at *index* is set to *value*.

Notes The *fstore* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

fstore_<n>***fstore_<n>***

Operation Store `f`loat into local variable

Format

<i>fstore_<n></i>

Forms *fstore_0* = 67 (0x43)
 fstore_1 = 68 (0x44)
 fstore_2 = 69 (0x45)
 fstore_3 = 70 (0x46)

Operand Stack ..., *value* →
 ...

Description The *<n>* must be an index into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type `f`loat. It is popped from the operand stack, and the value of the local variable at *<n>* is set to *value*.

Notes Each of the *fstore_<n>* instructions is the same as *fstore* with an *index* of *<n>*, except that the operand *<n>* is implicit.

fsub***fsub***

Operation Subtract float

Format

<i>fsub</i>

Forms *fsub* = 102 (0x66)

Operand ..., *value1*, *value2* →

Stack ..., *result*

Description Both *value1* and *value2* must be of type float. The values are popped from the operand stack. The float *result* is *value1* - *value2*. The *result* is pushed onto the operand stack.

For float subtraction, it is always the case that $a - b$ produces the same result as $a + (-b)$. However, for the *fsub* instruction, subtraction from zero is not the same as negation, because if x is $+0.0$, then $0.0 - x$ equals $+0.0$, but $-x$ equals -0.0 .

The Java Virtual Machine requires support of gradual underflow. Despite the fact that overflow, underflow, or loss of precision may occur, execution of an *fsub* instruction never throws a run-time exception.

getfield***getfield***

Operation Fetch field from object

Format	<i>getfield</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *getfield* = 180 (0xb4)

Operand ..., *objectref* →

Stack ..., *value*

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\text{indexbyte1} \ll 8) \mid \text{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a field (§5.1), which gives the name and descriptor of the field as well as a symbolic reference to the class in which the field is to be found. The referenced field is resolved (§5.4.3.2).

The *objectref*, which must be of type reference but not an array type, is popped from the operand stack. The *value* of the referenced field in *objectref* is fetched and pushed onto the operand stack.

Linking During resolution of the symbolic reference to the field, any of the
Exceptions errors pertaining to field resolution (§5.4.3.2) can be thrown.

Otherwise, if the resolved field is a static field, *getfield* throws an `IncompatibleClassChangeError`.

Run-time Otherwise, if *objectref* is `null`, the *getfield* instruction throws a
Exception `NullPointerException`.

Notes The *getfield* instruction cannot be used to access the `length` field of an array. The *arraylength* instruction (§*arraylength*) is used instead.

getstatic

getstatic

Operation Get static field from class

Format	<i>getstatic</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *getstatic* = 178 (0xb2)

Operand ..., →

Stack ..., *value*

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\text{indexbyte1} \ll 8) \mid \text{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a field (§5.1), which gives the name and descriptor of the field as well as a symbolic reference to the class or interface in which the field is to be found. The referenced field is resolved (§5.4.3.2).

On successful resolution of the field, the class or interface that declared the resolved field is initialized if that class or interface has not already been initialized (§5.5).

The *value* of the class or interface field is fetched and pushed onto the operand stack.

Linking Exceptions During resolution of the symbolic reference to the class or interface field, any of the exceptions pertaining to field resolution (§5.4.3.2) can be thrown.

Otherwise, if the resolved field is not a static (class) field or an interface field, *getstatic* throws an `IncompatibleClassChangeError`.

**Run-time
Exception**

Otherwise, if execution of this *getstatic* instruction causes initialization of the referenced class or interface, *getstatic* may throw an Error as detailed in §5.5.

goto

goto

Operation Branch always

Format	<i>goto</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

Forms *goto* = 167 (0xa7)

Operand No change
Stack

Description The unsigned bytes *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit *branchoffset*, where *branchoffset* is $(branchbyte1 \ll 8) \mid branchbyte2$. Execution proceeds at that offset from the address of the opcode of this *goto* instruction. The target address must be that of an opcode of an instruction within the method that contains this *goto* instruction.

goto_w

goto_w

Operation Branch always (wide index)

Format	<i>goto_w</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>
	<i>branchbyte3</i>
	<i>branchbyte4</i>

Forms *goto_w* = 200 (0xc8)

**Operand
Stack** No change

Description The unsigned bytes *branchbyte1*, *branchbyte2*, *branchbyte3*, and *branchbyte4* are used to construct a signed 32-bit *branchoffset*, where *branchoffset* is $(branchbyte1 \ll 24) \mid (branchbyte2 \ll 16) \mid (branchbyte3 \ll 8) \mid branchbyte4$. Execution proceeds at that offset from the address of the opcode of this *goto_w* instruction. The target address must be that of an opcode of an instruction within the method that contains this *goto_w* instruction.

Notes Although the *goto_w* instruction takes a 4-byte branch offset, other factors limit the size of a method to 65535 bytes (§4.11). This limit may be raised in a future release of the Java Virtual Machine.

i2b***i2b***

Operation	Convert int to byte
Format	<i>i2b</i>
Forms	<i>i2b</i> = 145 (0x91)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type int. It is popped from the operand stack, truncated to a byte, then sign-extended to an int <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>i2b</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> . The <i>result</i> may also not have the same sign as <i>value</i> .

i2c***i2c***

Operation	Convert int to char
Format	<i>i2c</i>
Forms	<i>i2c</i> = 146 (0x92)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type int. It is popped from the operand stack, truncated to char, then zero-extended to an int <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>i2c</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> . The <i>result</i> (which is always positive) may also not have the same sign as <i>value</i> .

i2d***i2d***

Operation	Convert <code>int</code> to <code>double</code>
Format	<i>i2d</i>
Forms	<i>i2d</i> = 135 (0x87)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>int</code> . It is popped from the operand stack and converted to a <code>double</code> <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>i2d</i> instruction performs a widening primitive conversion (JLS §5.1.2). Because all values of type <code>int</code> are exactly representable by type <code>double</code> , the conversion is exact.

i2f***i2f*****Operation** Convert `int` to `float`**Format**

<i>i2f</i>

Forms *i2f* = 134 (0x86)**Operand** ..., *value* →**Stack** ..., *result*

Description The *value* on the top of the operand stack must be of type `int`. It is popped from the operand stack and converted to a `float` *result* using the round to nearest rounding policy (§2.8). The *result* is pushed onto the operand stack.

Notes The *i2f* instruction performs a widening primitive conversion (JLS §5.1.2), but may result in a loss of precision because values of type `float` have only 24 significand bits.

i2l***i2l***

Operation	Convert <code>int</code> to <code>long</code>
Format	<i>i2l</i>
Forms	<i>i2l</i> = 133 (0x85)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>int</code> . It is popped from the operand stack and sign-extended to a <code>long</code> <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>i2l</i> instruction performs a widening primitive conversion (JLS §5.1.2). Because all values of type <code>int</code> are exactly representable by type <code>long</code> , the conversion is exact.

i2s***i2s***

Operation	Convert <code>int</code> to <code>short</code>
Format	<i>i2s</i>
Forms	<i>i2s</i> = 147 (0x93)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>int</code> . It is popped from the operand stack, truncated to a <code>short</code> , then sign-extended to an <code>int</code> <i>result</i> . The <i>result</i> is pushed onto the operand stack.
Notes	The <i>i2s</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> . The <i>result</i> may also not have the same sign as <i>value</i> .

iadd***iadd***

Operation Add int

Format

<i>iadd</i>

Forms *iadd* = 96 (0x60)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type `int`. The values are popped from the operand stack. The `int` *result* is *value1* + *value2*. The *result* is pushed onto the operand stack.

The result is the 32 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type `int`. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical sum of the two values.

Despite the fact that overflow may occur, execution of an *iadd* instruction never throws a run-time exception.

iaload***iaload***

Operation	Load int from array	
Format	<table border="1"><tr><td><i>iaload</i></td></tr></table>	<i>iaload</i>
<i>iaload</i>		
Forms	<i>iaload</i> = 46 (0x2e)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type int. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The int <i>value</i> in the component of the array at <i>index</i> is retrieved and pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>iaload</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>iaload</i> instruction throws an ArrayIndexOutOfBoundsException.	

iand***iand***

Operation Bitwise AND int

Format

<i>iand</i>

Forms *iand* = 126 (0x7e)

Operand ..., *value1*, *value2* →
Stack ..., *result*

Description Both *value1* and *value2* must be of type int. They are popped from the operand stack. An int *result* is calculated by taking the bitwise AND (conjunction) of *value1* and *value2*. The *result* is pushed onto the operand stack.

iastore***iastore***

Operation	Store into int array
Format	<i>iastore</i>
Forms	<i>iastore</i> = 79 (0x4f)
Operand Stack	..., <i>arrayref</i> , <i>index</i> , <i>value</i> → ...
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type int. Both <i>index</i> and <i>value</i> must be of type int. The <i>arrayref</i> , <i>index</i> , and <i>value</i> are popped from the operand stack. The int <i>value</i> is stored as the component of the array indexed by <i>index</i> .
Run-time Exceptions	If <i>arrayref</i> is null, <i>iastore</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>iastore</i> instruction throws an ArrayIndexOutOfBoundsException.

iconst_<i>***iconst_<i>***

Operation	Push int constant
Format	<i>iconst_<i></i>
Forms	<i>iconst_m1</i> = 2 (0x2) <i>iconst_0</i> = 3 (0x3) <i>iconst_1</i> = 4 (0x4) <i>iconst_2</i> = 5 (0x5) <i>iconst_3</i> = 6 (0x6) <i>iconst_4</i> = 7 (0x7) <i>iconst_5</i> = 8 (0x8)
Operand	... →
Stack	..., <i>
Description	Push the int constant <i> (-1, 0, 1, 2, 3, 4 or 5) onto the operand stack.
Notes	Each of this family of instructions is equivalent to <i>bipush <i></i> for the respective value of <i>, except that the operand <i> is implicit.

idiv***idiv***

Operation	Divide int
Format	<i>idiv</i>
Forms	<i>idiv</i> = 108 (0x6c)
Operand Stack	..., <i>value1</i>, <i>value2</i> → ..., <i>result</i>
Description	Both <i>value1</i> and <i>value2</i> must be of type int. The values are popped from the operand stack. The int <i>result</i> is the value of the Java programming language expression <i>value1</i> / <i>value2</i> (JLS §15.17.2). The <i>result</i> is pushed onto the operand stack. An int division rounds towards 0; that is, the quotient produced for int values in n/d is an int value q whose magnitude is as large as possible while satisfying $d \cdot q \leq n$. Moreover, q is positive when $n \geq d$ and n and d have the same sign, but q is negative when $n \geq d$ and n and d have opposite signs. There is one special case that does not satisfy this rule: if the dividend is the negative integer of largest possible magnitude for the int type, and the divisor is -1, then overflow occurs, and the result is equal to the dividend. Despite the overflow, no exception is thrown in this case.
Run-time Exception	If the value of the divisor in an int division is 0, <i>idiv</i> throws an <code>ArithmeticException</code> .

if_acmp<cond>

if_acmp<cond>

Operation Branch if reference comparison succeeds

Format	<i>if_acmp<cond></i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

Forms *if_acmpeq* = 165 (0xa5)
 if_acmpne = 166 (0xa6)

Operand ..., *value1*, *value2* →
Stack ...

Description Both *value1* and *value2* must be of type reference. They are both popped from the operand stack and compared. The results of the comparison are as follows:

- *if_acmpeq* succeeds if and only if *value1* = *value2*
- *if_acmpne* succeeds if and only if *value1* ≠ *value2*

If the comparison succeeds, the unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is calculated to be (*branchbyte1* << 8) | *branchbyte2*. Execution then proceeds at that offset from the address of the opcode of this *if_acmp<cond>* instruction. The target address must be that of an opcode of an instruction within the method that contains this *if_acmp<cond>* instruction.

Otherwise, if the comparison fails, execution proceeds at the address of the instruction following this *if_acmp<cond>* instruction.

if_icmp<cond>

if_icmp<cond>

Operation

Branch if int comparison succeeds

Format	<i>if_icmp<cond></i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

- Forms
- if_icmpeq* = 159 (0x9f)
if_icmpne = 160 (0xa0)
if_icmplt = 161 (0xa1)
if_icmpge = 162 (0xa2)
if_icmpgt = 163 (0xa3)
if_icmple = 164 (0xa4)

Operand Stack

..., *value1*, *value2* →
...

Description

Both *value1* and *value2* must be of type `int`. They are both popped from the operand stack and compared. All comparisons are signed. The results of the comparison are as follows:

- *if_icmpeq* succeeds if and only if *value1* = *value2*
- *if_icmpne* succeeds if and only if *value1* ≠ *value2*
- *if_icmplt* succeeds if and only if *value1* < *value2*
- *if_icmple* succeeds if and only if *value1* ≤ *value2*
- *if_icmpgt* succeeds if and only if *value1* > *value2*
- *if_icmpge* succeeds if and only if *value1* ≥ *value2*

If the comparison succeeds, the unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is calculated to be $(branchbyte1 \ll 8) \mid branchbyte2$. Execution then proceeds at that offset from the address of the opcode of this *if_icmp<cond>* instruction. The target address must

be that of an opcode of an instruction within the method that contains this *if_icmp<cond>* instruction.

Otherwise, execution proceeds at the address of the instruction following this *if_icmp<cond>* instruction.

if<cond>

if<cond>

Operation

Branch if int comparison with zero succeeds

Format	<i>if<cond></i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

- Forms
- ifeq* = 153 (0x99)

ifne = 154 (0x9a)

iflt = 155 (0x9b)

ifge = 156 (0x9c)

ifgt = 157 (0x9d)

ifle = 158 (0x9e)

Operand

..., *value* →

Stack

...

Description

The *value* must be of type `int`. It is popped from the operand stack and compared against zero. All comparisons are signed. The results of the comparisons are as follows:

- *ifeq* succeeds if and only if *value* = 0
- *ifne* succeeds if and only if *value* ≠ 0
- *iflt* succeeds if and only if *value* < 0
- *ifle* succeeds if and only if *value* ≤ 0
- *ifgt* succeeds if and only if *value* > 0
- *ifge* succeeds if and only if *value* ≥ 0

If the comparison succeeds, the unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is calculated to be $(branchbyte1 \ll 8) \mid branchbyte2$. Execution then proceeds at that offset from the address of the opcode of this *if<cond>* instruction. The target address must be

that of an opcode of an instruction within the method that contains this *if*<*cond*> instruction.

Otherwise, execution proceeds at the address of the instruction following this *if*<*cond*> instruction.

ifnonnull

ifnonnull

Operation Branch if reference not null

Format	<i>ifnonnull</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

Forms *ifnonnull* = 199 (0xc7)

Operand ..., *value* →
Stack ...

Description The *value* must be of type `reference`. It is popped from the operand stack. If *value* is not `null`, the unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is calculated to be $(branchbyte1 \ll 8) \mid branchbyte2$. Execution then proceeds at that offset from the address of the opcode of this *ifnonnull* instruction. The target address must be that of an opcode of an instruction within the method that contains this *ifnonnull* instruction.

Otherwise, execution proceeds at the address of the instruction following this *ifnonnull* instruction.

ifnull

ifnull

Operation Branch if reference is null

Format	<i>ifnull</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

Forms *ifnull* = 198 (0xc6)

Operand ..., *value* →
Stack ...

Description The *value* must of type reference. It is popped from the operand stack. If *value* is null, the unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is calculated to be $(branchbyte1 \ll 8) \mid branchbyte2$. Execution then proceeds at that offset from the address of the opcode of this *ifnull* instruction. The target address must be that of an opcode of an instruction within the method that contains this *ifnull* instruction.

Otherwise, execution proceeds at the address of the instruction following this *ifnull* instruction.

iinc***iinc***

Operation Increment local variable by constant

Format

<i>iinc</i>
<i>index</i>
<i>const</i>

Forms *iinc* = 132 (0x84)

**Operand
Stack** No change

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The *const* is an immediate signed byte. The local variable at *index* must contain an `int`. The value *const* is first sign-extended to an `int`, and then the local variable at *index* is incremented by that amount.

Notes The *iinc* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index and to increment it by a two-byte immediate signed value.

iload

iload

Operation Load int from local variable

Format	<i>iload</i>
	<i>index</i>

Forms *iload* = 21 (0x15)

Operand ... →

Stack ..., *value*

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The local variable at *index* must contain an int. The *value* of the local variable at *index* is pushed onto the operand stack.

Notes The *iload* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

iload_<n>***iload_<n>***

Operation Load int from local variable

Format

<i>iload_<n></i>

Forms *iload_0* = 26 (0x1a)
 iload_1 = 27 (0x1b)
 iload_2 = 28 (0x1c)
 iload_3 = 29 (0x1d)

Operand ... →
Stack ..., *value*

Description The <*n*> must be an index into the local variable array of the current frame (§2.6). The local variable at <*n*> must contain an int. The *value* of the local variable at <*n*> is pushed onto the operand stack.

Notes Each of the *iload_<n>* instructions is the same as *iload* with an *index* of <*n*>, except that the operand <*n*> is implicit.

imul***imul***

Operation Multiply int

Format

<i>imul</i>

Forms *imul* = 104 (0x68)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type `int`. The values are popped from the operand stack. The `int` *result* is *value1* * *value2*. The *result* is pushed onto the operand stack.

The result is the 32 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type `int`. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical multiplication of the two values.

Despite the fact that overflow may occur, execution of an *imul* instruction never throws a run-time exception.

ineg***ineg*****Operation** Negate int**Format**

<i>ineg</i>

Forms *ineg* = 116 (0x74)**Operand** ..., *value* →**Stack** ..., *result***Description** The *value* must be of type int. It is popped from the operand stack. The int *result* is the arithmetic negation of *value*, *-value*. The *result* is pushed onto the operand stack.

For int values, negation is the same as subtraction from zero. Because the Java Virtual Machine uses two's-complement representation for integers and the range of two's-complement values is not symmetric, the negation of the maximum negative int results in that same maximum negative number. Despite the fact that overflow has occurred, no exception is thrown.

For all int values *x*, *-x* equals *(~x)+1*.

instanceof***instanceof***

Operation Determine if object is of given type

Format	<i>instanceof</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *instanceof* = 193 (0xc1)

Operand ..., *objectref* →

Stack ..., *result*

Description The *objectref*, which must be of type reference, is popped from the operand stack. The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\text{indexbyte1} \ll 8) \mid \text{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a class, array, or interface type.

If *objectref* is null, the *instanceof* instruction pushes an int *result* of 0 onto the operand stack.

Otherwise, the named class, array, or interface type is resolved (§5.4.3.1). If *objectref* is a value of the type given by the resolved class, array, or interface type, the *instanceof* instruction pushes an int *result* of 1 onto the operand stack; otherwise, it pushes an int *result* of 0.

Whether *objectref* is a value of the type given by the resolved class, array, or interface type is determined according to the rules given for §*checkcast*.

Linking Exceptions During resolution of the symbolic reference to the class, array, or interface type, any of the exceptions documented in §5.4.3.1 can be thrown.

Notes The *instanceof* instruction is very similar to the *checkcast* instruction (§*checkcast*). It differs in its treatment of null, its

behavior when its test fails (*checkcast* throws an exception, *instanceof* pushes a result code), and its effect on the operand stack.

invokedynamic

invokedynamic

Operation Invoke a dynamically-computed call site

Format	<i>invokedynamic</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>
	0
	0

Forms *invokedynamic* = 186 (0xba)

Operand ..., [*arg1*, [*arg2* ...]] →

Stack ...

Description First, the unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The run-time constant pool entry at the index must be a symbolic reference to a dynamically-computed call site (§5.1). The values of the third and fourth operand bytes must always be zero.

The symbolic reference is resolved (§5.4.3.6) *for this specific invokedynamic instruction* to obtain a reference to an instance of `java.lang.invoke.CallSite`. The instance of `java.lang.invoke.CallSite` is considered "bound" to this specific *invokedynamic* instruction.

The instance of `java.lang.invoke.CallSite` indicates a *target method handle*. The *nargs* argument values are popped from the operand stack, and the target method handle is invoked. The invocation occurs as if by execution of an *invokevirtual* instruction

that indicates a run-time constant pool index to a symbolic reference R where:

- R is a symbolic reference to a method of a class;
- for the symbolic reference to the class in which the method is to be found, R specifies `java.lang.invoke.MethodHandle`;
- for the name of the method, R specifies `invokeExact`;
- for the descriptor of the method, R specifies the method descriptor in the dynamically-computed call site.

and where it is as if the following items were pushed, in order, onto the operand stack:

- a reference to the target method handle;
- the *nargs* argument values, where the number, type, and order of the values must be consistent with the method descriptor in the dynamically-computed call site.

Linking Exceptions

During resolution of the symbolic reference to a dynamically-computed call site, any of the exceptions pertaining to dynamically-computed call site resolution can be thrown.

Notes

If the symbolic reference to the dynamically-computed call site can be resolved, it implies that a non-null reference to an instance of `java.lang.invoke.CallSite` is bound to the *invokedynamic* instruction. Therefore, the target method handle, indicated by the instance of `java.lang.invoke.CallSite`, is non-null.

Similarly, successful resolution implies that the method descriptor in the symbolic reference is semantically equal to the type descriptor of the target method handle.

Together, these invariants mean that an *invokedynamic* instruction which is bound to an instance of `java.lang.invoke.CallSite` never throws a `NullPointerException` or a `java.lang.invoke.WrongMethodTypeException`.

invokeinterface

invokeinterface

Operation Invoke interface method

Format	<i>invokeinterface</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>
	<i>count</i>
	<i>0</i>

Forms *invokeinterface* = 185 (0xb9)

Operand Stack ..., *objectref*, [*arg1*, [*arg2* ...]] →
...

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The run-time constant pool entry at the index must be a symbolic reference to an interface method (§5.1), which gives the name and descriptor (§4.3.3) of the interface method as well as a symbolic reference to the interface in which the interface method is to be found. The named interface method is resolved (§5.4.3.4).

The resolved interface method must not be an instance initialization method, or the class or interface initialization method (§2.9.1, §2.9.2).

The *count* operand is an unsigned byte that must not be zero. The *objectref* must be of type reference and must be followed on the operand stack by *nargs* argument values, where the number, type, and order of the values must be consistent with the descriptor of the

resolved interface method. The value of the fourth operand byte must always be zero.

Let *c* be the class of *objectref*. A method is selected with respect to *c* and the resolved method (§5.4.6). This is the *method to be invoked*.

If the method to be invoked is synchronized, the monitor associated with *objectref* is entered or reentered as if by execution of a *monitorenter* instruction (§*monitorenter*) in the current thread.

If the method to be invoked is not native, the *nargs* argument values and *objectref* are popped from the operand stack. A new frame is created on the Java Virtual Machine stack for the method being invoked. The *objectref* and the argument values are consecutively made the values of local variables of the new frame, with *objectref* in local variable 0, *arg1* in local variable 1 (or, if *arg1* is of type long or double, in local variables 1 and 2), and so on. The new frame is then made current, and the Java Virtual Machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

If the method to be invoked is native and the platform-dependent code that implements it has not yet been bound (§5.6) into the Java Virtual Machine, then that is done. The *nargs* argument values and *objectref* are popped from the operand stack and are passed as parameters to the code that implements the method. The parameters are passed and the code is invoked in an implementation-dependent manner. When the platform-dependent code returns:

- If the native method is synchronized, the monitor associated with *objectref* is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread.
- If the native method returns a value, the return value of the platform-dependent code is converted in an implementation-dependent way to the return type of the native method and pushed onto the operand stack.

Linking Exceptions During resolution of the symbolic reference to the interface method, any of the exceptions pertaining to interface method resolution (§5.4.3.4) can be thrown.

Otherwise, if the resolved method is static, the *invokeinterface* instruction throws an *IncompatibleClassChangeError*.

Note that *invokeinterface* may refer to private methods declared in interfaces, including nestmate interfaces.

Run-time Exceptions Otherwise, if *objectref* is null, the *invokeinterface* instruction throws a *NullPointerException*.

Otherwise, if the class of *objectref* does not implement the resolved interface, *invokeinterface* throws an *IncompatibleClassChangeError*.

Otherwise, if the selected method is neither public nor private, *invokeinterface* throws an *IllegalAccessError*.

Otherwise, if the selected method is abstract, *invokeinterface* throws an *AbstractMethodError*.

Otherwise, if the selected method is native and the code that implements the method cannot be bound, *invokeinterface* throws an *UnsatisfiedLinkError*.

Otherwise, if no method is selected, and there are multiple maximally-specific superinterface methods of *C* that match the resolved method's name and descriptor and are not abstract, *invokeinterface* throws an *IncompatibleClassChangeError*.

Otherwise, if no method is selected, and there are no maximally-specific superinterface methods of *C* that match the resolved method's name and descriptor and are not abstract, *invokeinterface* throws an *AbstractMethodError*.

Notes The *count* operand of the *invokeinterface* instruction records a measure of the number of argument values, where an argument value of type *long* or type *double* contributes two units to the *count* value and an argument of any other type contributes one

unit. This information can also be derived from the descriptor of the selected method. The redundancy is historical.

The fourth operand byte exists to reserve space for an additional operand used in certain of Oracle's Java Virtual Machine implementations, which replace the *invokeinterface* instruction by a specialized pseudo-instruction at run time. It must be retained for backwards compatibility.

The *nargs* argument values and *objectref* are not one-to-one with the first *nargs*+1 local variables. Argument values of types `long` and `double` must be stored in two consecutive local variables, thus more than *nargs* local variables may be required to pass *nargs* argument values to the invoked method.

The selection logic allows a non-abstract method declared in a superinterface to be selected. Methods in interfaces are only considered if there is no matching method in the class hierarchy. In the event that there are two non-abstract methods in the superinterface hierarchy, with neither more specific than the other, an error occurs; there is no attempt to disambiguate (for example, one may be the referenced method and one may be unrelated, but we do not prefer the referenced method). On the other hand, if there are many abstract methods but only one non-abstract method, the non-abstract method is selected (unless an abstract method is more specific).

invokespecial

invokespecial

Operation	Invoke instance method; direct invocation of instance initialization methods and methods of the current class and its supertypes			
Format	<table><tr><td><i>invokespecial</i></td></tr><tr><td><i>indexbyte1</i></td></tr><tr><td><i>indexbyte2</i></td></tr></table>	<i>invokespecial</i>	<i>indexbyte1</i>	<i>indexbyte2</i>
<i>invokespecial</i>				
<i>indexbyte1</i>				
<i>indexbyte2</i>				
Forms	<i>invokespecial</i> = 183 (0xb7)			
Operand Stack	..., <i>objectref</i> , [<i>arg1</i> , [<i>arg2</i> ...]] → ...			
Description	The unsigned <i>indexbyte1</i> and <i>indexbyte2</i> are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The run-time constant pool entry at the index must be a symbolic reference to a method or an interface method (§5.1), which gives the name and descriptor (§4.3.3) of the method or interface method as well as a symbolic reference to the class or interface in which			

the method or interface method is to be found. The named method is resolved (§5.4.3.3, §5.4.3.4).

If all of the following are true, let *c* be the direct superclass of the current class:

- The resolved method is not an instance initialization method (§2.9.1).
- The symbolic reference names a class (not an interface), and that class is a superclass of the current class.
- The ACC_SUPER flag is set for the `c` class file (§4.1).

Otherwise, let *c* be the class or interface named by the symbolic reference.

The actual method to be invoked is selected by the following lookup procedure:

1. If *c* contains a declaration for an instance method with the same name and descriptor as the resolved method, then it is the method to be invoked.
2. Otherwise, if *c* is a class and has a superclass, a search for a declaration of an instance method with the same name and descriptor as the resolved method is performed, starting with the direct superclass of *c* and continuing with the direct superclass of that class, and so forth, until a match is found or no further superclasses exist. If a match is found, then it is the method to be invoked.
3. Otherwise, if *c* is an interface and the class `Object` contains a declaration of a `public` instance method with the same name and descriptor as the resolved method, then it is the method to be invoked.
4. Otherwise, if there is exactly one maximally-specific method (§5.4.3.3) in the superinterfaces of *c* that matches the resolved method's name and descriptor and is not abstract, then it is the method to be invoked.

The *objectref* must be of type *reference* and must be followed on the operand stack by *nargs* argument values, where the number,

type, and order of the values must be consistent with the descriptor of the selected instance method.

If the method is synchronized, the monitor associated with *objectref* is entered or reentered as if by execution of a *monitorenter* instruction (§*monitorenter*) in the current thread.

If the method is not native, the *nargs* argument values and *objectref* are popped from the operand stack. A new frame is created on the Java Virtual Machine stack for the method being invoked. The *objectref* and the argument values are consecutively made the values of local variables of the new frame, with *objectref* in local variable 0, *arg1* in local variable 1 (or, if *arg1* is of type long or double, in local variables 1 and 2), and so on. The new frame is then made current, and the Java Virtual Machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

If the method is native and the platform-dependent code that implements it has not yet been bound (§5.6) into the Java Virtual Machine, that is done. The *nargs* argument values and *objectref* are popped from the operand stack and are passed as parameters to the code that implements the method. The parameters are passed and the code is invoked in an implementation-dependent manner. When the platform-dependent code returns, the following take place:

- If the native method is synchronized, the monitor associated with *objectref* is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread.
- If the native method returns a value, the return value of the platform-dependent code is converted in an implementation-dependent way to the return type of the native method and pushed onto the operand stack.

Linking Exceptions

During resolution of the symbolic reference to the method, any of the exceptions pertaining to method resolution (§5.4.3.3) can be thrown.

Otherwise, if the resolved method is an instance initialization method, and the class in which it is declared is not the class

symbolically referenced by the instruction, a `NoSuchMethodError` is thrown.

Otherwise, if the resolved method is a class (static) method, the *invokespecial* instruction throws an `IncompatibleClassChangeError`.

Run-time Exceptions

Otherwise, if *objectref* is null, the *invokespecial* instruction throws a `NullPointerException`.

Otherwise, if step 1, step 2, or step 3 of the lookup procedure selects an abstract method, *invokespecial* throws an `AbstractMethodError`.

Otherwise, if step 1, step 2, or step 3 of the lookup procedure selects a native method and the code that implements the method cannot be bound, *invokespecial* throws an `UnsatisfiedLinkError`.

Otherwise, if step 4 of the lookup procedure determines there are multiple maximally-specific superinterface methods of *c* that match the resolved method's name and descriptor and are not abstract, *invokespecial* throws an `IncompatibleClassChangeError`.

Otherwise, if step 4 of the lookup procedure determines there are no maximally-specific superinterface methods of *c* that match the resolved method's name and descriptor and are not abstract, *invokespecial* throws an `AbstractMethodError`.

Notes

The difference between the *invokespecial* instruction and the *invokevirtual* instruction (§*invokevirtual*) is that *invokevirtual* invokes a method based on the class of the object. The *invokespecial* instruction is used to directly invoke instance initialization methods (§2.9.1) as well as methods of the current class and its supertypes.

The *invokespecial* instruction was named *invokenonvirtual* prior to JDK release 1.0.2.

The *nargs* argument values and *objectref* are not one-to-one with the first *nargs*+1 local variables. Argument values of types `long` and `double` must be stored in two consecutive local variables, thus

more than *nargs* local variables may be required to pass *nargs* argument values to the invoked method.

The *invokespecial* instruction handles invocation of a non-abstract interface method, referenced either via a direct superinterface or via a superclass. In these cases, the rules for selection are essentially the same as those for *invokeinterface* (except that the search starts from a different class).

invokestatic***invokestatic***

Operation Invoke a class (static) method

Format

<i>invokestatic</i>
<i>indexbyte1</i>
<i>indexbyte2</i>

Forms *invokestatic* = 184 (0xb8)

Operand ..., [*arg1*, [*arg2* ...]] →

Stack ...

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\text{indexbyte1} \ll 8) \mid \text{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a method or an interface method (§5.1), which gives the name and descriptor (§4.3.3) of the method or interface method as well as a symbolic reference to the class or interface in which

the method or interface method is to be found. The named method is resolved (§5.4.3.3, §5.4.3.4).

The resolved method must not be an instance initialization method, or the class or interface initialization method (§2.9.1, §2.9.2).

The resolved method must be *static*, and therefore cannot be *abstract*.

On successful resolution of the method, the class or interface that declared the resolved method is initialized if that class or interface has not already been initialized (§5.5).

The operand stack must contain *nargs* argument values, where the number, type, and order of the values must be consistent with the descriptor of the resolved method.

If the method is *synchronized*, the monitor associated with the resolved *Class* object is entered or reentered as if by execution of a *monitorenter* instruction (§*monitorenter*) in the current thread.

If the method is not *native*, the *nargs* argument values are popped from the operand stack. A new frame is created on the Java Virtual Machine stack for the method being invoked. The *nargs* argument values are consecutively made the values of local variables of the new frame, with *arg1* in local variable 0 (or, if *arg1* is of type *long* or *double*, in local variables 0 and 1) and so on. The new frame is then made current, and the Java Virtual Machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

If the method is *native* and the platform-dependent code that implements it has not yet been bound (§5.6) into the Java Virtual Machine, that is done. The *nargs* argument values are popped from the operand stack and are passed as parameters to the code that implements the method. The parameters are passed and the code is invoked in an implementation-dependent manner. When the platform-dependent code returns, the following take place:

- If the *native* method is *synchronized*, the monitor associated with the resolved *Class* object is updated and possibly exited

as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread.

- If the native method returns a value, the return value of the platform-dependent code is converted in an implementation-dependent way to the return type of the native method and pushed onto the operand stack.

**Linking
Exceptions**

During resolution of the symbolic reference to the method, any of the exceptions pertaining to method resolution (§5.4.3.3) can be thrown.

Otherwise, if the resolved method is an instance method, the *invokestatic* instruction throws an *IncompatibleClassChangeError*.

**Run-time
Exceptions**

Otherwise, if execution of this *invokestatic* instruction causes initialization of the referenced class or interface, *invokestatic* may throw an *Error* as detailed in §5.5.

Otherwise, if the resolved method is native and the code that implements the method cannot be bound, *invokestatic* throws an *UnsatisfiedLinkError*.

Notes

The *nargs* argument values are not one-to-one with the first *nargs* local variables. Argument values of types *long* and *double* must be stored in two consecutive local variables, thus more than *nargs* local variables may be required to pass *nargs* argument values to the invoked method.

invokevirtual***invokevirtual***

Operation Invoke instance method; dispatch based on class

Format	<i>invokevirtual</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *invokevirtual* = 182 (0xb6)

Operand Stack ..., *objectref*, [*arg1*, [*arg2* ...]] →
...

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\text{indexbyte1} \ll 8) \mid \text{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a method (§5.1), which gives the name and descriptor (§4.3.3) of the method as well as a symbolic reference to the class in which the method is to be found. The named method is resolved (§5.4.3.3).

*If the resolved method is not signature polymorphic (§2.9.3), then the *invokevirtual* instruction proceeds as follows.*

Let *c* be the class of *objectref*. A method is selected with respect to *c* and the resolved method (§5.4.6). This is the *method to be invoked*.

The *objectref* must be followed on the operand stack by *nargs* argument values, where the number, type, and order of the values must be consistent with the descriptor of the selected instance method.

If the method to be invoked is synchronized, the monitor associated with *objectref* is entered or reentered as if by execution of a *monitorenter* instruction (§*monitorenter*) in the current thread.

If the method to be invoked is not native, the *nargs* argument values and *objectref* are popped from the operand stack. A new

frame is created on the Java Virtual Machine stack for the method being invoked. The *objectref* and the argument values are consecutively made the values of local variables of the new frame, with *objectref* in local variable 0, *arg1* in local variable 1 (or, if *arg1* is of type long or double, in local variables 1 and 2), and so on. The new frame is then made current, and the Java Virtual Machine pc is set to the opcode of the first instruction of the method to be invoked. Execution continues with the first instruction of the method.

If the method to be invoked is native and the platform-dependent code that implements it has not yet been bound (§5.6) into the Java Virtual Machine, that is done. The *nargs* argument values and *objectref* are popped from the operand stack and are passed as parameters to the code that implements the method. The parameters are passed and the code is invoked in an implementation-dependent manner. When the platform-dependent code returns, the following take place:

- If the native method is synchronized, the monitor associated with *objectref* is updated and possibly exited as if by execution of a *monitorexit* instruction (§monitorexit) in the current thread.
- If the native method returns a value, the return value of the platform-dependent code is converted in an implementation-dependent way to the return type of the native method and pushed onto the operand stack.

*If the resolved method is signature polymorphic (§2.9.3), and declared in the java.lang.invoke.MethodHandle class, then the invokevirtual instruction proceeds as follows, where *D* is the descriptor of the method symbolically referenced by the instruction.*

First, a reference to an instance of `java.lang.invoke.MethodType` is obtained as if by resolution of a symbolic reference to a method type (§5.4.3.5) with the same parameter and return types as *D*.

- If the named method is `invokeExact`, the instance of `java.lang.invoke.MethodType` must be semantically equal to

the type descriptor of the receiving method handle *objectref*. The *method handle to be invoked* is *objectref*.

- If the named method is `invoke`, and the instance of `java.lang.invoke.MethodType` is semantically equal to the type descriptor of the receiving method handle *objectref*, then the *method handle to be invoked* is *objectref*.
- If the named method is `invoke`, and the instance of `java.lang.invoke.MethodType` is not semantically equal to the type descriptor of the receiving method handle *objectref*, then the Java Virtual Machine attempts to adjust the type descriptor of the receiving method handle, as if by invocation of the `asType` method of `java.lang.invoke.MethodHandle`, to obtain an exactly invokable method handle *m*. The *method handle to be invoked* is *m*.

The *objectref* must be followed on the operand stack by *nargs* argument values, where the number, type, and order of the values must be consistent with the type descriptor of the method handle to be invoked. (This type descriptor will correspond to the method descriptor appropriate for the kind of the method handle to be invoked, as specified in §5.4.3.5.)

Then, if the method handle to be invoked has bytecode behavior, the Java Virtual Machine invokes the method handle as if by execution of the bytecode behavior associated with the method handle's kind. If the kind is 5 (`REF_invokeVirtual`), 6 (`REF_invokeStatic`), 7 (`REF_invokeSpecial`), 8 (`REF_newInvokeSpecial`), or 9 (`REF_invokeInterface`), then a frame will be created and made current *in the course of executing the bytecode behavior*; however, this frame is not visible, and when the method invoked by the bytecode behavior completes (normally or abruptly), the *frame of its invoker* is considered to be the frame for the method containing this *invokevirtual* instruction.

Otherwise, if the method handle to be invoked has no bytecode behavior, the Java Virtual Machine invokes it in an implementation-dependent manner.

*If the resolved method is signature polymorphic and declared in the `java.lang.invoke.VarHandle` class, then the *invokevirtual* instruction proceeds as follows, where *N* and *D* are the name*

and descriptor of the method symbolically referenced by the instruction.

First, a reference to an instance of `java.lang.invoke.VarHandle.AccessMode` is obtained as if by invocation of the `valueFromMethodName` method of `java.lang.invoke.VarHandle.AccessMode` with a `String` argument denoting *N*.

Second, a reference to an instance of `java.lang.invoke.MethodType` is obtained as if by invocation of the `accessModeType` method of `java.lang.invoke.VarHandle` on the instance *objectref*, with the instance of `java.lang.invoke.VarHandle.AccessMode` as the argument.

Third, a reference to an instance of `java.lang.invoke.MethodHandle` is obtained as if by invocation of the `varHandleExactInvoker` method of `java.lang.invoke.MethodHandles` with the instance of `java.lang.invoke.VarHandle.AccessMode` as the first argument and the instance of `java.lang.invoke.MethodType` as the second argument. The resulting instance is called the *invoker method handle*.

Finally, the *nargs* argument values and *objectref* are popped from the operand stack, and the invoker method handle is invoked. The invocation occurs as if by execution of an *invokevirtual* instruction that indicates a run-time constant pool index to a symbolic reference *R* where:

- *R* is a symbolic reference to a method of a class;
- for the symbolic reference to the class in which the method is to be found, *R* specifies `java.lang.invoke.MethodHandle`;
- for the name of the method, *R* specifies `invoke`;
- for the descriptor of the method, *R* specifies a return type indicated by the return descriptor of *D*, and specifies a first parameter type of `java.lang.invoke.VarHandle` followed by

the parameter types indicated by the parameter descriptors of *D* (if any) in order.

and where it is as if the following items were pushed, in order, onto the operand stack:

- a reference to the instance of `java.lang.invoke.MethodHandle` (the invoker method handle);
- *objectref*;
- the *nargs* argument values, where the number, type, and order of the values must be consistent with the type descriptor of the invoker method handle.

Linking Exceptions

During resolution of the symbolic reference to the method, any of the exceptions pertaining to method resolution (§5.4.3.3) can be thrown.

Otherwise, if the resolved method is a class (static) method, the *invokevirtual* instruction throws an `IncompatibleClassChangeError`.

Otherwise, if the resolved method is signature polymorphic and declared in the `java.lang.invoke.MethodHandle` class, then during resolution of the method type derived from the descriptor in the symbolic reference to the method, any of the exceptions pertaining to method type resolution (§5.4.3.5) can be thrown.

Otherwise, if the resolved method is signature polymorphic and declared in the `java.lang.invoke.VarHandle` class, then any linking exception that may arise from invocation of the invoker method handle can be thrown. No linking exceptions are thrown from invocation of the `valueFromMethodName`, `accessModeType`, and `varHandleExactInvoker` methods.

**Run-time
Exceptions**

Otherwise, if *objectref* is null, the *invokevirtual* instruction throws a `NullPointerException`.

Otherwise, if the resolved method is not signature polymorphic:

- If the selected method is abstract, *invokevirtual* throws an `AbstractMethodError`.
- Otherwise, if the selected method is native and the code that implements the method cannot be bound, *invokevirtual* throws an `UnsatisfiedLinkError`.
- Otherwise, if no method is selected, and there are multiple maximally-specific superinterface methods of *C* that match the resolved method's name and descriptor and are not abstract, *invokevirtual* throws an `IncompatibleClassChangeError`.
- Otherwise, if no method is selected, and there are no maximally-specific superinterface methods of *C* that match the resolved method's name and descriptor and are not abstract, *invokevirtual* throws an `AbstractMethodError`.

Otherwise, if the resolved method is signature polymorphic and declared in the `java.lang.invoke.MethodHandle` class, then:

- If the method name is `invokeExact`, and the obtained instance of `java.lang.invoke.MethodType` is not semantically equal to the type descriptor of the receiving method handle *objectref*, the *invokevirtual* instruction throws a `java.lang.invoke.WrongMethodTypeException`.
- If the method name is `invoke`, and the obtained instance of `java.lang.invoke.MethodType` is not a valid argument to the `asType` method of `java.lang.invoke.MethodHandle` invoked on the receiving method handle *objectref*, the *invokevirtual* instruction throws a `java.lang.invoke.WrongMethodTypeException`.

Otherwise, if the resolved method is signature polymorphic and declared in the `java.lang.invoke.VarHandle` class, then any run-time exception that may arise from invocation of the invoker method handle can be thrown. No run-time exceptions are thrown from invocation of the `valueFromMethodName`, `accessModeType`, and `varHandleExactInvoker` methods, except `NullPointerException` if *objectref* is null.

Notes

The *nargs* argument values and *objectref* are not one-to-one with the first *nargs*+1 local variables. Argument values of types `long` and `double` must be stored in two consecutive local variables, thus more than *nargs* local variables may be required to pass *nargs* argument values to the invoked method.

It is possible that the symbolic reference of an *invokevirtual* instruction resolves to an interface method. In this case, it is possible that there is no overriding method in the class hierarchy, but that a non-abstract interface method matches the resolved method's descriptor. The selection logic matches such a method, using the same rules as for *invokeinterface*.

ior***ior***

Operation Bitwise OR int

Format

<i>ior</i>

Forms *ior* = 128 (0x80)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type int. They are popped from the operand stack. An int *result* is calculated by taking the bitwise inclusive OR of *value1* and *value2*. The *result* is pushed onto the operand stack.

irem***irem***

Operation	Remainder int
Format	<i>irem</i>
Forms	<i>irem</i> = 112 (0x70)
Operand Stack	..., <i>value1</i> , <i>value2</i> → ..., <i>result</i>
Description	Both <i>value1</i> and <i>value2</i> must be of type int. The values are popped from the operand stack. The int <i>result</i> is <i>value1</i> - (<i>value1</i> / <i>value2</i>) * <i>value2</i>. The <i>result</i> is pushed onto the operand stack. The result of the <i>irem</i> instruction is such that $(a/b) * b + (a \% b)$ is equal to <i>a</i>. This identity holds even in the special case in which the dividend is the negative int of largest possible magnitude for its type and the divisor is -1 (the remainder is 0). It follows from this rule that the result of the remainder operation can be negative only if the dividend is negative and can be positive only if the dividend is positive. Moreover, the magnitude of the result is always less than the magnitude of the divisor.
Run-time Exception	If the value of the divisor for an int remainder operator is 0, <i>irem</i> throws an ArithmeticException.

ireturn***ireturn***

Operation Return int from method

Format

<i>ireturn</i>

Forms *ireturn* = 172 (0xac)

Operand ..., *value* →

Stack [empty]

Description The current method must have return type boolean, byte, char, short, or int. The *value* must be of type int. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread. If no exception is thrown, *value* is popped from the operand stack of the current frame (§2.6) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

Prior to pushing *value* onto the operand stack of the frame of the invoker, it may have to be converted. If the return type of the invoked method was byte, char, or short, then *value* is converted from int to the return type as if by execution of *i2b*, *i2c*, or *i2s*, respectively. If the return type of the invoked method was boolean, then *value* is narrowed from int to boolean by taking the bitwise AND of *value* and 1.

The interpreter then returns control to the invoker of the method, reinstating the frame of the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *ireturn* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains

a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *ireturn* throws an `IllegalMonitorStateException`.

ishl***ishl***

Operation Shift left int

Format

<i>ishl</i>

Forms *ishl* = 120 (0x78)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type int. The values are popped from the operand stack. An int *result* is calculated by shifting *value1* left by *s* bit positions, where *s* is the value of the low 5 bits of *value2*. The *result* is pushed onto the operand stack.

Notes This is equivalent (even if overflow occurs) to multiplication by 2 to the power *s*. The shift distance actually used is always in the range 0 to 31, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x1f.

ishr***ishr***

Operation	Arithmetic shift right int	
Format	<table border="1"><tr><td><i>ishr</i></td></tr></table>	<i>ishr</i>
<i>ishr</i>		
Forms	<i>ishr</i> = 122 (0x7a)	
Operand	..., <i>value1</i> , <i>value2</i> →	
Stack	..., <i>result</i>	
Description	Both <i>value1</i> and <i>value2</i> must be of type int. The values are popped from the operand stack. An int <i>result</i> is calculated by shifting <i>value1</i> right by <i>s</i> bit positions, with sign extension, where <i>s</i> is the value of the low 5 bits of <i>value2</i> . The <i>result</i> is pushed onto the operand stack.	
Notes	The resulting value is $\text{floor}(\text{value1} / 2^s)$, where <i>s</i> is <i>value2</i> & 0x1f. For non-negative <i>value1</i> , this is equivalent to truncating int division by 2 to the power <i>s</i> . The shift distance actually used is always in the range 0 to 31, inclusive, as if <i>value2</i> were subjected to a bitwise logical AND with the mask value 0x1f.	

istore

istore

Operation Store `int` into local variable

Format	<i>istore</i>
	<i>index</i>

Forms *istore* = 54 (0x36)

Operand Stack ..., *value* →
...

Description The *index* is an unsigned byte that must be an index into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type `int`. It is popped from the operand stack, and the value of the local variable at *index* is set to *value*.

Notes The *istore* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

istore_<n>***istore_<n>***

Operation Store `int` into local variable

Format

<i>istore_<n></i>

Forms

istore_0 = 59 (0x3b)

istore_1 = 60 (0x3c)

istore_2 = 61 (0x3d)

istore_3 = 62 (0x3e)

Operand

..., *value* →

Stack

...

Description

The *<n>* must be an index into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type `int`. It is popped from the operand stack, and the value of the local variable at *<n>* is set to *value*.

Notes

Each of the *istore_<n>* instructions is the same as *istore* with an *index* of *<n>*, except that the operand *<n>* is implicit.

isub***isub***

Operation Subtract int

Format

<i>isub</i>

Forms *isub* = 100 (0x64)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type int. The values are popped from the operand stack. The int *result* is *value1* - *value2*. The *result* is pushed onto the operand stack.

For int subtraction, $a - b$ produces the same result as $a + (-b)$. For int values, subtraction from zero is the same as negation.

The result is the 32 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type int. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical difference of the two values.

Despite the fact that overflow may occur, execution of an *isub* instruction never throws a run-time exception.

iushr***iushr***

Operation	Logical shift right <code>int</code>	
Format	<table><tr><td><i>iushr</i></td></tr></table>	<i>iushr</i>
<i>iushr</i>		
Forms	<i>iushr</i> = 124 (0x7c)	
Operand Stack	..., <i>value1</i> , <i>value2</i> → ..., <i>result</i>	
Description	Both <i>value1</i> and <i>value2</i> must be of type <code>int</code> . The values are popped from the operand stack. An <code>int</code> <i>result</i> is calculated by shifting <i>value1</i> right by <i>s</i> bit positions, with zero extension, where <i>s</i> is the value of the low 5 bits of <i>value2</i> . The <i>result</i> is pushed onto the operand stack.	
Notes	If <i>value1</i> is positive and <i>s</i> is <i>value2</i> & 0x1f, the result is the same as that of <i>value1</i> >> <i>s</i> ; if <i>value1</i> is negative, the result is equal to the value of the expression (<i>value1</i> >> <i>s</i>) + (2 << ~ <i>s</i>). The addition of the (2 << ~ <i>s</i>) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 31, inclusive.	

ixor***ixor***

Operation Bitwise XOR int

Format

<i>ixor</i>

Forms *ixor* = 130 (0x82)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type int. They are popped from the operand stack. An int *result* is calculated by taking the bitwise exclusive OR of *value1* and *value2*. The *result* is pushed onto the operand stack.

jsr

jsr

Operation Jump subroutine

Format	<i>jsr</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>

Forms *jsr* = 168 (0xa8)

Operand ... →

Stack ..., *address*

Description The *address* of the opcode of the instruction immediately following this *jsr* instruction is pushed onto the operand stack as a value of type `returnAddress`. The unsigned *branchbyte1* and *branchbyte2* are used to construct a signed 16-bit offset, where the offset is $(branchbyte1 \ll 8) \mid branchbyte2$. Execution proceeds at that offset from the address of this *jsr* instruction. The target address must be that of an opcode of an instruction within the method that contains this *jsr* instruction.

Notes Note that *jsr* pushes the address onto the operand stack and *ret* (§*ret*) gets it out of a local variable. This asymmetry is intentional.

In Oracle's implementation of a compiler for the Java programming language prior to Java SE 6, the *jsr* instruction was used with the *ret* instruction in the implementation of the `finally` clause (§3.13, §4.10.2.5).

jsr_w

jsr_w

Operation Jump subroutine (wide index)

Format	<i>jsr_w</i>
	<i>branchbyte1</i>
	<i>branchbyte2</i>
	<i>branchbyte3</i>
	<i>branchbyte4</i>

Forms *jsr_w* = 201 (0xc9)

Operand ... →
Stack ..., *address*

Description The *address* of the opcode of the instruction immediately following this *jsr_w* instruction is pushed onto the operand stack as a value of type *returnAddress*. The unsigned *branchbyte1*, *branchbyte2*, *branchbyte3*, and *branchbyte4* are used to construct a signed 32-bit offset, where the offset is $(branchbyte1 \ll 24) \mid (branchbyte2 \ll 16) \mid (branchbyte3 \ll 8) \mid branchbyte4$. Execution proceeds at that offset from the address of this *jsr_w* instruction. The target address must be that of an opcode of an instruction within the method that contains this *jsr_w* instruction.

Notes Note that *jsr_w* pushes the address onto the operand stack and *ret* (§*ret*) gets it out of a local variable. This asymmetry is intentional.

In Oracle's implementation of a compiler for the Java programming language prior to Java SE 6, the *jsr_w* instruction was used with the *ret* instruction in the implementation of the finally clause (§3.13, §4.10.2.5).

Although the *jsr_w* instruction takes a 4-byte branch offset, other factors limit the size of a method to 65535 bytes (§4.11). This limit may be raised in a future release of the Java Virtual Machine.

l2d

l2d

Operation	Convert long to double
Format	<i>l2d</i>
Forms	<i>l2d</i> = 138 (0x8a)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type <code>long</code> . It is popped from the operand stack and converted to a <code>double</code> <i>result</i> using the round to nearest rounding policy (§2.8). The <i>result</i> is pushed onto the operand stack.
Notes	The <i>l2d</i> instruction performs a widening primitive conversion (JLS §5.1.2) that may lose precision because values of type <code>double</code> have only 53 significand bits.

l2f***l2f*****Operation** Convert `long` to `float`**Format**

<i>l2f</i>

Forms *l2f* = 137 (0x89)**Operand** ..., *value* →**Stack** ..., *result***Description** The *value* on the top of the operand stack must be of type `long`. It is popped from the operand stack and converted to a `float` *result* using the round to nearest rounding policy (§2.8). The *result* is pushed onto the operand stack.**Notes** The *l2f* instruction performs a widening primitive conversion (JLS §5.1.2) that may lose precision because values of type `float` have only 24 significand bits.

l2i***l2i***

Operation	Convert long to int
Format	<i>l2i</i>
Forms	<i>l2i</i> = 136 (0x88)
Operand Stack	..., <i>value</i> → ..., <i>result</i>
Description	The <i>value</i> on the top of the operand stack must be of type long. It is popped from the operand stack and converted to an int <i>result</i> by taking the low-order 32 bits of the long value and discarding the high-order 32 bits. The <i>result</i> is pushed onto the operand stack.
Notes	The <i>l2i</i> instruction performs a narrowing primitive conversion (JLS §5.1.3). It may lose information about the overall magnitude of <i>value</i> . The <i>result</i> may also not have the same sign as <i>value</i> .

ladd***ladd*****Operation** Add long**Format**

<i>ladd</i>

Forms *ladd* = 97 (0x61)**Operand Stack** ..., *value1*, *value2* →
 ..., *result***Description** Both *value1* and *value2* must be of type long. The values are popped from the operand stack. The long *result* is *value1* + *value2*. The *result* is pushed onto the operand stack.

The result is the 64 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type long. If overflow occurs, the sign of the result may not be the same as the sign of the mathematical sum of the two values.

Despite the fact that overflow may occur, execution of an *ladd* instruction never throws a run-time exception.

laload***laload***

Operation Load long from array

Format

<i>laload</i>

Forms *laload* = 47 (0x2f)

Operand ..., *arrayref*, *index* →

Stack ..., *value*

Description The *arrayref* must be of type reference and must refer to an array whose components are of type long. The *index* must be of type int. Both *arrayref* and *index* are popped from the operand stack. The long *value* in the component of the array at *index* is retrieved and pushed onto the operand stack.

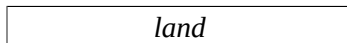
Run-time If *arrayref* is null, *laload* throws a NullPointerException.

Exceptions Otherwise, if *index* is not within the bounds of the array referenced by *arrayref*, the *laload* instruction throws an `ArrayIndexOutOfBoundsException`.

land***land***

Operation Bitwise AND long

Format



Forms *land* = 127 (0x7f)

Operand ..., *value1*, *value2* →

Stack ..., *result*

Description Both *value1* and *value2* must be of type long. They are popped from the operand stack. A long *result* is calculated by taking the bitwise AND of *value1* and *value2*. The *result* is pushed onto the operand stack.

lastore***lastore***

Operation Store into long array

Format

<i>lastore</i>

Forms *lastore* = 80 (0x50)

Operand Stack ..., *arrayref*, *index*, *value* →
...

Description The *arrayref* must be of type reference and must refer to an array whose components are of type long. The *index* must be of type int, and *value* must be of type long. The *arrayref*, *index*, and *value* are popped from the operand stack. The long *value* is stored as the component of the array indexed by *index*.

Run-time If *arrayref* is null, *lastore* throws a NullPointerException.

Exceptions Otherwise, if *index* is not within the bounds of the array referenced by *arrayref*, the *lastore* instruction throws an `ArrayIndexOutOfBoundsException`.

lcmp***lcmp*****Operation** Compare long**Format**

<i>lcmp</i>

Forms *lcmp* = 148 (0x94)**Operand Stack** ..., *value1*, *value2* →
..., *result***Description** Both *value1* and *value2* must be of type long. They are both popped from the operand stack, and a signed integer comparison is performed. If *value1* is greater than *value2*, the int value 1 is pushed onto the operand stack. If *value1* is equal to *value2*, the int value 0 is pushed onto the operand stack. If *value1* is less than *value2*, the int value -1 is pushed onto the operand stack.

lconst_<l>***lconst_<l>***

Operation Push long constant

Format

<i>lconst_<l></i>

Forms *lconst_0* = 9 (0x9)
lconst_1 = 10 (0xa)

Operand ... →

Stack ..., <l>

Description Push the long constant <l> (0 or 1) onto the operand stack.

ldc***ldc***

Operation Push item from run-time constant pool

Format	<i>ldc</i>
	<i>index</i>

Forms *ldc* = 18 (0x12)

Operand ... →

Stack ..., *value*

Description The *index* is an unsigned byte that must be a valid index into the run-time constant pool of the current class (§2.5.5). The run-time constant pool entry at *index* must be loadable (§5.1), and not any of the following:

- A numeric constant of type `long` or `double`.
- A symbolic reference to a dynamically-computed constant whose field descriptor is `J` (denoting `long`) or `D` (denoting `double`).

If the run-time constant pool entry is a numeric constant of type `int` or `float`, then the *value* of that numeric constant is pushed onto the operand stack as an `int` or `float`, respectively.

Otherwise, if the run-time constant pool entry is a string constant, that is, a reference to an instance of class `String`, then *value*, a reference to that instance, is pushed onto the operand stack.

Otherwise, if the run-time constant pool entry is a symbolic reference to a class or interface, then the named class or interface is resolved (§5.4.3.1) and *value*, a reference to the `Class` object representing that class or interface, is pushed onto the operand stack.

Otherwise, the run-time constant pool entry is a symbolic reference to a method type, a method handle, or a dynamically-computed constant. The symbolic reference is resolved (§5.4.3.5,

§5.4.3.6) and *value*, the result of resolution, is pushed onto the operand stack.

**Linking
Exceptions**

During resolution of a symbolic reference, any of the exceptions pertaining to resolution of that kind of symbolic reference can be thrown.

ldc_w

ldc_w

Operation Push item from run-time constant pool (wide index)

Format	<i>ldc_w</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *ldc_w* = 19 (0x13)

Operand ... →

Stack ..., *value*

Description The unsigned *indexbyte1* and *indexbyte2* are assembled into an unsigned 16-bit index into the run-time constant pool of the current class (§2.5.5), where the value of the index is calculated as $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The index must be a valid index into the run-time constant pool of the current class. The run-time constant pool entry at the index must be loadable (§5.1), and not any of the following:

- A numeric constant of type `long` or `double`.
- A symbolic reference to a dynamically-computed constant whose field descriptor is `J` (denoting `long`) or `D` (denoting `double`).

If the run-time constant pool entry is a numeric constant of type `int` or `float`, or a string constant, then *value* is determined and pushed onto the operand stack according to the rules given for the *ldc* instruction.

Otherwise, the run-time constant pool entry is a symbolic reference to a class, interface, method type, method handle, or dynamically-computed constant. It is resolved and *value* is determined and pushed onto the operand stack according to the rules given for the *ldc* instruction.

Linking	During resolution of a symbolic reference, any of the exceptions
Exceptions	pertaining to resolution of that kind of symbolic reference can be thrown.
Notes	The <i>ldc_w</i> instruction is identical to the <i>ldc</i> instruction (§ <i>ldc</i>) except for its wider run-time constant pool index.

ldc2_w

ldc2_w

Operation Push long or double from run-time constant pool (wide index)

Format	<i>ldc2_w</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *ldc2_w* = 20 (0x14)

Operand ... →

Stack ..., *value*

Description The unsigned *indexbyte1* and *indexbyte2* are assembled into an unsigned 16-bit index into the run-time constant pool of the current class (§2.5.5), where the value of the index is calculated as $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The index must be a valid index into the run-time constant pool of the current class. The run-time constant pool entry at the index must be loadable (§5.1), and in particular one of the following:

- A numeric constant of type long or double.
- A symbolic reference to a dynamically-computed constant whose field descriptor is J (denoting long) or D (denoting double).

If the run-time constant pool entry is a numeric constant of type long or double, then the *value* of that numeric constant is pushed onto the operand stack as a long or double, respectively.

Otherwise, the run-time constant pool entry is a symbolic reference to a dynamically-computed constant. The symbolic reference is resolved (§5.4.3.6) and *value*, the result of resolution, is pushed onto the operand stack.

Linking During resolution of a symbolic reference to a dynamically-computed constant, any of the exceptions pertaining to dynamically-computed constant resolution can be thrown.

Exceptions

Notes Only a wide-index version of the *ldc2_w* instruction exists; there is no *ldc2* instruction that pushes a long or double with a single-byte index.

ldiv***ldiv*****Operation** Divide long**Format**

<i>ldiv</i>

Forms *ldiv* = 109 (0x6d)**Operand Stack** ..., *value1*, *value2* →
..., *result***Description** Both *value1* and *value2* must be of type long. The values are popped from the operand stack. The long *result* is the value of the Java programming language expression *value1* / *value2*. The *result* is pushed onto the operand stack.

A long division rounds towards 0; that is, the quotient produced for long values in n / d is a long value q whose magnitude is as large as possible while satisfying $|d \cdot q| \leq |n|$. Moreover, q is positive when $|n| \geq |d|$ and n and d have the same sign, but q is negative when $|n| \geq |d|$ and n and d have opposite signs.

There is one special case that does not satisfy this rule: if the dividend is the negative integer of largest possible magnitude for the long type and the divisor is -1, then overflow occurs and the result is equal to the dividend; despite the overflow, no exception is thrown in this case.

Run-time Exception If the value of the divisor in a long division is 0, *ldiv* throws an `ArithmeticException`.

lload

lload

Operation Load long from local variable

Format	<i>lload</i>
	<i>index</i>

Forms *lload* = 22 (0x16)

Operand ... →
Stack ..., *value*

Description The *index* is an unsigned byte. Both *index* and *index*+1 must be indices into the local variable array of the current frame (§2.6). The local variable at *index* must contain a long. The *value* of the local variable at *index* is pushed onto the operand stack.

Notes The *lload* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

lload_<n>***lload_<n>***

Operation Load long from local variable

Format

<i>lload_<n></i>

Forms *lload_0* = 30 (0x1e)
 lload_1 = 31 (0x1f)
 lload_2 = 32 (0x20)
 lload_3 = 33 (0x21)

Operand ... →
Stack ..., *value*

Description Both <*n*> and <*n*>+1 must be indices into the local variable array of the current frame (§2.6). The local variable at <*n*> must contain a long. The *value* of the local variable at <*n*> is pushed onto the operand stack.

Notes Each of the *lload_<n>* instructions is the same as *lload* with an *index* of <*n*>, except that the operand <*n*> is implicit.

lmul***lmul***

Operation Multiply long

Format

<i>lmul</i>

Forms *lmul* = 105 (0x69)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type `long`. The values are popped from the operand stack. The `long result` is *value1* * *value2*. The *result* is pushed onto the operand stack.

The result is the 64 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type `long`. If overflow occurs, the sign of the result may not be the same as the sign of the mathematical multiplication of the two values.

Despite the fact that overflow may occur, execution of an *lmul* instruction never throws a run-time exception.

lneg***lneg*****Operation** Negate long**Format**

<i>lneg</i>

Forms *lneg* = 117 (0x75)**Operand** ..., *value* →**Stack** ..., *result*

Description The *value* must be of type long. It is popped from the operand stack. The long *result* is the arithmetic negation of *value*, *-value*. The *result* is pushed onto the operand stack.

For long values, negation is the same as subtraction from zero. Because the Java Virtual Machine uses two's-complement representation for integers and the range of two's-complement values is not symmetric, the negation of the maximum negative long results in that same maximum negative number. Despite the fact that overflow has occurred, no exception is thrown.

For all long values *x*, *-x* equals *(~x)+1*.

lookupswitch

lookupswitch

Operation Access jump table by key match and jump

Format	<i>lookupswitch</i>
	<0-3 byte pad>
	<i>defaultbyte1</i>
	<i>defaultbyte2</i>
	<i>defaultbyte3</i>
	<i>defaultbyte4</i>
	<i>npairs1</i>
	<i>npairs2</i>
	<i>npairs3</i>
	<i>npairs4</i>
	<i>match-offset pairs...</i>

Forms *lookupswitch* = 171 (0xab)

Operand ..., *key* →
Stack ...

Description A *lookupswitch* is a variable-length instruction. Immediately after the *lookupswitch* opcode, between zero and three bytes must act as padding, such that *defaultbyte1* begins at an address that is a multiple of four bytes from the start of the current method (the opcode of its first instruction). Immediately after the padding follow a series of signed 32-bit values: *default*, *npairs*, and then *npairs* pairs of signed 32-bit values. The *npairs* must be greater than or equal to 0. Each of the *npairs* pairs consists of an *int match* and a signed 32-bit *offset*. Each of these signed 32-bit values is

constructed from four unsigned bytes as $(\text{byte1} \ll 24) \mid (\text{byte2} \ll 16) \mid (\text{byte3} \ll 8) \mid \text{byte4}$.

The table *match-offset* pairs of the *lookupswitch* instruction must be sorted in increasing numerical order by *match*.

The *key* must be of type `int` and is popped from the operand stack. The *key* is compared against the *match* values. If it is equal to one of them, then a target address is calculated by adding the corresponding *offset* to the address of the opcode of this *lookupswitch* instruction. If the *key* does not match any of the *match* values, the target address is calculated by adding *default* to the address of the opcode of this *lookupswitch* instruction. Execution then continues at the target address.

The target address that can be calculated from the *offset* of each *match-offset* pair, as well as the one calculated from *default*, must be the address of an opcode of an instruction within the method that contains this *lookupswitch* instruction.

Notes

The alignment required of the 4-byte operands of the *lookupswitch* instruction guarantees 4-byte alignment of those operands if and only if the method that contains the *lookupswitch* is positioned on a 4-byte boundary.

The *match-offset* pairs are sorted to support lookup routines that are quicker than linear search.

lor***lor***

Operation Bitwise OR long

Format

<i>lor</i>

Forms *lor* = 129 (0x81)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type long. They are popped from the operand stack. A long *result* is calculated by taking the bitwise inclusive OR of *value1* and *value2*. The *result* is pushed onto the operand stack.

lrem***lrem*****Operation** Remainder long**Format**

<i>lrem</i>

Forms *lrem* = 113 (0x71)**Operand Stack** ..., *value1*, *value2* →
 ..., *result***Description** Both *value1* and *value2* must be of type long. The values are popped from the operand stack. The long *result* is *value1* - (*value1* / *value2*) * *value2*. The *result* is pushed onto the operand stack.

The result of the *lrem* instruction is such that $(a/b)*b + (a\%b)$ is equal to *a*. This identity holds even in the special case in which the dividend is the negative long of largest possible magnitude for its type and the divisor is -1 (the remainder is 0). It follows from this rule that the result of the remainder operation can be negative only if the dividend is negative and can be positive only if the dividend is positive; moreover, the magnitude of the result is always less than the magnitude of the divisor.

Run-time Exception If the value of the divisor for a long remainder operator is 0, *lrem* throws an `ArithmeticException`.

lreturn***lreturn***

Operation Return long from method

Format

<i>lreturn</i>

Forms *lreturn* = 173 (0xad)

Operand ..., *value* →

Stack [empty]

Description The current method must have return type `long`. The *value* must be of type `long`. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread. If no exception is thrown, *value* is popped from the operand stack of the current frame (§2.6) and pushed onto the operand stack of the frame of the invoker. Any other values on the operand stack of the current method are discarded.

The interpreter then returns control to the invoker of the method, reinstating the frame of the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *lreturn* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *lreturn* throws an `IllegalMonitorStateException`.

lshl***lshl***

Operation Shift left long

Format

<i>lshl</i>

Forms

lshl = 121 (0x79)

Operand

..., *value1*, *value2* →

Stack

..., *result*

Description

The *value1* must be of type long, and *value2* must be of type int. The values are popped from the operand stack. A long *result* is calculated by shifting *value1* left by *s* bit positions, where *s* is the low 6 bits of *value2*. The *result* is pushed onto the operand stack.

Notes

This is equivalent (even if overflow occurs) to multiplication by 2 to the power *s*. The shift distance actually used is therefore always in the range 0 to 63, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x3f.

lshr***lshr***

Operation Arithmetic shift right long

Format

<i>lshr</i>

Forms *lshr* = 123 (0x7b)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description The *value1* must be of type long, and *value2* must be of type int. The values are popped from the operand stack. A long *result* is calculated by shifting *value1* right by *s* bit positions, with sign extension, where *s* is the value of the low 6 bits of *value2*. The *result* is pushed onto the operand stack.

Notes The resulting value is $\text{floor}(\text{value1} / 2^s)$, where *s* is *value2* & 0x3f. For non-negative *value1*, this is equivalent to truncating long division by 2 to the power *s*. The shift distance actually used is therefore always in the range 0 to 63, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x3f.

lstore

lstore

Operation Store long into local variable

Format	<i>lstore</i>
	<i>index</i>

Forms *lstore* = 55 (0x37)

Operand Stack ..., *value* →
 ...

Description The *index* is an unsigned byte. Both *index* and *index*+1 must be indices into the local variable array of the current frame (§2.6). The *value* on the top of the operand stack must be of type long. It is popped from the operand stack, and the local variables at *index* and *index*+1 are set to *value*.

Notes The *lstore* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

lstore_<n>***lstore_<n>***

Operation	Store long into local variable
Format	<i>lstore_<n></i>
Forms	<i>lstore_0</i> = 63 (0x3f) <i>lstore_1</i> = 64 (0x40) <i>lstore_2</i> = 65 (0x41) <i>lstore_3</i> = 66 (0x42)
Operand Stack	..., <i>value</i> → ...
Description	Both < <i>n</i> > and < <i>n</i> >+1 must be indices into the local variable array of the current frame (§2.6). The <i>value</i> on the top of the operand stack must be of type long. It is popped from the operand stack, and the local variables at < <i>n</i> > and < <i>n</i> >+1 are set to <i>value</i> .
Notes	Each of the <i>lstore_<n></i> instructions is the same as <i>lstore</i> with an <i>index</i> of < <i>n</i> >, except that the operand < <i>n</i> > is implicit.

lsub***lsub*****Operation** Subtract long**Format**

<i>lsub</i>

Forms *lsub* = 101 (0x65)**Operand Stack** ..., *value1*, *value2* →
..., *result***Description** Both *value1* and *value2* must be of type `long`. The values are popped from the operand stack. The `long result` is *value1* - *value2*. The *result* is pushed onto the operand stack.

For `long` subtraction, $a - b$ produces the same result as $a + (-b)$. For `long` values, subtraction from zero is the same as negation.

The result is the 64 low-order bits of the true mathematical result in a sufficiently wide two's-complement format, represented as a value of type `long`. If overflow occurs, then the sign of the result may not be the same as the sign of the mathematical difference of the two values.

Despite the fact that overflow may occur, execution of an *lsub* instruction never throws a run-time exception.

lushr***lushr***

Operation Logical shift right long

Format

<i>lushr</i>

Forms *lushr* = 125 (0x7d)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description The *value1* must be of type long, and *value2* must be of type int. The values are popped from the operand stack. A long *result* is calculated by shifting *value1* right logically by *s* bit positions, with zero extension, where *s* is the value of the low 6 bits of *value2*. The *result* is pushed onto the operand stack.

Notes If *value1* is positive and *s* is *value2* & 0x3f, the result is the same as that of *value1* >> *s*; if *value1* is negative, the result is equal to the value of the expression (*value1* >> *s*) + (2L << ~*s*). The addition of the (2L << ~*s*) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 63, inclusive.

lxor***lxor***

Operation Bitwise XOR long

Format

<i>lxor</i>

Forms *lxor* = 131 (0x83)

Operand Stack ..., *value1*, *value2* →
 ..., *result*

Description Both *value1* and *value2* must be of type long. They are popped from the operand stack. A long *result* is calculated by taking the bitwise exclusive OR of *value1* and *value2*. The *result* is pushed onto the operand stack.

monitorenter***monitorenter***

Operation Enter monitor for object

Format

<i>monitorenter</i>

Forms *monitorenter* = 194 (0xc2)

Operand ..., *objectref* →

Stack ...

Description The *objectref* must be of type reference.

Each object is associated with a monitor. A monitor is locked if and only if it has an owner. The thread that executes *monitorenter* attempts to gain ownership of the monitor associated with *objectref*, as follows:

- If the entry count of the monitor associated with *objectref* is zero, the thread enters the monitor and sets its entry count to one. The thread is then the owner of the monitor.
- If the thread already owns the monitor associated with *objectref*, it reenters the monitor, incrementing its entry count.
- If another thread already owns the monitor associated with *objectref*, the thread blocks until the monitor's entry count is zero, then tries again to gain ownership.

Run-time Exception If *objectref* is null, *monitorenter* throws a `NullPointerException`.

Notes A *monitorenter* instruction may be used with one or more *monitorexit* instructions (§*monitorexit*) to implement a synchronized statement in the Java programming language (§3.14). The *monitorenter* and *monitorexit* instructions are not used in the implementation of synchronized methods, although they can be used to provide equivalent locking semantics. Monitor entry on invocation of a synchronized method, and monitor exit

on its return, are handled implicitly by the Java Virtual Machine's method invocation and return instructions, as if *monitorenter* and *monitorexit* were used.

The association of a monitor with an object may be managed in various ways that are beyond the scope of this specification. For instance, the monitor may be allocated and deallocated at the same time as the object. Alternatively, it may be dynamically allocated at the time when a thread attempts to gain exclusive access to the object and freed at some later time when no thread remains in the monitor for the object.

The synchronization constructs of the Java programming language require support for operations on monitors besides entry and exit. These include waiting on a monitor (`Object.wait`) and notifying other threads waiting on a monitor (`Object.notifyAll` and `Object.notify`). These operations are supported in the standard package `java.lang` supplied with the Java Virtual Machine. No explicit support for these operations appears in the instruction set of the Java Virtual Machine.

monitorexit***monitorexit***

Operation Exit monitor for object

Format

<i>monitorexit</i>

Forms *monitorexit* = 195 (0xc3)

Operand ..., *objectref* →

Stack ...

Description The *objectref* must be of type reference.

The thread that executes *monitorexit* must be the owner of the monitor associated with the instance referenced by *objectref*.

The thread decrements the entry count of the monitor associated with *objectref*. If as a result the value of the entry count is zero, the thread exits the monitor and is no longer its owner. Other threads that are blocking to enter the monitor are allowed to attempt to do so.

Run-time If *objectref* is null, *monitorexit* throws a `NullPointerException`.

Exceptions Otherwise, if the thread that executes *monitorexit* is not the owner of the monitor associated with the instance referenced by *objectref*, *monitorexit* throws an `IllegalMonitorStateException`.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the second of those rules is violated by the execution of this *monitorexit* instruction, then *monitorexit* throws an `IllegalMonitorStateException`.

Notes One or more *monitorexit* instructions may be used with a *monitorenter* instruction (§*monitorenter*) to implement a synchronized statement in the Java programming language (§3.14). The *monitorenter* and *monitorexit* instructions are not

used in the implementation of synchronized methods, although they can be used to provide equivalent locking semantics.

The Java Virtual Machine supports exceptions thrown within synchronized methods and synchronized statements differently:

- Monitor exit on normal synchronized method completion is handled by the Java Virtual Machine's return instructions. Monitor exit on abrupt synchronized method completion is handled implicitly by the Java Virtual Machine's *athrow* instruction.
- When an exception is thrown from within a synchronized statement, exit from the monitor entered prior to the execution of the synchronized statement is achieved using the Java Virtual Machine's exception handling mechanism (§3.14).

multianewarray

multianewarray

Operation Create new multidimensional array

Format	<i>multianewarray</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>
	<i>dimensions</i>

Forms *multianewarray* = 197 (0xc5)

Operand ..., *count1*, [*count2*, ...] →

Stack ..., *arrayref*

Description The *dimensions* operand is an unsigned byte that must be greater than or equal to 1. It represents the number of dimensions of the array to be created. The operand stack must contain *dimensions* values. Each such value represents the number of components in a dimension of the array to be created, must be of type *int*, and must be non-negative. The *count1* is the desired length in the first dimension, *count2* in the second, etc.

All of the *count* values are popped off the operand stack. The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The run-time constant pool entry at the index must be a symbolic reference to a class, array, or interface type. The named class, array, or interface type is resolved (§5.4.3.1). The resulting entry must be an array class type of dimensionality greater than or equal to *dimensions*.

A new multidimensional array of the array type is allocated from the garbage-collected heap. If any *count* value is zero, no subsequent dimensions are allocated. The components of the array in the first dimension are initialized to subarrays of the type of the second dimension, and so on. The components of the last allocated dimension of the array are initialized to the default initial value

(§2.3, §2.4) for the element type of the array type. A reference *arrayref* to the new array is pushed onto the operand stack.

**Linking
Exceptions**

During resolution of the symbolic reference to the class, array, or interface type, any of the exceptions documented in §5.4.3.1 can be thrown.

Otherwise, if the current class does not have permission to access the element type of the resolved array class, *multianewarray* throws an `IllegalAccessError`.

**Run-time
Exception**

Otherwise, if any of the *dimensions* values on the operand stack are less than zero, the *multianewarray* instruction throws a `NegativeArraySizeException`.

Notes

It may be more efficient to use *newarray* or *anewarray* (§*newarray*, §*anewarray*) when creating an array of a single dimension.

The array class referenced via the run-time constant pool may have more dimensions than the *dimensions* operand of the *multianewarray* instruction. In that case, only the first *dimensions* of the dimensions of the array are created.

new

new

Operation Create new object

Format	<i>new</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *new* = 187 (0xbb)

Operand ... →

Stack ..., *objectref*

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a class or interface type. The named class or interface type is resolved (§5.4.3.1) and should result in a class type. Memory for a new instance of that class is allocated from the garbage-collected heap, and the instance variables of the new object are initialized to their default initial values (§2.3, §2.4). The *objectref*, a reference to the instance, is pushed onto the operand stack.

On successful resolution of the class, it is initialized if it has not already been initialized (§5.5).

Linking During resolution of the symbolic reference to the class or
Exceptions interface type, any of the exceptions documented in §5.4.3.1 can
be thrown.

Otherwise, if the symbolic reference to the class or interface type resolves to an interface or an abstract class, *new* throws an `InstantiationError`.

Run-time Exception Otherwise, if execution of this *new* instruction causes initialization of the referenced class, *new* may throw an `Error` as detailed in JLS §15.9.4.

Notes The *new* instruction does not completely create a new instance; instance creation is not completed until an instance initialization method (§2.9.1) has been invoked on the uninitialized instance.

newarray

newarray

Operation Create new array

Format	<i>newarray</i>
	<i>atype</i>

Forms *newarray* = 188 (0xbc)

Operand ..., *count* →

Stack ..., *arrayref*

Description The *count* must be of type `int`. It is popped off the operand stack. The *count* represents the number of elements in the array to be created.

 The *atype* is a code that indicates the type of array to create. It must take one of the following values:

Table 6.5.newarray-A. Array type codes

Array Type	<i>atype</i>
T_BOOLEAN	4
T_CHAR	5
T_FLOAT	6
T_DOUBLE	7
T_BYTE	8
T_SHORT	9
T_INT	10
T_LONG	11

A new array whose components are of type *atype* and of length *count* is allocated from the garbage-collected heap. A reference *arrayref* to this new array object is pushed into the operand stack. Each of the elements of the new array is initialized to the default initial value (§2.3, §2.4) for the element type of the array type.

Run-time Exception If *count* is less than zero, *newarray* throws a `NegativeArraySizeException`.

Notes In Oracle's Java Virtual Machine implementation, arrays of type `boolean` (*atype* is `T_BOOLEAN`) are stored as arrays of 8-bit values and are manipulated using the *baload* and *bastore* instructions (*\$baload*, *\$bastore*) which also access arrays of type `byte`. Other implementations may implement packed `boolean` arrays; the *baload* and *bastore* instructions must still be used to access those arrays.

nop

nop

Operation Do nothing

Format

<i>nop</i>

Forms *nop* = 0 (0x0)

Operand No change
Stack

Description Do nothing.

pop***pop***

Operation Pop the top operand stack value

Format

<i>pop</i>

Forms

pop = 87 (0x57)

Operand

..., *value* →

Stack

...

Description

Pop the top value from the operand stack.

The *pop* instruction must not be used unless *value* is a value of a category 1 computational type (§2.11.1).

pop2

pop2

Operation Pop the top one or two operand stack values

Format

<i>pop2</i>

Forms *pop2* = 88 (0x58)

Operand Stack Form 1:
 ..., *value2*, *value1* →

 ...

 where each of *value1* and *value2* is a value of a category 1 computational type (§2.11.1).

 Form 2:
 ..., *value* →

 ...

 where *value* is a value of a category 2 computational type (§2.11.1).

Description Pop the top one or two values from the operand stack.

putfield

putfield

Operation Set field in object

Format	<i>putfield</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *putfield* = 181 (0xb5)

Operand Stack ..., *objectref*, *value* →
...

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a field (§5.1), which gives the name and descriptor of the field as well as a symbolic reference to the class in which the field is to be found. The referenced field is resolved (§5.4.3.2).

The type of a *value* stored by a *putfield* instruction must be compatible with the descriptor of the referenced field (§4.3.2). If the field descriptor type is `boolean`, `byte`, `char`, `short`, or `int`, then the *value* must be an `int`. If the field descriptor type is `float`, `long`, or `double`, then the *value* must be a `float`, `long`, or `double`, respectively. If the field descriptor type is a class type or an array type, then the *value* must be a value of the field descriptor type. If the field is `final`, it must be declared in the current class, and the instruction must occur in an instance initialization method of the current class (§2.9.1).

The *value* and *objectref* are popped from the operand stack.

The *objectref* must be of type reference but not an array type.

If the *value* is of type `int` and the field descriptor type is one of `byte`, `char`, `short`, or `boolean`, then the `int` *value* is converted to the field descriptor type as follows. If the field descriptor type is `byte`, `char`, or `short`, then the `int` *value* is truncated to a value

of the field descriptor type, *value*'. If the field descriptor type is boolean, then the int *value* is narrowed by taking the bitwise AND of *value* and 1, resulting in *value*'. The referenced field in *objectref* is set to *value*'.

Otherwise, the referenced field in *objectref* is set to *value*.

**Linking
Exceptions**

During resolution of the symbolic reference to the field, any of the exceptions pertaining to field resolution (§5.4.3.2) can be thrown.

Otherwise, if the resolved field is a static field, *putfield* throws an `IncompatibleClassChangeError`.

Otherwise, if the resolved field is `final`, it must be declared in the current class, and the instruction must occur in an instance initialization method of the current class. Otherwise, an `IllegalAccessException` is thrown.

**Run-time
Exception**

Otherwise, if *objectref* is null, the *putfield* instruction throws a `NullPointerException`.

putstatic

putstatic

Operation Set static field in class

Format	<i>putstatic</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>

Forms *putstatic* = 179 (0xb3)

Operand Stack ..., *value* →
...

Description The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the run-time constant pool of the current class (§2.6), where the value of the index is $(\textit{indexbyte1} \ll 8) \mid \textit{indexbyte2}$. The run-time constant pool entry at the index must be a symbolic reference to a field (§5.1), which gives the name and descriptor of the field as well as a symbolic reference to the class or interface in which the field is to be found. The referenced field is resolved (§5.4.3.2).

On successful resolution of the field, the class or interface that declared the resolved field is initialized if that class or interface has not already been initialized (§5.5).

The type of a *value* stored by a *putstatic* instruction must be compatible with the descriptor of the referenced field (§4.3.2). If the field descriptor type is `boolean`, `byte`, `char`, `short`, or `int`, then the *value* must be an `int`. If the field descriptor type is `float`, `long`, or `double`, then the *value* must be a `float`, `long`, or `double`, respectively. If the field descriptor type is a class type or an array type, then the *value* must be a value of the field descriptor type. If the field is `final`, it must be declared in the current class or

interface, and the instruction must occur in the class or interface initialization method of the current class or interface (§2.9.2).

The *value* is popped from the operand stack.

If the *value* is of type `int` and the field descriptor type is one of `byte`, `char`, `short`, or `boolean`, then the `int` *value* is converted to the field descriptor type as follows. If the field descriptor type is `byte`, `char`, or `short`, then the `int` *value* is truncated to a value of the field descriptor type, *value'*. If the field descriptor type is `boolean`, then the `int` *value* is narrowed by taking the bitwise AND of *value* and 1, resulting in *value'*. The referenced field in the class or interface is set to *value'*.

Otherwise, the referenced field in the class or interface is set to *value*.

Linking Exceptions

During resolution of the symbolic reference to the class or interface field, any of the exceptions pertaining to field resolution (§5.4.3.2) can be thrown.

Otherwise, if the resolved field is not a static (class) field or an interface field, *putstatic* throws an `IncompatibleClassChangeError`.

Otherwise, if the resolved field is `final`, it must be declared in the current class or interface, and the instruction must occur in the class or interface initialization method of the current class or interface. Otherwise, an `IllegalAccessError` is thrown.

Run-time Exception

Otherwise, if execution of this *putstatic* instruction causes initialization of the referenced class or interface, *putstatic* may throw an `Error` as detailed in §5.5.

Notes

A *putstatic* instruction may be used only to set the value of an interface field on the initialization of that field. Interface fields may be assigned to only once, on execution of an interface variable initialization expression when the interface is initialized (§5.5, JLS §9.3.1).

ret

ret

Operation Return from subroutine

Format	<i>ret</i>
	<i>index</i>

Forms *ret* = 169 (0xa9)

Operand Stack No change

Description The *index* is an unsigned byte between 0 and 255, inclusive. The local variable at *index* in the current frame (§2.6) must contain a value of type `returnAddress`. The contents of the local variable are written into the Java Virtual Machine's pc register, and execution continues there.

Notes Note that *jsr* (§jsr) pushes the address onto the operand stack and *ret* gets it out of a local variable. This asymmetry is intentional.

In Oracle's implementation of a compiler for the Java programming language prior to Java SE 6, the *ret* instruction was used with the *jsr* and *jsr_w* instructions (§jsr, §jsr_w) in the implementation of the `finally` clause (§3.13, §4.10.2.5).

The *ret* instruction should not be confused with the *return* instruction (§return). A *return* instruction returns control from a method to its invoker, without passing any value back to the invoker.

The *ret* opcode can be used in conjunction with the *wide* instruction (§wide) to access a local variable using a two-byte unsigned index.

return***return***

Operation Return void from method

Format

<i>return</i>

Forms *return* = 177 (0xb1)

Operand ... →

Stack [empty]

Description The current method must have return type void. If the current method is a synchronized method, the monitor entered or reentered on invocation of the method is updated and possibly exited as if by execution of a *monitorexit* instruction (§*monitorexit*) in the current thread. If no exception is thrown, any values on the operand stack of the current frame (§2.6) are discarded.

The interpreter then returns control to the invoker of the method, reinstating the frame of the invoker.

Run-time Exceptions If the Java Virtual Machine implementation does not enforce the rules on structured locking described in §2.11.10, then if the current method is a synchronized method and the current thread is not the owner of the monitor entered or reentered on invocation of the method, *return* throws an `IllegalMonitorStateException`. This can happen, for example, if a synchronized method contains a *monitorexit* instruction, but no *monitorenter* instruction, on the object on which the method is synchronized.

Otherwise, if the Java Virtual Machine implementation enforces the rules on structured locking described in §2.11.10 and if the first of those rules is violated during invocation of the current method, then *return* throws an `IllegalMonitorStateException`.

saload***saload***

Operation	Load short from array	
Format	<table border="1"><tr><td><i>saload</i></td></tr></table>	<i>saload</i>
<i>saload</i>		
Forms	<i>saload</i> = 53 (0x35)	
Operand Stack	..., <i>arrayref</i> , <i>index</i> → ..., <i>value</i>	
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type short. The <i>index</i> must be of type int. Both <i>arrayref</i> and <i>index</i> are popped from the operand stack. The component of the array at <i>index</i> is retrieved and sign-extended to an int <i>value</i> . That <i>value</i> is pushed onto the operand stack.	
Run-time Exceptions	If <i>arrayref</i> is null, <i>saload</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>saload</i> instruction throws an ArrayIndexOutOfBoundsException.	

sastore***sastore***

Operation	Store into short array
Format	<i>sastore</i>
Forms	<i>sastore</i> = 86 (0x56)
Operand Stack	..., <i>arrayref</i> , <i>index</i> , <i>value</i> → ...
Description	The <i>arrayref</i> must be of type reference and must refer to an array whose components are of type short. Both <i>index</i> and <i>value</i> must be of type int. The <i>arrayref</i> , <i>index</i> , and <i>value</i> are popped from the operand stack. The int <i>value</i> is truncated to a short and stored as the component of the array indexed by <i>index</i> .
Run-time Exceptions	If <i>arrayref</i> is null, <i>sastore</i> throws a NullPointerException. Otherwise, if <i>index</i> is not within the bounds of the array referenced by <i>arrayref</i> , the <i>sastore</i> instruction throws an <i>ArrayIndexOutOfBoundsException</i> .

sipush

sipush

Operation Push short

Format	<i>sipush</i>
	<i>byte1</i>
	<i>byte2</i>

Forms *sipush* = 17 (0x11)

Operand ... →

Stack ..., *value*

Description The immediate unsigned *byte1* and *byte2* values are assembled into an intermediate short, where the value of the short is $(byte1 \ll 8) \mid byte2$. The intermediate value is then sign-extended to an int *value*. That *value* is pushed onto the operand stack.

swap***swap***

Operation	Swap the top two operand stack values
Format	<i>swap</i>
Forms	<i>swap</i> = 95 (0x5f)
Operand Stack	..., <i>value2</i> , <i>value1</i> → ..., <i>value1</i> , <i>value2</i>
Description	Swap the top two values on the operand stack. The <i>swap</i> instruction must not be used unless <i>value1</i> and <i>value2</i> are both values of a category 1 computational type (§2.11.1).
Notes	The Java Virtual Machine does not provide an instruction implementing a swap on operands of category 2 computational types.

tableswitch

tableswitch

Operation

Access jump table by index and jump

Format	<i>tableswitch</i>
	<0-3 byte pad>
	<i>defaultbyte1</i>
	<i>defaultbyte2</i>
	<i>defaultbyte3</i>
	<i>defaultbyte4</i>
	<i>lowbyte1</i>
	<i>lowbyte2</i>
	<i>lowbyte3</i>
	<i>lowbyte4</i>
	<i>highbyte1</i>
	<i>highbyte2</i>
	<i>highbyte3</i>
	<i>highbyte4</i>
	<i>jump offsets...</i>

Forms

tableswitch = 170 (0xaa)

Operand

..., *index* →

Stack

...

Description

A *tableswitch* is a variable-length instruction. Immediately after the *tableswitch* opcode, between zero and three bytes must act as padding, such that *defaultbyte1* begins at an address that is a multiple of four bytes from the start of the current method (the opcode of its first instruction). Immediately after the padding are bytes constituting three signed 32-bit values: *default*, *low*, and *high*. Immediately following are bytes constituting a series of *high* - *low* + 1 signed 32-bit offsets. The value *low* must be less than or equal to *high*. The *high* - *low* + 1 signed 32-bit offsets are treated

as a 0-based jump table. Each of these signed 32-bit values is constructed as $(byte1 \ll 24) \mid (byte2 \ll 16) \mid (byte3 \ll 8) \mid byte4$.

The *index* must be of type `int` and is popped from the operand stack. If *index* is less than *low* or *index* is greater than *high*, then a target address is calculated by adding *default* to the address of the opcode of this *tableswitch* instruction. Otherwise, the offset at position *index* - *low* of the jump table is extracted. The target address is calculated by adding that offset to the address of the opcode of this *tableswitch* instruction. Execution then continues at the target address.

The target address that can be calculated from each jump table offset, as well as the one that can be calculated from *default*, must be the address of an opcode of an instruction within the method that contains this *tableswitch* instruction.

Notes

The alignment required of the 4-byte operands of the *tableswitch* instruction guarantees 4-byte alignment of those operands if and only if the method that contains the *tableswitch* starts on a 4-byte boundary.

wide

wide

Operation Extend local variable index by additional bytes

Format 1	<i>wide</i>
	< <i>opcode</i> >
	<i>indexbyte1</i>
	<i>indexbyte2</i>

where <*opcode*> is one of *iload*, *fload*, *aload*, *lload*, *dload*, *istore*, *fstore*, *astore*, *lstore*, *dstore*, or *ret*

Format 2	<i>wide</i>
	<i>iinc</i>
	<i>indexbyte1</i>
	<i>indexbyte2</i>
	<i>constbyte1</i>
	<i>constbyte2</i>

Forms *wide* = 196 (0xc4)

Operand Stack Same as modified instruction

Description The *wide* instruction modifies the behavior of another instruction. It takes one of two formats, depending on the instruction being modified. The first form of the *wide* instruction modifies one of the instructions *iload*, *fload*, *aload*, *lload*, *dload*, *istore*, *fstore*, *astore*, *lstore*, *dstore*, or *ret* (§*iload*, §*fload*, §*aload*, §*lload*, §*dload*, §*istore*, §*fstore*, §*astore*, §*lstore*, §*dstore*, §*ret*). The second form applies only to the *iinc* instruction (§*iinc*).

In either case, the *wide* opcode itself is followed in the compiled code by the opcode of the instruction *wide* modifies. In either form, two unsigned bytes *indexbyte1* and *indexbyte2* follow the modified opcode and are assembled into a 16-bit unsigned index to a local variable in the current frame (§2.6), where the value

of the index is $(indexbyte1 \ll 8) \mid indexbyte2$. The calculated index must be an index into the local variable array of the current frame. Where the *wide* instruction modifies an *lload*, *dload*, *lstore*, or *dstore* instruction, the index following the calculated index ($index + 1$) must also be an index into the local variable array. In the second form, two immediate unsigned bytes *constbyte1* and *constbyte2* follow *indexbyte1* and *indexbyte2* in the code stream. Those bytes are also assembled into a signed 16-bit constant, where the constant is $(constbyte1 \ll 8) \mid constbyte2$.

The widened bytecode operates as normal, except for the use of the wider index and, in the case of the second form, the larger increment range.

Notes

Although we say that *wide* "modifies the behavior of another instruction," the *wide* instruction effectively treats the bytes constituting the modified instruction as operands, denaturing the embedded instruction in the process. In the case of a modified *iinc* instruction, one of the logical operands of the *iinc* is not even at the normal offset from the opcode. The embedded instruction must never be executed directly; its opcode must never be the target of any control transfer instruction.

Opcode Mnemonics by Opcode

THIS chapter gives the mapping from Java Virtual Machine instruction opcodes, including the reserved opcodes (§6.2), to the mnemonics for the instructions represented by those opcodes.

Opcode value 186 was not used prior to Java SE 7.

Constants	Loads	Stores
00 (0x00) <i>nop</i>	21 (0x15) <i>iload</i>	54 (0x36) <i>istore</i>
01 (0x01) <i>aconst_null</i>	22 (0x16) <i>lload</i>	55 (0x37) <i>lstore</i>
02 (0x02) <i>iconst_m1</i>	23 (0x17) <i>fload</i>	56 (0x38) <i>fstore</i>
03 (0x03) <i>iconst_0</i>	24 (0x18) <i>dload</i>	57 (0x39) <i>dstore</i>
04 (0x04) <i>iconst_1</i>	25 (0x19) <i>aload</i>	58 (0x3a) <i>astore</i>
05 (0x05) <i>iconst_2</i>	26 (0x1a) <i>iload_0</i>	59 (0x3b) <i>istore_0</i>
06 (0x06) <i>iconst_3</i>	27 (0x1b) <i>iload_1</i>	60 (0x3c) <i>istore_1</i>
07 (0x07) <i>iconst_4</i>	28 (0x1c) <i>iload_2</i>	61 (0x3d) <i>istore_2</i>
08 (0x08) <i>iconst_5</i>	29 (0x1d) <i>iload_3</i>	62 (0x3e) <i>istore_3</i>
09 (0x09) <i>lconst_0</i>	30 (0x1e) <i>lload_0</i>	63 (0x3f) <i>lstore_0</i>
10 (0x0a) <i>lconst_1</i>	31 (0x1f) <i>lload_1</i>	64 (0x40) <i>lstore_1</i>
11 (0x0b) <i>fconst_0</i>	32 (0x20) <i>lload_2</i>	65 (0x41) <i>lstore_2</i>
12 (0x0c) <i>fconst_1</i>	33 (0x21) <i>lload_3</i>	66 (0x42) <i>lstore_3</i>
13 (0x0d) <i>fconst_2</i>	34 (0x22) <i>fload_0</i>	67 (0x43) <i>fstore_0</i>
14 (0x0e) <i>dconst_0</i>	35 (0x23) <i>fload_1</i>	68 (0x44) <i>fstore_1</i>
15 (0x0f) <i>dconst_1</i>	36 (0x24) <i>fload_2</i>	69 (0x45) <i>fstore_2</i>
16 (0x10) <i>bipush</i>	37 (0x25) <i>fload_3</i>	70 (0x46) <i>fstore_3</i>
17 (0x11) <i>sipush</i>	38 (0x26) <i>dload_0</i>	71 (0x47) <i>dstore_0</i>
18 (0x12) <i>ldc</i>	39 (0x27) <i>dload_1</i>	72 (0x48) <i>dstore_1</i>
19 (0x13) <i>ldc_w</i>	40 (0x28) <i>dload_2</i>	73 (0x49) <i>dstore_2</i>
20 (0x14) <i>ldc2_w</i>	41 (0x29) <i>dload_3</i>	74 (0x4a) <i>dstore_3</i>
	42 (0x2a) <i>aload_0</i>	75 (0x4b) <i>astore_0</i>
	43 (0x2b) <i>aload_1</i>	76 (0x4c) <i>astore_1</i>
	44 (0x2c) <i>aload_2</i>	77 (0x4d) <i>astore_2</i>
	45 (0x2d) <i>aload_3</i>	78 (0x4e) <i>astore_3</i>
	46 (0x2e) <i>iaload</i>	79 (0x4f) <i>iastore</i>
	47 (0x2f) <i>laload</i>	80 (0x50) <i>lastore</i>
	48 (0x30) <i>faload</i>	81 (0x51) <i>fastore</i>
	49 (0x31) <i>daload</i>	82 (0x52) <i>dastore</i>
	50 (0x32) <i>aaload</i>	83 (0x53) <i>aastore</i>
	51 (0x33) <i>baload</i>	84 (0x54) <i>bastore</i>
	52 (0x34) <i>caload</i>	85 (0x55) <i>castore</i>
	53 (0x35) <i>saload</i>	86 (0x56) <i>sastore</i>

OPCODE MNEMONICS BY OPCODE

Stack	Math	Conversions
87 (0x57) <i>pop</i>	96 (0x60) <i>iadd</i>	133 (0x85) <i>i2l</i>
88 (0x58) <i>pop2</i>	97 (0x61) <i>ladd</i>	134 (0x86) <i>i2f</i>
89 (0x59) <i>dup</i>	98 (0x62) <i>fadd</i>	135 (0x87) <i>i2d</i>
90 (0x5a) <i>dup_x1</i>	99 (0x63) <i>dadd</i>	136 (0x88) <i>l2i</i>
91 (0x5b) <i>dup_x2</i>	100 (0x64) <i>isub</i>	137 (0x89) <i>l2f</i>
92 (0x5c) <i>dup2</i>	101 (0x65) <i>lsub</i>	138 (0x8a) <i>l2d</i>
93 (0x5d) <i>dup2_x1</i>	102 (0x66) <i>fsub</i>	139 (0x8b) <i>f2i</i>
94 (0x5e) <i>dup2_x2</i>	103 (0x67) <i>dsub</i>	140 (0x8c) <i>f2l</i>
95 (0x5f) <i>swap</i>	104 (0x68) <i>imul</i>	141 (0x8d) <i>f2d</i>
	105 (0x69) <i>lmul</i>	142 (0x8e) <i>d2i</i>
	106 (0x6a) <i>fmul</i>	143 (0x8f) <i>d2l</i>
	107 (0x6b) <i>dmul</i>	144 (0x90) <i>d2f</i>
	108 (0x6c) <i>idiv</i>	145 (0x91) <i>i2b</i>
	109 (0x6d) <i>ldiv</i>	146 (0x92) <i>i2c</i>
	110 (0x6e) <i>fdiv</i>	147 (0x93) <i>i2s</i>
	111 (0x6f) <i>ddiv</i>	
	112 (0x70) <i>irem</i>	
	113 (0x71) <i>lrem</i>	
	114 (0x72) <i>frem</i>	
	115 (0x73) <i>drem</i>	
	116 (0x74) <i>ineg</i>	
	117 (0x75) <i>lneg</i>	
	118 (0x76) <i>fneg</i>	
	119 (0x77) <i>dneg</i>	
	120 (0x78) <i>ishl</i>	
	121 (0x79) <i>lshl</i>	
	122 (0x7a) <i>ishr</i>	
	123 (0x7b) <i>lshr</i>	
	124 (0x7c) <i>iushr</i>	
	125 (0x7d) <i>lushr</i>	
	126 (0x7e) <i>iand</i>	
	127 (0x7f) <i>land</i>	
	128 (0x80) <i>ior</i>	
	129 (0x81) <i>lor</i>	
	130 (0x82) <i>ixor</i>	
	131 (0x83) <i>lxor</i>	
	132 (0x84) <i>iinc</i>	

Comparisons

148 (0x94)	<i>lcmp</i>
149 (0x95)	<i>fcmpl</i>
150 (0x96)	<i>fcmpg</i>
151 (0x97)	<i>dcmpi</i>
152 (0x98)	<i>dcmpg</i>
153 (0x99)	<i>ifeq</i>
154 (0x9a)	<i>ifne</i>
155 (0x9b)	<i>iflt</i>
156 (0x9c)	<i>ifge</i>
157 (0x9d)	<i>ifgt</i>
158 (0x9e)	<i>ifle</i>
159 (0x9f)	<i>if_icmpeq</i>
160 (0xa0)	<i>if_icmpne</i>
161 (0xa1)	<i>if_icmplt</i>
162 (0xa2)	<i>if_icmpge</i>
163 (0xa3)	<i>if_icmpgt</i>
164 (0xa4)	<i>if_icmple</i>
165 (0xa5)	<i>if_acmpeq</i>
166 (0xa6)	<i>if_acmpne</i>

Control

167 (0xa7)	<i>goto</i>
168 (0xa8)	<i>jsr</i>
169 (0xa9)	<i>ret</i>
170 (0xaa)	<i>tableswitch</i>
171 (0xab)	<i>lookupswitch</i>
172 (0xac)	<i>ireturn</i>
173 (0xad)	<i>lreturn</i>
174 (0xae)	<i>freturn</i>
175 (0xaf)	<i>dreturn</i>
176 (0xb0)	<i>areturn</i>
177 (0xb1)	<i>return</i>

References

178 (0xb2)	<i>getstatic</i>
179 (0xb3)	<i>putstatic</i>
180 (0xb4)	<i>getfield</i>
181 (0xb5)	<i>putfield</i>
182 (0xb6)	<i>invokevirtual</i>
183 (0xb7)	<i>invokespecial</i>
184 (0xb8)	<i>invokestatic</i>
185 (0xb9)	<i>invokeinterface</i>
186 (0xba)	<i>invokedynamic</i>
187 (0xbb)	<i>new</i>
188 (0xbc)	<i>newarray</i>
189 (0xbd)	<i>anewarray</i>
190 (0xbe)	<i>arraylength</i>
191 (0xbf)	<i>athrow</i>
192 (0xc0)	<i>checkcast</i>
193 (0xc1)	<i>instanceof</i>
194 (0xc2)	<i>monitorenter</i>
195 (0xc3)	<i>monitorexit</i>

Extended

196 (0xc4)	<i>wide</i>
197 (0xc5)	<i>multianewarray</i>
198 (0xc6)	<i>ifnull</i>
199 (0xc7)	<i>ifnonnull</i>
200 (0xc8)	<i>goto_w</i>
201 (0xc9)	<i>jsr_w</i>

Reserved

202 (0xca)	<i>breakpoint</i>
254 (0xfe)	<i>impdep1</i>
255 (0xff)	<i>impdep2</i>

Appendix A. Limited License Grant

ORACLE AMERICA, INC. IS WILLING TO LICENSE THIS SPECIFICATION TO YOU ONLY UPON THE CONDITION THAT YOU ACCEPT ALL OF THE TERMS CONTAINED IN THIS AGREEMENT ("AGREEMENT"). PLEASE READ THE TERMS AND CONDITIONS OF THIS AGREEMENT CAREFULLY.

Specification: JSR-401 Java SE 26

Version: 26

Status: Final Release

Release: March 2026

Copyright © 1997, 2026, Oracle America, Inc.

All rights reserved.

LIMITED LICENSE GRANTS

1. License for Evaluation Purposes. Specification Lead hereby grants you a fully-paid, non-exclusive, non-transferable, worldwide, limited license (without the right to sublicense), under Specification Lead's applicable intellectual property rights to view, download, use and reproduce the Specification only for the purpose of internal evaluation. This includes (i) developing applications intended to run on an implementation of the Specification, provided that such applications do not themselves implement any portion(s) of the Specification, and (ii) discussing the Specification with any third party; and (iii) excerpting brief portions of the Specification in oral or written communications which discuss the Specification provided that such excerpts do not in the aggregate constitute a significant portion of the Specification.

2. License for the Distribution of Compliant Implementations. Specification Lead also grants you a perpetual, non-exclusive, non-transferable, worldwide, fully paid-up, royalty free, limited license (without the right to sublicense) under any applicable copyrights or, subject to the provisions of subsection 4 below, patent rights it may have covering the Specification to create and/or distribute an Independent Implementation of the Specification that: (a) fully implements the Specification including all its required interfaces and functionality; (b) does not modify, subset, superset or otherwise extend the Licensor Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within

the Licensor Name Space other than those required/authorized by the Specification or Specifications being implemented; and (c) passes the Technology Compatibility Kit (including satisfying the requirements of the applicable TCK Users Guide) for such Specification ("Compliant Implementation"). In addition, the foregoing license is expressly conditioned on your not acting outside its scope. No license is granted hereunder for any other purpose (including, for example, modifying the Specification, other than to the extent of your fair use rights, or distributing the Specification to third parties). Also, no right, title, or interest in or to any trademarks, service marks, or trade names of Specification Lead or Specification Lead's licensors is granted hereunder. Java, and Java-related logos, marks and names are trademarks or registered trademarks of Oracle America, Inc. in the U.S. and other countries.

3. Pass-through Conditions. You need not include limitations (a)-(c) from the previous paragraph or any other particular "pass through" requirements in any license You grant concerning the use of your Independent Implementation or products derived from it. However, except with respect to Independent Implementations (and products derived from them) that satisfy limitations (a)-(c) from the previous paragraph, You may neither: (a) grant or otherwise pass through to your licensees any licenses under Specification Lead's applicable intellectual property rights; nor (b) authorize your licensees to make any claims concerning their implementation's compliance with the Specification in question.

4. Reciprocity Concerning Patent Licenses.

a. With respect to any patent claims covered by the license granted under subparagraph 2 above that would be infringed by all technically feasible implementations of the Specification, such license is conditioned upon your offering on fair, reasonable and non-discriminatory terms, to any party seeking it from You, a perpetual, non-exclusive, non-transferable, worldwide license under Your patent rights which are or would be infringed by all technically feasible implementations of the Specification to develop, distribute and use a Compliant Implementation.

b. With respect to any patent claims owned by Specification Lead and covered by the license granted under subparagraph 2, whether or not their infringement can be avoided in a technically feasible manner when implementing the Specification, such license shall terminate with respect to such claims if You initiate a claim against Specification Lead that it has, in the course of performing its responsibilities as the Specification Lead, induced any other entity to infringe Your patent rights.

c. Also with respect to any patent claims owned by Specification Lead and covered by the license granted under subparagraph 2 above, where the infringement of

such claims can be avoided in a technically feasible manner when implementing the Specification such license, with respect to such claims, shall terminate if You initiate a claim against Specification Lead that its making, having made, using, offering to sell, selling or importing a Compliant Implementation infringes Your patent rights.

5. Definitions. For the purposes of this Agreement: "Independent Implementation" shall mean an implementation of the Specification that neither derives from any of Specification Lead's source code or binary code materials nor, except with an appropriate and separate license from Specification Lead, includes any of Specification Lead's source code or binary code materials; "Licensor Name Space" shall mean the public class or interface declarations whose names begin with "java", "javax", "com.oracle", "com.sun" or their equivalents in any subsequent naming convention adopted by Oracle America, Inc. through the Java Community Process, or any recognized successors or replacements thereof; and "Technology Compatibility Kit" or "TCK" shall mean the test suite and accompanying TCK User's Guide provided by Specification Lead which corresponds to the Specification and that was available either (i) from Specification Lead 120 days before the first release of Your Independent Implementation that allows its use for commercial purposes, or (ii) more recently than 120 days from such release but against which You elect to test Your implementation of the Specification.

This Agreement will terminate immediately without notice from Specification Lead if you breach the Agreement or act outside the scope of the licenses granted above.

DISCLAIMER OF WARRANTIES

THE SPECIFICATION IS PROVIDED "AS IS". SPECIFICATION LEAD MAKES NO REPRESENTATIONS OR WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT (INCLUDING AS A CONSEQUENCE OF ANY PRACTICE OR IMPLEMENTATION OF THE SPECIFICATION), OR THAT THE CONTENTS OF THE SPECIFICATION ARE SUITABLE FOR ANY PURPOSE. This document does not represent any commitment to release or implement any portion of the Specification in any product. In addition, the Specification could include technical inaccuracies or typographical errors.

LIMITATION OF LIABILITY

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL SPECIFICATION LEAD OR ITS LICENSORS BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION, LOST REVENUE,

PROFITS OR DATA, OR FOR SPECIAL, INDIRECT, CONSEQUENTIAL, INCIDENTAL OR PUNITIVE DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF OR RELATED IN ANY WAY TO YOUR HAVING, IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION, EVEN IF SPECIFICATION LEAD AND/OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

You will indemnify, hold harmless, and defend Specification Lead and its licensors from any claims arising or resulting from: (i) your use of the Specification; (ii) the use or distribution of your Java application, applet and/or implementation; and/or (iii) any claims that later versions or releases of any Specification furnished to you are incompatible with the Specification provided to you under this license.

RESTRICTED RIGHTS LEGEND

U.S. Government: If this Specification is being acquired by or on behalf of the U.S. Government or by a U.S. Government prime contractor or subcontractor (at any tier), then the Government's rights in the Software and accompanying documentation shall be only as set forth in this license; this is in accordance with 48 C.F.R. 227.7201 through 227.7202-4 (for Department of Defense (DoD) acquisitions) and with 48 C.F.R. 2.101 and 12.212 (for non-DoD acquisitions).

REPORT

If you provide Specification Lead with any comments or suggestions concerning the Specification ("Feedback"), you hereby: (i) agree that such Feedback is provided on a non-proprietary and non-confidential basis, and (ii) grant Specification Lead a perpetual, non-exclusive, worldwide, fully paid-up, irrevocable license, with the right to sublicense through multiple levels of sublicensees, to incorporate, disclose, and use without limitation the Feedback for any purpose.

GENERAL TERMS

Any action related to this Agreement will be governed by California law and controlling U.S. federal law. The U.N. Convention for the International Sale of Goods and the choice of law rules of any jurisdiction will not apply.

The Specification is subject to U.S. export control laws and may be subject to export or import regulations in other countries. Licensee agrees to comply strictly with all such laws and regulations and acknowledges that it has the responsibility to obtain such licenses to export, re-export or import as may be required after delivery to Licensee.

LIMITED LICENSE GRANT

This Agreement is the parties' entire agreement relating to its subject matter. It supersedes all prior or contemporaneous oral or written communications, proposals, conditions, representations and warranties and prevails over any conflicting or additional terms of any quote, order, acknowledgment, or other communication between the parties relating to its subject matter during the term of this Agreement. No modification to this Agreement will be binding, unless in writing and signed by an authorized representative of each party.

